



RS-Bank V. 6

**Проблемно-
ориентированный
язык RSL**

Компания "Эр-Стайл Софтлаб" не дает гарантий, что содержание настоящего Руководства соответствует тем или иным требованиям проектов в конкретных кредитных организациях и их целям. Компания оставляет за собой право перерабатывать настоящее издание, вносить в него изменения и дополнения, не уведомляя об этом частных лиц и организации.

Программное обеспечение, описанное в настоящем документе, поставляется строго по лицензионному соглашению. Авторские права компании "Эр-Стайл Софтлаб" защищены законом. Копирование и распространение программного обеспечения и документации к нему в какой бы то ни было форме и любыми средствами, включая фотокопирование и запись на магнитные носители, в отсутствие специального соглашения является противозаконным.

© АО Эр-Стайл Софтлаб, 2005 – 2023

Все права защищены.

Дата редакции: 08.09.2023

Оглавление

Введение	25
Элементы языка.....	27
Служебные слова.....	27
Имена	27
Область видимости имен.....	28
Комментарии.....	29
Объекты языка	29
Типы данных.....	30
Переменные с декларацией типа данных.....	30
Переменные без декларации типа данных.....	31
Символические константы и специальные значения переменных.....	31
Скалярные типы	32
Объектные типы	33
Константы.....	34
Выражения.....	36
Синтаксис.....	36
Семантика.....	37
Структура RSL-программы.....	41
Загрузка и кэширование модулей программы исполняющей системой RSL	44

Директива IMPORT	44
Конструкции языка RSL.....	47
Пустая инструкция	47
Инструкция "выражение"	47
Условная инструкция IF	47
Инструкция цикла WHILE	48
Инструкция цикла FOR	48
Инструкция прерывания BREAK и продолжения CONTINUE..	50
Инструкция возврата RETURN.....	50
Инструкция вывода.....	51
Формат управляющей строки	51
Определение переменных VAR.....	52
Определение классов и объектов	53
Определение символических констант CONST.....	57
Определение процедуры MACRO	57
Процедуры языка RSL.....	58
Передача параметров.....	58
Определение массивов.....	60
Конструкция ARRAY	60
Стандартный класс TArray.....	61

Поддержка технологии ActiveX в RSL	63
Использование специальных значений	65
Соответствие типов данных языка RSL и ActiveX	66
Поддержка стандартных коллекций	67
Обращение к свойствам объектов с параметрами	67
Доступ к объектам Object RSL из других языков	68
Обработка событий	69
Класс TRslEvHandler	69
Свойства класса TRslEvHandler	73
Свойство EvSource	73
Свойство TypeLib	74
Методы класса TRslEvHandler	74
Метод SetHandler	74
Метод RemHandler	74
Метод raise	75
Класс RslTimer	75
Методы класса RslTimer	76
Метод RemTimer	76
Метод SetTimer	77
Передача параметров	77
Поддержка модулей платформы .NET	78
Обработка ошибок, возникающих во время выполнения программы	80
Автоматическое создание объектов Object RSL	82
Конструкция WITH	82

Организация ввода/вывода.....	83
Спецификаторы форматирования.....	83
Формирование отчетов с использованием шаблонов.....	85
Стандартный класс TRepForm.....	86
Стандартный класс TPattFieldR.....	89
Поддержка интерактивного режима.....	89
Меню.....	89
Вертикальное меню.....	89
Горизонтальное меню.....	90
Диалоговые окна.....	91
Процедура обработки сообщений.....	92
Список обрабатываемых сообщений.....	93
Список обрабатываемых сообщений процедуры RunDialog.....	93
Список обрабатываемых сообщений процедуры RunScroll.....	94
Значение, возвращаемое процедурой обработки сообщений.....	95
Скроллинг.....	99
Пример программы обработки скроллинга.....	101
Создание источников данных с помощью класса ToolsDataAdapter.....	103
Метод AddColumn.....	109
Работа с файлами и таблицами баз данных.....	111
Использование стандартного класса Tbfile.....	111
Методы класса Tbfile.....	113
Свойства класса Tbfile.....	115

Использование стандартного класса TRecHandler	116
Использование конструкций FILE и RECORD	119
Работа с текстовыми файлами	122
Работа с файлами в формате DBF	124
Работа с записями таблиц как с записями переменной длины	125
Доступ к источникам данных с помощью библиотеки RSD ...	125
Описание библиотеки RSD	126
Класс RsdEnvironment	126
Свойства класса	127
Методы класса	127
Класс RsdConnection	128
Свойства класса	129
Методы класса	129
Класс RsdCommand	129
Свойства класса	131
Методы класса	131
Класс RsdRecordset	133
Свойства класса	134
Методы класса	135
Класс RsdError	137
Класс RsdField	138
Свойства класса	138
Методы класса	138
Класс RsdParameter	138
Свойства класса	139

Методы класса.....	139
Использование RSD в программах на языке RSL.....	139
Обработка ошибок при работе с таблицами баз данных.....	141
Обработка транзакций.....	144
Работа с текстовыми и двоичными файлами.....	147
Использование класса TStream.....	147
Свойства класса.....	148
Методы класса.....	148
Метод Write.....	148
Метод Read.....	150
Метод Write2.....	150
Метод Read2.....	151
Метод WriteVal.....	152
Метод ReadVal.....	152
Метод Copy.....	153
Метод SetPos.....	153
Метод GetPos.....	154
Метод SetSize.....	154
Метод GetSize.....	154
Метод Flush.....	154
Использование класса TStreamDoc.....	155
Свойства класса.....	156
Методы класса.....	156
Управление файлами и каталогами.....	159

Использование класса TDirList.....	160
Свойства класса.....	160
Методы класса.....	162
Использование "домашних" каталогов пользователей..	167
Использование RSCOM-объектов из программ на языке RSL.....	169
Описание стандартного модуля RCW.....	170
Процедуры.....	170
Процедура CreateMarkObj.....	170
Процедура ReadMarkObj.....	170
Процедура CreateObject.....	171
Процедура GetNamedChanel.....	172
Процедура ClrRmtStubs.....	172
Процедура PrintObject.....	172
Процедура LockRcwHost.....	173
Процедура UnlockRcwHost.....	173
Класс TRslChanel.....	173
Свойства класса TRslChanel.....	175
Методы класса TRslChanel.....	175
Класс TRcwSite.....	175
Обработка ошибок RSCOM в языке RSL.....	176
Обработка сообщений от RSCOM-объектов.....	176
Стандартные RSCOM-серверы.....	179
RSCOM-сервер rsax.d32.....	179

Методы класса TRsAxServer.....	179
RSCOM-сервер rcwhost.d32.....	181
Использование класса TRcwHost.....	182
Свойства класса TRcwHost.....	182
Методы класса TRcwHost.....	183
RSCOM-сервер rsclr.d32.....	185
Методы класса TClrHost.....	185
RSCOM-сервер rsjvm.d32.....	186
Класс TJavaHost.....	187
Методы класса.....	187
Класс TJavaObj.....	190
Строковый идентификатор метода.....	191
Кастомизация процесса создания Java объектов.....	191
Передача параметров между приложением хостом и Java кодом.....	192
Инструментальная библиотека rsjvm.jar.....	192
Класс RsJvm.....	193
Методы класса.....	193
Класс RSOject.....	195
Методы класса.....	195
Класс Connection.....	197
Класс RSMethod.....	199
Класс CppWrapGen.....	199
Соответствие типов данных и передача параметров.....	199
Передача объектов между RSCOM и JAVA.....	202
Вызов перегруженных методов.....	203

Поддержка метаинформации RSCOM объектом TJavaObj.....	205
Настроечный файл rsjvm.ini (2).....	205
Отладка Java кода при использовании rsjvm.d32.....	206
Рекомендации по использованию.....	207
Использование RSCOM из автономных Java приложений.....	208
Встроенные процедуры.....	209
Стандартные процедуры ввода данных с клавиатуры.....	209
Процедура GetInt.....	210
Процедура GetDouble.....	210
Процедура GetMoney.....	210
Процедура GetNumeric.....	210
Процедура GetString.....	210
Процедура GetStringR.....	210
Процедура GetTime.....	211
Процедура GetDate.....	211
Процедура GetTRUE.....	211
Стандартные процедуры вывода.....	211
Процедура Print.....	211
Процедура PrintGlobs.....	212
Процедура PrintFiles.....	212
Процедура PrintLn.....	212
Процедура PrintLocs.....	213
Процедура PrintModule.....	213

Процедура PrintProps.....	213
Процедура PrintRefs.....	213
Процедура PrintStack.....	213
Процедура PrintSymModule.....	214
Процедура Message.....	214
Процедура SetOutput.....	214
Процедура SetColumn.....	214
Процедура FlushColumn.....	215
Процедура ClearColumn.....	216
Процедура SetDefPrec.....	216
Процедура SetOutHandler.....	216
Процедура GetPRNInfo.....	217
Процедура SetPRNInfo.....	217
Преобразование типов значений переменных.....	217
Процедура ValType.....	217
Процедура Double.....	218
Процедура DoubleL.....	218
Процедура Int.....	218
Процедура String.....	219
Процедура Money.....	219
Процедура MkStr.....	219
Процедура Floor.....	219
Процедура Asize.....	220

Процедура Date.....	220
Процедура DateSplit.....	221
Процедура Time.....	221
Процедура TimeSplit.....	221
Процедура DtTm.....	222
Процедура DtTmSplit.....	222
Процедура RubToStr.....	222
Процедура RubToStrAlt.....	223
Процедура CurToStrAlt.....	223
Процедура MonName.....	223
Процедура NumToStr.....	224
Процедура SetAutoMoneyFloor.....	224
Процедура Round.....	224
Процедуры для работы с типом DoubleL.....	225
Процедура MakeDouble.....	225
Процедура MakeMoney.....	225
Процедура SplitMoney.....	225
Работа со строками.....	226
Процедура StrLen.....	226
Процедура Index.....	226
Процедура StrBrk.....	226
Процедура StrIsNumber.....	226
Процедура SubStr.....	227

Процедура StrSet.....	227
Процедура Trim.....	227
Процедура StrSplit.....	227
Процедура StrSplit2.....	228
Процедура StrUpr.....	228
Процедура StrLwr.....	229
Процедура CodeFor.....	229
Процедура StrFor.....	229
Процедура StrSubst.....	229
Процедура ToOEM.....	230
Процедура ToANSI.....	230
Параметры процедур.....	230
Процедура GetParm.....	230
Процедура IsOutParm.....	231
Процедура SetParm.....	231
Процедура Parmcount.....	231
Математические процедуры.....	232
Внешние программы.....	232
Процедура Run.....	232
Удаленный запуск макропрограмм.....	234
Процедура CallRemoteRsl.....	234
Обработка меню.....	235

Процедура Menu.....	235
Процедура RunMenu	236
Обработка диалоговых окон.....	237
Процедура AddScroll	237
Процедура RunDialog.....	240
Процедура DisableValidation	241
Процедура UpdateFields.....	241
Процедура SetFocus.....	242
Процедура MsgBox.....	242
Процедура DisableFields	242
Процедура EnableFields.....	242
Процедура SetTimer	242
Процедура MsgBoxEx.....	243
Обработка скроллинга.....	244
Макропроцедуры модуля RslScr.....	244
Переменные.....	244
Макропроцедуры.....	244
Процедура SetScroll.....	244
Процедура FindRow.....	245
Процедура FindCol.....	245
Процедура FillDown.....	245
Процедура FillUp.....	245
Процедура SetDlgFields.....	245
Процедура ScrollMes.....	246
Процедура UserFill.....	246

Макропроцедуры модуля rsIx.....	246
Процедура AddMultiAction.....	246
Процедура GetMultiCount.....	247
Процедура GoToScroll.....	247
Процедура RunScroll.....	248
Процедура UpdateFields.....	251
Процедура UpdateScroll.....	251
Контроль ввода записи в скроллинг.....	252
Процедура GetScrollFieldValue.....	252
Процедура SetScrollFieldValue.....	252
Процедура IsScrollEditMode.....	253
Работа с отладчиком RSL.....	254
Процедура Trace.....	254
Процедура DebugBreak.....	255
Файлы и структуры.....	255
Процедура ConvertDDF.....	255
Процедура CopyTblDef.....	256
Процедура ExistFile.....	256
Процедура Open.....	257
Процедура Close.....	258
Процедура ClearStructs.....	258
Процедура Create.....	258
Процедура Clone.....	259
Процедура ViewFile.....	260

Процедура DelFile.....	260
Процедура DropTable.....	260
Процедура Next.....	260
Процедура Prev.....	261
Процедура Rewind.....	261
Процедура Insert.....	261
Процедура Update.....	262
Процедура Delete.....	262
Процедура GetPos.....	263
Процедура GetDirect.....	263
Процедура GetEQ.....	263
Процедура GetGT.....	264
Процедура GetGE.....	264
Процедура GetLT.....	265
Процедура GetLE.....	266
Процедура SetRecordAddr.....	266
Процедура PackVarBuff.....	267
Процедура UnPackVarBuff.....	268
Процедура GetRecordSize.....	268
Процедура GetVarSize.....	268
Процедура FileName.....	268
Процедура NRecords.....	269
Процедура FldNumber.....	269

Процедура FldName	269
Процедура FldIndex	269
Процедура FldOffset	270
Процедура ClearRecord	270
Процедура SetBuff	270
Процедура Copy	270
Процедура CopyBlob	271
Процедура KeyNum	271
Процедура SetDelim	272
Процедура Status	272
Процедура ProcessTrn	272
Процедура LoopInTrn	274
Процедура AbortTrn	274
Процедура SelectFile	274
Процедура SelectFolder	275
Процедура NeedFreeDB	276
Процедура WriteBlob	276
Процедура ReadBlob	276
Управление файлами и каталогами	276
Процедура CopyFile	277
Процедура RenameFile	277
Процедура RemoveFile	277
Процедура ExistDir	278

Процедура MakeDir	278
Процедура RemoveDir	278
Процедура GetCurDir	279
Процедура SplitFile	279
Процедура MergeFile	280
Процедура FindPath	280
Процедура GetSysDir	280
Процедура GetIniFileValue	281
Процедура GetFileInfo	281
Классы и объекты	282
Процедура ActiveX	282
Процедура ClassKind	282
Процедура GenClassName	283
Процедура GenAttach	283
Процедура GenObject	283
Процедура GenRun	283
Процедура GenSetProp	284
Процедура GenGetProp	284
Процедура GetObjProps	284
Процедура GetObjMethods	284
Процедура IsEqClass	285
Процедура GenPropID	285
Процедура R2M	286

Процедура CallR2M.....	286
Процедура GenNumProps.....	286
Процедура ClrRmtOnRelease.....	287
Процедура GetNamedChanel.....	287
Индикаторы выполнения процессов.....	287
Процедура InitProgress.....	287
Процедура UseProgress.....	288
Процедура RemProgress.....	288
Процедура BegAction.....	288
Процедура EndAction.....	288
Другие процедуры.....	289
Процедура CheckBits.....	289
Процедура DateShift.....	289
Процедура ExecMacro.....	289
Процедура ExecMacro2.....	290
Процедура ExecMacroFile.....	290
Процедура ExecMacroModule.....	291
Процедура ReplaceMacro.....	291
Процедура ExecExp.....	292
Процедура GCollect.....	292
Процедура GetCallStack.....	293
Процедура GetEnv.....	293
Процедура GetMemAddrFrom.....	293

Процедура GetUIMode	294
Процедура InstLoadModule	294
Процедура IsWeakRef	294
Процедура WriteByte	294
Процедура ReadByte	295
Процедура RunError	295
Процедура Exit	296
Процедура SetExitFlag	296
Процедура MemSize	297
Процедура Version	297
Процедура CurrentLine	297
Процедура UserNumber	297
Процедура Random	298
Процедура SetDefMoneyPrec	298
Процедура ShowDictError	299
Процедура ShowRSCOMError	299
Процедура StartProg	299
Процедура StrongRef	300
Процедура SysGetProperty	300
Процедура SysPutProperty	300
Процедура System	301
Процедура IsStandAlone	301
Процедура TestEvent	302

Процедура AddEvent.....	302
Процедура IsGUI.....	303
Процедура ErrPrint.....	303
Процедура ErrBox.....	303
Процедура ModuleFileName.....	304
Процедура ModuleName.....	304
Процедура CmdArgs.....	305
Процедура GetUserName.....	305
Процедура IsSQL.....	305
Процедура UnderRCWHost.....	305
Процедура GetLocaleInfo.....	305
Процедура GetLangId.....	306
Процедура ZeroValue.....	307
Процедура StartShellProgram.....	307
Средство разработки расширений для языка RSL (DLM SDK).....	309
Создание и использование DLM-модулей.....	309
Передача параметров и возврат значений.....	311
Трансляция макропрограмм.....	313
Отладка макропрограмм.....	315
Запуск отладчика.....	316
Основное окно отладчика.....	316
Отладка макропрограмм.....	317

Точки останова	319
Список точек останова	320
Список импортируемых модулей	321
Окно проверки	321
Стек вызовов процедур	322
Окно переменных	323
Окно выражений	324
Ввод выражения	325
Редактирование выражения	326
Настройка отладчика	327
Список "горячих" клавиш отладчика	328
Приложение	329
Сводка синтаксиса RSL	329
Алфавитный указатель	333

Введение

Настоящее Руководство содержит описание языка RSL, являющегося неотъемлемой частью систем, разработанных программистами компании R-Style Softlab, при помощи которого пользователь имеет возможность создавать собственные программы. Изучение этого Руководства предполагает знание языков программирования высокого уровня.

Наличие хорошо структурированного языка, тесно связанного с базой данных, значительно расширяет возможности системы. Язык может быть использован для создания силами работников банка специфических отчетов, отсутствующих в системе, или других процедур, реализующих дополнительные процедуры.

Язык интерпретатора RSL – это язык высокого уровня. Он достаточно прост в изучении и применении и одинаково хорош как для квалифицированного, так и для неквалифицированного пользователей системы, так как имеет широкий спектр функциональных возможностей. В отличие от других подобных ему языков, язык интерпретатора интегрирован с системами, разработанными программистами компании R-Style Softlab. Он вызывается непосредственно из меню системы, обеспечивает доступ к базе данных, использует одинаковые с ней форматы данных, работает с экранными формами.

RSL – современный язык, разработанный с учетом нынешних требований к программному обеспечению. В компании R-Style Softlab ведется постоянная работа по его развитию и совершенствованию.

При ссылке на информацию, расположенную в настоящем руководстве, указывается номер страницы (в скругленном прямоугольнике )²³), соответствующий нужному Вам разделу, или гиперссылка. Ссылка на раздел другого руководства дается с указанием названия главы и раздела, содержащего требуемую информацию.

Элементы языка

В настоящей главе приведено описание элементов языка RSL.

Служебные слова

Служебное, или зарезервированное, слово – это имя, с которым в языке жестко сопоставлены определенные смысловые значения, и которое не может быть использовано ни для каких других целей. Ниже приведен список служебных слов языка RSL (они записываются в любом регистре).

and	if	record
array	import	return
const	macro	this
class	not	true
elif	NULL	var
end	onerror	with
false	or	while
file	local	private

Кроме этого, существует отдельный список ключевых слов, действующих в пределах определений FILE и RECORD.

btr	key	txt
dbf	mem	sort
dialog	write	

Значение каждого служебного слова поясняется при описании конструкций языка.

Имена

Под **именем (идентификатором)** в программе понимается не являющаяся служебным словом последовательность букв (латинских и русских) и цифр, начинающаяся с буквы; символ "_" (подчеркивание) рассматривается как буква. Длина идентификатора в RSL не должна превышать 80 символов. Прописные и строчные буквы не рассматриваются как различные символы, то есть регистр игнорируется. Имена используются для идентификации объектов RSL.

Кроме обычных идентификаторов, в RSL применяются имена специальных переменных, которые могут содержать любые символы, включая специальные, без ограничений. Такие

имена должны быть заключены в фигурные скобки "{}", которые также входят в имя переменной.

Специальные переменные являются глобальными и используются обычно для передачи значений из вызывающих модулей, написанных на языках С или С++, в программу на языке RSL. Для обозначения полей в структурах и файлах, описанных в словаре базы данных, используются составные имена. Составные имена состоят из имени, идентифицирующего файл или структуру, после которого следует точка и название поля либо индекс поля в круглых скобках.

Пример.

<i>Счет</i>	- простое имя
<i>Сумма010</i>	- простое имя
<i>{Debet account 52.5}</i>	- имя специальной переменной
<i>Клиенты.Name</i>	- составное имя, ссылающееся на поле Name в структуре Клиенты
<i>Клиенты(10)</i>	- составное имя, ссылающееся на десятое поле в структуре Клиенты

Если имя переменной совпадает с именем переменной среды, оно инициализируется ее значением.

Пример.

<i>PATH</i>	- переменная будет проинициализирована значением системной переменной PATH, содержащей список каталогов для поиска программ
-------------	---

Присвоение значений этим переменным не изменяет содержимого системных переменных.

Область видимости имен

Глобальными именами считаются те, которые объявлены вне любой процедуры RSL. Имена специальных переменных, независимо от места их появления, считаются глобальными. Для размещения глобальных объектов используется специальная область памяти, через которую осуществляется доступ к их значениям из любого места программы RSL или из вызывающего внешнего модуля.

Локальными именами считаются те, которые объявлены в любой процедуре RSL. Локальные объекты создаются заново при каждом вызове процедуры, в которой были объявлены их имена, и доступны только в этой процедуре, а также во всех вложенных в нее процедурах RSL. Следует отметить, что **определенные явно** имена локальных объектов могут совпадать с именами глобальных объектов. В этом случае имя локального объекта перекрывает имя глобального, и доступ к глобальному объекту будет заблокирован.

Каждая RSL-программа образует глобальную область видимости. Каждая процедура, содержащаяся в программе, образует свою, вложенную область видимости. Из вложенной области видимости доступны имена этой области и всех вышележащих. В каждой области видимости можно "перекрывать" имена из вышележащих областей, то есть во вложенной области могут быть определены переменные с такими же именами, как и в вышележащих.

При входе в область видимости переменные конструируются, при выходе разрушаются. Таким образом, время жизни переменной ограничено областью ее видимости.

Комментарии

Любой фрагмент текста, заключенный в скобки вида `/* */`, является комментарием. Язык RSL разрешает наличие вложенных комментариев.

Пример.

```
/* ****  
    Комментарий на языке RSL  
  /*  
      Вложенный комментарий  
  */  
  ****  
*/
```

Данный пример содержит два комментария, один из которых вложен в другой.

Объекты языка

Объект RSL представляет собой совокупность информации, с которой оперирует язык. Объектами языка RSL являются:

- **Переменная** – объект, содержащий значение одного из типов данных, который может изменять величину и тип хранимого значения. Это происходит при присвоении переменной нового значения инструкцией присваивания (при этом старое значение теряется).

Пример.

```
aa=10                      /* aa хранит значение Integer */  
aa="Привет"                /* aa хранит значение String */
```

- **Объект типа FILE** хранит информацию о таблице в базе данных (структуру таблицы и текущие значения полей).

Примечание.

Объект типа **FILE** является устаревшей конструкцией и поддерживается для совместимости с ранее разработанными приложениями. При работе с таблицами рекомендуется использовать конструкцию **Tbfile**.

- **Объект типа RECORD** хранит информацию о структуре файла или диалоговой панели. Объект типа RECORD, в отличие от объекта типа FILE, не имеет явной связи с файлом. Если это необходимо, то связь с файлом должна быть задана при помощи процедур RSL или неявно в вызывающем модуле, написанном на языке C или C++.
- **Объект типа DBFFILE** хранит информацию о файле формата DBF (структуру файла и текущие значения полей).
- **Объект типа TXTFILE** хранит информацию о текстовом файле (структуру файла и текущие значения полей).
- **Объект типа ARRAY** представляет собой вектор переменных, который можно индексировать.
- **Объект типа CLASS RSL** представляет собой совокупность свойств и методов и служит для создания экземпляра класса – объекта.

Все объекты RSL требуют явного определения их имени.

Типы данных

Тип данных RSL определяет функциональные возможности, предоставляемые этими данными, их область значений и средства RSL, предназначенные для работы с этими данными.

Данные, поддерживаемых в RSL типов хранятся в переменных языка RSL, а также в свойствах экземпляров RSL классов.

При присвоении значения переменной, если тип значения не соответствует типу, который может храниться в переменной, выполняется [приведение типов](#)⁽³⁷⁾ данных по правилам RSL. Состав типов и правила их преобразования могут зависеть от конкретной версии RSL.

Все поддерживаемые типы данных разделяются на [скалярные](#)⁽³²⁾ и [объектные](#)⁽³³⁾.

Тип данных, хранящихся в переменной RSL, можно определить во время выполнения RSL-программы при помощи специальной процедуры [ValType](#)⁽²¹⁷⁾. Эта процедура принимает в качестве параметра переменную RSL и возвращает целочисленный код типа данных, который в ней хранится. Код типа выражается [символическими константами](#)⁽³¹⁾ вида **V_TypeName**.

По умолчанию переменные RSL могут содержать значения любых типов данных RSL.

Если структура описана во внутреннем словаре, то все поля этой структуры получают те типы, которые были заданы в словаре.

RSL не предполагает обязательную декларацию типов данных, в зависимости от конкретной задачи разработчик может явно [декларировать тип переменной](#)⁽³⁰⁾ либо [не делать этого](#)⁽³¹⁾.

Переменные с декларацией типа данных

При декларации переменной можно явно указать, что переменная может содержать значение любого типа. Для этого используется специальное ключевое слово **Variant**.

Кроме автоматического определения типа, в Object RSL имеется возможность явно декларировать типы:

- переменных;
- параметров макропроцедур и методов;
- возвращаемых значений макропроцедур и методов.

Рассмотрим **правила декларирования типов для различных лексем языка RSL**:

- Декларируемый тип [переменной или формального параметра](#) указывается через двоеточие после объявления. Кроме того, для декларирования типов данных можно использовать и имена классов.

Пример.

Определим переменную для хранения ссылки на объект типа "MyClass", который является именем класса:

```
Var obj: MyClass
```

Переменные, которые являются ссылками на файл или структуру, нельзя явно декларировать.

- Тип возвращаемого значения макропроцедуры или метода класса указывается после двоеточия, которое следует за списком формальных параметров, а в случае их отсутствия – непосредственно после имени макропроцедуры или метода класса.

Пример.

```
Macro MyStrLen (str: String): Integer  
    Return strlen (str);  
End;
```

Переменные без декларации типа данных

Декларация типа переменных в RSL необязательна. Декларация типа не приводит к повышению производительности работы RSL-программы и не уменьшает требований к системным ресурсам.

Любая переменная без декларации типа эквивалентна декларации с использованием ключевого слова **Variant**. В этом случае переменная может содержать значение любого типа.

Пример.

```
Var p1;  
Var p2:Variant;  
p1 = 10;  
p2 = "Test string";  
p2 = p1;
```

Символические константы и специальные значения переменных

Используя символические константы, переменным можно задать специальные значения:

- Если переменная не содержит значения ни одного из поддерживаемых типов данных, то считается, что переменная содержит специальное значение "Пусто". Процедура [ValType](#)²¹⁷ для такой переменной возвращает код, соответствующий символической константе V_UNDEF.

Для специального значения "Пусто" в RSL имеется символическая константа **Null**, которую можно присвоить любой переменной RSL, не зависимо от декларируемого типа переменной.

- Для задания специального значения "Нулевое значение" в RSL применяется символическая константа **NullVal**.

Примечание.

В случае использования библиотеки RSD при передаче параметра в программах на языке RSI константа **NullVal** задает значение "Пусто".

- Для задания специального значения "Умалчиваемое значение" в RSL применяется символическая константа **OptVal**.

Все перечисленные специальные значения служат для передачи в методы **ActiveX** объектов [специальных значений COM-автоматизации](#) ⁽⁶⁵⁾.

Пример.

```
Var p:SpecVal;  
P = NullVal;
```

В RSL имеются литеральные константы для задания значений логической величины **True** и **False**.

Скалярные типы

Предусмотрены следующие скалярные типы данных:

Название типа	Ключевое слово для декларации	Код типа	Принимаемые значения
Целое число	Integer	V_INTEGER	От -2147483648 до 2147483647
Число с плавающей точкой	Double	V_DOUBLE	15 значащих десятичных цифр
Число с плавающей точкой длинное	DoubleL	V_DOUBLEL	15 значащих десятичных цифр (аналог Double)
Строка символов	String	V_STRING	
Логическая величина	Bool	V_BOOL	True, False
Дата	Date	V_DATE	
Время	Time	V_TIME	
Дата и время	DateTime	V_DTTM	
Адрес памяти	MemAddr	V_MEMADDR	
Ссылка на процедуру RSL	ProcRef	V_PROC	
Ссылка на метод объекта RSL	MethodRef	V_R2M	
Двоично-десятичное число с 4-мя знаками после запятой	Decimal	V_DECIMAL	28 значащих цифр (аналог Numeric)
Двоично-десятичное число	Numeric	V_NUMERIC	28 значащих цифр

Название типа	Ключевое слово для декларации	Код типа	Принимаемые значения
с плавающей точкой. Используется для вещественных величин (денежных сумм, курсов валют, процентных ставок и пр.)			
Тип для денежных величин	Money	V_MONEY	28 значащих цифр (аналог Numeric)
Длинный тип для денежных величин	MoneyL	V_MONEYL	28 значащих цифр (аналог Numeric)
Специальное значение "Нулевое значение", или "Умалчиваемое значение"	SpecVal	Специальной константы нет. Код типа: 26	OptVal или NullVal
Специальное значение "Пусто"	-----	V_UNDEF	Null

Особенности реализации типов

Некоторые из ранее используемых типов имеют синтаксические эквиваленты:

- лексемы типа **DoubleL** являются синонимом типа **Double**;
- лексемы **MoneyL** являются синонимом **Money**;
- лексемы **Decimal** являются синонимом **Numeric**.

Для внутреннего представления переменных типа **Money** используется тип **Numeric**.

Эти типы, используемые в ранних версиях RSL, оставлены только для совместимости исходного кода на RSL.

Объектные типы

Объектные типы данных подразделяются на специализированные объекты и обобщённые объекты (**GenObject**):

- Обобщённые объекты создаются либо непосредственным вызовом конструктора RSL-класса, либо при помощи стандартной процедуры [GenObject](#)²⁸³, принимающей имя RSL-класса в качестве строкового параметра. И конструктор, и процедура **GenObject** возвращают ссылку на созданный объект.
- В настоящее время специализированные объекты являются устаревшими. В новом коде рекомендуется использовать обобщённые объекты, реализованные на замену специализированных объектов.

Предусмотрены следующие объектные типы данных:

Название типа	Ключевое слово для декларации	Код типа	Тип аналогичного обобщённого объекта
Ссылка на специализированный объект "файл базы данных"	BtFileRef	V_FILE	TBfile
Ссылка на специализированный объект "структура в памяти"	StrucRef	V_STRUC	TRecHandler
Ссылка на специализированный объект "массив значений"	ArrayRef	V_ARRAY	TArray
Ссылка на специализированный объект "текстовый файл"	TxtFileRef	V_TXTFILE	Нет
Ссылка на специализированный объект "Файл dBase"	DbfFileRef	V_DBFFILE	Нет
Ссылка на обобщённый объект RSL класса	Объект для любого объектного типа, или <Имя RSL класс>	V_GENOBJ	

Пример.

//Пример создания обобщённого объекта RSL-класса и переменных для ссылки на него:

```
Class X
End;
Var p1:Object;
Var p2:X;
p1 = X;
p2 = p1;
```

Константы

Под константой понимается значение одного из типов данных.

Символьные константы имеют тип **String** и задаются в виде последовательности символов, заключённых в кавычки.

Пример.

```
"a"
"F"
"абв_12"
```

```
"\n"  
"\t"
```

Максимальная длина символьной константы – 2047 символов. Если задана более длинная строка, то генерируется ошибка RSL. Для ввода строки длиннее 2047 символов необходимо использовать операцию '+' для "склеивания" символьных констант во время выполнения RSL-программы. Получаемые таким образом динамические строки не имеют ограничения по длине.

Пример.

//пример получения строки длиной 24056 байт:

```
macro GetLongString  
a = "1234567890 ";  
a = a + a + a;  
a = a + a + a;  
a = a + a + a;  
a = a + a + a;  
a = a + a + a;  
a = a + a + a;  
a = a + a + a;  
a = a + a + a;  
return a;  
end;  
str = GetLongString;  
len = "\n\nДлина вставленной строки равна " + string (strlen  
(str));
```

Последовательности, начинающиеся с обратной косой черты, ничем не отличаются от таковых в языке C:

- \n – новая строка;
- \r – возврат каретки;
- \t – символ табуляции;
- \f – перевод формата;
- \xHH, \XHH – символ, заданный кодом (здесь HH – две шестнадцатеричные цифры);
- \\ – одиночный символ '\.

Целочисленные константы типа **Integer** имеют диапазон допустимых значений от -2'147'483'648 до 2'147'483'648. Они задаются следующим образом: 2345, 1236 и т.п.

Числа с плавающей точкой имеют тип **Double** и могут находиться в диапазоне от 1.7e-308 до 1.7e308. При их записи используются точка и латинская буква "e", например: 4356.234, 345., .1234, 1231.2341e-23.

При записи денежных сумм в начале ставят знак доллара "\$". Эти константы имеют тип **Money** и могут принимать максимальное значение 9'999'999'999'999.99. Например, \$146, \$765.23, -\$12.34.

Денежная константа, в которой не задана ни одна цифра, считается недействительной. При компиляции будет выдана ошибка: "Неверная денежная константа".

Для записи шестнадцатеричных констант используется знак "#". Например, #F2; #125ab2;

Логические константы типа **Bool** могут принимать только два значения: **TRUE** – истина, **FALSE** – ложь.

Язык RSL позволяет определить символические константы при помощи определения **CONST**. В языке RSL predefinedены следующие символические константы:

- **TRUE** – задает значение “истина” типа **Bool**.
- **FALSE** – задает значение “ложь” типа **Bool**.
- **NULL** – задает значение типа **V_UNDEF**

Предопределенные константы предусмотрены также при работе с [диалоговыми окнами](#) ⁹¹.

Выражения

Синтаксис

Выражения представляют собой последовательность операндов, разделенных знаками операций.

Пример.

<i>Name</i>	– выражение, состоящее из одного операнда – простого имени;
<i>MgFun (10)</i>	– выражение, состоящее из одного операнда – сложного имени;
<i>a+b</i>	– сумма двух переменных;
<i>b*10+c</i>	– произведение и сумма переменных.

В приведенной ниже таблице операции располагаются в порядке возрастания их приоритета при вычислении выражений. Приоритет операций в каждой группе одинаков.

Таблица видов операций

Обозначение	Операция
" = "	Присваивание
" == "	Равно
" != "	Не равно
" < "	Меньше
" > "	Больше
" <= "	Меньше или равно
" >= "	Больше или равно
" + "	Сложение
" _ "	Вычитание

" OR "	Дизъюнкция
" * "	Умножение
" / "	Деление
" AND "	Конъюнкция
" _ "	Унарный минус
" + "	Унарный плюс
" NOT "	Отрицание
" @ "	Получение ссылки на процедуру

Для того чтобы явно задать порядок вычисления, необходимо применять круглые скобки.

Пример.

$a + b * c$ эквивалентно $a + (b * c)$

Все операции, кроме операции присваивания, имеют левую ассоциативность. Это означает, что операции с одинаковым приоритетом выполняются слева направо. Например, выражение $a + b + c$ интерпретируется как $(a + b) + c$. Для использования другого порядка вычисления необходимо использовать скобки.

Операция присваивания имеет правую ассоциативность. То есть выражение $a = b = 10$ интерпретируется как $a = (b = 10)$.

Последовательность вычисления операндов в выражении не определена. Исключение составляют операции "AND" и "OR". Для них гарантируется строгий порядок вычисления операндов: сначала вычисляется значение левого операнда, и если после этого результат операции уже известен, вычисление правого операнда не производится.

Все перечисленные операции используются при работе с любыми переменными, за исключением переменных, хранящих ссылки на объекты RSL классов, которые можно сравнивать друг с другом или с константой **NULL** с помощью операций **==** и **!=**.

С помощью операций **==** и **!=** также можно сравнивать между собой объекты RSL-классов и целые числа, при этом операция **==** всегда возвращает значение **FALSE**, а операция **!=** возвращает значение **TRUE**. Данная возможность предусмотрена в связи с необходимостью поддержки технологий работы с данными СУБД Oracle.

Семантика

Для типов данных в RSL предусмотрены операции преобразования из одного типа данных в другой. Когда в арифметической операции участвуют два операнда разных типов, для которых не определена явная процедура вычисления операции, применяется автоматическое преобразование типа одного из операндов к типу другого операнда.

Преобразуется тип операнда с меньшим весом типа к типу операнда с большим весом типа. Ниже представлены варианты преобразования типов в соответствии с их весами. Тип слева преобразуется в тип справа:

Integer → Double → DoubleL → Numeric → String

Явные преобразования выполняются при помощи специальных процедур: **string**, **int**, **double**, **money**, **decimal**, **numeric**.

Преобразование также происходит при присвоении значения переменной с явно декларированным типом данных.

Пример.

```
var a : String;
a = 10; /* a будет содержать значение строковое "10".*/
```

При преобразовании значения типа **Money** в тип **Double** результат представляет собой число в рублях с копейками, отделенными точкой. При преобразовании значения типа **Double** или **Integer** в тип **Money** действует обратное правило: число с десятичной точкой преобразуется в соответствующее значение в копейках.

Пример.

```
var a : Double;
    b : Money;
a = $12.34; /* a примет значение 12.34. */
b = a;      /* b примет значение 1234. */
```

Существует возможность преобразования строкового типа данных **String** в типы данных **Date** или **Time**.

Пример.

```
var d:Date;
    t:Time;
d = "31.12.2010"; /*d примет значение 31.12.2010*/
t = "23:59:59"; /*t примет значение 23:59:59*/
```

Логические операции **AND**, **OR** и **NOT** определены только для операндов типа **Bool**. Все арифметические операции определены для всех числовых типов данных (**Integer**, **Double**, **Money**). Тип результата бинарных операций будет таким же, как у операнда с большим приоритетом, за исключением особых случаев с операндами типа **Money**.

Операция "+" определена для операндов типа **String** как конкатенация (соединение строк).

Пример.

```
aa = "Stroka1" + "Stroka2"; /* Переменная aa имеет тип
                             String и содержит строку
                             "Stroka1Stroka2" */
```

Операция "-" определена для операндов типа **Date**. Результат имеет тип **Integer** и представляет собой количество дней между двумя датами.

Пример.

```
n = Завтра - Вчера; /*Переменная n получает
                     тип Integer и значение2*/
```

Операции "+" и "-" определены для операндов типа:

- **Date** и **Integer**. Результатом операции "+" и "-" с операндами этих типов является значение с типом **Date**.

Пример.

```

Сегодня = date;           /* Переменные в левой
Завтра   = Сегодня + 1;    части имеют тип Date */
Вчера    = Сегодня - 1;

```

- **Time.** Результатом операции "+" и "-" между операндами этого типа является значение типа **Time**.

Пример.

```

n = time(10,0) + time(0,15); /*Переменная n
                             получает тип Time и
                             значение 10:15*/

```

- **Money.** Результатом операции "+" и "-" с операндами этого типа является значение типа **Money**.

Пример.

```

Дебет = $100;              /* Оборот получит тип
Кредит = $33;              Money и значение $67.
Оборот = Дебет - Кредит;   Результат получит тип
Результат = Дебет/Кредит; Double и значение
                           3.3333(3) */

```

Операция "**" определена для операндов типа:

- **Money** и **Integer.** Результатом операции "**" между операндами этих типов является значение типа **Money**.
- **Money** и **Double.** Результатом операции "**" между операндами этих типов является значение типа **Double**.

Операция "/" определена для операндов типа:

- **Money** и **Integer.** Результатом операции "/" между операндами этих типов является значение типа **Money**.
- **Money** и **Double.** Результатом операции "/" между операндами этих типов является значение типа **Double**.
- **Integer.** Результатом операции "/" между операндами этого типа является значение типа **Integer**, равное целой части от деления.

Пример.

```

a = 5;
b = 3;
c = a/b; /* Переменная c получает тип Integer и значение 1*/

```

Операции "**" и "/" определены для операндов типа **Money**. Результатом операций "**" и "/" с операндами типа **Money** будет значение типа **Double**.

Операции сравнения определены для всех типов данных. Для данных типа **Bool** определены операции сравнения равно "==" и не равно "!=". Результатом операций сравнения является значение типа **Bool** – ложь или истина.

Для переменных с типами, соответствующими константам V FREF, V SREF, V AREF, V_TREF, V_DREF и V_MEMADDR, определены операции сравнения с **NULL**.

Для переменных типа, соответствующего константе V_MEMADDR, предусмотрена операция сравнения друг с другом.

Структура RSL-программы

Программа на языке RSL представляет собой последовательность конструкций языка, разделенных знаком ";".

Конструкции языка – это либо определения объектов RSL, либо определения процедур, либо выполняемые инструкции. Программа, как и процедура, может заканчиваться словом END. В конструкции, которая располагается перед словом END, символ ";" может отсутствовать.

Выполняться RSL-программа начинает с первой встретившейся выполняемой инструкцией, которая может находиться как в текущем файле, так и в файле, подключенном посредством директивы **IMPORT** ⁽⁴⁴⁾.

Вот пример самой простой программы на RSL:

```
[ Привет, Мир! ]
```

Этот пример выводит в стандартный выходной поток строку "Привет, Мир!".

Ниже представлен более сложный пример, в котором вычисляется самое длинное имя клиента, хранящееся в нашей базе данных.

Пример.

```
file f (client);
var n = 0,
    maxLen = 0,
    curLen,
    maxName = "";
macro Report
[ Самое длинное название клиента ####
  #
]
(maxLen:1,maxName)
end;
while (next (f))
  n = n + 1;
  message ("Обработано записей ",n);
  curLen = strlen (f.Name_Client);
  if (curLen > maxLen)
    maxLen = curLen;
    maxName = f.Name_Client
  end
end;
Report;
```

Конструкции языка, использованные в этом и последующих примерах, описываются в разделе ["Конструкции языка RSL"](#) ⁽⁴⁷⁾.

Текст программы должен находиться в текстовом файле с расширением "mac". Содержимое файла с таким расширением образует макромодуль.

Примечание.

Длина имени макрофайла не должна превышать 24 символа.

В принципе, всю RSL-программу, содержащуюся в модуле, можно рассматривать как процедуру инициализации, имя которой – это имя файла, в котором находится программа. Синтаксически единственным отличием программы от процедуры является отсутствие у

программы заголовка MACRO. Заканчиваться программа может, как и процедура, словом END.

Пример.

```
[ Это пример программы ]  
end
```

Все, что расположено после слова END, игнорируется интерпретатором.

В текст макропрограммы можно включать **комментарии**, которые задаются следующим образом: `//<текст комментария>` или `/*<текст комментария>*/`

Макромодуль, или сокращенно модуль, может включать в себя определения переменных, объектов, процедур, а также выполняемые инструкции. Все инструкции модуля неявно образуют процедуру инициализации модуля.

Пример.

```
/* Пример макрофайла test.mac */  
a = 10  
macro Mac1  
    b = 20  
end  
class MyClass  
    var Prop = 100;  
    macro Method  
        var aa = 200;  
    end;  
    var p = 10;    /* Начало конструктора класса */  
    println ( p )  
end;  
/* Начало процедуры инициализации test макромодуля test.mac  
*/  
println ( a );  
end;                /* Конец процедуры инициализации */  
/* Переменная a и процедура Mac1 являются глобальными.  
Инструкция println образует процедуру инициализации модуля.  
*/
```

По умолчанию все переменные, процедуры и классы, объявленные в макромодуле автоматически приобретают статус глобальных. Они становятся доступными не только внутри макромодуля, но и во всех остальных модулях.

Чтобы запретить извне доступ к переменной, процедуре модуля или классу, определенному в модуле, используется модификатор **private**. Таким образом, переменная или процедура становятся недоступными извне и видны только в рамках данного модуля.

Примечание.

С точки зрения C++ этот модификатор можно рассматривать, как **protected**, либо как **private** с автоматическим объявлением всех классов наследников друзьями базового класса.

Если **private** стоит перед определением свойства или метода класса, то это свойство или метод становятся недоступными извне класса.

Пример.

```
class MyClass  
    var prop1 = 500;  
    private var prop2 = 1000;
```

```
end;
obj = MyClass;
println ( obj.prop1 ); /* Свойство доступно */
println ( obj.prop2 ); /* Свойство недоступно */
```

Если этот модификатор используется при определении переменной, то для ее декларации обязательно должно присутствовать [ключевое слово VAR](#)⁵².

Следует иметь в виду, что обращение через **this** к свойствам и методом считается обращением извне **объекта**. Поэтому этот код приведёт к ошибке.

Пример.

```
class X
  private var prop1;
  this.prop1 = 1; // Ошибка
end;
```

В языке RSL используется также модификатор **local**, который при установке перед объектом или переменной указывает, что данный объект RSL, переменная или процедура является локальным объектом процедуры инициализации или конструктора объекта.

Внимание!

Этот модификатор применим только для процедур инициализации модуля и конструктора класса.

Локальные переменные модуля доступны только локальным процедурам модуля. Невозможно обратиться к локальной переменной внутри любой другой процедуры модуля.

Пример.

```
local var str;           //Локальная переменная модуля, видима
                           только
                           //в теле текущего модуля или в
                           локальной процедуре
macro MyProc1()          //Глобальная процедура модуля
  str = "Hello!";         //Ошибка! str недоступна
end;
local macro MyMacro1()    //Локальная процедура модуля
  str = "Hello!";         //Правильно!
end;
MyMacro;
println ( str );
```

Если модификатор **local** стоит перед свойством класса, то по аналогии с модулем, оно перестает быть свойством класса как таковым и становится локальной переменной конструктора, то есть может быть доступно только для инструкций конструктора.

Пример.

```
class MyClass
  var prop1 = 200;
  local var lVar = 300;
  macro Method1
    println ( lVar );      //Локальная переменная из
    метода недоступна
  end;
  local macro LocProc
    println ( lVar );      //Локальная переменная
    доступна
```

```
end;                                     //из локальной процедуры
класса
LocProc;
end;
```

Процедуры RSL могут быть вызваны как явно при помощи инструкции вызова процедуры, так и неявно вызывающим модулем, написанным на языке C, C++. Примером неявно вызываемой процедуры является **BtrError**, которая вызывается неявно при возникновении ошибок в процессе обращения к таблицам баз данных.

Загрузка и кэширование модулей программы исполняющей системой RSL

В процессе выполнения RSL-программы исполняющая система RSL создает в памяти так называемые поименованные экземпляры интерпретатора (ПЭИ). В них загружаются системные и пользовательские RSL-модули, используемые в программе.

ПЭИ с именем *default* создается всегда и для каждой программы. В прикладной системе могут также дополнительно определяться другие ПЭИ с произвольными именами.

ПЭИ состоит из постоянной части и отгружаемой. Постоянная часть ПЭИ создается при первом использовании программы, после чего она остается в кэше. При последующих запусках RSL-программ, использующих данный ПЭИ, постоянная часть берется из кэша, что существенно ускоряет выполнение RSL программы. Отгружаемая часть выгружается сразу после выполнения.

В постоянной части ПЭИ сохраняются системные модули, определяемые прикладной системой.

Чтобы пользовательские модули попали в постоянную часть всех существующих ПЭИ, их необходимо включить в модуль *autoinc.mac*. Если пользовательский модуль нужно включить в какой-либо определенный ПЭИ, его нужно включить в модуль, имя которого совпадает с именем ПЭИ, например *default.mac*.

Пример.

```
/*Пример содержимого модуля autoinc.mac: */
IMPORT a /* В результате модуль a будет включен в */
/* постоянную часть всех существующих ПЭИ
*/
```

Постоянная часть ПЭИ может быть отгружена исполняющей системой, например, в том случае, если изменился код одного из используемых модулей.

Директива IMPORT

В тексте программы может находиться директива **IMPORT**. Использование этой директивы позволяет включить в текст макромодуля информацию из других файлов, что дает возможность вынести часто используемые процедуры в отдельный файл и потом при необходимости подключать этот файл директивой **IMPORT**.

Пример.

```
IMPORT blnc, common;
```

Директива **IMPORT** должна находиться вне определения макропроцедур.

Директива **IMPORT** позволяет подключать:

- другие RSL-модули, при этом имя модуля должно указываться без расширения (к нему будет по умолчанию добавлено расширение *MAC*);
- DLM-модули модули, созданные с помощью инструмента [DLM SDK](#)³⁰⁹, которые имеют расширения *.D32*, *.DLM*;
- встроенные в прикладную систему стандартные модули, которые написаны на языках C или C++.

Внимание!

Импортируемые файлы должны иметь разные названия. Нельзя импортировать файлы с одинаковым названием и разным расширением.

Имена модулей, подключаемых с помощью этой директивы, могут содержать русские буквы.

Пример.

```
IMPORT BankInter, Баланс;
```

Поиск указанных в директиве **IMPORT** файлов выполняется системой в следующей последовательности:

- среди имен стандартных модулей, написанных на языке C/C++;
- среди имен DLM-модулей;
- среди имен текстовых файлов, имеющих расширение *MAC*. В этом случае система сначала осуществляет поиск файла с прекомпилированным кодом, который имеет такое же имя и расширение *RSM*, а затем ищет соответствующий ему макрофайл. Если для макрофайла существует актуальный файл с прекомпилированным кодом, то будет загружен соответствующий файл с расширением *RSM*. В противном случае загрузится файл с расширением *MAC*.

Конструкции языка RSL

RSL предоставляет программисту достаточно широкие возможности по управлению данными. Количество конструкций языка невелико, но вполне позволяет писать хорошо структурированные и эффективные программы. В качестве конструкций языка используются:

- выполняемые инструкции;
- определение объектов и процедур.

При описании конструкций языка используются следующие соглашения:

- синтаксические элементы, которые являются необязательными, заключаются в квадратные скобки [...];
- синтаксические элементы, которые могут повторяться произвольное число раз или отсутствовать, заключаются в фигурные скобки {...}.

Пустая инструкция

Пустая инструкция состоит только из разделителя – точки с запятой. Она используется там, где по правилам языка могла бы находиться какая-либо инструкция, а по логике программы там ничего выполнять не надо.

Пример.

```
if ( aa != 10 );    /* Пустая инструкция */  
  else  
    bb = aa  
  end
```

Инструкция "выражение"

Данная инструкция представляет собой любое выражение RSL. Вычисленное значение выражения отбрасывается.

Примеры.

```
a;  
a + b;  
a + (b = 12);  
a = 10;  
d + f2 (12, c = b = 20);  
a = b = c = d = 10;
```

Условная инструкция IF

Условная инструкция в RSL имеет следующий вид:

```
IF ('условное выражение') список инструкций  
{ ELIF ('условное выражение') список инструкций }  
[ELSE список инструкций ] END
```

Количество конструкций ELIF не ограничено.

Пример.

```
if ( a < 10 )  
    a = 10  
elif ( a < 20 )  
    a = 20  
elif ( (a < 100) and (a > 50) )  
    a = 0  
else  
    a = b;  
    b = 0  
end
```

Первым анализируется условное выражение после IF. Если оно истинно, выполняется список инструкций. В противном случае последовательно, сверху вниз анализируются условные выражения после ELIF, если они есть. Если результатом сравнения какого-либо из них является истина, выполняется список инструкций, который следует за данным условным выражением. Остальные условия при этом не анализируются.

Если в конструкции присутствует ELSE, то соответствующий список инструкций выполняется в том случае, если все условные выражения ложны.

Инструкция цикла WHILE

Для организации циклов в RSL используется инструкция WHILE.

Цикл WHILE имеет следующий синтаксис:

```
WHILE ('условное выражение')  
список инструкций  
END
```

Список инструкций выполняется до тех пор, пока остается истинным условное выражение. Если условие ложно до входа в цикл, то список инструкций не выполняется ни разу.

Пример.

```
a = 0;  
while ( a < 10 )  
    a = a + 1;  
    println (a)  
end
```

Инструкция цикла FOR

Цикл FOR имеет следующий синтаксис:

```
FOR (loopVar, beginExpr, endExpr, deltaExpr)  
  
список инструкций
```


end

Инструкция может использоваться для выполнения трёх различных задач:

- Для реализации бесконечного цикла. В этом случае не нужно задавать ни одного параметра цикла. Можно не указывать и скобки.

Пример.

```
For
    println ("Loop")
end
```

- Для реализации перебора элементов стандартных коллекций COM и RSL (т.е. коллекций, для которых можно создать объект-enumerатор). В этом случае в качестве параметров инструкции FOR передаётся только переменная цикла **loopVar**, которая последовательно получает значения элементов коллекции и объект, для которого можно запросить объект-enumerатор.

Для перебора всех элементов массива RSL можно использовать один из приведённых ниже примеров. По своей результативности они идентичны, однако использование инструкции FOR в данном случае более наглядно и естественно.

Пример 1.

```
ar = TArray;
...
ob = ar.createEnum;
while (ob.next)
    println (ob.item)
end;
```

Пример 2.

```
for (item, ar)
    println (item)
end;
```

- Для выполнения цикла заданное количество раз. В этом случае в инструкции используются три параметра:

- **beginExpr** – начальное значение счётчика цикла;
- **endExpr** – конечное значение счетчика;
- **deltaExpr** – величина, на которую изменяется счётчик цикла при каждой итерации.

Значения этих параметров приводятся к типу **V_INTEGER**. Значение счётчика цикла присваивается переменной **loopVar**. Все параметры инструкции FOR в этом случае, кроме **loopVar**, являются необязательными:

- Если не задан параметр **deltaExpr**, то он принимается равным "1" в случае, если значения **endExpr** больше чем или равно **beginExpr**, и "-1" – если **endExpr** меньше **beginExpr**.
- Если параметр **endExpr** не задан, то в качестве конечного значения счетчика принимается значение, максимальное для типа **V_INTEGER** – "2147483647".
- Если не задан параметр **beginExpr**, то вместо него принимается "0".

Пример.

```
for (val, 1, 100)
    println (val)
end;
```

Следует иметь в виду, что переменная цикла **loopVar** может быть объявлена непосредственно в цикле. Областью видимости объявленной таким образом переменной является не только вложенный блок, в котором она объявлена, а **вся** программа.

Пример.

```
for (var i : Numeric, 1, 10)
    var d : String = "80";
    println (i);
end;
println (i);
```

Инструкция прерывания BREAK и продолжения CONTINUE

Инструкция BREAK прерывает выполнение цикла [FOR](#)⁴⁸ или [WHILE](#)⁴⁸ и передаёт управление инструкции, следующей за инструкцией цикла.

Инструкция CONTINUE предназначена для непосредственного перехода к следующей итерации цикла без выполнения инструкций следующих после инструкции continue.

Пример.

```
For
    if (testMacro > 0)
        break;
    end;
    if (otherTest < 0)
        continue
    end;
    DoWork;
end;
```

Если инструкция BREAK или CONTINUE вызвана вне инструкции цикла, то её действие эквивалентно инструкции return без параметров.

Инструкция возврата RETURN

Инструкция RETURN применяется для выхода из процедуры или завершения всей RSL-программы. Во втором случае она должна быть указана вне любой процедуры.

Применяется следующая синтаксическая форма:

RETURN [выражение];

Выражение, заданное после слова RETURN, определяет возвращаемое процедурой значение. Если в инструкции завершения RSL-программы в качестве выражения задать

текст, содержащийся в символьной константе или переменной типа **String**, он будет выведен на стандартное устройство вывода.

Инструкция вывода

Инструкция вывода позволяет определить в тексте программы форму для вывода в стандартный выходной поток. Выводимая форма заключается в квадратные скобки:

[Этот текст без изменений будет выведен в стандартный выходной поток]

Внутри текста формы могут находиться поля, в которые необходимо выводить посчитанные значения из переменных языка RSL. Поля внутри формы задаются последовательностью символов '#'. Значения, которые необходимо вывести в поля, указываются сразу после формы в круглых скобках.

*[Номер счета #####
Остаток #####]
(Account, Summa)*

Синтаксическая форма инструкции выглядит следующим образом:

['управляющая строка'] ['(фактические параметры)']

Список фактических параметров заключается в круглые скобки. Он представляет собой список выражений, разделенных запятыми, в котором при необходимости задаются [спецификаторы форматирования](#) ⁸³.

Формат управляющей строки

Управляющая строка представляет собой последовательность печатных и управляющих символов. Управляющие символы имеют более высокий приоритет, чем [спецификаторы форматирования](#) ⁸³.

В языке RSL применяется только один управляющий символ форматирования – это знак "#". Все остальные символы считаются печатными и выводятся на устройство вывода без изменений. Последовательность символов '#' задает поле для вывода в него соответствующего значения из списка параметров. Количество символов '#' задает ширину поля. Совокупность управляющих символов и спецификаторов форматирования определяет характеристики вывода значений (позицию в форме, ширину, выравнивание, количество десятичных знаков и т.д.). Количество полей в форме должно соответствовать количеству параметров, указанных для данной формы.

Пример.

```
[ ##### ] (123.45)      /* Вывод поля шириной 10
                           символов */
```

Одиночный управляющий символ "#" задает не ширину поля вывода, равную 1, а позицию табуляции, то есть точку, относительно которой происходит выравнивание поля вывода. В этом случае ширина поля вывода задается спецификатором. Если в соответствии с заданным спецификатором поле выравнивается по левому краю ("l"), позиция табуляции задает левую границу поля вывода. Если поле выравнивается по правому краю ("r"), позиция табуляции задает правую границу поля вывода. Если поле выравнивается по центру ("c"), позиция табуляции задает центральную позицию поля вывода.

Пример.

```
a = "Пример";
[
    |
    #
    #
    # ] ( a:l, a:r, a:c)
```

В выходной поток будет выведено:

```
Пример
Пример
Пример
```

Определение переменных VAR

Имена переменных RSL необходимо объявлять явно, используя определение **VAR**.

Синтаксическая форма определения VAR имеет следующий вид:

```
[local | private]VAR идентификатор [: имя типа ] [ '=' выражение ]
{ ',' идентификатор [: имя типа ] [ '=' выражение ] }
```

Список идентификаторов может содержать произвольное количество имен переменных. Инициализация заключается в присвоении идентификатору переменной значения какого-либо выражения. Допускается инициализировать не все объявленные переменные. Конструкции, относящиеся к отдельным переменным, должны быть разделены запятой.

После того, как определение VAR будет обнаружено компилятором в тексте программы хотя бы один раз, любая необъявленная явно переменная в текущем RSL-модуле приведет к сообщению об ошибке.

Пример.

```
var
    Остаток40 : money,
    Дата = date;
```

При необходимости переменная RSL может быть объявлена внутри инструкций [if](#) ⁽⁴⁷⁾, [for](#) ⁽⁴⁸⁾, [while](#) ⁽⁴⁸⁾. Кроме этого, переменную цикла **for** допустимо декларировать непосредственно в цикле, в круглых скобках. Областью видимости объявленных таким образом переменных является не только вложенный блок, в котором они объявлены, а **вся** программа.

Пример.

```
var a = 10;
if (a < 100)
    var b : Integer = 70;
    println (b)
end;
while (true)
    var c = b;
    println (c);
    break
end;
for (var i : Numeric, 1, 10)
    var d : String = "80";
    println (i);
end;
println (i);
```

Определение классов и объектов

В языке RSL имеется возможность создавать классы и объекты. Классы и объекты, например, используются для поддержки программирования визуальной среды, в котором очень удобен объектно-ориентированный подход: такие ее специфические черты, как отчеты, кнопки, поля редактирования, переключатели всегда можно представить в качестве объектов со своими свойствами и методами.

Синтаксическая форма определения классов выглядит следующим образом:

```
[local | private] Class [ 'идентификатор базового класса' ] идентификатор [ ' (список формальных параметров)' ]
```

```
<свойства класса>;
```

```
<методы класса>;
```

```
End
```

Идентификатор класса – это имя конструктора класса, которое используется при создании экземпляра данного класса. Использование идентификатора класса в выражении создает экземпляр данного класса – объект.

Список формальных параметров содержит разделенные запятыми идентификаторы переменных, которые являются параметрами инициализатора объекта класса.

Все локальные переменные, находящиеся внутри определения класса, являются свойствами класса, а все локальные макропроцедуры – методами. Любой код внутри определения класса, не включенный в методы, является кодом конструктора.

Приведем пример, в котором задается класс "Персона", имеющий два формальных параметра **p1** и **p2** и содержащий:

- два свойства – "Имя" и "Фамилия";
- метод, выводящий на печать информацию, описываемую объектом;
- код для инициализации объекта.

Пример.

```
Class Персона (p1, p2)
    Var Имя, Фамилия;
Macro Отчет
    Println ("Имя:", Имя);
    Println ("Фамилия:", Фамилия);
End;
Имя = p1;
Фамилия = p2;
End;
```

Чтобы использовать заданный класс, необходимо создать экземпляр класса – объект: *Obj = Персона ("Иван", "Петров")*;

В этой инструкции объект одновременно и создается, и инициализируется. Ссылка на объект присваивается переменной *Obj*.

В языке RSL предусмотрено автоматическое создание объектов классов при первом обращении к декларированной переменной класса. Если конструктор класса имеет параметры, то в этом случае при создании объекта параметры будут иметь значение **NULL**.

Пример.

```
Class TestClass
    Var prop = 10;
End;
Var Ob:TestClass; /* Декларируем переменную типа TestClass */
Println (Ob.prop); /* Объект автоматически создается */
```

Object RSL позволяет наследовать классы объектов от других классов. Имя базового класса указывается в круглых скобках после ключевого слова CLASS. Для инициализации базового класса необходимо вызвать предопределенный метод, название которого образуется путем добавления к имени класса приставки "Init". Вызов инициализатора базового класса может располагаться в любом месте определения дочернего класса.

Пример.

```
Унаследуем класс "Сотрудник" от класса "Персона", добавив к
нему свойство "Должность":
Class ( Персона ) Сотрудник ( п_Имя, п_Фамилия, п_Должность)
    Var Должность;
    Macro Отчет
        Отчет;
        Println ("Должность:", Должность);
    End;
    InitПерсона (п_Имя, п_Фамилия);
    Должность = п_Должность;
End;
```

Множественное наследование в Object RSL не поддерживается.

Обращение к свойствам и методам класса осуществляется по имени.

Пример.

```
Obj.Имя;
Obj.Отчет;
```

Кроме того, Object RSL поддерживает обращение к свойствам RSL-классов не только по имени, но и по индексу.

Пример.

```
Class Test
    Var prop1, prop2;
End;
Ob = Test;
A = ob.prop1; /* Доступ к свойству по имени */
B = ob (0); /* Доступ к свойству по индексу */
```

Для совместимости со стандартом автоматизации в Object RSL добавлена поддержка свойства по умолчанию. Свойством по умолчанию считается свойство, имя которого в выражении можно не указывать.

Пример.

```
/* Объект ob содержит свойство по умолчанию item. Для
обращения к свойству item можно записать:*/
ob. (10) = 100;
```

```
/* вместо: ob.item (10) = 100;*/
```

Если метод имеет параметры, то они указываются в круглых скобках аналогично параметрам [макропроцедур RSL](#) ⁽⁵⁷⁾.

Пример.

```
A = ob .met (3);
```

Классы Object RSL могут иметь и свойства с параметрами. В настоящее время такие классы можно создать, например, при помощи [специального инструмента DLM SDK](#) ⁽³⁰⁹⁾. Параметры свойства аналогично параметрам метода указываются в круглых скобках.

Пример.

```
A = ob .prop (3);
```

Классы Object RSL могут содержать деструктор, определяемый пользователем. Для его использования в определении класса необходимо указать метод с предопределенным именем **Destructor**. Деструктор вызывается автоматически при "разрушении" объекта.

Пример.

```
Class Test
  macro Destructor
    printlm ("Destructor called");
  end;
end;
```

Если деструктор не определен, то все выделенные при создании объекта ресурсы освобождаются автоматически.

В коде методов все методы и свойства доступны непосредственно по имени. При этом в каждый метод класса передается скрытый параметр **this**, представляющий собой ссылку на объект, для которого вызывается метод или свойство.

Ссылки на объекты хранятся в переменных RSL. При очистке переменной происходит удаление ссылки на объект. Переменная очищается перед присвоением ей нового значения либо при выходе из области видимости. Таким образом, объект класса будет существовать и его невозможно будет удалить до тех пор, пока на него есть хотя бы одна ссылка. При удалении последней ссылки на объект удаляется сам объект.

В языке RSL также предусмотрено использование так называемых "слабых" ссылок, которые не управляют временем жизни самого объекта, то есть объект можно удалить, если на него существуют "слабые" ссылки. Механизм "слабых" ссылок в настоящее время поддерживается только для объектов классов, созданных на языке RSL. "Слабая" ссылка реализуется встроенным классом **WeakRef**. Конструктор класса выглядит следующим образом:

```
WeakRef (obj:Object) : Object
```

Конструктор возвращает объект, который представляет собой "слабую" ссылку на объект **obj**. Этот объект в выражениях используется точно так же, как объект **obj**. При попытке использовать "слабую" ссылку на объект, который уже удален, возникает ошибка времени исполнения.

В языке RSL предусмотрен тип данных **MethodRef**, представляющий собой ссылку на метод объекта. Значение данного типа можно получить при помощи процедуры [R2M](#) ⁽²⁸⁶⁾. Используя ссылку на метод, можно вызвать этот метод при помощи процедур [ExecMacro](#) ⁽²⁸⁹⁾, [ExecMacro2](#) ⁽²⁹⁰⁾, для этого следует передать в качестве первого параметра переменную типа ссылка на процедуру, или при помощи процедуры [CallR2M](#) ⁽²⁸⁶⁾.

Все методы классов являются виртуальными. Добавление в дочерний класс метода с именем, которое уже используется для одного из методов базового класса, вызовет замену метода родительского класса методом дочернего класса. Применение наследования с использованием виртуальных методов позволяет модифицировать функциональность базовых классов. Кроме того, в Object RSL можно заменять методы конкретного экземпляра класса. Для этого используется стандартная процедура [GenAttach](#)²⁸³.

RSL-классы можно наследовать от классов, определенных в DLM-модулях, которые создаются с помощью [специального инструмента DLM SDK](#)³⁰⁹, а также от встроенных классов, таких как **TBFile**, **TDirList**¹⁶⁰ и т.п. Такие наследуемые классы являются внешними по отношению к RSL-классам.

Особенностью использования RSL-классов, унаследованных от внешних классов, является невозможность обращения напрямую к методам родительского класса после того, как в дочернем классе эти методы были переопределены. Для этих целей служит стандартное свойство RSL-классов **_extObj**, возвращающее ссылку на внешний объект. Это свойство позволяет переопределять методы базового класса в дочернем и сохраняет возможность вызова методов базового класса.

Пример.

```
class (TBFile) TBfileEx (name,mode,key)
  InitTBFile (name,mode,key);
  macro GetDirect (pos)
    return _extObj.GetDirect (pos);
  onerror
    println (status);
    if (status == 43)
      return false;
    end;
  RunError;
end;
end;
ob = TBfileEx ("client.dbt","w",0);
ob.next;
pos = ob.getPos;
println (ob.GetDirect (pos));
```

При использовании пользовательских RSL-классов предусмотрено преобразование объектов в строку. Для этого необходимо определить в классе метод с именем **ToString**, возвращающий символьную строку.

Пример.

```
class Test
  macro ToString
    println ("Это мой объект Test")
  end;
end;
println (Test);
// Выводит:
Это мой объект Test
```


Определение символических констант CONST

Все символические константы должны быть объявлены явно при помощи определения CONST. Кроме объявления имени, необходимо проинициализировать его каким-либо значением.

Список идентификаторов константы будет соответствовать типу заданной величины.

Синтаксическая форма определения CONST имеет следующий вид:

```
[local | private]CONST идентификатор [: имя типа ] '=' выражение  
{ ',' идентификатор [: имя типа ] '=' выражение }
```

Список идентификаторов может содержать произвольное количество проинициализированных имен констант, разделенных запятыми. Попытка изменить значение любой из объявленных констант приведет к сообщению об ошибке при компиляции.

Пример.

```
const  
    Количество = 35,  
    ФИО = "Иванов Иван Иванович";
```

Определение процедуры MACRO

Определение MACRO используется для описания процедуры RSL:

```
[local | private] MACRO идентификатор [ '('(список формальных параметров)')' ] [: имя  
типа ]  
    список определений и инструкций  
END
```

Идентификатор процедуры – это ее имя, которое используется в инструкции вызова процедуры для выполнения.

Список формальных параметров заключается в круглые скобки. Он имеет следующий вид:

```
идентификатор [ : имя типа ] { , идентификатор [ : имя типа ] }
```

Этот список содержит разделенные запятыми идентификаторы переменных, которым при вызове данной процедуры будут присвоены значения фактических параметров. Список определений и инструкций может содержать любые инструкции и определения языка RSL, в том числе и другие определения MACRO. Таким образом, структура процедуры повторяет структуру RSL-программы.

Пример.

```
/* Пример определения процедуры, печатающей */  
/* содержимое файла: */  
MACRO Содержимое (ff)  
    WHILE (next (ff))
```

```
[      #      |##### |      #      ]
      (ff.Balance,      ff.Name_Part,      ff.Part)
      END;
END;
```

Процедуры языка RSL

Процедуры являются неотъемлемой частью любой программы, написанной на RSL. Они являются фрагментом программы со своим именем, к которому можно обратиться для выполнения необходимых действий.

Кроме того, допускается использовать стандартные процедуры, входящие в состав языка RSL. Обращение к встроенным процедурам так же осуществляется по имени.

Для косвенного вызова процедуры (определенной пользователем или стандартной) служит стандартная процедура [ExecMacro](#)²⁸⁹.

Процедуры RSL могут присутствовать в выражениях, при этом вместо процедуры в выражение будет подставлено значение, возвращаемое процедурой.

Если процедура явно не возвращает значение, то возвращаемое значение будет иметь тип **V_UNDEF**.

Передача параметров

При описании процедуры может быть указан список параметров.

```
macro My (a,b,c,d)
```

Здесь **a,b,c,d** являются формальными параметрами, создаваемыми и инициализируемыми значениями фактических аргументов при входе в процедуру. После завершения процедуры эти переменные прекращают свое существование.

Нумерация параметров начинается с нуля. При вызове процедуры после ее имени задаются значения фактических параметров, которые заменят формальные при выполнении.

Передача параметров в процедуре производится по значению, однако параметры типа **ARRAY**, **RECORD** и **FILE**, а также все объекты RSL-классов передаются по ссылке.

Так как передача параметров производится по значению, поэтому изменение формальных параметров в вызванной процедуре не приводит к изменению значений соответствующих им фактических аргументов в вызывающей. Однако пользователю предоставляется возможность изменить значения фактических параметров. Для этого необходимо выполнение следующих условий:

- при указании типа параметров перед декларацией типа должен находиться символ "@";
- фактические параметры при вызове процедуры должны быть переданы по ссылке.

Пример.

```
/* Пример модификации параметров:*/
macro Test (p1:@variant, p2:@integer)
  p1 = 10;
  p2 = 20;
end;
```

```
var v1, v2;
Test (@v1, @v2);
/* В результате переменная v1 будет равна 10, а v2 равна 20.
*/
```

Изменить значение фактических параметров можно также при помощи процедуры [SetParm](#)²³¹. В этом случае в качестве изменяемого фактического параметра должна быть переменная. Если же это условие не выполняется, значение фактического параметра не изменится, при этом система не сгенерирует сообщения об ошибке.

Пример.

```
setparm (0,10+aa)      /* Замена первого параметра в
                        вызывающей процедуре (с номером
                        0) на значение 10+aa */

setparm (1,Local (bb)); /* Замена второго параметра в
                        вызывающей процедуре (с номером
                        1) на значение Local (bb) */
```

Пример процедуры, изменяющей значения фактических параметров в вызывающей процедуре. См. подробнее.

```
macro Demo (number , text)
    setparm (0,10);
    setparm (1,"Привет")
end;
```

Теперь после вызова:

```
var id, name;
Demo (id, name)
```

переменная **id** получит значение 10, а переменная **name** – значение “Привет”.

Если количество фактических параметров меньше количества формальных, то вместо недостающих параметров передаются неопределенные значения типа V_UNDEF. Если количество фактических параметров больше количества формальных, то к избыточным параметрам можно получить доступ через обращение к процедуре [GetParm](#)²³⁰.

При описании процедуры можно вообще не указывать формальные параметры. В этом случае доступ к фактическим параметрам производится только через обращение к процедуре **GetParm**.

Пример.

```
getparm (0,param1);    /* Присвоить переменной param1
                        значение первого параметра (с
                        номером 0) из вызывающей процедуры
                        */

getparm (1,param2);    /* Присвоить переменной param2
                        значение второго параметра (с
                        номером 1) из вызывающей процедуры
                        */
```

Наличие процедуры **GetParm** позволяет писать процедуры, принимающие произвольное количество параметров.

См. Пример процедуры, вычисляющей и возвращающей сумму всех своих параметров.

```
macro Calc
  var sum = 0, i, val;
  while (getparm (i,val))
    sum = sum + val;
    i = i + 1;
  end;
  return sum
end
```

Теперь можно вызывать процедуру **Calc** так:

```
sum = Calc (1,2,3,4,5);
sum = Calc (10,20);
```

Определение массивов

В языке RSL можно объявлять только одномерные массивы. Для этого предусмотрены следующие конструкции:

- [конструкция ARRAY](#) ⁽⁶⁰⁾;
- [стандартный класс TArray](#) ⁽⁶¹⁾.

Примечание.

При определении массивов предпочтительнее использовать класс **TArray**. Конструкция **ARRAY** в настоящее время поддерживается для совместимости с ранее разработанными программами.

Конструкция ARRAY

Синтаксическая форма определения **ARRAY** имеет следующий вид:

[local | private] ARRAY идентификатор {',' идентификатор }

Список идентификаторов может содержать произвольное количество имен, разделенных запятыми. Идентификатор массива – это имя объекта, через которое можно ссылаться на определенный массив. В выражении идентификатор массива преобразуется к переменной типа **V_ARRAY** (ссылка на массив). Благодаря этому, идентификатор массива можно присваивать переменным и передавать в качестве параметров процедурам.

Элементы массива могут хранить значения переменных любых типов, включая тип **V_ARRAY** (ссылка на массив). Их нумерация начинается с 0. Размер массива изменяется динамически. Он может быть задан неявно в момент присвоения значений элементам.

Пример.

```
/* Определение пустых массивов */
array MyVar, First, Second;
/* Присвоение значений элементам массива First*/
First (0) = "Имя";
First (1) = Second;
/* Присвоение значений элементам массива Second */
Second (0) = 1;
Second (1) = 23;
Second (2) = 567;
```

```

/* Определение размера массива MyVar */
asize (MyVar, 10 );
/* Присвоение значения элементу массива MyVar */
MyVar (9) = First;
[#####] (MyVar (9) (1) (2) );
/*В стандартный выходной поток */ будет выведено: 567 */
/*Конструкция MyVar (9) (1) (2) эквивалентна конструкции MyVar
(9,1,2).*/

```

Текущий размер массива можно определить явно при помощи процедуры [ASize](#)²²⁰.

Пример.

```
cc = asize (aa);
```

Используя процедуру **ASize** с двумя параметрами, можно изменить размер уже существующего массива. При этом старое содержимое удаляется, и распределяется новый массив заданного размера. Рекомендуется использовать эту возможность, чтобы уменьшить количество перераспределений памяти.

Пример.

```

asize (aa,10);    /* Установить для массива aa новый размер
10 */
i = 0;
while (i < 10)    /* Цикл присвоения элементам новых
значений */
    aa (i) = I;
    i = i + 1;
end;

```

Если в левой и правой части инструкции присваивания стоят массивы или переменные типа V_ARRAY, то происходит копирование массивов.

Пример.

```

dd = aa;    /* dd - переменная типа V_ARRAY ссылается на
массив aa */

bb = dd;    /* Создается копия массива dd */

```

Стандартный класс TArray

Стандартный класс **TArray** языка RSL используется для реализации динамического массива. Он представляет собой объектную альтернативу стандартной конструкции языка ARRAY. Имеется возможность замены конструкции ARRAY на новую конструкцию **TArray**.

Пример.

```

Array myArray; /* Прежняя конструкция */
myArray = TArray; /* Замена прежней конструкции на новую */

```

Синтаксическая форма конструктора **TArray** имеет следующий вид:

TArray (MarshalByVal: Bool, rootSz: Integer, delta: Integer)

Память под элементы массива выделяется постранично. Параметр [MarshalByVal](#)⁶³ задает одноименное свойство для нового объекта. Оставшиеся параметры конструктора класса **TArray** задают количество элементов массива на первой странице (значение **rootSz**) и на последующих страницах (значение **delta**), под которые выделяется память. По умолчанию

первая страница может содержать 50 элементов, последующие страницы – по 10. Данные значения можно изменить, указав параметры конструктору класса.

Пример.

```
/* Опишем массив, в котором первая страница содержит 100 элементов, а все следующие – по 5:*/  
MyArray = TArray ( true, 100, 5 );
```

В качестве параметра можно указать ссылку на уже существующий массив. В этом случае создается новый массив, содержащий копию элементов массива, обозначенного в качестве параметра.

Доступ к элементам массива осуществляется посредством указания индекса элемента в круглых скобках после переменной, ссылающейся на объект класса **TArray**.

Пример.

```
FirstArray = TArray;  
/* Присвоение значения */  
FirstArray (0) = 100;  
/* Создается копия массива FirstArray */  
SecondArray = TArray (FirstArray);
```

Объекты класса **TArray** содержат свойство **Size**, значением которого является текущий размер массива. Присваивая свойству **Size** новое значение, можно изменять размер массива. Увеличивается размер массива динамически при обращении к элементу, стоящему за последним элементом массива.

Пример.

```
Println ( "Текущий размер массива:", FirstArray.Size );
```

Объекты класса **Tarray** имеют наименованные свойства – элементы массива, к которым можно обращаться только по индексу. Для доступа к ним к свойству **Size** добавилось еще одно свойство – **Value(N)** – значение N-го элемента массива.

Следует отметить, что с появлением у объектов свойств с параметрами возникла проблема, связанная с одинаковым синтаксисом указания индекса и свойства с параметрами. Поэтому для разрешения неоднозначности предусмотрена возможность указания индекса свойства объекта в квадратных скобках.

Пример.

```
Ob = Tarray;
Ob [5] = 100;

/* Пример обращения к элементам массива */
class TTT /* Определяем класс
TTT */
    var prop = Tarray;
    prop (5) = 200;
    macro Me
        return prop;
    end;
end;
ob = TTT; /* Создаем экземпляр класса TTT */
/* Распечатаем значение пятого элемента массива, */
/* на который ссылается свойство prop тремя способами: */
println (ob.prop() (5));
println (ob.prop .value (5));
println (ob.prop [5]);
```

Объекты класса **TArray** имеют свойство **MarshalByVal** типа **Bool**, которое определяет, каким образом объекты данного класса передаются в **RSCOM**. Если свойству присвоено значение **TRUE**, то объекты передаются по значению. По умолчанию свойство имеет значение **FALSE**, и, соответственно, объекты класса **TArray** по умолчанию передаются по ссылке.

Кроме перечисленных выше свойств, объекты класса **TArray** имеют метод **Sort (callback:Variant, data:Variant):Bool**. Метод предназначен для сортировки элементов массива в соответствии с порядком, определяемым пользовательским обработчиком callback. Обработчик используется для сравнения двух элементов массива и имеет следующий прототип:

```
macro Compare (left:Variant, right:Variant, data:Variant):Integer
```

Алгоритм работы обработчика следующий:

- если левый элемент (**left**) больше правого (**right**), обработчик возвращает значение, которое больше **0**;
- если левый элемент (**left**) меньше правого (**right**), обработчик возвращает значение, которое меньше **0**;
- если элементы равны, обработчик возвращает **0**.

Указанный обработчик может быть задан в виде строки с именем глобальной процедуры RSL, ссылки на процедуру или ссылки на метод класса.

Параметр дата (**data**) передаётся из процедуры сортировки в пользовательский обработчик.

В случае успешного завершения метод возвращает **TRUE**; в случае возникновения ошибки или при неверно заданных параметрах метод возвращает **FALSE**.

Поддержка технологии ActiveX в RSL

В Object RSL предусмотрена поддержка ActiveX-объектов, являющихся объектами автоматизации и зарегистрированными в системе.. Пользователи Object RSL могут создавать ActiveX-объекты и обращаться к их свойствам и методам из RSL-программы.

Чтобы воспользоваться этой возможностью, необходимо импортировать модуль **rslx** и вызвать конструктор ActiveX-объектов. Синтаксическая форма конструкции **ActiveX** выглядит следующим образом:

ACTIVEX (имя объекта [, имя удаленного компьютера [, ссылка на объект]])

В качестве первого параметра передается идентификатор необходимого класса COM-объекта (progID). Вторым (необязательным) параметром является имя компьютера, на котором создается объект. В качестве третьего параметра можно передать значение **TRUE**, в этом случае конструктор будет пытаться вернуть ссылку на уже существующий объект, а не создавать его новый экземпляр.

Пример.

```
1) Запишем строку в MS Word:
ob = ActiveX ("Word.basic");
ob.FileNew;
ob.Insert ("Hello, World!");
ob.AppShow;

2) Подключимся к объекту автоматизации Excel.Application и
добавим рабочую книгу Excel. Если приложение Excel уже было
запущено, то его новая копия запускаться не будет.
import rslx;
ob = ActiveX ("Excel.Application", NULL, TRUE);
ob.Visible = TRUE;
ob.Workbooks.add;
MsgBox ("Нажмите любую клавишу для выхода из Excel");
ob.Quit
```

Способ обращения к свойствам и методам объекта автоматизации ничем не отличается от обращения к свойствам и методам объекта RSL.

Следует иметь в виду, что некоторые методы объектов ActiveX возвращают строку символов, включающую нули. В RSL конец строки определяется по нулевому символу, поэтому информация таких строк доступна только до первого нуля. Для корректного отображения полученной строки результат может быть выведен с помощью нультерминированной строки. С этой целью следует использовать значение **nullVal**, заданное в качестве последнего параметра свойства или метода объекта ActiveX. Указанное значение не передается в свойства или методы объекта и служит лишь сигналом о том, что возвращаемое значение типа **VT_BSTR**⁶⁶ необходимо вернуть в виде массива строк.

Пример.

```
import rcw;
cpwin;

const cdlOFNAllowMultiselect = #200;
const cdlOFNExplorer = #80000;
const cdlOFNFileMustExist = #1000;
const NoChangeDir = #8;
var srv = CreateObject ("rsax", "TRsAxServer", "RsAxServer",
false);
var oDlg = srv.CreateComObject ("MSComDlg.CommonDialog");

oDlg.Flags =
cdlOFNAllowMultiselect+cdlOFNFileMustExist+cdlOFNExplorer+No
ChangeDir;
oDlg.MaxFileSize = 32767;
oDlg.ShowOpen();
```



```
var result = oDlg.FileName (nullVal);
var n;
for (n, result)
    println (n)
end;
```

ActiveX является стандартным классом языка RSL, и ActiveX-объект может быть создан с помощью процедуры [GenObject](#)²⁸³. В этом случае синтаксис определения имени класса выглядит следующим образом:

"ActiveX\\<имя объекта>[\\<имя удаленного компьютера>]"

Пример.

Создадим объект *Word.Basic*:

```
ob = GenObject ("ActiveX\\Word.basic")
```

Доступ к объектам, созданным с помощью этой конструкции, будет только локальным. Если необходимо создавать или запускать ActiveX-объект не только локально, но и удаленно, для этого следует воспользоваться RSCOM-технологией: вместо встроенной конструкции **ActiveX** использовать стандартный RSCOM-сервер [Rsax](#)¹⁷⁹.

Использование специальных значений

Для поддержки специальных значений, используемых в COM-автоматизации, в языке RSL предусмотрены два значения:

- **OptVal** – значение используется для передачи опциональных параметров.
- **NullVal** – значение используется для передачи специального значения NULL из COM-автоматизации.

При передаче этих значений в методы объектов COM-автоматизации выполняются следующие преобразования:

OptVal -> *VT_ERROR*, *scode* = *DISP_E_PARAMNOTFOUND*

NullVal -> *VT_NULL*

Значение NULL языка RSL транслируется следующим образом: *Null* -> *VT_EMPTY*.

Пример.

```
import rcw;
ob = CreateObject ("rsax","TRsAxServer","RsAxServer",false);
var ex = ob.CreateComObject ("Excel.Application",false);
ex.visible = true;
var wb = ex.WorkBooks.add;
var sh = wb.ActiveSheet;
sh.Range ("A1").value = $100;

// В качестве первого параметра использовать значение по
умолчанию.
var MySheet=wb.Sheets.Add (OptVal,sh);
MySheet.Name = "My sheet";
```

Соответствие типов данных языка RSL и ActiveX

При использовании ActiveX-объектов в программе на языке RSL выполняется преобразование типов данных языка RSL в типы данных ActiveX и наоборот по правилам, приведенным в таблице:

Тип RSL	Тип ActiveX
NullVal	VT_NULL
OptVal	VT_ERROR (DISP_E_PARAMNOTFOUND)
Null	VT_EMPTY
Integer	VT_I4, VT_UI1, VT_I1, VT_UI2, VT_I2, VT_UI4
Double	VT_R8, VT_R4
Money	VT_CY
Bool	VT_BOOL
String	VT_BSTR
Date	VT_DATE
Time	VT_DATE
Dttm	VT_DATE
Object	VT_DISPATCH
TArray	SAFEARRAY(VARIANT)

Типы данных языка RSL, для которых представлено несколько типов ActiveX, при конвертировании в ActiveX преобразуются в первый тип из списка.

При преобразовании типа данных ActiveX VT_DATE в тип данных RSL действует следующее правило:

- если в переменной типа VT_DATE содержится непустое время и непустая дата, результатом будет RSL-тип **Dttm**;
- если в переменной типа VT_DATE содержится пустое время и непустая дата, результатом будет тип **Date**;
- если в переменной типа VT_DATE содержится пустая дата и непустое время, результатом будет тип **Time**.

Пустым временем считается время "0:0:0,0", пустой датой – дата 30.12.1899.

Поддержка стандартных коллекций

Object RSL позволяет использовать стандартные коллекции объектов автоматизации. Для создания таких коллекций и доступа к их содержимому существует специальный стандарт, в значительной степени облегчающий их использование.

Так, например, в приложении Excel есть коллекция рабочих книг. Пусть переменная **ob** содержит ссылку на объект **Excel.Application**. Для создания коллекции язык RSL позволяет создать объект-перечислитель при помощи предопределенного метода **CreateEnum**, который автоматически добавляется к каждому ActiveX-объекту.

Пример.

```
En = ob.Workbooks.CreateEnum;  
While (En.Next )  
    MsgBox (En.Item.Name)  
End;
```

Для навигации по коллекции используется метод **Next** объекта-перечислителя. Когда коллекция просмотрена до конца, данный метод возвращает значение FALSE. Ссылка на очередной элемент хранится в свойстве **Item**. Еще один метод объекта-перечислителя – **Reset**. После его вызова можно всю итерацию повторить снова.

В качестве стандартной коллекции может выступать объект **TArray**.

Пример.

```
Ar = TArray;  
Ar (0) = "Element 1";  
Ar (1) = "Elemrnt 2";  
En = Ar.createEnum;  
While (En.next)  
    Println (En.Item)  
End;
```

Обращение к свойствам объектов с параметрами

Кроме обычных свойств, объекты автоматизации могут содержать свойства с параметрами, к которым можно обращаться из Object RSL.

Пример.

```
ob = ActiveX ("XArray.XArray");  
ob.ReDim (0,10,0,20); /* У массива будет две размерности 0-  
10 и 0- 20 */  
i = 0;  
while (i < 10)  
    ob.Value (i,0) = "String "+i;  
    i = i + 1  
end;
```

Объект **XArray.Xarray** реализует двухмерный массив. Элементы этого массива хранятся в свойстве **Value**, а индексы элементов определяются параметрами данного свойства.

При поддержке свойств с параметрами может потребоваться внести изменения в существующие программы.

Пример.

Предположим, что есть класс Object RSL **MyClass**:

```
Class MyClass
    Var prop = Tarray;
End;
Ob = MyClass;
```

Раньше присвоить значения элементам массива можно было следующим образом:

```
Ob.prop (0) = "Строка1";
Ob.prop (1) = "Строка2";
```

Теперь же в такой записи значения 0 и 1 будут истолкованы как параметры свойства prop, и после выполнения приведенных инструкций это свойство будет содержать не массив из двух элементов, а значение Строка2. Чтобы все-таки присвоить значения элементам массива, следует сделать следующую запись:

```
Ob.prop () (0) = "Строка1";
Ob.prop () (1) = "Строка2";
```

Для разрешения неоднозначности при доступе к элементам массива также можно использовать квадратные скобки.

```
Ob.prop [0] = "Строка1";
Ob.prop [1] = "Строка2";
```

Используя свойство Value, с помощью которого можно обращаться к элементам массива, приведенные выше инструкции можно переписать иначе:

```
Ob.prop.Value (0) = "Строка";
Ob.prop.Value (1) = "Строка2";
```

Доступ к объектам Object RSL из других языков

Все объекты классов языка Object RSL являются объектами автоматизации и, следовательно, могут быть доступны из других языков программирования.

Пример.

Рассмотрим пример обращения к объекту Object RSL из Visual BASIC.

Объект Object RSL может быть передан в процедуру Visual BASIC, например, в качестве параметра. Предположим, что на языке Visual BASIC реализован простой сервер автоматизации, к которому можно обратиться, указав его идентификатор **VbServer.VbClass**. Пусть этот сервер экспортирует следующую процедуру:

```
Public Sub TetRSL (ob As Object)
    ob.Test "From Basic", " Parm1 ", "Parm2"
End Sub
```

Теперь на языке *Object RSL* создадим два объекта: один — класса *RSL*, другой — класса *Visual BASIC*. Вызов метода **Test** класса *Object RSL* в языке *Visual BASIC* осуществляется следующим образом:

```
import rslx;
class TestRSLClass
    macro Test (p1,p2,p3)
        MsgBox ("Вызван метод Test с параметрами: ", p1,
" ", p2, " ", p3)
    end;
end;
vb = ActiveX ("VbServer.VbClass");
rsl = TestRSLClass;
vb.TestRSL (rsl);
```

Обработка событий

Объекты автоматизации могут иметь не только свойства и методы, но и события, то есть способы уведомления клиента объекта о происходящих в объекте изменениях. *Object RSL* позволяет обрабатывать эти события, то есть принимать сообщения от объектов автоматизации и выполнять необходимые действия.

Для этого предусмотрены следующие классы:

- [TrslEvHandler](#) ⁽⁶⁹⁾.
- [RslTimer](#) ⁽⁷⁵⁾.

Класс TRslEvHandler

Класс **TRslEvHandler** предназначен для обработки событий от ActiveX-объектов. Он может использоваться несколькими способами: от него можно унаследовать класс — обработчик событий, можно создать экземпляр класса **TRslEvHandler** и передать в качестве параметра конструктора объект — обработчик событий и т.п. Можно также создать экземпляр класса **TRslEvHandler** и не передавать объект-обработчик, в этом случае для обработки событий будут использоваться глобальные макропроцедуры.

Чтобы объект класса **TRslEvHandler** мог обрабатывать события от некоторого ActiveX-объекта, данный ActiveX-объект должен быть добавлен во внутреннюю коллекцию объекта класса **TRslEvHandler** путем присвоения его свойству **EvSource**.

Нет необходимости создавать глобальный экземпляр класса **TRslEvHandler**, RSL автоматически создает его с именем **RslEvHandler**.

Для создания экземпляра данного класса **TRslEvHandler** используется конструктор **TRslEvSource** в следующем формате:

```
TRslEvSource ( [ handlerObj ] [, doAddRef ] )
```

Параметры конструктора задают следующие значения:

- **handlerObj** — объект-обработчик событий, методы которого будут использоваться для обработки событий. Если параметр не задан или задано значение NULL, то в качестве объекта-обработчика выступает вся RSL-программа, а для обработки событий будут использоваться глобальные макропроцедуры.

- **doAddRef** – значение типа Bool. Если параметр равен TRUE, то конструктор класса автоматически инкрементирует счетчик ссылок на объект **handlerObj**. В противном случае инкремент выполняться не будет. Это полезно в том случае, когда объект класса **TRslEvSource** создается как свойство класса обработчика. Если параметр не задан, то по умолчанию его значение принимается равным TRUE.

Класс **TRslEvSource** имеет следующие свойства:

- Свойство [EvSource](#)⁽⁷³⁾.
- Свойство [TypeLib](#)⁽⁷⁴⁾.

Класс **TRslEvSource** имеет следующие методы:

- Метод [SetHandler](#)⁽⁷⁴⁾.
- Метод [RemHandler](#)⁽⁷⁴⁾.
- Метод [raise](#)⁽⁷⁵⁾.

Примеры использования класса TRslEvHandler.

В примерах будем обрабатывать событие NewWorkBook с идентификатором 1565 приложения Microsoft Excel.

Пример 1:

```
/* Используем глобальный объект RslEvHandler, создаваемый
автоматически. */
import rslx;
ob = ActiveX ("Excel.Application");
ob.Visible = true;
RslEvHandler.evSource ("Excel", "Application") = ob;
ob.Workbooks.add;
macro Excel_NewWorkBook (wb)
  println ("New Book: ",wb.Name)
End;
MsgBox ("OK");
RslEvHandler.evSource ("Excel") = NULL;
ob.Quit;
```

Пример 2:

```
/* Создадим специальный класс для обработки событий и
отдельный экземпляр класса TRslEvHandler. */
import rslx;
class THandler
  macro Excel_NewWorkBook (wb)
    println ("New Book: ",wb.Name)
  End;
end;
handler = TRslEvHandler (THandler);
ob = ActiveX ("Excel.Application");
ob.Visible = true;
handler.evSource ("Excel", "Application") = ob;
ob.Workbooks.add;
MsgBox ("OK");
handler.evSource ("Excel") = NULL;
ob.Quit;
```

Пример 3:

/ Создадим специальный класс для обработки событий и отдельный экземпляр класса TRslEventHandler как свойство класса обработчика. Чтобы не было циклических ссылок, вторым параметром конструктору TRslEventHandler передаем false. */*

```
import rslx;
class THandler
    evObj = TRslEventHandler (this, false);
    macro Excel_NewWorkBook (wb)
        println ("New Book: ",wb.Name)
    End;
end;
handler = THandler;
ob = ActiveX ("Excel.Application");
ob.Visible = true;
handler.evObj.evSource ("Excel", "Application") = ob;
ob.Workbooks.add;
MsgBox ("OK");
handler.evObj.evSource ("Excel") = NULL;
ob.Quit;
```

Пример 4:

/ Создадим специальный класс для обработки событий, наследуя его от класса TRslEventHandler. */*

```
import rslx;
class (TRslEventHandler) THandler
    macro Excel_NewWorkBook (wb)
        println ("New Book: ",wb.Name)
    End;
end;
handler = THandler;
ob = ActiveX ("Excel.Application");
ob.Visible = true;
handler.evSource ("Excel", "Application") = ob;
ob.Workbooks.add;
MsgBox ("OK");
handler.evSource ("Excel") = NULL;
ob.Quit;
```

Пример 5:

/ Создадим специальный класс для обработки событий, наследуя его от класса TRslEventHandler. В отличие от примера 4, применяем*

*вторую форму использования evSource, не использующую обращений к библиотеки типов. Такая форма может применяться и для остальных примеров. */*

```
import rslx;
class (TRslEventHandler) THandler
    macro Excel_NewWorkBook (wb)
        println ("New Book: ",wb.Name)
    End;
end;
handler = THandler;
ob = ActiveX ("Excel.Application");
```

```

ob.Visible = true;
handler.evSource ("Excel", "{00024413-0000-0000-C000-000000000046}", 1) = ob;
handler.SetHandler ("Excel", "Excel_NewWorkBook", 1565);
ob.Workbooks.add;
MsgBox ("OK");
handler.evObj.evSource ("Excel") = NULL;
ob.Quit;

```

Пример 6:

/ Рассмотрим пример использования Object RSL в качестве скрипта на HTML-странице. Здесь обрабатывается событие onClick, закрепленное за кнопкой, которая размещена на HTML-странице. Как и в случае с Excel, для связи источника с обработчиком событий используется глобальная переменная RslEventHandler. */*

```

<HTML>
<BODY>
<P> Демонстрация обработки сообщения onClick </P>
<BUTTON ID="CmdButton" STYLE="margin:20%;padding:40">
Press Me
</BUTTON>
<SCRIPT LANGUAGE="RSLScript">

macro CmdButton_onClick
    MsgBox ("Нажата кнопка ",event.srcElement.id );
end;
RSLEventHandler.EvSource ("CmdButton") = CmdButton;
</SCRIPT>
</BODY>
</HTML>

```

В качестве обработчиков событий от элементов объектной модели Internet explorer можно также использовать ссылки на процедуру и ссылки на метод объекта.

```

<HTML>
<BODY>
<P> Демонстрация обработки сообщения onClick </P>
<BUTTON ID="CmdButton1" STYLE="margin:10%;padding:40%">
Button1
</BUTTON>
<BUTTON ID="CmdButton2" STYLE="margin:10%;padding:40%">
Button2
</BUTTON>
<SCRIPT LANGUAGE="RSLScript">
class HandlerClass
    macro OnClick
        MsgBox ("Нажата кнопка ",event.srcElement.id );
    end;
end;
var handler = HandlerClass;
macro RslTestOnClick
    MsgBox ("Нажата кнопка ",event.srcElement.id );
end;
/* Используем в качестве обработчика глобальную процедуру */

```



```
CmdButton1.onclick = @RslTestOnClick;  
/* Используем в качестве обработчика метод объекта */  
CmdButton2.onclick = R2M(handler, "OnClick");  
</SCRIPT>  
</BODY>  
</HTML>
```

Свойства класса **TRslEvHandler**

Свойство **EvSource**

Свойство **EvSource** класса **TRslEvSource** обеспечивает доступ к внутренней коллекции объектов – источников событий. Чтобы поместить объект источник во внутреннюю коллекцию, необходимо присвоить объект-источник этому свойству.

Возможны два способа использования данного свойства.

Рассмотрим первый способ:

EvSource (pref [, intfName]) = ob

Параметры при этом задают следующие значения:

- **pref** – уникальная строка для данного объекта класса **TRslEvHandler**, используемая при формировании имени объекта – источника событий во внутренней коллекции. Имя метода для обработки события образуется в формате `pref_<имя события>`.
- **intfName** – имя интерфейса событий для объекта-источника в библиотеки типов. Данный параметр является необязательным. Его необходимо указывать только для тех объектов, для которых RSL не может получить данную информацию автоматически, например, для объектов Microsoft Excel. В некоторых случаях требуется еще явно указать имя библиотеки типов при помощи свойства **TypeLib**.
- **ob** – ActiveX-объект, который является источником событий.

Второй способ более эффективен, так как вообще не требует использования библиотеки типов, но для его применения необходимо вручную задать имена обработчиков и соответствующие им идентификаторы событий:

EvSource (pref, GUID, numProc) = ob

Параметр **pref** используется так же, как при первом способе. Остальные параметры задают следующие значения:

- **GUID** – задает уникальный идентификатор интерфейса событий в виде строки, например, "{00024413-0000-0000-C000-000000000046}".
- **numProc** – количество процедур обработчиков, которые будут добавлены при помощи метода **SetHandler**.

Этот способ также позволяет динамически подключать и отключать обработчики событий во время выполнения RSL-программы. Идентификаторы событий можно узнать из документации на используемый ActiveX-объект или из анализа библиотеки типов.

Для того чтобы изъять объект источник событий из коллекции и тем самым прекратить обработку событий, необходимо присвоить свойству **EvSource** с соответствующим параметром **pref** значение NULL. Это, например, необходимо сделать перед вызовом метода **Quit** объекта **Exel.Application**.

Свойство **TypeLib**

Свойству **TypeLib** класса **TRslEvSource** можно присвоить имя файла, содержащего библиотеку типа для объектов источников. Это необходимо делать только в том случае, если RSL не может получить необходимую информацию автоматически.

Пример.

```
TypeLib = fileName
```

Методы класса **TRslEvHandler**

Метод **SetHandler**

Метод **SetHandler** класса **TRslEvSource** устанавливает связь между событием и методом обработки.

Имя для метода обработки может быть выбрано произвольно, но при его формировании удобнее следовать правилу, принятому при использовании первого варианта работы со свойством **EvSource**.

Пример.

```
SetHandler ( pref, proc, id )
```

При этом применяются следующие параметры:

- **pref** – строка с именем объекта – источника событий во внутренней коллекции. Объект с данным именем должен быть предварительно присвоен свойству **EvSource** с использованием второго способа.
- **proc** – строка с именем метода обработчика события в объекте обработчике событий.
- **id** – целочисленный идентификатор события в объекте-источнике.

Метод **RemHandler**

Метод **RemHandler** класса **TRslEvSource** разрывает связь события и процедуры обработчика, установленной при помощи метода **SetHandler**.

Синтаксическая форма метода выглядит следующим образом:

```
RemHandler ( pref, id )
```

При этом применяются следующие параметры:

- **pref** – строка с именем объекта – источника событий во внутренней коллекции. Объект с данным именем должен быть предварительно присвоен свойству **EvSource** с использованием второго способа.
- **proc** – строка с именем метода обработчика события в объекте обработчике событий.
- **id** – целочисленный идентификатор события в объекте-источнике.

Метод raise

Данный метод класса **TRslEvSource** применяется в тех случаях, когда источниками событий являются не только внешние объекты автоматизации, но и объекты **Object RSL**. Для этого нужно создать класс **Object RSL** – наследник стандартного класса **TRslEvHandler**.

В этом случае событие произойдет в результате вызова метода **raise**, который в качестве первого параметра принимает имя события. Остальные параметры передаются как есть в процедуру обработки события.

Пример.

```
/*Приведем пример класса, который порождает событие onEvent1
при вызове метода Test*/
import rslx;
class (TRslEvHandler) Source
    macro Test
        raise ("onEvent1", "Это параметр события");
    end;
end;
/* Объект-обработчик выглядит так же, как в примере с Excel
*/
class (TRslEvHandler) Handler ( ob )
    EvSource ("OB") = ob;
    macro OB_onEvent1 ( par )
        MsgBox ("Событие OnEvent1 с параметром: ", par)
    end;
end;
ob = Source;
ev = Handler (ob);
ob.Test
```

Класс RslTimer

Класс **RslTimer** предназначен для вызова пользовательских обработчиков событий через заданные интервалы времени, т.е. позволяет использовать таймеры. Для работы таймера необходимо, чтобы поток управления находился в состоянии обработки пользовательского ввода.

Внимание!

Методы класса, в которых используются таймеры, не выполняются в RSL модулях, исполняемых на терминале при помощи RSCOM сервера rcwhost.d32.

Точность вызова таймера составляет 1 секунду. В случае необходимости большей точности срабатывания таймера, следует использовать системный таймер. Таймер, время которого равно нулю, вызывается сразу перед входом потока в состояние ожидания пользовательского ввода. Такой таймер предназначен только для однократного срабатывания, его следует удалить в обработчике таймера при первом срабатывании.

Класс **RslTimer** вызывается без параметров:

```
RslTimer ()
```

Для класса **RslTimer** предусмотрены следующие методы:

- [RemTimer](#)⁷⁶.

- [SetTimer](#)⁷⁷.

Методы класса RslTimer

Метод RemTimer

RemTimer (id:Integer):Bool

Метод позволяет удалить таймер с идентификатором **id**. Допускается вызывать этот метод из обработчика таймера.

Пример 1.

```
class TestTimer
  var count = 1;
  var t = RslTimer ();
  t.setTimer (2000, 1, R2M(this, "TimerProc"));
  t.setTimer (5000, 2, R2M(this, "TimerProc"));

  macro TimerProc (ind)
    message ("Timer id = ", ind, ", count = ", count);
    count = count + 1;
    if (count == 10)
      t.remTimer (1);
    end;
  end;
end;

var ob = TestTimer;
MsgBox (1);
```

Пример 2.

```
class (TForm) MyForm (owner:object, nm:string)
  InitTForm(owner,nm);
  setTemplate("alpha4.lbr", "FRM001");

  var Button1 = TControl(this, "Button1");
  var Label1 = TControl(this, "Label1");
  var myTimer = RslTimer;

  Button1.addHandler (1, R2M(this, "OnClick"));
  macro MyTimerProc (id)
    Label1.Text = "Текст установлен из таймера";
    myTimer.remTimer (1);
  end;

  macro OnClick (Sender: Object)
    myTimer.setTimer (0, 1, R2M(this, "MyTimerProc"));
  end;
end;
```

Метод SetTimer

SetTimer (timeout:Integer, id:Integer, handler:Variant [, isSys:Bool]):Bool

Метод позволяет установить новый таймер.

Параметры:

timeout – время в миллисекундах, через которое должен срабатывать таймер.

id – произвольный идентификатор задаваемого таймера. Введенное значение передается в пользовательский обработчик и в метод [RemTime](#)⁽⁷⁶⁾_r⁽⁷⁶⁾.

handler – пользовательский обработчик. В качестве обработчика могут быть заданы: имя глобальной или локальной процедуры, переменная типа V_PROC или V_R2M.

isSys – признак использования системного таймера. Возможные значения:

- TRUE – системный таймер используется.
- FALSE – системный таймер не используется.

Примечание.

Указанный признак поддерживается только на платформе Win32.

Возвращаемое значение:

Метод возвращает одно из значений:

- **TRUE** – в случае успешного выполнения.
- **FALSE** – в случае возникновения ошибки.

Передача параметров

В Object RSL параметры передаются процедурам по значению. Тем не менее, вызванная процедура может изменить значение фактического параметра в процедуре, ее вызвавшей, — это делается при помощи стандартной процедуры [SetParm](#)⁽²³¹⁾. Что же касается методов объектов автоматизации, то они могут принимать параметры по ссылке. Чтобы передать параметры иным способом, в Object RSL предусмотрены специальные модификаторы (атрибуты):

- **:i** – атрибут используется для передачи параметров из Object RSL по ссылке;
- **:iv** – атрибут используется для передачи параметров из Object RSL как ссылок на вариант, так как некоторые методы ActiveX-объектов могут принимать параметры как ссылки на вариант, благодаря чему эти методы могут менять не только значение параметра, но и его тип;
- **:v** – атрибут используется для передачи параметров по значению в метод ActiveX-объекта. Необходимость в данном атрибуте возникает в Visual RSL, который имеет специальную настройку, согласно которой все параметры передаются методам ActiveX-объектов по умолчанию по ссылке.

Пример.

/* Пусть метод объекта автоматизации представляет процедуру на языке Visual Basic: */

```
Sub BasicProc (ByRef par1 As String, ByRef par2 As Variant)
    Par1 = "New string";
    Par2 = 10;
End sub
```

/* При вызове этого метода необходимо передать параметры по ссылке: переменная Val1 передается как ссылка на строку, а переменная Val2 - как ссылка на вариант: */

```
Val1 = "Old string 1";
Val2 = "Old string 2";
Ob.BasicProc (Val1:i, Val2:iv)
```

Рассмотрим порядок передачи методам объектов автоматизации в качестве параметров Object RSL объектов типа **TArray**. По умолчанию объекты типа **TArray** передаются так же, как и любые другие объекты классов Object RSL. Так, в программе на языке Visual Basic массив **TArray** будет выглядеть как объект, у которого есть свойства **size** и **value**. Однако стандарт автоматизации предусматривает для массивов свой тип данных **SAFEARRAY**. Поэтому при необходимости можно сделать так, чтобы при передаче из **Object RSL** объекта типа **TArray** этот объект автоматически конвертировался в тип данных **SAFEARRAY**. Для этого нужно будет присвоить свойству **CvtToSafeArray** объекта **TArray** значение TRUE.

Пример.

```
Ar = Tarray;
Ar.value (0) = "value for array";
Ar. CvtToSafeArray = true;
```

При передаче в качестве параметра методам объектов автоматизации массив **Ar** будет автоматически преобразовываться в тип **SAFEARRAY**.

Поддержка модулей платформы .NET

В языке RSL реализована поддержка модулей платформы .NET.

Для создания объектов CLR-классов служит процедура **ClrObj**.

ClrObj (asName:string, className:string) : object;

Процедура создаёт объект класса **className** в сборке с именем **asName**. Закрытые сборки ищутся в каталоге запуска приложения и в подкаталоге с именем CLR.

Доступ к объектам, созданным с помощью этой процедуры, будет только локальным.

Если необходимо создавать или запускать CLR-объект не только локально, но и удаленно, следует воспользоваться [RSCOM-технологией](#)¹⁶⁹ вместо встроенной процедуры **ClrObj** использовать стандартный RSCOM-сервер [Rsclr](#)¹⁸⁵.

Объекты, созданные процедурой **ClrObj**, можно использовать так же, как и ActiveX-объекты, созданные процедурой **ActiveX**. Например, можно таким же образом обрабатывать события от CLR-объектов.

Пример.

```
ob = ClrObj ("ClassLib1", "ClassLib1.Class1");
RslEventHandler.EvSource ("Test") = ob;
```

```

ob.Prop1 = "Data for Prop1";
println (ob.Prop1);
v = 10;
println (ob.Method1 (1,@v));
println (v);
ob.Prop1 = "Data for Prop1";
RslEvHandler.EvSource ("Test") = NULL;
macro Test_TestEvent (src, prm)
    println ("Test Event");
end;
macro Test_TestEvent2 (src, prm)
    println ("Test Event2");
end;

// В примере используется сборка с именем ClassLib1.
Исходный текст на/ языке C#: //
using System;
using System.Runtime.InteropServices;
using System.Reflection;
[assembly: AssemblyVersion("1.0.*")]
namespace ClassLib1
{
    [ClassInterfaceAttribute(ClassInterfaceType.AutoDual)]
    public class TestEventArgs : EventArgs
    {
        private readonly string data;
        public TestEventArgs (string parm)
        { this.data = parm; }
        public string EventText
        {
            get { return data;}
        }
    }

    public delegate void TestEventHandler(object sender,
TestEventArgs e);
    [GuidAttribute("3444502B-0E87-414b-8279-D20FB40FA087")]
    [InterfaceTypeAttribute(ComInterfaceType.InterfaceIsIDispatch)]
    public interface Class1Events
    {
        void TestEvent(object sender, TestEventArgs e);
        void TestEvent2(object sender, TestEventArgs e);
    }

    [GuidAttribute("F94F09D3-EDE7-49e1-8B91-A22689153599")]
    public interface Class1Intf
    {
        int Method1(int p1, ref short p2);
        string Prop1 { get; set; }
    }

    [ClassInterfaceAttribute(ClassInterfaceType.None)]
    [ComSourceInterfacesAttribute("ClassLib1.Class1Events,ClassLib1")]
    public class Class1 : Class1Intf
    {
        public event TestEventHandler TestEvent;
        public event TestEventHandler TestEvent2;
        protected virtual void OnTestEvent (TestEventArgs e)
        {
            if (TestEvent != null)
            {
                TestEvent (this, e);
            }
        }
    }
}

```

```

    }
    protected virtual void OnTestEvent2 (TestEventArgs e)
    {
        if (TestEvent2 != null)
            {TestEvent2 (this, e);}
    }
    public Class1()
    {
    }
    public int Method1(int p1, ref short p2)
    {
        TestEventArgs e = new TestEventArgs ("This is a
data for TestEvent");
        OnTestEvent(e);
        short v = p2;
        p2 = 100;
        return p1 + v;
    }
    public string Prop1
    {
        get { return fld1; }
        set
        {
            fld1 = value;
            TestEventArgs e = new TestEventArgs ("This is a
data for TestEvent2");
            OnTestEvent2(e);
        }
    }
    private string fld1;
}
}

```

Обработка ошибок, возникающих во время выполнения программы

В Object RSL можно осуществить "перехват" любой ошибки, возникающей во время выполнения, не допустив аварийного завершения RSL-программы. Для этого любая макропроцедура RSL или метод класса может иметь обработчик ошибок. Записывается это следующим образом:

Macro Test

<инструкции RSL>

OnError

<инструкции для обработки ошибок>

End;

Если какая-либо инструкция в теле макропроцедуры **Test** или инструкция в процедурах, вызванных из **Test**, генерирует ошибку, то управление передается на первую инструкцию для обработки ошибок после ключевого слова **OnError**, и аварийного завершения RSL-программы не происходит.

Если обработчику ошибок нужна информация о произошедшей ошибке, после ключевого слова **OnError** в скобках необходимо указать имя переменной (эта переменная может не декларироваться в теле макропроцедуры при помощи ключевого слова VAR), которая после

возникновения сбоя получит ссылку на специальный объект класса **TrslError**, содержащий информацию о данной ошибке.

Объект класса **TrslError** имеет следующие свойства:

- **Code** – код ошибки.
- **Message** – строка, описывающая ошибку.
- **Module** – название модуля RSL, вызвавшего ошибку.
- **Line** – строка модуля, в которой произошла ошибка.
- **AxCode** – код ошибки ActiveX-объекта.
- **AxMes** – строка с информацией об ошибке ActiveX-объекта.

Пример.

```
Macro Test2
<инструкции RSL>
OnError ( er )
    MsgBox (er.Message,"| Модуль: ",er.Module,"| строка: ",er.Line)
End;
```

Пример.

```
macro Demo
    ob = ActiveX ("Test.TestClass");
onError (er)
    println (er.message);
    if (er.AxCode)
        println (er.AxMes);
    end
end /* Demo */
```

В обработчике **OnError** можно получить информацию об ошибке пользователя при помощи обращения к полю **err**.

Пример.

```
onError (erObj)
    if (IsEqClass ("MyError", erObj.err))
        println ("Пользовательская ошибка с кодом ",
erObj.err.erCode);
    else
        println (erObj.message);
```

Чтобы в обработчике заново сгенерировать ту же самую ошибку для передачи ее другому обработчику в цепочке вызовов макропроцедур или обработчику RSL, необходимо вызвать стандартную RSL-процедуру **RunError** без параметров.

В макромодуле, как и в макропроцедуре, можно реализовать обработчик ошибок. Ключевое слово **end** при этом не обязательно.

Пример.

```
Import rslx;
Macro Test
```

```
end;  
OnError (er)  
    MsgBox (er.Message)  
End      /* End может отсутствовать */
```

Передача управления обработчику **OnError** происходит также при прерывании пользователем RSL-программы по **[Ctrl+Break]** и при завершении программы процедурой **Exit**²⁹⁶. В этом случае в обработчик передается специальный код ошибки: при нажатии **[Ctrl+Break]** равный 17, при вызове процедуры **Exit** равный 0. При этом в самой программе ошибка при выполнении программы не генерируется.

Автоматическое создание объектов Object RSL

Объекты классов создаются автоматически при первом обращении к декларированной переменной типа класса Object RSL.

Пример.

```
Class TestClass  
    Var prop = 10;  
End;  
Var Ob:TestClass; /* Декларируем переменную типа TestClass */  
Println (Ob.prop); /* Объект автоматически создается */
```

Конструкция WITH

Конструкция WITH применяется для обращения к конкретным экземплярам класса.

Синтаксическая форма конструкции WITH выглядит следующим образом:

```
WITH (идентификатор)  
  
    <список инструкций>  
  
END
```

Идентификатор – это имя переменной, содержащей ссылку на объект RSL-класса. Список инструкций выполняется для указанного объекта.

Пример.

```
With ( this )  
Println ("-----");  
Отчет;  
Println ("-----");  
End;
```

В приведенном примере для объекта **this** выполняется метод **Отчет** и печатаются две пунктирные линии.

Организация ввода/вывода

Ввод данных в RSL возможен:

- интерактивным способом – ввод данных с клавиатуры, использование [диалоговых окон](#) ⁽⁹¹⁾, [скроллингов](#) ⁽⁹⁹⁾, [вертикальных](#) ⁽⁸⁹⁾ и [горизонтальных меню](#) ⁽⁹⁰⁾;
- из таблиц базы данных, текстовых и файлов формата DBF.

Вывод осуществляется:

- в таблицы баз данных;
- в файлы – текстовые и DBF-файлы;
- на стандартное устройство – файл, в который выводят процедуры [Print](#) ⁽²¹¹⁾, [PrintLn](#) ⁽²¹²⁾ и [инструкция вывода](#) ⁽⁵¹⁾; имя этого файла определяется программой, вызывающей RSL-программу. При выводе на стандартное устройство используются [спецификаторы форматирования](#) ⁽⁸³⁾.

Язык Object RSL позволяет создавать отчеты с использованием шаблонов. Для этого предусмотрены стандартные классы [TRepForm](#) ⁽⁸⁶⁾ и [TPattFieldR](#) ⁽⁸⁹⁾.

Спецификаторы форматирования

Спецификаторы форматирования определяют вид, в котором данные выводятся процедурами [Print](#) ⁽²¹¹⁾, [PrintLn](#) ⁽²¹²⁾ и [инструкцией вывода](#) ⁽⁵¹⁾. Эти спецификаторы позволяют форматировать как строку в памяти (при помощи процедуры [String](#) ⁽²¹⁹⁾), так и данные, отображаемые процедурой [Message](#) ⁽²¹⁴⁾.

Список спецификаторов форматирования указывается сразу же после выражения, к которому он относится. Он начинается с двоеточия и не должен иметь пропусков. Спецификаторы в этом списке отделяются друг от друга двоеточиями.

Список спецификаторов форматирования включает в себя:

- число, указывающее ширину поля вывода. Если значение выражения будет меньше заданной ширины, то оно будет дополнено пробелами. Если ширина поля вывода не указана данным спецификатором или специальными символами управляющей строки, а также если задана ширина, равная нулю, значение выражения выводится полностью;
- число, задающее количество цифр после десятичной точки для данных типа **Double**. Если это значение равно нулю, дробная часть не выводится;
- один или несколько символов форматирования, которые могут задаваться в любом порядке.

При форматировании данных используются следующие символы форматирования:

- **"l"** – выравнивание по левому краю поля вывода; такое выравнивание по умолчанию выполняется при выводе символьных строк;

- "r" – выравнивание по правому краю поля вывода; такое выравнивание по умолчанию выполняется при выводе чисел;
- "c" – выравнивание по центру поля вывода;
- "a" – при выводе чисел применение апострофов для отделения тысяч и так далее;
- "e" – выводит значения переменных, содержащих пустые значения, в виде пустой строки. По умолчанию для таких переменных выводится слово **Undefined**.
- "m" – для типа **Money** вывод значения прописью; для типа **Date** вывод значения с названием месяца словом; для типа **Time** вывод сотых долей секунды; для остальных типов данных этот спецификатор игнорируется;
- "z" – не выводит нулевые значения переменных типа **Integer**, **Money** и **Double**, а также значений типа **V_UNDEF**; спецификатор игнорируется, если не определена ширина поля вывода;
- "f" – форматирует строку по шаблону, установленному в прикладной системе и соответствующему типу форматируемого параметра. С его помощью, например, форматируются поля типа **SNR**, используемые для хранения и редактирования номеров лицевых счетов, а также денежные суммы, которые форматируются в "финансовом" формате. Даты с этим спецификатором выводятся с ведущим нулем.
- "o" – при использовании с числовыми полями заполняет поле вывода слева нулями, а не пробелами.
- "s" – позволяет передать в методы автоматизации COM ссылку на тип **Short**. Так как язык RSL не поддерживает типа данных **Short**, по умолчанию передаётся ссылка на **Integer**.

Примечание.

Для передачи ссылки на **variant** применяется модификатор "v".

Пример.

```
Var testVal = 10;
Obj.TestMethod (@TestVal:s); // Передача ссылки на short
Obj.TestMethod (@TestVal:v); // Передача ссылки на variant
```

- "x" – используется при вводе целых чисел в шестнадцатеричном формате.

Для управления выводом значений, длина которых превышает длину поля вывода, дополнительно используются следующие символы форматирования:

- "t" – обрезать значение по ширине поля вывода; это действие по умолчанию выполняется при выводе слишком длинных символьных строк;
- "d" – вывести в поле вывода символы "***"; так выводятся по умолчанию слишком длинные числа;
- "w" – вывести продолжение на следующей строке или строках; это действие выполняется только для слишком длинных значений строкового типа. При этом формат строк продолжения дублируется, строки повторяются до тех пор, пока самое длинное из значений не будет выведено полностью.

Примечание.

Применение спецификатора "w" к целочисленному выражению приводит к выводу числа в шестнадцатеричном формате.

- `"*"` – позволяет задать ширину поля вывода и количество цифр после десятичной точки в виде переменных. Указанный символ используется вместо константного значения спецификатора, при этом значение спецификатора следует указать следующим параметром в списке параметров.

Пример 1.

```
print (a + b:10:5:c)
// В этом примере для значения выражения a+b
указаны //следующие спецификаторы форматирования: ширина
поля //вывода, равная 10 символам, вывод 5 знаков после
запятой, //печатать значения переменной по центру поля
вывода.
```

Пример 2.

```
start = time;
i = 1000000;
while (i)
    i = i - 1;
end;
println ("Время выполнения: ", time - start : m)
// Вывод:
Время выполнения: 0:00:06.87
println(date:f); // 10го сентября 2005 года будет выведено:
01.09.2005
```

Пример 3.

```
val = 134;
println (val:x);
```

Пример 4.

```
Инструкции с константными спецификаторами:
println (123.123456789:20:5);
[#] ("Это длинная строка для вывода на нескольких
строках":w:6);

могут быть записаны так:
println (123.123456789:*:*, 20, 5);
[#] ("Это длинная строка для вывода на нескольких
строках":w:*, 6);
```

Формирование отчетов с использованием шаблонов

При подготовке отчетов на языке Object RSL используются стандартные классы:

- [TRepForm](#)⁸⁶.
- [TPattFieldR](#)⁸⁹.

Стандартный класс TRepForm

Класс **TRepForm** предназначен для формирования отчетов на языке Object RSL с использованием шаблонов, подготовленных в отдельных текстовых файлах. Для использования этого класса в макропрограмму необходимо импортировать DLM-модуль *prnfrm.d32* при помощи директивы **Import**:

```
Import prnfrm;
```

Файл шаблонов представляет собой текстовый файл, содержащий одну или несколько форм отчета. По умолчанию для этого файла используется расширение **.tpl*. Файл ищется во всех каталогах, указанных для поиска макрофайлов.

Любая форма, входящая в файл шаблонов, имеют следующий вид:

```
<FORM:FormName>
```

Набор полей формы

```
</FORM>
```

Здесь теги *<FORM:FormName>* и *</FORM>* отделяют одну форму от другой в файле шаблона и задают, соответственно, начало и конец формы, а *FormName* означает имя формы. Если в файле шаблона присутствует только одна форма, то теги *<FORM:FormName>* и *</FORM>* в файл включать не нужно.

Поле является последовательность символов '#'. Непосредственно перед ним можно задать атрибуты в следующем формате:

```
<!--fieldName:format:pN-->#####
```

Атрибуты поля необязательны. К ним относятся:

- **fieldName** – имя поля; если задан этот атрибут, то поле является именованным;
- **format** – один из следующих атрибутов форматирования содержимого поля:
 - **c** – выравнивание по центру поля;
 - **l** – выравнивание по левому краю поля;
 - **r** – выравнивание по правому краю поля.
- **pN** – атрибут, связывающий с полем некоторое число N. Данный атрибут предназначен для вывода данных во внешние источники, например, если данные выводятся в MS Excel, то этот атрибут задает номер колонки таблицы.

Пример.

```
/* Создадим файл шаблона test.tpl, в котором содержатся две
формы TEST1 и TEST2:*/
<FORM:TEST1>
    Test Report
    -----
    | --<!--id:r:p1-->#####--| --<!--Name:c:p3--
    >#####--|
    -----
```

```

</FORM>
<FORM:TEST2>

<!--Signature1:r:p1-->#####
<!--Signature2:c:p3-->#####
</FORM>

```

Для удобства редактирования формы с поименованными полями встроенный редактор режим скрытия имен полей, который вызывается с помощью клавиши [F12].

В процессе формирования отчета необходимые значения в поля формы помещаются при помощи класса **TRepForm**.

Конструктор класса **TRepForm** выглядит следующим образом:

TRepForm (fileName:String [, useUnnamed:Bool, formName:String, delSpace:Bool])

Конструктор загружает форму с именем **formName** из файла с шаблонами форм с именем **fileName**. Имя формы следует указывать с учетом регистра. Если файл шаблона содержит только одну форму, то параметр **formName** не задается.

Если второй необязательный параметр **useUnnamed** задан и равен TRUE, то в качестве полей в форме могут быть использованы наименованные поля. В противном случае наименованные поля игнорируются. По умолчанию этот параметр принимает значение FALSE.

Если параметр **delSpace** задан и равен TRUE, то из текста шаблона удаляются все лишние пробельные символы (пробелы, табуляции, переводы каретки).

Класс **TRepForm** имеет следующие свойства:

- **Value.**

Свойство, позволяющее читать и писать в поле шаблона с именем **name** или индексом **id**. Свойство используется следующим образом:

```
Value ( name : String | id : Integer )
```

- **Index.**

Свойство возвращает индекс поля с именем **name**. Если поля с указанным именем не существует, возвращает -1. Свойство используется следующим образом:

```
Index ( name : String )
```

- **Field.**

Свойство возвращает ссылку на объект класса [TPattFieldR](#)⁽⁸⁹⁾, содержащий информацию о поле, которое задается именем **name** или индексом **id**. Свойство используется следующим образом:

```
Field ( name : String | id : Integer )
```

Класс **TRepForm** также имеет метод **Write**, который используется следующим образом:
Write ([outObj : object])

Если параметр **outObj** не задан, то метод выводит шаблон формы с заполненными полями в стандартный выходной поток. Если требуется, чтобы формы выводились каким-либо иным способом, то следует создать класс выходного объекта и передать объект этого класса в качестве параметра **outObj** методу **Write**. Объект **outObj**, в свою очередь, должен иметь два метода с предопределенными именами:

- **newLine** – метод предназначен для отделения одной формы от другой;
- **writeFields** – метод предназначен для вывода полей формы. Метод используется следующим образом: **writeFields (name:String, col:Integer, data:Variant, flags:Integer, w:Integer, p:Integer)**

Параметры метода **writeFields**:

- **name** – имя поля;
- **col** – число, заданное в поле с помощью атрибута **p**;
- **data** – данные в поле;
- **flags** – флажки;
- **w** – ширина поля;
- **p** – число знаков после точки.

Пример.

```
/* Создадим файл шаблона test.tpl следующего содержания: */
<FORM:TEST1>
    Test Report
    -----
    |--<!--id:r:p1-->#####--|--<!--Name:c:p3--
    >#####--|
    -----
</FORM>
/* Пример RSL-программы, использующей формирование отчета
при помощи файла шаблона test.tpl. */
import prnfrm;
class TOutObj
    macro newLine
        println;
    end;
    macro writeField (name:string, col:integer, data:variant,
        flags:integer, w:integer, p:integer)
        println ("name = ",name," col = ",col," data = ", data)
    end
end;

var ob = TRepForm ("test",false,"TEST1");
var out = TOutObj;

ob.value("id") = "123";
ob.value("name") = "Name1";

ob.Write;
ob.Write(out);
ob.Write(out);
```


Стандартный класс **TPattFieldR**

Экземпляры класса **TPattFieldR** создаются автоматически для каждого поля, заданного в файле шаблоне. Каждый объект содержит информацию для одного поля. Доступ к экземплярам класса осуществляется посредством свойства **Field** класса **TRepForm**.

Класс **TPattFieldR** обладает следующими свойствами:

- **Value** – свойство позволяет читать и писать значение в связанное с объектом поле.
- **Name** – свойство содержит имя поля и доступно только для чтения.
- **Attr** – свойство позволяет читать и устанавливать новые значения для атрибутов данного поля. Возможны следующие значения данного свойства:
 - **1** – выравнивание по левому краю;
 - **2** – выравнивание по правому краю;
 - **3** – выравнивание по центру.

Поддержка интерактивного режима

В состав языка RSL входят процедуры, позволяющие организовать интерактивный режим работы, значительно облегчающий процесс общения с компьютером.

Пользователь имеет возможность создавать [меню](#)⁽⁸⁹⁾, предназначенные для выбора конкретного варианта из заранее заданного списка, производить запрос на ввод данных любого типа.

С помощью языка RSL пользователь также имеет возможность организовывать [скроллинг по файлам](#)⁽⁹⁹⁾.

Кроме этого, для ввода и корректировки данных можно разработать специальные [диалоговые окна](#)⁽⁹¹⁾.

Меню

Если разрабатываемая программа имеет несколько вариантов выполнения, то самый простой способ организовать ветвление – дать пользователю возможность выбора варианта из меню.

В Object RSL предусмотрена возможность создавать как [вертикальное](#)⁽⁸⁹⁾, так и [горизонтальное](#)⁽⁹⁰⁾ меню.

Вертикальное меню

Для создания вертикального меню необходимо в программе создать массив строк с вариантами выбора и вызвать процедуру [Menu](#)⁽²³⁵⁾.

Пример.

```
array m;
m(0) = "Первый вариант решения";
m(1) = "Другой вариант";
m(2) = "Еще один вариант";
вариант = Menu (m, "Какой вариант желаете выбрать?");
if (вариант == 0)
    println ("Выбран первый вариант");
elif (вариант == 1)
    println ("Выбран другой вариант");
elif (вариант == 2)
    println ("Выбран еще один вариант");
end;
```

Процедура **Menu** возвращает индекс выбранного элемента меню.

Горизонтальное меню

Горизонтальное меню создается и сохраняется в специальной библиотеке. Для создания, редактирования и тестирования горизонтальных меню предусмотрены специальные команды в режиме редактирования ресурсов.

Описание горизонтального меню начинается с ключевого слова **MENU**, после которого указывается имя меню и комментарий. Далее в блоках **POPUP-END** необходимо описать ниспадающее меню. Ключевое слово **END** заканчивает описание.

Имя и комментарий для ниспадающего меню указываются после ключевого слова **POPUP**. Далее внутри блока **POPUP-END** следует описать пункты меню **ITEM** и разделители пунктов **DELIM**. Для каждого пункта необходимо задать:

- имя – название пункта меню на экране;
- целочисленный идентификатор – код, используемый для обработки выбранного пункта меню;

Примечание.

Целочисленный идентификатор не должен быть равен 256, так как данный код используется для обработки нажатия клавиши [Esc].

- комментарий – текст, отображаемый в статус-строке при выборе пункта меню.

Пример.

```
MENU Демо, "Демонстрационное меню"
    POPUP "~Команды", "Команды демонстрационного меню"
        ITEM "~Печать", 102, "Печать отчета"
        DELIM
        ITEM "~Выход", 101, "Окончание работы"
    END
END
```

Для вызова горизонтального меню из RSL-программы используется процедура [**RunMenu**](#)²³⁶.

Диалоговые окна

Если необходимо предоставить пользователю возможность просматривать и корректировать сложную совокупность данных, то лучше всего для этого воспользоваться диалоговыми окнами.

Форма диалогового окна разрабатывается в специальном редакторе диалоговых окон и сохраняется в файле-библиотеке с расширением .LBR.

Для языка RSL диалоговая форма выглядит как структура. Чтение значений из полей этой структуры позволяет получать значения, введенные пользователем в поля диалоговой панели. И наоборот, запись в поля структуры новых значений позволяет определить содержимое полей диалоговой панели.

Чтобы в программе можно было работать с диалоговым окном, необходимо определить идентификатор диалога:

Record dlg (demo) dialog;

Идентификатор **dlg** связан с объектом типа RECORD, имеющим название **demo** и задающим структуру диалогового окна из библиотеки диалогов. Теперь, при помощи процедуры [RunDialog](#)²⁴⁰ можно начать работу с диалогом:

RunDialog (dlg)

Имеется возможность одновременной работы с диалоговыми панелями, хранящимися в разных файлах библиотек ресурсов. Для этого при определении диалога указывается путь к альтернативному файлу библиотеки диалогов.

Пример.

```
Record dlg (demo, "mydir\mylib.lbr") dialog;
```

Идентификатор диалога может также являться объектом класса **TRecHandler** или его наследником. Чтобы создать такой идентификатор, необходимо создать экземпляр класса **TRecHandler**, в качестве первого параметра которого передано имя диалоговой панели из библиотеки диалогов, в качестве второго – имя LBR-файла с диалоговыми панелями, а в качестве третьего – значение TRUE. Если второй параметр не задан, то используется имя библиотеки по умолчанию. Далее следует задать начальные значения полям диалогового окна. Все сопутствующие процедуры (**RunDialog**, **UpdateFields** и т.п.) при этом могут принимать ссылку на объект типа **TRecHandler** или его наследника.

Пример.

```
dlg = TRecHandler ( "Test", NULL, TRUE );
dlg.rec.name1 = val1;
dlg.rec.name2 = val2;
...
RunDialog ( dlg );
```

При работе с диалоговым окном определены клавиши перемещения курсора, а также следующие функциональные клавиши:

[Esc] – завершение работы с диалоговым окном без сохранения введенных пользователем значений.

Примечание.

Для того чтобы отменить закрытие диалогового окна при нажатии клавиши **[Esc]**, необходимо выполнить обработку сообщения **DLG_KEY**. Процедура

обработки сообщений для кода указанной клавиши возвращает значение **CM_IGNORE**.

[F9] – завершение работы с диалоговым окном и сохранение введенных значений.

Примечание.

Для того чтобы отменить закрытие диалогового окна при нажатии клавиши **[F9]**, необходимо выполнить обработку сообщения **DLG_KEY**. Процедура обработки сообщений для кода указанной клавиши возвращает значение **CM_IGNORE**.

[Enter] – завершение редактирования текущего поля и перевод фокуса ввода в следующее поле. Если эта клавиша нажимается в последнем поле редактирования, то это эквивалентно нажатию на **[F9]**.

Если необходимо специальным образом обрабатывать нажатия каких-либо других клавиш в диалоговой панели, проверять корректность введенной информации и так далее, для разрешения подобных задач пользователь может написать специальную [процедуру обработки сообщений](#) ⁽⁹²⁾.

Процедура обработки сообщений

Процедура обработки сообщений должна вызываться после того, как пользователь произвел какие-либо действия с диалоговым окном. Обработывая эти сообщения, можно влиять на функционирование диалогового окна.

Следует отметить, что процедура [RunDialog](#) ⁽²⁴⁰⁾ генерирует сообщения только в том случае, если пользователем определена специальная процедура для их обработки.

Имя процедуры может быть произвольным, например, **EvProc**. Ее имя или ссылку на нее необходимо указать вторым параметром при вызове **RunDialog**.

```
RunDialog (dlg, "EvProc")
```

```
RunDialog (dlg,@EvProc)
```

Эта процедура должна быть определена где-либо в тексте программы следующим образом:

macro EvProc (obj, cmd, id, key)

Параметры этой процедуры имеют следующие значения:

obj – источник данных (тип **Object**). В качестве параметра может быть указан:

- для процедуры обработки сообщений [окна прокрутки](#) – источник, переданный в процедуру [RunScroll](#) ⁽²⁴⁸⁾ или [AddScroll](#) ⁽²³⁷⁾;
- для процедуры обработки сообщений [диалоговой панели](#) – объект класса [TrecHandler](#) ⁽¹¹⁶⁾ или производный от него.

cmd – [код сообщения](#) ⁽⁹³⁾ (каждый код имеет предопределенную константу); тип **Integer**.

id – номер текущего поля в диалоговой панели или номер колонки в области прокрутки (тип **Integer**).

key – код нажатой клавиши (тип **Integer**) появляется в том случае, если код сообщения **cmd** равен **DLG_KEY** или **DLG_MOUSE**.

Значение, возвращаемое процедурой, зависит от типа сообщения. Если процедура не обрабатывает сообщение, она возвращает

CM_DEFAULT или не возвращает ничего, что равносильно значению **CM_DEFAULT**.

Список обрабатываемых сообщений

Выделяют следующие списки сообщений:

- Список обрабатываемых сообщений процедуры [RunDialog](#)⁹³.
- Список обрабатываемых сообщений процедуры [RunScroll](#)⁹⁴.

Список обрабатываемых сообщений процедуры RunDialog

DLG_PREINIT – это первое посылаемое сообщение. Указанное сообщение посылается сразу после создания диалоговой панели. При его обработке процедурой [AddScroll](#)²³⁷ создается область прокрутки данных.

DLG_INREC – это сообщение посылается при установке фокуса ввода в текущее поле.

DLG_KEY – это сообщение посылается в том случае, если пользователь нажал клавишу на клавиатуре.

DLG_OUTREC – это сообщение посылается при переносе фокуса из текущего поля. Если необходимо запретить перенос фокуса, из обработчика возвращается значение **CM_CANCEL**.

DLG_TIMER – это сообщение посылается по истечении интервала времени, установленного при последнем вызове процедуры [SetTimer](#)²⁴².

DLG_DESTROY – это сообщение посылается непосредственно перед удалением диалогового окна с экрана. Указанное сообщение не посылается в процедуру обработки сообщений скроллинга, встроенного в диалоговую панель с помощью процедуры **AddScroll**. В параметре **key** процедуры обработки сообщений передается статус завершения диалогового окна:

- **0** – закрытие диалогового окна без сохранения изменений; по умолчанию соответствует нажатию клавиши **[Esc]**. Процедуры [RunDialog](#)²⁴⁰ и [RunScroll](#)²⁴⁸ возвращают значение **FALSE**.
- **1** – закрытие диалогового окна с сохранением изменений или выбор требуемой записи из окна прокрутки. Для диалогового окна по умолчанию соответствует нажатию клавиши **[F9]** или возвращаемому значению **CM_SAVE**. Для скроллинга, запущенного с помощью процедуры **RunScroll**, соответствует значению **CM_SELECT**, возвращаемому при обработке сообщения **DLG_KEY**. Процедуры **RunDialog** и **RunScroll** возвращают значение **TRUE**.

DLG_MOUSE – это сообщение посылается, если пользователь использует в своей работе мышь. Процедура обработки сообщений получит следующие значения параметра **key**:

- **0** – нажатие левой кнопки мыши;
- **2** – отпускание левой кнопки мыши;

- 7 – двойной щелчок кнопкой мыши.

DLG_SWITCH – это сообщение посылается один раз в пользовательский обработчик диалоговой панели, созданной с помощью процедуры [RunDialog](#)²⁴⁰, при переключении между панелью и скроллингом. Если процедура **EvProc** указана в качестве значения параметра **proc** в процедуре [RunDialog](#)²⁴⁰ при обработке сообщения **DLG_SWITCH**, то она может вернуть значение **CM_IGNORE** и, тем самым, отменить переключение.

Указанные ниже сообщения посылаются в процедуру обработки после нажатия клавиши, зарегистрированной вызовом процедуры [AddMultiAction](#)²⁴⁶.

DLG_MSELSTART – это сообщение посылается один раз в начале обработки выбранных записей. При его обработке процедурой [GetMultiCount](#)²⁴⁷ возвращается количество выделенных записей.

DLG_MSEL – это сообщение посылается для каждой выделенной записи. Обрабатываемая запись становится текущей в объекте [RsdRecordSet](#)¹³³, переданном в процедурах **RunScroll** или **AddScroll**.

DLG_MSELEND – это сообщение посылается один раз в конце обработки выделенных записей. При его обработке возможно удаление индикатора выполнения.

[Возвращаемое значение](#)⁹⁵ процедуры зависит от типа обрабатываемого сообщения.

Список обрабатываемых сообщений процедуры **RunScroll**

DLG_INIT – это сообщение посылается после отправки **DLG_PREINIT**, непосредственно перед отображением диалога на экране. Указанное сообщение не посылается в процедуру обработки сообщений скроллинга, встроенного в диалоговую панель с помощью процедуры **AddScroll**. При его обработке можно произвести необходимую инициализацию и установить фокус ввода в нужное поле. Последняя операция выполнится с помощью процедуры [SetFocus](#)²⁴².

DLG_INLOOP – это сообщение используется для отслеживания переключения циклов выборки сообщений и посылается при входе диалоговой панели и области прокрутки в цикл выборки сообщений. Указанное отслеживание необходимо для диалоговых панелей со встроенной областью прокрутки. Такие панели состоят из композиции панелей, поэтому для них используется два разных цикла выборки сообщений: цикл выборки сообщений панели и области прокрутки. При переходе фокуса ввода между полями диалоговой панели и встроенной областью прокрутки происходит переключение используемых циклов выборки сообщений и используемых процедур обработки сообщений.

DLG_OUTLOOP – это сообщение используется для отслеживания переключения циклов выборки сообщений и посылается при выходе диалоговой панели и области прокрутки из цикла выборки сообщений. См. описание значения [DLG_INLOOP](#)⁹⁴.

DLG_BUTTON – это сообщение посылается, если пользователь щелкнул кнопкой мыши на экранной кнопке.

DLG_REMFOCUS – это сообщение посылается перед переносом фокуса из поля, указанного в обработчике.

DLG_SETFOCUS – это сообщение посылается непосредственно перед установкой фокуса ввода в текущее поле.

DLG_SAVE – это сообщение посылается перед запросом на выход из диалога с сохранением внесенных изменений. Указанное действие происходит в случае нажатия клавиши **[F9]** или в случае, когда обработчик сообщений **DLG_KEY**, **DLG_TIMER** или **DLG_BUTTON** вернул значение **CM_SAVE**.

DLG_SWITCH – это сообщение посылается один раз в пользовательский обработчик скроллинга, созданного с помощью процедуры [RunScroll](#)²⁴⁸, при переключении между скроллингом и панелью. Если процедура **EvProc** указана в качестве значения параметра **proc** в процедуре **RunScroll** при обработке сообщения **DLG_SWITCH**, то она может вернуть значение **CM_IGNORE** и, тем самым, отменить переключение.

Значение, возвращаемое процедурой обработки сообщений

Значение, возвращаемое процедурой **EvProc**, зависит от того, какое из сообщений обрабатывается, и влияет на дальнейшую работу с диалоговым окном.

При обработке сообщения **DLG_PREINIT** процедура **EvProc** может вернуть значение **CM_UPDATE_ADDSCROLL** – обновить скроллинг, добавленный с помощью процедуры [AddScroll](#)²³⁷. В этом случае автоматически закрываются все ресурсы, созданные процедурой **AddScroll**, затем заново вызывается обработчик диалоговых окон с событием **DLG_PREINIT**, и выполняется процедура **AddScroll**. Следует иметь в виду, что при выводе найденного значения необходимо закрыть и создать заново объект класса **RecordSet**. Обновление скроллинга выполняется при нажатии указанной клавиши.

Пример.

Примерный вид RSL-кода (обработчик для RunDialog):

```
var rs;
...
/*Обработка нажатия клавиш в диалоговом окне*/
macro MsgProc (dlgPanel, cmd, id, key)
...
var CM_FLAG = CM_DEFAULT;
/*возвращаемое значение по умолчанию*/

    if (cmd == DLG_PREINIT)
        ...
        rs = RsdRecordSet(RsdCommand(str), RSDVAL_CLIENT,
RSDVAL_STATIC );
        ...
        AddScroll ( ..., rs, ... );
    end;

    if( key == K_F5)
        rs.Close(); //закрываем рекордсет
        rs = NULL; //на всякий случай, разрушаем объект
        CM_FLAG = CM_UPDATE_ADDSCROLL; //обновить скроллинг
- полная перезагрузка, с вызовом AddScroll на событии
DLG_PREINIT

    end;

    return CM_FLAG;
end;
```

При обработке сообщения **DLG_INIT** процедурой **EvProc** может возвращаться одно из следующих значений:

- **CM_CANCEL** – завершить работу с диалоговым окном; процедура [RunScroll](#)²⁴⁸ возвращает значение **FALSE**.
- **CM_DEFAULT** – установить фокус ввода на первое поле панели.
- **CM_IGNORE** – запретить автоматическую установку фокуса ввода; в этом случае фокус ввода следует установить в нужное поле с помощью процедуры **SetFocus**.

При обработке сообщения **DLG_KEY** процедура **EvProc** может вернуть одно из следующих значений:

- **CM_CANCEL** – завершить работу с диалогом, не сохраняя введенные данные.
- **CM_IGNORE** – игнорировать нажатие клавиши или экранной кнопки (это выполняется по умолчанию для кнопки).
- **CM_INSERT** – выполнить вставку новой записи в область прокрутки. Указанное значение можно использовать для области прокрутки, встроенной в диалоговую панель, т.к. в этом случае вставка записи в область прокрутки по клавише **[F9]** недоступна – в диалоговой панели эта клавиша используется для выхода из панели с сохранением внесенных изменений.
- **CM_SAVE** – завершить работу с диалогом и сохранить введенные данные.
- **CM_SELECT** – закрытие окна прокрутки, запущенного процедурой **RunScroll**, с выбором текущей записи.

При обработке сообщений **DLG_OUTREC**, **DLG_REMFOCUS**, **DLG_SAVE** может возвращаться значение **CM_CANCEL** – запретить перенос фокуса, а также выйти из диалогового окна (для сообщения **DLG_SAVE**).

При обработке сообщения **DLG_BUTTON** процедура **EvProc** может вернуть одно из следующих значений:

- **CM_CANCEL** – завершить работу с диалоговым окном без сохранения изменений.
- **CM_SAVE** – завершить работу с диалоговым окном и сохранить выполненные изменения.
- **CM_UPDATE_ADDSCROLL** – обновить скроллинг, добавленный с помощью процедуры **AddScroll**.

При обработке сообщения **DLG_TIMER** процедура **EvProc** может вернуть одно из следующих значений:

- **CM_CANCEL** – завершить работу с диалоговым окном без сохранения изменений.
- **CM_SAVE** – завершить работу с диалоговым окном и сохранить выполненные изменения.

При обработке сообщения **DLG_MSELSTART** процедура **EvProc** возвращает одно из следующих значений:

- **CM_MSEL_STOP_KEEP** – прервать обработку выделенных записей, сохранив выделение текущей записи.

- **CM_MSEL_STOP_CLEARALL** – прервать обработку выделенных записей и снять выделение со всех записей.

При обработке сообщения **DLG_MSEL** процедура **EvProc** может вернуть одно из следующих значений:

- **CM_DEFAULT** – продолжить обработку выделенных записей, сохранив выделение текущей записи.
- **CM_MSEL_CONT_CLEAR** – продолжить обработку выделенных записей, сняв выделение с текущей записи.
- [CM_MSEL_STOP_KEEP](#) ⁽⁹⁶⁾.
- **CM_MSEL_STOP_CLEAR** – прервать обработку выделенных записей, выделение с текущей записи снять.
- [CM_MSEL_STOP_CLEARALL](#) ⁽⁹⁷⁾.

Если процедура **EvProc** не обрабатывает никакого сообщения, она должна вернуть **CM_DEFAULT**, то есть выполнить действия по умолчанию.

Если при обработке сообщений полям структуры диалога присваивается новое значение, то для того, чтобы измененные значения отобразились на экране, необходимо вызвать стандартную процедуру [UpdateFields](#) ⁽²⁴¹⁾.

Для установки фокуса ввода в нужное поле применяется процедура [SetFocus](#) ⁽²⁴²⁾. Эта процедура посылает сообщение **DLG_REMFOCUS** в процедуру обработки сообщений от диалогового окна.

Пример.

Пример программы для обработки сообщений диалогового окна:

```
macro EvenMacro(dlg, cmd, id, key)
/* Установим фокус ввода на поле Dat */
if (cmd == DLG_INIT)
SetFocus (dlg, "Dat")
return CM_IGNORE;
/* Нажата клавиша F3 */
elif (cmd == DLG_KEY)
if (key == 317)
dlg.Int = 500;
dlg.Lab = "NewLabel";
dlg.Str =
"Это новая строка";
UpdateFields (dlg);
/* Нажата клавиша Enter */
elif ( (key == 13) and
(id == 6 ) )
SetFocus (dlg, 0);
return CM_IGNORE;
end;
/* Нажата экранная кнопка */
elif (cmd == DLG_BUTTON)
/* Выйти с сохранением */
if (trim (fldname (dlg, id)) == "~O~k")
return CM_SAVE;
/* Выйти без сохранения */
elif (trim (fldname (dlg, id)) == "~C~ancel")
return CM_CANCEL;
end;
end;
```

```

/* Проверка правильности ввода данных */
if (cmd == DLG_REMFOCUS)
    if ( (id == 1) and (dlg.Int < 100) )
        MsgBox ("Значение должно быть больше 100");
        return CM_CANCEL;
    end;
elif (cmd==DLG_SETFOCUS)
    message ("Focus installed");
elif (cmd == DLG_SAVE)
    if (dlg.Int < 100)
        SetFocus (dlg, 1);
        MsgBox ("Значение
        должно быть |>100");
        return CM_CANCEL;
    end;
elif (cmd==DLG_DESTROY)
    message ("Dialog is destroyed");
end;
return CM_DEFAULT;
end

```

Пример работы с диалоговым окном в объектно-ориентированном стиле:

```

const K_F3 = 317, K_F4 = 318;
macro MyDlg (dlg,cmd,id ,key)
    return dlg.MsgHandler (cmd,id,key)
end;
class (TRecHandler) TestDialogBase (resName)
    InitTRecHandler (resName,NULL,true);
    macro onF3
        MsgBox ("Default F3 handler");
    end;
    macro onF4
        MsgBox ("Default F4 handler ")
    end;
    macro MsgHandler (cmd,id,key)
        if (cmd == DLG_KEY)
            if (key == K_F3)    onF3;
            elif (key == K_F4)  onF4;
        end;
    end;
    end;
    macro Run
        RunDialog (this,@MyDlg);
    end;
end;
class (TestDialogBase) TestDialog (par)
    InitTestDialogBase ("Test");
    var prop = par;
    rec.f1 = "Test string";
    rec.f2 = "Other Test string";
    macro onF3
        rec.f1 = prop;
        rec.f2 = "22222222222";
        UpdateFields (this,1);
        MsgBox ("New F3 handler");
        prop = "New prop Value"
    end;
    macro MsgHandler (cmd,id,key)
        trace ("Cmd = ", cmd);
        return MsgHandler (cmd,id,key);
    end;
end;

```

```
dlg = TestDialog ("Value for f1 field");  
dlg.Run
```

Скроллинг

В качестве источника данных скроллинга могут выступать следующие объекты:

- объект класса [RsdRecordSet](#)⁽¹³³⁾ из стандартной библиотеки RSD;
- объект класса, производного от инструментального класса [ToolsDataAdapter](#)⁽¹⁰³⁾.

С помощью языка RSL пользователь может организовать просмотр на экране Btrieve-файла с возможностью "прокручивания" его вперед и назад.

Для решения такой задачи служит макромодуль **Rs/Scr**, в котором определены специальные процедуры и переменные, с помощью которых организовывается работа со скроллингом.

Основным компонентом модуля является специальная процедура обработки сообщения от диалоговой панели [ScrollMes](#)⁽²⁴⁶⁾. Эта процедура обрабатывает нажатия клавиш **[PgUp]**, **[PgDown]** и клавиш управления курсором для выполнения прокрутки записей из файла.

Для корректной работы процедуре **ScrollMes** необходима инициализирующая информация:

- файл, по которому производится скроллинг;
- соответствие полей диалоговой панели полям в файле.

Эта информация задается при помощи процедуры **SetScroll**:

SetScroll (FILE, DlgName, FileName, ...)

Параметр **FILE** задает идентификатор файла, по которому будет производиться прокрутка.

DlgName – имя первой колонки в диалоговой панели.

FileName – имя поля в файле **FILE**, данные из которого должны выводиться в колонку с именем **DlgName**. Если имя поля в файле совпадает с именем колонки, можно указать вместо имени **FileName** значение NULL. Пары "**DlgName-FileName**" должны быть указаны для каждой колонки в скроллинге. Процедура **SetScroll** должна вызываться перед вызовом [RunDialog](#)⁽²⁴⁰⁾.

Область прокрутки в диалоговой панели должна быть создана как совокупность колонок. Каждая колонка образуется из набора редактируемых полей. Названия полей в колонке должны быть одинаковыми за исключением окончания. Окончанием в названии поля должен быть номер строки в колонке. Первая строка должна иметь номер 0. Например: **ColName0** – имя первой строки в колонке, **ColName1** – имя второй строки в колонке и т.д.

Под именем колонки подразумевается имя поля без окончания. В данном случае именем колонки будет **ColName**. Именно это имя колонки передается процедуре **SetScroll**.

Таким образом, для создания скроллинга на языке RSL необходимо:

- с помощью редактора ресурсов создать диалоговую панель;

- используя директиву **IMPORT**, импортировать в RSL-программу модуль **RsIScr**;
- непосредственно перед вызовом процедуры **RunDialog** вызвать макропроцедуру **SetScroll** для задания соответствия имен полей в файле и диалоговой панели;
- вызвать процедуру **RunDialog** и задать процедуру обработки сообщений;
- в процедуре обработки сообщений вызывать predeterminedенную процедуру **ScrollMes** для всех необработанных сообщений.

Пример.

Область прокрутки в диалоговой панели состоит из двух колонок с названиями **Blnc** и **NameB**. В этом случае вызов процедуры **SetScroll** может быть таким:

```
SetScroll ( ff,
            "Blnc",      "Balance",
            "NameB",     "Name_Part")
```

После вызова процедуры **SetScroll** запуск на выполнение диалоговой панели с областью прокрутки производится обращением к стандартной процедуре **RunDialog**. При этом если не указывается пользовательская процедура обработки сообщений от диалоговой панели, то необходимо указать имя стандартной процедуры **ScrollMes**. Если создается пользовательская процедура обработки сообщений, то в ней необходимо для всех необработанных сообщений вызывать стандартную процедуру **ScrollMes**.

Стандартная процедура **ScrollMes** обрабатывает нажатие на клавишу **[Enter]**. При нажатии на данную клавишу диалоговая панель закрывается, выбранная запись в области прокрутки делается текущей записью в файле и процедура **RunDialog** возвращает **TRUE**.

Пользователю необходимо быть внимательным в том случае, когда необходимо отобразить на экране диалог с областью прокрутки из процедуры обработки сообщений от диалоговой панели, так же содержащей область прокрутки. Так как макромодуль **RsIScr** содержит ряд глобальных переменных, то рекурсивный вызов может уничтожить данные для текущей диалоговой панели. Чтобы избежать этого, нужно сохранять значения глобальных переменных.

Пример.

```
/* Данный пример содержит фрагмент макропрограммы, в
котором рекурсивно используется диалоговая панель с областью
скроллинга.*/
var SaveApos, SaveNrec, SaveNfields, SaveScrff, InUse = FALSE;
array saveDlgFields;
array SaveFileFields;

SaveApos = Apos (0);
SaveNrec = Nrec;
SaveNfields = Nfields;
SaveScrff = Scrff;
SaveDlgFields = DlgFields;
SaveFileFields = FileFields;
SetScroll (Scrff,
           "Blnc",      "Balance",
           "NameB",     "Name_Part"
           );

RunDialog (new, "ScrollMes")
Nrec = SaveNrec;
Nfields = SaveNfields;
Scrff = SaveScrff;
DlgFields = SaveDlgFields;
FileFields = SaveFileFields;
FillDown (dl, SaveApos);
```

См. [пример программы обработки скроллинга](#) ¹⁰¹.

Кроме перечисленных выше клавиш, используемых при работе со скроллингом, применяются следующие:

[Esc] – Закрытие окна прокрутки без сохранения введенных пользователем значений.

Примечание.

Для того чтобы отменить закрытие окна прокрутки при нажатии клавиши **[Esc]**, необходимо выполнить обработку сообщения **DLG_KEY**. Процедура обработки сообщений для кода указанной клавиши возвращает значение **CM_IGNORE**.

[F9] – Вставка новой записи.

Пример программы обработки скроллинга

Данный пример представляет собой текст законченной макропрограммы, демонстрирующей использование области скроллинга в диалоговой панели. Кроме этого, в нем демонстрируется техника замены процедур, которая позволяет организовать отбор записей для показа в области скроллинга.

Так как код из макромодуля *rs/scr.mac* для навигации по файлу использует только процедуры **next**, **prev**, **rewind**, то, подменяя эти процедуры на специальные, обеспечивается навигация только по заданному подмножеству записей.

В данном примере по нажатию клавиши **[F5]** производится запрос максимального и минимального номера счета для отображения в области скроллинга. Предполагается, что в редакторе диалоговых панелей созданы основная панель **BLNC**, содержащая колонки с именами **Blnc**, **NameB** и **Num** и вспомогательная панель **RBdlg** для задания интервала счетов. В примере так же показано, как можно использовать процедуру **UserFill**, которая заменяется на **MyFill**. Процедура **MyFill** заполняет колонку **Num**, используя первые два символа из колонки **Blnc**. Колонки **Blnc** и **NameB** заполняются данными из таблицы с именем "balance.dbt" во внутреннем словаре.

Пример.

```

/*
    Пример использования скроллинга по файлу с
    отбором записей из заданного интервала
*/
import rslscr; /* Поддержка скроллинга */
file ff (balance) key 0; /* Рабочий файл */
/* Отображать на экране будем только заданный
    интервал счетов */
var fromB = "", /* Минимальное значение */
    toB = ""; /* Максимальное значение */

/* -----
    Процедуры MyNext, MyPrev, MyRewind будут
    использоваться вместо стандартных next,
    prev, rewind для обеспечения отбора записей
    из заданного интервала
    ----- */

var first = TRUE;
macro MyNext (ff)
    /* Проверка на превышение максимального значения */
    macro Test
        if (toB and (ff.Balance > toB)) return FALSE end;
        return TRUE
    end;
    if (first and fromB)
        first = FALSE;
/* Установка минимального значения для первой записи */
        ff.Balance = fromB;
        if (getGE (ff)) return Test end
    else
        if (next (ff)) return Test end
    end;
    return FALSE
end;

macro MyPrev (ff)
/* Проверка достижения минимального значения */
    macro Test
        if (fromB and (ff.Balance < fromB)) return FALSE;
    end;
    return TRUE;
    end;
    if (first and toB)
        first = FALSE;
/* Установка максимального значения для последней записи */
        ff.Balance = toB;
        if (getLE (ff)) return Test end;
    else
        if (prev (ff)) return Test end;
    end;
    return FALSE
end;

macro MyRewind (ff)
    rewind (ff);
    first = TRUE
end;

macro MyFill (dlg)
    var str,i = 0;
    while (i < Nrec)
        str = dlg (FldIndex (dlg,"Blnc" + i));
        dlg (FldIndex (dlg,"Num" + i)) = substr (str,1,2);
        i = i + 1
    end
end;
end;

```

```

/* Выполняем замену стандартных процедур next,prev,rewind */
ReplaceMacro ("next","MyNext");
ReplaceMacro ("prev","MyPrev");
ReplaceMacro ("rewind","MyRewind");
ReplaceMacro ("UserFill","MyFill");

record dlg (blnc) dialog; /* Основной диалог */
record rb (RBdlg) dialog; /* Панель ввода интервала счетов
*/

const F4 = 318,F5 = 319;
macro MyMesProc (dl,cmd,id,key)
  var row;
  if (cmd == DLG_KEY)
    if (key == F4)
      /* Сделаем выбранную запись текущей в файле */
      row = FindRow (dl,id);
      if ((row != -1) and GetDirect (Scrff,Apos (row)))
        MsgBox ("Выбран счет " + Scrff.Balance);
      end;
      return CM_IGNORE
    elif (key == F5)
      /* Запрос нового интервала счетов */
      rb.fromB = fromB;
      rb.toB = toB;
      if (RunDialog (rb))
        fromB = rb.fromB;
        toB = rb.toB;
        /* Выводим на экран новый интервал счетов */
        rewind (Scrff);
        if (next (Scrff))
          FillDown (dl,GetPos (Scrff))
        end;
      end;
    end;
  end;
  return ScrollMes (dl,cmd,id,key)
end;

SetScroll (ff,
  "Blnc",      "Balance",
  "NameB",     "Name_Part"
);

dlg.Comment = "Тест прокрутки";
dlg.CurDate = date;

if (RunDialog (dlg,"MyMesProc"))
  println (ff.Balance," ",ff.Name_Part);
  println (dlg.Comment," ",dlg.Curdate)
end;

```

Создание источников данных с помощью класса ToolsDataAdapter

Инструментальный класс **ToolsDataAdapter** используется для создания произвольных источников данных, не зависящих от библиотеки RSD, при этом процесс создания осуществляется посредством процедуры [RunScroll](#)⁽²⁴⁸⁾, для которой в качестве источников данных могут выступать следующие объекты:

- объект класса, производного от инструментального класса **ToolsDataAdapter**.
- объект класса [RsdRecordset](#)⁽¹³³⁾ из стандартной библиотеки RSD.

Пример инициализации класса ToolsDataAdapter.

```
class ToolsDataAdapter
```

```

// Производный класс вызывает этот метод для
задания //объекта, хранящего текущую запись набора данных.
macro setCurrentRecord (rec: TRecHandler)
end;

// Методы, переопределяемые в производном классе:

// Метод возвращает массив с дополнительной информацией
о //визуальных атрибутах колонок:
macro getColumnsInfo : TArray
end;

// Методы, предназначенные для получения
информации, //необходимой для отображения окна с информацией
об ошибке, //произошедшей в источнике данных:
macro getLastStatus : integer
end;
macro getFileName : string
end;

// Методы, используемые для навигации по набору данных:
macro moveFirst : bool
end;
macro moveLast : bool
end;
macro moveNext : bool
end;
macro movePrev : bool
end;
macro moveToBookmark (bmk) : bool
end;

// Метод, позволяющий получить закладку для текущей записи:
macro getBookmark : variant
end;

// Методы для модификации данных:
macro RecordInsert: bool
end;
macro RecordUpdate: bool
end;
macro RecordDelete: bool
end;
end;

```

Примером использования класса **ToolsDataAdapter** может служить реализация следующих производных классов, предназначенных для создания источников данных для Btrieve-файла, который представлен классом **TBFile**:

- **BtrAdapterBase** – базовый класс для **BtrAdapter** и **BtrAdapterEx**.
- **BtrAdapter** – позволяет только просматривать содержимое существующих полей объекта класса **TBFile**. В качестве текущей записи класс **BtrAdapter** использует непосредственно объект класса **TBFile**.

- **BtrAdapterEx** – позволяет создавать вычисляемые колонки. Класс **BtrAdapterEx** создает вспомогательный объект класса **TRecHandler**, который содержит дополнительные колонки и используется для работы с текущей записью. Между объектами **TBFile** и **TRecHandler** выполняется обмен информацией.

Для объекта класса **TBFile**, непосредственно переданного в классы **BtrAdapter**, **BtrAdapterEx**, в скроллинг выводятся все поля. Имеется возможность управлять перечнем отображаемых полей. С этой целью необходимо:

- Создать класс, производный от классов **BtrAdapter** или **BtrAdapterEx**, и переопределить в нем метод **prepareColumns** указанных классов.
- Каждое поле (колонку), которое должно отображаться в скроллинге, необходимо определить с помощью метода [AddColumn](#)¹⁰⁹, вызванного в методе **prepareColumns**.
- Для класса с вычисляемыми полями, производного от **BtrAdapterEx**, следует переопределить метод **calculate**. В указанном методе необходимо указать правило задания значения вычисляемым колонкам. Значения должны быть помещены в текущую запись, которая хранится в свойстве **curRecord** базового класса.

Пример реализации классов BtrAdapterBase, BtrAdapter и BtrAdapterEx.

```
/*Содержимое макрофайла btrdata.mac*/
class (ToolsDataAdapter) BtrAdapterBase (bf)
  InitToolsDataAdapter;
  var bfile = bf;
  var curRecord;
  var cols;
  macro getColumnsInfo
    return cols
  end;
  macro getLastStatus
    return status;
  end;
  macro getFileName
    return bfile.fileName;
  end;
  macro AddColumn (name, head, width, kind, dec)
    if (cols == null)
      cols = TArray;
    end;
    var cind = cols.size;
    cols.value (cind)      = name;
    cols.value (cind + 1)  = head;
    cols.value (cind + 2)  = width;
    cols.value (cind + 3 ) = kind;
    cols.value (cind + 4 ) = dec;
    cols.value (cind + 5 ) = 0
  end;
  macro prepareColumns
  end;
  macro initCurRecord
    prepareColumns;
    curRecord = bfile
  end;
```

```

macro initAdapter
    initCurRecord;
    setCurrentRecord (curRecord)
end;
initAdapter;
end;
class (BtrAdapterBase) BtrAdapter (bf)
    InitBtrAdapterBase (bf);
macro moveFirst: bool
    bfile.rewind;
    return bfile.Next;
end;
macro moveLast: bool
    bfile.rewind;
    return bfile.Prev
end;
macro moveNext: bool
    return bfile.Next ()
end;
macro movePrev: bool
    return bfile.Prev ()
end;
macro moveToBookmark (bmk): bool
    return bfile.getDirect (bmk)
end;
macro getBookmark: variant
    return bfile.getPos ()
end;
macro RecordInsert: bool
    return bfile.insert (null, true);
end;
macro RecordUpdate: bool
    return bfile.update (null, true);
end;
macro RecordDelete: bool
    return bfile.Delete;
end;
end;
class (BtrAdapterBase) BtrAdapterEx (bf)
    InitBtrAdapterBase (bf);
var fields;
var undefTypePresent = false;
macro AddColumn (name, head, width, kind, dec, type,
size, decPoint)
    AddColumn (name, head, width, kind, dec);
    if (fields == null)
        fields = TArray;
    end;
var find = fields.size;
fields.value (find) = name;
fields.value (find + 1) = type;
fields.value (find + 2) = size;
fields.value (find + 3) = decPoint;
fields.value (find + 4) = width;

```

```

        if (type == null)
            undefTypePresent = true
        end
    end;
macro initCurRecord
    macro setFromMeta (ind, ar)
        for (var i, 0, ar.size / 5 - 1)
            if (fields.value (ind * 5) == ar.value (i * 5))
                fields.value (ind * 5 + 1) = ar.value (i * 5
+ 1);
                fields.value (ind * 5 + 2) = ar.value (i * 5
+ 2);
                fields.value (ind * 5 + 3) = ar.value (i * 5
+ 3);
                if (fields.value (ind * 5 + 4) == null)
                    fields.value (ind * 5 + 4) = ar.value (i *
5 + 4)
                end;
            return
        end
    end;
    RunError ("Не найдено поле " + fields.value (ind *
5));
end;
prepareColumns;
if ((fields == null) or (fields.size == 0))
    curRecord = TRecHandler (bfile.tblName,
bfile.GetFldInfo, false);
    return
end;
if (undefTypePresent)
    var ar = bfile.GetFldInfo;
    for (var i, 0, fields.size / 5 - 1)
        if (fields.value (i * 5 + 1) == null)
            setFromMeta (i, ar)
        end
    end
end;
curRecord = TRecHandler (bfile.tblName, fields,
false);
end;
macro calculate
end;
macro moveFirst: bool
    bfile.rewind;
    var res = bfile.Next;
    if (res)
        copy (curRecord, bfile);
        calculate
    end;
    return res
end;
macro moveLast: bool
    bfile.rewind;
    var res = bfile.Prev;
    if (res)

```

```
        copy (curRecord, bfile);
        calculate
    end;
    return res
end;
macro moveNext: bool
    var res = bfile.Next ();
    if (res)
        copy (curRecord, bfile);
        calculate
    end;
    return res
end;
macro movePrev: bool
    var res = bfile.Prev ();
    if (res)
        copy (curRecord, bfile);
        calculate
    end;
    return res
end;
macro moveToBookmark (bmk): bool
    var res = bfile.getDirect (bmk);
    if (res)
        copy (curRecord, bfile);
        calculate
    end;
    return res
end;
macro getBookmark
    return bfile.getPos ()
end;
macro RecordInsert: bool
    copy (bfile, curRecord);
    return bfile.insert (null, true);
end;
macro RecordUpdate: bool
    copy (bfile, curRecord);
    return bfile.update (null, true);
end;
macro RecordDelete: bool
    return bfile.Delete;
end;
end;
```

Классы **BtrAdapter**, **BtrAdapterEx** и **BtrAdapterBase** не входят в состав дистрибутива, но на основе данного примера возможна реализация других пользовательских источников данных для процедуры [RunScroll](#)²⁴⁸.

Метод AddColumn

AddColumn (name:String, head:String, width:Integer, kind:Integer, dec:Integer, type:Integer, size:Integer, decPoint:Integer)

Метод **AddColumn()** предназначен для задания колонок, отображаемых в области прокрутки, а также их атрибутов и вычисляемых полей.

Параметры:

name – программное имя колонки. Указанное имя должно соответствовать имени поля в объекте **TBfile**, что необходимо для установления связи колонки с полем. Для задания вычисляемой колонки необходимо, чтобы заданное имя не соответствовало ни одному полю в объекте **TBfile**.

head – отображаемое наименование колонки. Если параметр не задан, используется программное имя колонки, заданное параметром **name**.

width – начальная ширина колонки (в символах).

kind – тип колонки.

dec – отображаемое количество знаков после точки (для числовых типов данных).

type – тип данных колонки. В качестве значения параметра указывается любой необъектный тип данных RSL. Параметр задается только для вычисляемых колонок.

size – размер поля типа **V_STRING** в символах. Параметр задается только для вычисляемых колонок.

decPoint – хранимое количество знаков после точки для типа Numeric.

Пример 1.

```
import btrdata, rsd;

class (BtrAdapter) BtrAdapterUser (bf)
  InitBtrAdapter (bf);
  macro prepareColumns
    //      name,      head,      width,  kind,  dec
    AddColumn ("iNumPlan", "№",      2,      0,      0 );
    AddColumn ("Balance", null,      10,      0,      0 );
    AddColumn ("Name_Part", null,     80,      0,      0 );
  end;
end;
```

Пример 2.

```
class (BtrAdapterEx) BtrAdapterCalc (bf)
  InitBtrAdapterEx (bf);
  macro prepareColumns
    //      name,      head,      width,  kind,  dec,
    type,  size, decPoint
    AddColumn ("iNumPlan", "№",      2,      0,      0 );
    AddColumn ("Balance", null,     10,      0,      0 );
```

```

        AddColumn ("Calc",          "Вычислено", 20,      2,      0,
V_STRING, 100, 0);
        AddColumn ("Name_Part", null,          80,      0,      0 );
    end;
    var flag = 0;
    macro calculate
        curRecord.rec.Calc = string (curRecord.rec.Balance) +
" MyCalc " + flag
    end;
end;

```

Пример 3.

```

cn = RsdConnection("DSN=Northwind;USER ID=sa;PASSWORD=sa");
cmd = RsdCommand (cn,"select * from Customers");
//Создаём источник данных одним из способов/
//ob = RSDRecordset(cmd , RSDVAL_CLIENT, RSDVAL_STATIC );
//ob = BtrAdapter (tbfile ("balance", "w"));
//ob = BtrAdapterUser (tbfile ("balance", "w"));
//ob = BtrAdapterEx (tbfile ("balance", "w"));
ob = BtrAdapterCalc (tbfile ("balance", "w"));
macro SMsgProc (ob, cmd, id, key)
    var a;
    if (cmd == DLG_KEY)
        if (key == 13)
            return CM_SELECT
        end;
    end;
end;
if (RunScroll (ob, 0, null, "Test", @SMsgProc, "Заголовок",
"Status", false, -1, -1, 60, 30))
    println ("Запись выбрана")
end;

```

Работа с файлами и таблицами баз данных

Для работы с файлами и таблицами баз данных в Object RSL предусмотрены следующие конструкции:

- стандартный класс [Tbfile](#)⁽¹¹¹⁾;
- стандартный класс [TRecHandler](#)⁽¹¹⁶⁾;
- конструкции [FILE](#)⁽¹¹⁹⁾ и [RECORD](#)⁽¹¹⁹⁾.

Примечание.

Для работы с таблицами баз данных предпочтительнее использовать классы [Tbfile](#) и [TRecHandler](#). Конструкция [FILE](#) в настоящее время поддерживается для совместимости с ранее разработанными программами.

См. Особенности при использовании СУБД Oracle

При работе с прикладными программами, разработанными в компании R-Style Softlab, следует обратить особое внимание на отличие в текстах макропрограмм имен таблиц базы данных и их полей от реальных имен таблиц и полей, видимых при работе в среде СУБД Oracle. Это обусловлено необходимостью обеспечения совместимости с разработанным ранее программным кодом.

В прикладной программе, помимо стандартного описания базы данных, предоставляемого СУБД Oracle, используется внутренний системный словарь, в котором хранится описание структур и ключей всех таблиц данных в специальном формате. С помощью этого словаря устанавливается соответствие между именами реальных таблиц и их полей и именами таблиц и полей, которые используются в текстах макропрограмм.

Обращение к таблицам базы данных в рамках прикладной программы должно осуществляться в соответствии с описанием таблиц во внутреннем словаре. Поэтому при декларировании объектов, представляющих собой ссылки на таблицы базы данных, и передаче их в RSL-процедуру нужно указывать имена таблиц во внутреннем формате, то есть как во внутреннем словаре:

<имя таблицы во внутреннем словаре>.dbt

Реальное имя таблицы в базе данных имеет формат:

d<имя таблицы во внутреннем словаре>_dbt

Имена полей таблиц баз данных, описанные во внутреннем словаре и используемые при написании RSL-программ, в реальных таблицах базы данных имеют вид:

t_<имя поля во внутреннем словаре>

Использование стандартного класса Tbfile

Стандартный класс **Tbfile** предназначен для работы с таблицами баз данных и представляет собой объектную альтернативу стандартной конструкции языка **FILE**.

Синтаксис конструктора **Tbfile** имеет следующий вид:

TBfile (dicName:string [, attrStr:String, keyNum:Integer ,btrFName: String, altDicName:String, cloneName:String]):Object

Параметры конструктора **Tbfile** задают следующие значения:

DicName – имя внутреннего словаря базы данных. Параметр указывается в том случае, если необходимо использовать словарь, отличный от используемого по умолчанию.

AttrStr – строка атрибутов открытия таблицы. В качестве параметра **AttrStr** могут быть заданы следующие символы:

- **R** – открыть таблицу только для чтения (принимается по умолчанию).
- **W** – открыть таблицу для чтения и записи.
- **E** – открыть таблицу для чтения и записи в монопольном режиме.
- **A** – открыть таблицу в ускоренном режиме; данный атрибут используется только для временных таблиц.
- **C** – открыть таблицу; если таблица не существует, то она будет создана.
- **C+** – создать временную таблицу; если таблица с таким именем существует, то она будет замещена вновь созданной.
- **O** – выполнить клонирование; если клонирование производится не в открытую таблицу, имя требуемой таблицы необходимо задать в параметре **cloneName**.
- **P+** – создать постоянную таблицу; если таблица с таким именем существует, то она будет замещена вновь созданной.

Если этот параметр не задан, таблица открывается только для чтения.

Для открытия таблицы с переменной частью записи, используемой как поле типа **BLOB**, дополнительно к перечисленным атрибутам используется символ **"B"**.

KeyNum – текущий индекс в таблице. Если параметр не указан, текущим индексом является 0. Для организации доступа к записям таблицы в физической последовательности необходимо указать значение данного параметра равным -1.

btrFName – имя исходной таблицы. Параметр используется при выполнении операции клонирования. Если параметр не задан, используется имя, переданное в параметре **DicName**. В случае отсутствия расширения в указанном имени, оно считается равным **.dbt**.

altDicName – имя альтернативного словаря.

cloneName – таблица для записи результата клонирования. Возможны следующие случаи:

- если в имени таблицы не указано расширение, оно считается равным **.dbt**;
- если имя таблицы не задано, используется имя исходной таблицы;

- если таблица с указанным именем не существует, она создается в первом каталоге из списка каталогов поиска таблиц (.dbt).

Для создания экземпляра класса необходимо вызвать конструктор **Tbfile**. В процессе работы конструктора, помимо других действий, таблица (файл) открывается и к ней применяется метод **rewind**.

Примечание.

При выполнении операции клонирования конструктор возвращает объект, связанный с клонированным файлом. Дальнейшая работа производится с указанным файлом.

Пример.

```
/*Таблица account открывается только для чтения по нулевому
индексу*/
Ob = Tbfile ("Account.dbt");
/* Таблица account открывается для записи по второму индексу
*/
Ob = Tbfile ("Account.dbt", "W", 2);
/* Открывается временная таблица test на основании описания
таблицы "person.dbt" из внутреннего словаря */
Ob = TBFile ("person","C+", 0, "test");
/* Открывается постоянная таблица test на основании описания
таблицы "person.dbt" из внутреннего словаря*/
Ob = TBFile ("person","P+", 0, "test");
```

Для объектов класса **TbFile** определен [ряд методов](#) ¹¹³.

Объект класса **Tbfile** имеет [ряд свойств](#) ¹¹⁵.

Методы класса Tbfile

Для объектов класса **TbFile** определены следующие методы:

AddFilter (cond:string) – с помощью этого метода можно наложить ограничения на выбираемые записи. Параметром является строка, которая добавляется ко всем генерируемым при работе с таблицей SQL-запросам. Строка добавляется без изменений в условие **WHERE**, поэтому она должна удовлетворять всем требованиям синтаксиса SQL.

DropFilter ():bool – снимает ограничения, наложенные методом **AddFilter**. При отсутствии фильтра, установленного методом **AddFilter**, метод возвращает значение FALSE.

Пример.

```
file = TBfile ("partyown");
file.AddFilter("t_partykind=3");
.....
file.DropFilter();
```

GetFldInfo ():TArray – метод возвращает объект класса **TArray** с метаданной о полях файла. Размер массива равен количеству полей, умноженному на пять. Таким образом, для каждого поля используются пять последовательных элементов массива, которые содержат (по порядку) следующую информацию:

- имя поля;
- тип;
- размер;

- количество знаков после точки (для числовых полей);
- 0 (зарезервирован).

GetKeyInfo ():TArray – метод возвращает объект класса **TArray** с метаданной о ключах файла. Размер массива равен количеству сегментов ключей, умноженному на пять. Следовательно, для каждого сегмента используются пять последовательных элементов массива, которые содержат:

- номер ключа;
- номер сегмента в ключе;
- номер поля, которому принадлежит сегмент;
- порядок сортировки: 1 – по возрастанию; 0 – по убыванию;
- 0 (зарезервирован).

Пример.

```
var f = tbfile ("balance");
macro PrintInfo (a)
    var sz = a.size / 5;
    var i;
    for (i, 0, sz - 1)
        [##### | ##### |
        ##### | #####]
        (a [i * 5],          a [i * 5 + 1],          a
        [i * 5 + 2],          a [i * 5 + 3])
    end
end;

var fldInfo = f.GetFldInfo;
printInfo (fldInfo);
[-----];
var keyInfo = f.GetKeyInfo;
printInfo (keyInfo);
```

Кроме этих методов, для объектов класса **Tbfile** также реализованы методы, аналогичные процедурам для работы с таблицами базы данных:

- | | | | |
|-------------|---------------|-----------------|-------------|
| • next | • prev | • getpos | • getdirect |
| • insert | • update | • delete | • getEQ |
| • getGE | • getLE | • getGT | • getLT |
| • rewind | • fldnumber | • fldindex | • fldname |
| • fldoffset | • Nrecords | • RecSize | • VarSize |
| • Clear | • PackVarBuff | • UnPackVarBuff | • WriteBlob |
| • ReadBlob | • FileName | | |

Свойства класса **Tbfile**

Объект класса **Tbfile** имеет следующие свойства:

Rec – ссылка на объект типа RECORD, при обращении к которому осуществляется доступ к полям записи.

KeyNum – текущий индекс в таблице.

OpenMode – код открытия файла. Возможные значения:

- **0 (Normal)** – разрешено чтение и запись.
- **-1 (Accelerated)** – ускоренный режим открытия Btrieve-файлов. Может использоваться только для работы с временными файлами.
- **-2 (ReadOnly)** – разрешено только чтение.
- **-4 (Exclusive)** – монопольный доступ к файлу. Поддерживается только для работы с Btrieve файлами.

TblName – имя таблицы.

DicName – имя внутреннего словаря базы данных.

cnvMode – режим автоматической трансляции строковых полей между кодировками ANSI и OEM. Свойство может принимать следующие значения:

- **0** – автоматическая трансляция не выполняется;
- **1** – в базе данных кодировка OEM, в программе – ANSI;
- **2** – в базе данных кодировка ANSI, в программе – OEM.

Пример.

```
/*Пример доступа к полям записи таблицы: */  
Ob.rec.Account = "711034";  
Ob.rec.Rest = $12345.67;
```

Объекты класса **Tbfile** имеют свойство по умолчанию **item**. Параметр этого свойства указывает имя поля или номер поля записи таблицы в соответствии со структурой, описанной во внутреннем словаре.

Пример.

```
Ob = Tbfile ("account");  
Ob.item ("Sum") = $123.45;  
Ob.item (1) = "Comment";
```

Так как **item** является свойством по умолчанию, то можно записать и так:

```
Ob. ("Sum") = $123.45;  
Ob. (1) = "Comment";
```

Использование стандартного класса TRecHandler

Стандартный класс **TRecHandler** представляет собой объектную альтернативу стандартной конструкции языка RECORD.

Объект класса может быть создан следующими способами:

- Структура полей объекта может быть загружена из словаря базы данных или из библиотеки ресурсов (файл LBR).
- Структура может быть задана с помощью конструктора класса.

Конструктор класса может быть вызван одним из следующих способов:

TRecHandler (strName: String, fldInfo: TArray [, shared:Bool]):Object

Конструктор используется для создания объекта класса **TRecHandler**.

Параметры:

strName — уникальное имя структуры. Если структура с указанным именем уже существует, параметр **fldInfo** игнорируется.

fldInfo — массив, задающий поля структуры. Каждое поле структуры определяется пятью последовательными элементами указанного массива, которые содержат следующие сведения:

- Имя поля.
- Тип поля — могут быть заданы константы V_INTEGER, V_MONEY, V_NUMERIC, V_DOUBLE, V_STRING, V_DATE, V_TIME. Если передано другое значение, оно изменяется на V_INTEGER, поле получает размер 2 байта.
- Размер поля — может быть задан только для полей типа V_INTEGER, V_DOUBLE и V_STRING. Для поля типа V_INTEGER размер поля может быть 1, 2, 4 байта. Если передано другое значение, используется размер поля 2. Для поля типа V_DOUBLE — 4 или 8 (по умолчанию). Для поля с типом V_STRING задаётся размер символьной строки с учётом терминирующего нуль-символа. Для остальных типов полей размер поля определяется, исходя из типа поля; при этом переданное значение игнорируется.
- Количество знаков после точки — может быть задано только для полей типа V_NUMERIC и V_MONEY. Для остальных типов полей переданное значение игнорируется.
- Ширина поля (в символах) для вывода информации. Если значение равно -1, поле не отображается.

shared — признак того, что поиск структуры **strName** начинается в кэше. Возможные значения:

- **TRUE** – поиск выполняется в кэше (значение по умолчанию). Если структура найдена, параметр **fldInfo** игнорируется.
- **FALSE** – создается новая структура, в т.ч. и в случае наличия в кэше структуры с тем же именем.

TRecHandler (strName:String [, dicName:String] , isLbr:Bool):Object

Конструктор создает объект класса **TRecHandler**.

Параметры:

strName – имя структуры в словаре или панели в библиотеке ресурсов.

dicName – имя словаря базы данных или имя библиотеки ресурсов. Если имя не задано, используется словарь или библиотека, установленные в системе по умолчанию.

isLbr – признак, позволяющий определить, наименование какого элемента передано в параметре **strName**: панели или структуры. Возможные значения параметра:

- **TRUE** – параметр **strName** содержит имя панели в библиотеке ресурсов.
- **FALSE** (или не задан) – параметр **strName** указывает на имя структуры в словаре.

Пример.

```
macro AddField (ar, name, tp, sz, dec)
  ar [ar.size] = name;
  ar [ar.size] = tp;
  ar [ar.size] = sz;
  ar [ar.size] = dec;
  ar [ar.size] = 0;
end;

ar = TArray;
AddField (ar, "field1", V_NUMERIC, 0, 10);
AddField (ar, "field2", V_INTEGER, 4, 0);
AddField (ar, "field4", V_STRING, 40, 0);
ob = TRecHandler ("test", ar);
ob.rec.field1 = 12345.782345678;
ob.rec.field2 = 12345.782345678;
ob.rec.field4 = 12345.782345678;
printprops (ob.rec);
println (ob.item("field1"):0:10);
println (ob.item("field2"):0:10);
println (ob.item("field4"):0:10);
```

В отличие от класса **Tbfile**, класс **TRecHandler** не связан непосредственно с таблицей базы данных, однако его объекты могут использоваться для хранения в памяти копии записи из таблицы.

Пример.

```
/* Создается экземпляр класса Tbfile. В качестве параметра
конструктору класса передается файл account.dbt, открываемый
для чтения по нулевому индексу */;
```

```

Ob = Tbfile ("account");
/* Выполняется обращение к методу next */;
Ob.next;
/* Создается экземпляр класса TRecHandler. В качестве
параметра конструктору класса передается структура файла
account.db */;
Rec = TRecHandler ("account");
/* Создается копия записи файла account.dbt */;
Copy (rec,ob);

```

Объекты **TRecHandler** могут использоваться совместно с объектами **Tbfile**.

Можно "наложить" объект класса **TRecHandler** на постоянную или переменную часть записи таблицы. Наложение структуры на запись таблицы предоставляет возможность считывать данные из записи таблицы, а так же модифицировать ее данные непосредственно в таблице посредством обращения к полям структуры.

Объекты класса **TRecHandler** имеют свойство **cnvMode**, которое задает режим автоматической трансляции строковых полей между кодировками ANSI и OEM. Свойство может принимать следующие значения:

- **0** – автоматическая трансляция не выполняется;
- **1** – в базе данных кодировка OEM, в программе – ANSI;
- **2** – в базе данных кодировка ANSI, в программе – OEM.

Объекты **TRecHandler** имеют метод **SetRecordAddr**, который позволяет связать существующий объект типа **TRecHandler** с объектом **Tbfile**:

```
SetRecordAddr ( file, ind, offs, isFix );
```

Параметр **file** задает ссылку на объект **Tbfile** или его наследника. Остальные параметры аналогичны параметрам конструктора.

В результате использования этого метода все процедуры, работающие с конструкцией FILE (**next**, **prev**, **SetRecordAddr** и т.д), могут принимать в качестве параметра ссылку на объект **Tbfile** или его наследника. Процедуры **fldname**, **fldindex** и др., принимающие ссылку на конструкцию RECORD, могут принимать ссылку на объект **TRecHandler** или его наследника.

Пример.

```

class (Tbfile) MyTb
    InitTbfile ("client.dbt","w",0);
end;
ob = MyTb;
class (TRecHandler) MyRec
    InitTRecHandler ("client.dbt");
end;
obr = MyRec;
obr.SetRecordAddr (ob,0,0,true);
ob.next;
println (obr.rec.name_client);

```

Использование конструкций FILE и RECORD

Конструкции **FILE** и **RECORD** используются для объявления объектов типа **FILE**, **DBFFILE**, **TEXTFILE** и **RECORD**. Эти определения практически одинаковы. Различие между ними заключается в следующем: конструкция **FILE** по умолчанию определяет объект типа **FILE**, а конструкция **RECORD** – объект типа **RECORD**.

Определение имеет следующий синтаксис:

[local | private] FILE идентификатор (имя объекта [, имя словаря]) [список параметров объекта]

[local | private] RECORD идентификатор (имя объекта [, имя библиотеки диалогов]) [список параметров объекта]

Идентификатор – это имя, через которое можно ссылаться на определяемый объект. В выражении идентификатор преобразуется к переменной типа, характеризующегося кодом **V_FILE**, **V_STRUC**, **V_TEXTFILE** или **V_DBFFILE**, в зависимости от типа определяемого объекта. Благодаря этому, идентификатор объекта можно присваивать переменным и передавать в качестве параметра процедурам.

В круглых скобках указывается:

- Для таблиц базы данных (объект типа **FILE**) – имя таблицы во внутреннем словаре системы с описанием базы данных. Если пользователь использует несколько словарей, то после запятой необходимо указать имя словаря, в котором описана данная таблица. В случае использования одного словаря этот параметр можно не указывать.
- Для текстовых и файлов формата DBF (объекты типа **TEXTFILE** и **DBFFILE**) – физическое имя файла. Если оно отсутствует, необходимо явно вызвать процедуру [*Open*](#)²⁵⁷ с указанием имени файла.
- Для структур (объект типа **RECORD**) – имя структуры во внутреннем словаре системы или имя диалоговой панели в библиотеке диалогов (если указан спецификатор **DIALOG**). Если пользователь использует несколько библиотек диалогов, то после запятой необходимо указать имя библиотеки, в которой описана диалоговая панель. В случае использования одной библиотеки данный параметр можно не указывать.

Если имя объекта включает недопустимые в идентификаторе символы, например, точку, его необходимо заключить в кавычки. Таким образом, если необходимо явно указать расширение файла, его имя необходимо заключить в кавычки.

Список параметров определения не обязателен. Он может содержать любое количество разделенных пробелами спецификаторов:

- **SORT <number>** – определение номера ключевого индекса таблицы (файла) (только для объектов типа **FILE**), где **number** – номер ключа.
- **KEY <number>** – конструкция, эквивалентная спецификатору **SORT**.
- **WRITE** – режим открытия таблицы (файла) на запись.
- **APPEND** – режим открытия текстового файла в режиме дозаписи.

- **MEM** – спецификатор структуры (умолчание для конструкции **RECORD**).
- **TXT** – спецификатор текстового файла.
- **DBF** – спецификатор файла формата DBF.
- **DIALOG** – параметр, задающий структуру диалогового окна.
- **BTR** – определяет объект типа **FILE** (умолчание для конструкции **FILE**).

Так как конструкции **FILE** и **RECORD** определяют одинаковые объекты, то справедливы следующие определения:

file id (name) равносильно record id (name) btr;

record id (name) равносильно file id (name) mem;

По умолчанию файлы и таблицы баз данных открываются в режиме "Только чтение". Чтобы в файл можно было вносить изменения, его необходимо открыть в режиме для записи, указав спецификатор **WRITE**.

Если в таблице базы данных отсутствует номер ключевого индекса, заданный с помощью параметра **SORT NUMBER**, то используется ключ с номером 0.

Пример.

1) Пример определения объекта типа **FILE** для доступа по записи к таблице, описанной во внутреннем словаре как *account.dbt*:

File Счета (account) write;

или

Record Счета (account) btr write;

2) Пример определения объекта типа **DBF** для доступа к файлу формата DBF *demo.dbf*:

File demo ("demo.dbf") dbf;

или

Record demo ("demo.dbf") dbf;

3) Пример определения объекта типа **TXT** для доступа к текстовому файлу *demo.txt*:

File tfile ("demo.txt") txt;

или

Record tfile ("demo.txt") txt;

4) Пример определения объекта типа **TXT** для доступа по добавлению записей к текстовому файлу *demo.txt*:

File ff ("demo") txt append;

или

Record tfile ("demo") txt append;

5) Пример определения объекта типа **RECORD**, имеющего структуру, описанную во внутреннем словаре как *account.dbt*:

File rec (account) mem;

или

Record rec (account);

Доступ к полям объекта осуществляется по именам полей, описанных в словаре данных или библиотеке диалогов, или по индексу поля (поля нумеруются, начиная с нуля).

Пример.

```
Номер = aa.Account;  
Имя   = Клиенты.Name;  
Номер = aa(0);  
Имя   = Клиенты(10);
```

Для таблиц базы данных возможен поиск записей по ключу при помощи процедур:

- getEQ;
- getGT;
- getGE;
- getLT;
- getLE.

Данные процедуры выполняют поиск записи таблицы, значение ключа для которой:

- равно указанному;
- строго больше указанного;
- больше или равно указанному;
- строго меньше указанного;
- меньше или равно указанному

соответственно перечисленным выше процедурам.

Чтобы задать значение ключа для поиска, необходимо присвоить нужные значения тем полям записи таблицы, которые соответствуют сегментам текущего ключа перед вызовом этих процедур.

Чтобы определить номер ключа, по которому открывается таблица, необходимо при определении таблицы указать спецификатор SORT. По умолчанию используется ключ с номером 0. Для доступа к записям таблицы в последовательности, задаваемой первичным ключом, значение ключа должно быть равно -1.

Пример.

```
/* Пример выборки записей из таблицы с использованием  
оптимизации по ключу. Предполагается, что таблица arhdos  
имеет ключ с номером 0 по полю Date_Carry. Программа  
выбирает  
записи, попадающие в заданный диапазон дат.*/  
/* Отбор записей по ключу */  
file from ("arhdos.dbt") sort 0;  
var fromdate, /* Минимальное значение диапазона*/  
    todate; /* Максимальное значение диапазона*/  
var first = TRUE;  
macro getarh (ff)  
    macro Test /* Проверка на превышение максимального  
значения */
```

```

        if (ff.Date_Carry > todate) first = TRUE; return
FALSE; end;
        return TRUE;
    end;
    if (first)
        first = FALSE;
        /* Установка минимального значения для поиска по ключу
первой записи */
        ff.Date_Carry = fromdate;
        if (getGE (ff)) return Test; end;
    else
        if (next (ff)) return Test; end;
    end
    return FALSE;
end;

/* Установка минимального и максимального значения для
отбора */
fromdate = date (1,1,1994);
todate   = date (1,4,1994);
var ndoc = 0;
while (getarh (from))
    ndoc = ndoc + 1;
    message (from.Date_Carry, "    ", ndoc);
end;
println (ndoc);
/* Задаем новый интервал */
fromdate = date (1,6,1994);
todate   = date (1,7,1994);
/* Обрабатываем новую выборку */
while (getarh (from))
    message (from.Date_Carry, "    ", ndoc);
end;

```

Работа с текстовыми файлами

Перед обработкой данных текстового файла необходимо определить его в RSL-программе, используя определение **FILE**.

Пример.

```
FILE aa ("c:\\out\\data.rep") txt;
```

Данная строка определяет идентификатор aa для текстового файла с именем "c:\\out\\data.rep", доступного только для чтения.

Чтобы определить файл для записи, используется следующее определение:

```
FILE aa ("c:\\out\\data.rep") txt write;
```

В этом случае файл создается автоматически. Если файл уже существовал, то он будет перезаписан.

Если необходимо осуществить дозапись в текстовый файл, его необходимо открывать с атрибутом **append**.

Пример.

```
FILE aa ("MyFile") txt append;
```

Для работы с текстовыми файлами применяются следующие процедуры:

- [Open](#)²⁵⁷.
- [Close](#)²⁵⁸.
- [Next](#)²⁶⁰.
- [Insert](#)²⁶¹.
- [Rewind](#)²⁶¹.

Можно явно открыть и закрыть файл при помощи процедуры **Open** и **Close**.

Пример.

```
if (open (aa))
    ....
close (aa);
end
```

Текстовый файл автоматически открывается при первой необходимости и закрывается при выходе из области видимости идентификатора файла. Поэтому явно вызывать процедуры **Open** и **Close** нет необходимости.

Процедура **Next** позволяет читать файл строка за строкой. Доступ к прочитанной информации осуществляется через фиксированное имя поля **str**.

Пример.

В приведенном примере читается файл, и его строки выводятся в выходной поток:

```
while (next (aa))
    println (aa.str)
end;
```

Если строки текстового файла состоят из полей, разделенных символами-разделителями, то к ним можно обращаться по индексу. Первое поле имеет индекс 0. Набор символов-разделителей можно задать при помощи процедуры [SetDelim](#)²⁷². По умолчанию разделителями являются символы пробел и табуляция.

Пример.

```
f1 = aa(0);
f2 = aa(1);
f3 = aa(5);
```

Запись в файл осуществляется строка за строкой в конец файла при помощи процедуры **Insert**. Данные для записи определяются при помощи поля с предопределенным именем **str** или поля с индексом 0. Допускается передать в качестве второго параметра процедуры **insert** непосредственно строку для записи в файл.

Пример.

```
aa.str = "Данные для записи";
insert (aa);
или
aa(0) = "Данные для записи";
insert (aa);
или
insert (aa, "Данные для записи");
```

Последовательность процедур **rewind** и **next** переустанавливает текущую позицию на начало текстового файла. Она полезна в том случае, если необходимо повторно прочитать файл.

При описании файла можно опустить физическое имя файла (**но не круглые скобки!**). В этом случае необходимо явно вызвать процедуру **Open** с указанием имени файла.

Пример.

```
File aa () txt;  
open (aa,"data.txt");
```

Максимальная длина буфера у конструкции TXT равна 1024 символа. Если необходимо считывать более длинные строки, то необходимо указать максимальный размер строки следующим образом:

```
file aa () txt 2048;
```

В этом примере задается максимальный размер строки, равный 2048 байт. Размер одного поля в строке, тем не менее, не должен превышать 1024 символа.

Работа с файлами в формате DBF

При работе с файлами формата DBF применяются те же принципы, что и для таблиц базы данных. Прежде всего, необходимо определить файл, используя определение **FILE**.

Пример.

```
FILE aa ("c:\\out\\data.dbf") dbf;
```

Данная строка определяет идентификатор **aa** для DBF-файла, доступного только для чтения, с именем c:\\out\\data.dbf.

Чтобы определить DBF-файл для записи, используется следующее определение:

```
FILE aa ("c:\\out\\data.dbf") dbf write;
```

Для работы с DBF-файлами применяются следующие процедуры:

- | | | |
|--|--|---|
| • Open ²⁵⁷ . | • Close ²⁵⁸ . | • Create ²⁵⁸ . |
| • Next ²⁶⁰ . | • Insert ²⁶¹ . | • Update ²⁶² . |
| • Delete ²⁶² . | • Rewind ²⁶¹ . | • GetPos ²⁶³ . |
| • GetDirect ²⁶³ . | • FldNumber ²⁶⁹ . | • FldIndex ²⁶⁹ . |
| • FldName ²⁶⁹ . | • Clone ²⁵⁹ . | |

Индексы для DBF-файлов не поддерживаются. Файл читается всегда в физической последовательности. Процедура **Insert** добавляет новую запись всегда в конец файла. Процедура **GetPos** возвращает номер записи в DBF-файле.

Если необходимо прочитать запись с конкретным номером, то можно использовать процедуру **GetDirect** без предварительного вызова **GetPos**. В этом случае вторым параметром для **GetDirect** необходимо передать номер интересующей записи в DBF-файле. (Первая запись имеет номер 1).

Файл читается запись за записью при помощи процедуры **Next**. Доступ к полям осуществляется по именам или индексам, аналогично таблицам базы данных.

Пример.

Этот пример читает весь файл и выводит строки в выходной поток:

```
while (next (aa))  
  println (aa.name," ",aa.id)  
end
```

DBF-файл автоматически открывается при первой необходимости и закрывается при выходе из области видимости идентификатора файла. Поэтому явно вызывать процедуры **Open** и **Close** нет необходимости.

При описании файла можно опустить физическое имя файла (**но не круглые скобки!**).

```
File aa () dbf;
```

В этом случае необходимо явно вызвать процедуру **Open** с указанием имени файла.

```
open (aa, "data.dbf");
```

Работа с записями таблиц как с записями переменной длины

В RSL предусмотрено два способа организации доступа к записям таблиц базы данных, имеющим переменную длину записи.

Первый способ подразумевает, что размер переменной части записи ограничивается пределом 64 Kb. В этом случае переменная часть записи считывается в буфер памяти вместе с постоянной частью. Доступ к данным переменной части осуществляется посредством наложения определенной пользователем структуры на переменную часть записи. Для этого используется процедура [SetRecordAddr](#)²⁶⁶.

Процедура **SetRecordAddr** позволяет наложить пользовательскую структуру и на постоянную часть записи. Это дает возможность иметь в одном файле записи с разной структурой, но для этого необходимо предусмотреть какой-либо способ идентификации типа записи.

Размер переменной части записи 64Kb соответствует ее теоретическому пределу. Как правило, процедуры системы RS-Bank работают непосредственно с записями такого размера, но иногда размер переменной части записи необходимо уменьшить. С этой целью ее "упаковывают". Для работы с "упакованными" записями в RSL предусмотрено две процедуры: [PackVarBuff](#)²⁶⁷ и [UnPackVarBuff](#)²⁶⁸.

Второй способ предполагает, что переменная часть записи рассматривается как специальное поле типа **BLOB** ("двоичный большой объект"), которое не имеет ограничений на размер записываемой информации.

Обычные операции чтения и записи таблицы распространяются только на постоянную часть, а для доступа к полю **BLOB** применяются специальные процедуры: [WriteBlob](#)²⁷⁶ и [ReadBlob](#)²⁷⁶.

Доступ к источникам данных с помощью библиотеки RSD

Библиотека RSD предназначена для универсального доступа из программ на языке RSL к источникам данных, поддерживающим SQL.

Данный раздел содержит:

- [описание библиотеки RSD](#) ⁽¹²⁶⁾;
- технологию [использования RSD в программах на языке RSL](#) ⁽¹³⁹⁾.

Описание библиотеки RSD

Физически библиотека RSD имеет несколько уровней. Каждый последующий уровень базируется на предыдущем (большую часть функциональности делегирует нижележащему уровню):

- **Системный уровень**, включающий в себя:
 - **драйвер источника данных**, который обеспечивает низкоуровневый доступ к сервисам библиотеки; его реализация специфична для источника данных, в настоящее время имеется драйвер для ODBC;
 - **драйвер универсального набора записей** (Recordset), который обеспечивает унифицированный навигационный доступ к данным.

Системный уровень реализован в виде *RDDrvO.dll* (драйвер ODBC) и *RDRSet.dll* (драйвер набора записей).

- **Объектно-ориентированный интерфейс доступа к данным**, не зависящий от конкретного источника. Данный уровень представляет собой C++-классы и реализован в виде *RsdC.dll*.
- **Интерфейс для доступа к данным из языка RSL**. Данный уровень представляет собой RSL-классы и реализован как *DLM Rsd.d32*.

Объектно-логическая модель библиотеки представлена на схеме, изображенной на [Рис. 1](#) ⁽¹²⁶⁾.

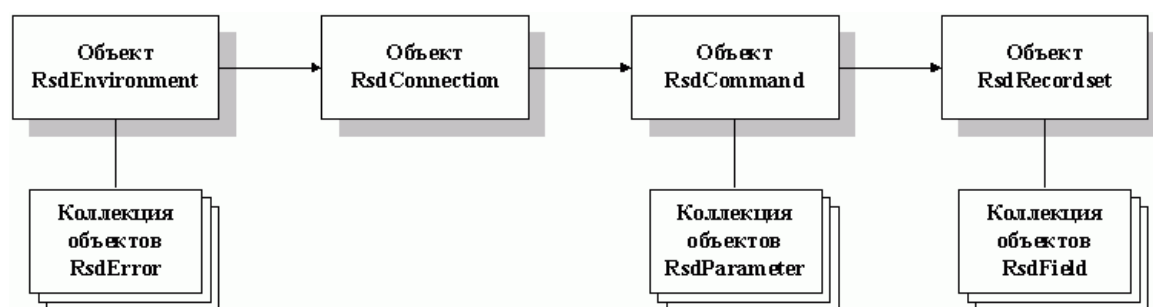


Рис. 1. Объектно-логическая модель библиотеки RSD.

Класс RsdEnvironment

При использовании RSD первым создается объект окружения **RsdEnvironment**. Он обеспечивает загрузку нужного драйвера, создание окружения для функционирования других компонент и обработку ошибок.

Конструктор класса вызывается следующим образом:

RsdEnvironment ([driver], [library])

При вызове конструктора используются параметры:

driver – имя интерфейса. По умолчанию используется имя "RDDrvO".

library – имя файла драйвера ODBC. По умолчанию используется имя "RDDrvO.dll".

В одной программе может быть создано несколько окружений с разными параметрами. Доступ к коллекции ошибок, возникающих при работе объекта, производится с помощью индексированного свойства **Error**.

В программах на языке RSL нет необходимости явно создавать окружение. Вместо этого можно использовать объект **RsdDefEnv**, который создается по умолчанию и подключается к драйверу для ODBC.

В этом и других объектах библиотеки используется "ленивая" активизация, то есть выполнение функций драйвера откладывается до момента реальной необходимости. То есть окружение фактически открывается только тогда, когда явно вызывается его метод **Open**, или когда открывается один из объектов, использующих это окружение (это может быть команда при вызове метода **Execute**).

Свойства класса

Для класса **RsdEnvironment** реализованы следующие свойства:

Driver – имя интерфейса ([RW|RO](#)¹²⁷). Свойство аналогично одноименному параметру конструктора.

Library – имя файла драйвера ODBC ([RW|RO](#)¹²⁷). Свойство аналогично одноименному параметру конструктора.

ErrorCount – количество ошибок в коллекции ([RO](#)¹²⁷).

Error (index):RsdError – свойство предоставляет доступ к ошибке с заданным номером из коллекции ошибок ([RO](#)¹²⁷).

Здесь и далее в описании свойств классов используются следующие обозначения:

- RO – значение свойства нельзя модифицировать,
- RW – значение свойства можно модифицировать,
- RW|RO – значение свойства можно модифицировать, когда объект не открыт, и нельзя модифицировать для открытого объекта (при попытке модификации генерируется ошибка).

Методы класса

Для класса **RsdEnvironment** реализованы следующие методы:

Open ([driver], [library]) – метод открывает окружение. При его вызове можно использовать те же параметры, что и в конструкторе.

Close () – метод закрывает окружение.

ClearErrors () – метод очищает коллекцию ошибок.

Класс **RsdConnection**

Объект класса **RsdConnection** создает соединение с конкретным источником данных. В одном окружении может существовать несколько соединений с разными параметрами. Каждое соединение характеризуется именем источника данных и (не обязательно) именем пользователя и паролем.

Существует также предопределенный объект класса **RsdConnection** с именем **RslDefCon**.

В качестве строки соединения допустимо указать имя источника данных либо передать полную спецификацию в виде: **"DSN=dsnname; USER ID=username; PASSWORD=userpasswd"**.

Пример.

```
Import RSD;
Env = RsdEnvironment ( "RDRvO" ); /* явно создается
окружение для драйвера ODBC */
Con1 = RsdConnection ( Env, "MyDatabase" ); /* соединение в
окружении Env */
Con2 = RsdConnection ( Env, "DSN=MyDB2;USER ID=usr2" ); /*
другое соединение в окружении Env, со своими параметрами */
Con3 = RsdConnection ( "MyDatabase", "user", "secret"); /*
используется окружение по умолчанию */
Con4 = RsdConnenction ( "DSN=MyDatabase;USER
ID=user;PASSWORD=secret"); /* полная спецификация
соединения; используется предопределенное окружение */
```

Конструктор класса может быть вызван следующими способами:

RsdConnection (env, [constr], [user], [password])

RsdConnection (env, [fullconstr])

RsdConnection ([constr,] [user,] [password])

RsdConnection ([fullconstr])

При вызове конструктора используются следующие параметры:

env:RsdEnvironment – объект окружения.

constr – строка соединения, задающая имя DSN.

user – имя пользователя базы данных, осуществляющего соединение.

password – пароль пользователя, осуществляющего соединение.

fullconstr – строка вида **"dsn=database;user id=usr;password=pwd"** или **"custom=customconstr"**. Во втором случае строка **"customconstr"** не анализируется, а напрямую передается источнику данных.

Так как свойства объектов RSD устанавливаются не только в конструкторе, то для соединения можно задать другое имя пользователя следующим образом: **Con.User = "user2"**;

Чтобы установить или получить окружение для объекта **RsdConnection**, можно использовать его свойство **Environment**.

Объект **RsdConnection** осуществляет транзакции. По умолчанию соединение работает в режиме "Autocommit", то есть каждое выполнение команды, использующей это соединение, завершается сохранением изменений в базе данных. В то же время в классе **RsdConnection** предусмотрены методы **BeginTrans**, **CommitTrans** и **RollbackTrans**, с помощью которых можно явно управлять транзакциями.

Свойства класса

Для класса **RsdConnection** реализованы следующие свойства:

Environment – имя окружения ([RWJRO](#)¹²⁷).

ConString – строка соединения ([RWJRO](#)¹²⁷).

User – имя пользователя, осуществляющего соединение ([RWJRO](#)¹²⁷).

Password – пароль пользователя, осуществляющего соединение ([RWJRO](#)¹²⁷).

Методы класса

Для класса **RsdConnection** реализованы следующие методы:

Open () – метод открывает соединение.

Close () – метод закрывает открытое соединение.

BeginTrans () – метод начинает транзакцию.

CommitTrans () – метод завершает транзакцию с сохранением изменений в базе данных.

IsInTrans () – метод возвращает **TRUE**, если вызывается внутри транзакции, **FALSE** – в противном случае.

RollbackTrans () – метод выполняет "откат" транзакции.

Класс RsdCommand

Объект класса **RsdCommand** представляет собой SQL-запрос к источнику данных. Этот объект инициализируется соединением либо текстом команды и выполняется методом **Execute**. К одному соединению может быть подключено несколько объектов команды **RsdCommand**.

Конструктор класса может быть вызван одним из следующих способов:

RsdCommand ([cmdText:String], [cmdType:Integer], [cursorType:Integer])

**RsdCommand ([con:Variant], [cmdText:String], [cmdType:Integer],
[cursorType:Integer])**

RsdCommand ([constring|fullconstring:String], [cmdText:String], [cmdType:Integer], [cursorType:Integer])

При вызове конструктора используются следующие параметры:

con – соединение.

constring – строка соединения, задающая имя DSN.

cmdText – текст команды.

cmdType – тип команды, передаваемой в параметре **cmdText**. Тип команды задается одной из следующих констант:

- **RsdCmdStoreProc** – в параметре **cmdText** передается имя хранимой процедуры.
- **RsdCmdTable** – в параметре **cmdText** передается имя таблицы базы данных.
- **RsdCmdText** – в параметре **cmdText** передается текст SQL-запроса.

cursorType – тип курсора, который будет создан при выполнении команды. Тип курсора задается одной из констант:

- **RSDVAL_STATIC** – статический курсор, который загружает данные, полученные при выполнении команды, в область памяти целиком.
- **RSDVAL_FORWARD_ONLY** – курсор удерживает только одну запись и поддерживает перемещение вперед по набору данных. Этот тип курсора рекомендуется использовать, если запрос создается с целью простого чтения данных без необходимости их дальнейших изменений.

fullconstr – строка вида `"dsn=database; user id=usr; password=pwd"` или `"custom=customconstr"`. Во втором случае строка `"customconstr"` не анализируется, а напрямую передается источнику данных.

В целях оптимизации команда производит локальное кэширование данных, им можно управлять с помощью свойства **BlockSize**.

Для задания параметров команды имеется коллекция объектов [RsdParameter](#)¹³⁸. Доступ к параметрам команды осуществляется по имени или по номеру (первый параметр в коллекции имеет номер 0).

Пример.

```
Import RSD;

. . . /* здесь определен объект соединения Con */

Cmd1 = RsdCommand( Con, "INSERT INTO clients(id, name)
VALUES(1,'Петров') " ); /* простая команда,
использующая соединение Con */
Cmd2 = RsdCommand( Con, "INSERT INTO clients(id, name)
VALUES(?,?) " ); /* параметризованная команда,
тоже использующая соединение Con. Для ее успешного
выполнения надо задать значения параметров */
Cmd2.AddParam("id", RSDBP_IN, 174 );
Cmd2.AddParam("name", RSDBP_IN, 'Петров-Сидоров' );
```

```

Cmd2.Execute; /* выполнение команды */

/* демонстрация различных способов задания параметров */
Cmd2.Value("id") = 175; /* использование индексированного
свойства Value */
Cmd2.Param(1).Value = 'Сидоров'; /* использование коллекции
параметров; можно использовать альтернативный способ
Cmd2.Param("name").Value = '...' */
Cmd2.Execute; /* выполнение команды уже с другими значениями
параметров */

Cmd3 = RsdCommand( "MyDb2", "UPDATE clients SET id=id+1000
WHERE id<100" ); /* задание строки соединения для
команды. Будет создан отдельный объект класса RsdConnection
в предопределенном окружении */

Cmd4 = RsdCommand( "SELECT * FROM
clients" ); /*использование подключения по умолчанию */
Cmd4.CmdText = "SELECT name FROM employees"; /* изменяем
текст команды */
Cmd1.Connection = RslDefCon; /* теперь Cmd1 использует
предопределенное соединение */

```

Свойства класса

Для класса **RsdCommand** реализованы следующие свойства:

Connection – соединение ([RW|RO](#)¹²⁷); тип Variant.

CmdText – текст команды; тип String. Свойство аналогично одноименному параметру конструктора класса ([RW|RO](#)¹²⁷).

ParamCount – количество параметров команды ([RO](#)¹²⁷); тип Integer.

Param:RsdParameter (index|name:Integer, String) – параметр команды, заданный индексом index или именем name ([RO](#)¹²⁷); тип Variant.

Value (index|name:Integer, String) – свойство, обеспечивающее непосредственный доступ к значению параметра, заданного индексом index или именем name ([RW](#)¹²⁷); тип Variant.

CursorType – тип курсора; тип Variant. Свойство аналогично одноименному параметру конструктора класса ([RW|RO](#)¹²⁷).

BlockSize – размер блока данных, которое команда прочитывает за одно обращение к серверу ([RW|RO](#)¹²⁷); тип Integer.

NullConversion – признак необходимости конвертирования в NULL специальных значений, используемых в базе данных ИБС RS-Bank (chr(1), '01.01.01'); тип Bool.

Методы класса

Для класса **RsdCommand** реализованы следующие методы:

Execute ([parm1:Variant [, parm2:Variant]...]) – метод выполняет команду с именованными параметрами **param1**, **param2** и т.д.

- **name** – наименование параметра;
- **dir** – характеристика параметра, которая задается одной из констант.
- **RSDBP_IN** – входящий параметр;
- **RSDBP_OUT** – параметр может использоваться для возвращения значения;
- **RSDBP_IN_OUT** – параметр может использоваться как входящий параметр, так и для возвращения значения. В случае возврата строкового значения длина этого значения определяется значением параметра **val**:
 - Если в качестве значения параметра **val** указана строка, длина которой превышает 74 символа, то метод может вернуть значение, превышающее длину значения параметра **val**. при превышении допустимой длины на экран выдается сообщение об ошибке.
 - В остальных случаях максимальная длина возвращаемой строки равна 74 символа.

В этом случае можно вернуть строку, длиной до 100 символов.

- **RSDBP_RETVAL** – параметр используется для возвращения значения хранимой процедуры, указанной в SQL-запросе.
- **val** – начальное значение параметра. Если параметр **dir** принимает значения **RSDBP_RETVAL** или **RSDBP_OUT**, то в параметре **val** должен задаваться тип возвращаемого значения (одна из констант **V_***).
- **len** – длина строкового параметра. Значение параметра может быть указано в случае, если параметр **dir** принимает значения **RSDBP_RETVAL** или **RSDBP_OUT**.

Для строковых параметров также может быть указана длина буфера.

```
cmd.addParam("p2", RSDBP RETVAL, V STRING, 100)
```

RefreshParams () — для команды с типом *RsdCmdStoredProc* метод заполняет коллекцию параметров команды в соответствии с аргументами хранимой процедуры.

Close – метод закрывает команду.

Класс **RsdRecordset**

Если в результате выполнения команды создается набор записей, возможен навигационный доступ к нему через объект **RsdRecordset**.

При создании объекта **RsdRecordset** имеется возможность непосредственно задать подключение и текст команды, в этом случае объект команды создается неявно. Также вместо подключения можно задать строку соединения. Таким образом, конструктор класса может быть вызван одним из следующих способов:

RsdRecordset ([cmd], [CursorLocation], [CursorType])

RsdRecordset ([cmdText], [CursorLocation], [CursorType])

RsdRecordset ([constring|fullconstring], [cmdText], [CursorLocation], [CursorType])

Конструктор вызывается с параметрами:

cmd:RsdCommand – объект класса [RsdCommand](#)¹²⁹. Параметр передается по ссылке.

cmdText – текст команды. Параметр аналогичен одноименному параметру конструктора класса [RsdCommand](#)¹²⁹.

constring – строка соединения, задающая имя DNS.

fullconstring – строка вида `"dsn=database; user id=usr; password=pwd"` или `"custom=customconstr"`. Во втором случае строка `"customconstr"` не анализируется, а напрямую передается источнику данных.

CursorLocation – местоположение курсора, которое задается одной из следующих констант:

- **RSDVAL_SERVER** – курсор создается на сервере.
- **RSDVAL_CLIENT** – курсор создается в памяти клиентского приложения.
- **RSDVAL_CLIENT_IF_NEEDED** – если сервер поддерживает серверные курсоры, то курсор создается на сервере, если нет, то в памяти клиентского приложения.

CursorType – тип курсора. Данный параметр аналогичен одноименному параметру конструктора класса [RsdCommand](#)¹²⁹.

Пример.

```
Import Rsd;
/* здесь определяются объекты соединения Con и команды Cmd1,
Cmd2 */
Rs1 = RsdRecordset( Cmd1 ); /* создаем объект для получения
набора записей из команды Cmd1 */
Rs2 = RsdRecordset( Cmd1 ); /* еще один набор,
подключенный к команде Cmd1. По объектам Rs1 и Rs2 можно
перемещаться независимо */
```

```
Rs1.Command = Cmd2; /* заново подключаемся к команде Cmd2 */
Rs3 = RsdRecordset( Con, "SELECT * FROM clients" );
/* инициализируем соединением и текстом команды */
Rs4 = RsdRecordset ( "MyDatabase", "SELECT name FROM
employees"); /* неявно создаются объекты соединения и
команды */
Rs5 = RsdRecordset( "DSN=MyDb2;USER
ID=scott;PASSWORD=tiger",
"SELECT id, name FROM clients ORDER BY name" ); /* строку
соединения можно специфицировать полностью */
```

Для доступа к значениям и атрибутам полей имеется коллекция объектов [RsdField](#)⁽¹³⁸⁾.

Для работы с набором данных объект **RsdRecordset** реализует простую навигационную модель – по данным он перемещается с помощью методов **MoveNext**, **MovePrev**, **MoveFirst**, **MoveLast**. Для получения данных используется либо коллекция полей, либо индексированное свойство **Value**.

Пример.

Простейшая процедура получения данных выглядит так:

```
Import RSD;
Rs5 = RsdRecordset( "DSN=MyDb2;USER
ID=scott;PASSWORD=tiger", "SELECT id, name FROM clients
ORDER BY name" );
while( Rs5.MoveNext ) /* итерация по всем записям */
[#####] ( Rs5.Fld(0).Value,
Rs5.Fld("name").Value ); /* используется коллекция
полей */
end;
while( Rs5.MovePrev ) /* можно перемещаться и в обратном
направлении */
[#####] ( Rs5.Value("id"),
Rs5.Value(1) );
/* используется индексированное свойство Value */
end;
```

Свойства класса

Для класса **RsdRecordset** реализованы следующие свойства:

AutoRefresh – признак автоматического обновления записи; тип **RsdCommand**.

Command – объект команды, с помощью которого получен набор данных ([RWIRO](#)⁽¹²⁷⁾); тип **RsdCommand**.

CursorLocation – местоположение курсора. Свойство аналогично одноименному параметру конструктора класса ([RWIRO](#)⁽¹²⁷⁾).

CursorType – тип курсора ([RWIRO](#)⁽¹²⁷⁾). Свойство аналогично одноименному параметру конструктора класса **RsdCommand**.

BOF – признак выхода за начало набора данных; тип **Bool**. Свойство принимает одно из значений:

- **TRUE** – признак установлен.
- **FALSE** – признак не установлен.

Свойство доступно только для чтения.

Bookmark – закладка текущего элемента набора данных.

DeleteCommand – пользовательская команда для удаления записи; тип `RsdCommand`.

EOF – признак достижения конца набора данных; тип `Bool`. Свойство принимает одно из значений:

- **TRUE** – признак установлен.
- **FALSE** – признак не установлен.

Свойство доступно только для чтения.

FldCount – количество полей в наборе данных ([RO](#)¹²⁷).

Fld (index|name) – свойство, обеспечивающее доступ к полю с номером `index` либо с именем `name` ([RO](#)¹²⁷); тип `RsdField`.

InsertCommand – пользовательская команда для вставки записи; тип `RsdCommand`.

InsupdCommand – пользовательская команда для получения обновленных данных из базы данных после вставки записи; тип `RsdCommand`.

MaxPages – максимальное количество страниц локального кэша, загружаемое в память ([RW|RO](#)¹²⁷).

PageSize – число записей в странице локального кэша, используемого при загрузке данных в область памяти.

RecCount – количество записей в наборе данных ([RO](#)¹²⁷). Значение, возвращаемое свойством, зависит от типа курсора: если курсор статический, то значение свойства равно количеству записей, если курсор динамический, то значение равно `-1`.

Value (index|name [, nullflag] [, type]) – свойство, обеспечивающее доступ к значению поля с номером `index` либо с именем `name` ([RW](#)¹²⁷). Свойство также имеет параметры:

- **nullflag** – переменная, в которой возвращается признак нулевого значения SQL, если поле содержит такое значение.
- **type** – код типа данных RSL (константа с префиксом `V_...`), задающий желаемый тип значения поля.

UpdateCommand – пользовательская команда для изменения записи; тип `RsdCommand`.

Методы класса

Для класса **RsdRecordset** реализованы следующие методы:

AddNew () – вставка новой записи в набор данных.

AddUserCmdParam(NameParm:String, NameField:String, VersionValue:Integer) – метод предназначен для добавления специальных параметров к свойствам

AutoRefresh, **DeleteCommand**, **InsertCommand**, **InsupdCommand**, **RefreshCommand**, **UpdateCommand**. Метод вызывается с параметрами.

- **NameParm** – имя параметра.
- **NameField** – имя поля.
- **VersionValue** – версия значения. Возможные значения параметра:
 - **RSDRVER_OLDVAL** – версия записи до изменения.
 - **RSDRVER_NEWVAL** – версия записи после изменения.

Примечание.

Совместное указание значений **RSDRVER_OLDVAL** и **RSDRVER_NEWVAL** актуально только для свойства **UpdateCommand**. Для остальных свойств можно указывать любое из двух значений.

- **RSDRVER_CHANGE** – версия измененной записи. Значение используется только для свойства **UpdateCommand**.

Пример.

```
macro DeleteData
  con = CreateMsAccessConnection();
  cmd = RsdCommand(con,"select o.OrderId, o.ShipCity,
c.CompanyName"
  "from Orders o, Customers c "
  "where o.CustomerId = c.CustomerId");
  rs = RsdRecordset (cmd,RSDVAL_CLIENT, RSDVAL_STATIC);
  // пользовательская команда для удаления записи
  cmdDel = RsdCommand (con, "delete from Orders where
OrderId = ?");
  cmdDel.AddUserCmdParam("par1", "OrderId",
RSDRVER_OLDVAL);
  rs.DeleteCommand = cmdDel;
  rs.MoveFirst;
  rs.Delete;
end;
```

CancelEdit () – отмена изменений в данных. Метод **CancelEdit** применяется для выхода из режима редактирования после неудачного выполнения команды **Update**.

Close () – метод закрывает набор данных.

Delete () – удаление записи из набора данных.

Edit () – начало редактирования текущей записи набора данных.

Move (NumRec, MoveDirect) – переход к заданному элементу набора данных. Метод вызывается с параметрами:

- **NumRec** – закладка либо номер элемента, к которому нужно перейти.
- **MoveDirect** – направление перехода, задаваемое одной из констант.

- **RELATIVE** – переход на **NumRec** элементов относительно текущего элемента.
- **ABSOLUTE** – переход на **NumRec** элементов относительно начала набора данных.
- **BOOKMARK** – переход к закладке, заданной в параметре **NumRec**.

MoveNext ():Bool – переход к следующему элементу набора данных.

MovePrev ():Bool – переход к предыдущему элементу набора данных.

MoveFirst ():Bool – переход к первому элементу набора данных.

MoveLast ():Bool – переход к последнему элементу набора данных.

Open () – метод открывает набор данных.

Update () – сохранение изменений в данных.

Методы **MoveNext**, **MovePrev**, **MoveFirst** и **MoveLast** возвращают значение **TRUE** при успешном завершении и **FALSE**, если произошла ошибка.

Если в процессе редактирования или ввода новой записи возникла ошибка, то процесс останавливается в том же состоянии (ввода или редактирования). Следовательно, следует:

- либо попытаться завершить операцию успешно, тогда выход из режима ввода или редактирования будет выполнен автоматически;
- либо явно выйти из этих режимов – для чего воспользоваться методом **CancelEdit**.

Пример.

```
Insertor.AddNew( );
Insertor.Value( "MyField" ) = "10";
if (not UpdateCatch( Insertor ))
Insertor.Value( "MyField" ) = "11";
UpdateCatch( Insertor );
// Либо отменить
// Insertor.CancelEdit;
end;
```

Класс RsdError

Класс **RsdError** предназначен для доступа к SQL-ошибкам, возникающим в процессе работы с базой данных.

Для класса реализованы свойства:

Descr – описание ошибки ([RO](#)¹²⁷).

Code – код ошибки ([RO](#)¹²⁷).

Source – тип объекта, в котором произошла ошибка ([RO](#)¹²⁷). Свойство может принимать значения:

- **Connection** – ошибка в соединении.
- **Command** – ошибка при выполнении команды.
- **Recordset** – ошибка при работе с набором данных.

SQLState – код состояния SQL, установленный после возникновения ошибки ([RO](#)¹²⁷).

NativeError – код ошибки драйвера ([RO](#)¹²⁷).

Класс RsdField

Класс **RsdField** предназначен для доступа к полям данных, полученных из базы в результате выполнения команд.

Свойства класса

Для класса реализованы следующие свойства:

Name – имя поля ([RO](#)¹²⁷).

Value – значение поля ([RO](#)¹²⁷).

BlobFilename – имя файла, в котором будет осуществляться чтение/запись содержимого поля типа BLOB ([RW](#)¹²⁷).

NullVal – значение, которое будет возвращаться в RSL в качестве нулевого, если в поле содержится нулевое значение SQL ([RW](#)¹²⁷).

Методы класса

Для класса реализованы следующие методы:

Read (Var, Count) – чтение **Count** байтов из поля типа BLOB и запись в переменную **Var**.

Read (RecHandler) – чтение значения поля типа BLOB и запись его в структуру типа **RecordHandler**.

Write (Var, Count) – запись **Count** байтов из переменной **Var** в поле типа BLOB.

Write (RecHandler) – запись содержимого структуры типа **RecHandler** в поле типа BLOB.

При работе с полями типа BLOB следует использовать либо свойство **BlobFilename**, либо методы **Read** и **Write**.

Класс RsdParameter

Класс **RsdParameter** предназначен для доступа к именованным параметрам, используемым при задании SQL-запросов в объекте класса [RsdCommand](#)¹²⁹.

Свойства класса

Для класса реализованы следующие свойства:

Direction – характеристика параметра ([RW](#)⁽¹²⁷⁾), которая задается одной из следующих констант:

- **RSDBP_IN** – входящий параметр.
- **RSDBP_OUT** – параметр может использоваться для возвращения значения.
- **RSDBP_RETVAL** – параметр используется для возвращения значения хранимой процедуры, указанной в SQL-запросе.

Name – наименование параметра ([RO](#)⁽¹²⁷⁾).

Type – тип параметра ([RO](#)⁽¹²⁷⁾).

Value – значение параметра ([RW](#)⁽¹²⁷⁾).

Методы класса

Для класса реализованы следующие методы:

SetSelfAlloc([...:Variant]):Variant – управление внутренним буфером.

SetStatusPtr(pStatus:Variant) – управление внутренним буфером.

Использование RSD в программах на языке RSL

Типичная схема использования RSD в RSL состоит из следующих шагов:

- **Создание объекта окружения** (необязательный шаг). Если окружение не создается явно, библиотека конструирует нужный объект при первой необходимости. В данной реализации окружение по умолчанию использует ODBC-драйвер.
- **Создание объекта соединения** (необязательный шаг). При отсутствии явно созданного соединения порождается объект со строкой связи RSTest.
- **Создание объекта команды** (необязательный шаг).
- **Создание объекта набора записей**, если в результате выполнения команды предполагается получить набор данных.

Пример.

Таким образом, минимальная программа на RSL с использованием RSD может состоять из строк:

```
import RSD;
cmd = RsdCommand("INSERT INTO T(ID, NAME) VALUES(101,
'RSLTest1')");
```

Обработка ошибок производится самими классами RSL. Фатальные ошибки приводят к аварийному останову. Успешность выполнения метода RSD-класса можно проверить с помощью коллекции ошибок объекта окружения. Также можно применить стандартную технику обработки ошибок.

Пример.

```
onerror(e)
    println( e.message );
    i = 0;
/* env - объект окружения. Создан в программе ранее или
является окружением по умолчанию: RslDefEnv */
    while( i < env.ErrorCount )
        println( env.error(i).descr );
        i = i + 1;
end;
```

Рассмотрим примеры использования библиотеки RSD. В примерах будет предполагаться, что в системе создан DSN RSTest и существует таблица, созданная с помощью запроса SQL:

```
CREATE TABLE T (ID INTEGER, NAME VARCHAR(20) )
```

Примеры.**Пример 1. Выполнение простой команды (вставка в таблицу).**

```
// Явное создание всех объектов.
import RSD;
env = RsdEnvironment( "RDDrvO", "RDDrvO.dll" ); /*
создание окружения */
con = RsdConnection( env, "RSTest" ); /*      создание
соединения */
cmd = RsdCommand( con, "INSERT INTO T
VALUES(99,'RSLTEST')" );
/* создание команды */
cmd.execute; /* выполнение команды */
/* блок обработки ошибок: */
i = 0;
while( i < env.ErrorCount )
[error № ##: #####](i,
env.error(i).descr);
i=i+1;
end;
// Краткий вариант
import RSD;
cmd = RsdCommand( "RSTest", "INSERT INTO T
VALUES(99,'RSLTEST')" );
cmd.execute;
env = cmd.connection.environment;
/* получение объекта окружения по умолчанию/
/* блок обработки ошибок: */
i = 0;
```

```

while( i<env.ErrorCount )
[error № ##: #####] (i,
env.error(i).descr);
i=i+1;
end;
/* В последующих примерах предложение import и обработка
ошибок будут опущены.*/
Пример 2. Выполнение параметризованной команды.
cmd = RsdCommand( "DSN=RSTest;USER
ID=username;PASSWORD=passwd",
"INSERT INTO T VALUES(?, 'testrs12')" );
cmd.addParam( "id" );
cmd.value("id") = 45;
/* можно было объединить в одну строку:
cmd.addParam("id") = 45 или cmd.addParam("id", NULL, 45) !!!
*/
/* эквивалентные формы: cmd.value(0)=45,
cmd.param("id").value=45,... */
cmd.execute;

Пример 3. Выполнение запроса с получением результатов (самая
краткая форма).
rs = RsdRecordset("DSN=RSTest;USER
ID=username;PASSWORD=passwd", "SELECT ID, NAME FROM T" );
while( rs.moveNext )
[id: ##### name: #####] (rs.value(0),
rs.value(1) );
/* эквивалентные формы: rs.fld(0).value, rs.value("id"), ...
*/
end;

```

Следует отметить, что при задании строки соединения в объектах **RsdCommand** и **RsdRecordset** для каждого вновь создаваемого объекта будет неявно создаваться новый объект **RsdConnection**, что может привести к снижению производительности. При неоднократном использовании одного и того же соединения лучшим выходом будет создать один объект **RsdConnection** и использовать его во всех командах.

Обработка ошибок при работе с таблицами баз данных

Все процедуры, работающие с таблицами базы данных, при возникновении критических ошибок аварийно завершают выполнение RSL-программы. В остальных случаях они возвращают статус завершения:

- **TRUE** – в случае успешного завершения;
- **FALSE** – в случае неуспешного завершения.

Если процедура завершилась неуспешно, для таблиц можно использовать процедуру [Status](#)²⁷² для получения информации об ошибке.

Пример.

```

var ercode, ertext;
if (not next (ff))
    ercode = status (ertext);

```

```
println ("Ошибка (" , icode, "): " , ertext)
end
```

Данный пример иллюстрирует вывод в стандартный выходной поток сообщения об ошибке в случае ее возникновения при выполнении процедуры **next**.

Другой способ обработки ошибок обращения к таблицам данных предполагает определение в тексте программы процедуры с именем **BtrError**. Формат вызова процедуры должен быть следующим:

```
macro BtrError (oper, kod, name)
```

Данная процедура будет вызываться неявно в случае возникновения ошибки, связанной с выполнением операций. Процедура вызывается для всех ошибок, за исключением ошибки "Конец файла" (**BEeof**).

При вызове процедуры **BtrError** в первом параметре передается код операции; во втором – код ошибки; в третьем – имя таблицы базы данных, при обработке которой произошла ошибка.

Процедура **BtrError** генерирует ряд кодов ошибок.

- 1 – "Код операции".
- 2 – "Ввод/Вывод".
- 3 – "Файл не открыт".
- 4 – "Значение ключа не найдено".
- 5 – "Значение ключа дублируется".
- 6 – "Неверный номер ключа".
- 7 – "Изменился номер ключа".
- 8 – "Неверная позиция".
- 9 – "Конец файла".
- 12 – "Файл не найден".
- 20 – "Менеджер записей не активен".
- 21 – "Мал буфер ключа".
- 22 – "Некорректная длина буфера".
- 28 – "Неверная длина записи".
- 29 – "Неверная длина ключа".
- 36 – "Менеджер не может обрабатывать транзакции".
- 37 – "Транзакция уже начата".
- 39 – "Транзакция не была начата".
- 41 – "Контекстно недопустимая операция".
- 43 – "Неверный номер записи".
- 46 – "Обновление файла, доступного для чтения".
- 59 – "Файл уже существует".
- 80 – "Запись конкурентно изменена".
- 84 – "Запись захвачена".

- 85 – "Файл захвачен".

Процедура должна возвращать значение типа **Integer**. От возвращаемого значения зависит дальнейшая работа системной процедуры, вызвавшей ошибку:

- **0** – процедура вернет FALSE, указывая на неуспешное завершение (это эквивалентно действию, выполняемому по умолчанию);
- **1** – ошибка игнорируется, процедура вернет TRUE, указывая на успешное завершение;
- **2** – процедура вернет FALSE, указывая на неуспешное завершение, и на экран будет выдано сообщение об ошибке;
- **3** – процедура вернет FALSE, указывая на неуспешное завершение, и выполнение программы будет прервано.

В RSL предусмотрена специальная конструкция **OnError** для обработки ошибок, которая позволяет не допустить аварийного завершения RSL-программы в случае возникновения во время ее выполнения ошибки.

Синтаксическая форма конструктора **OnError** имеет следующий вид:

OnError ['(ссылка на объект класса TrslError)']

<инструкции для обработки ошибок>

Если любая инструкция в теле макропроцедуры генерирует ошибку времени выполнения, аварийного завершения программы не происходит, а последовательно выполняются инструкции для обработки ошибок, указанные после ключевого слова **OnError**.

Если в обработке ошибок нужна информация о случившейся ошибке, то в качестве параметра **OnError** необходимо указать имя переменной, которая после возникновения ошибки получит ссылку на специальный объект класса **TrslError**, содержащий информацию об ошибке.

Примечание.

Переменная, которая в случае сбоя получает ссылку на объект класса **TrslError**, не должна декларироваться в теле макропроцедуры при помощи ключевого слова **var**.

Пример.

```
OnError (er)
    MsgBox      (er.Message, "|      Модуль: ", er.Module, "|
строка: ", er.Line);
```

Объект класса **TrslError** имеет свойства:

Code – код ошибки.

Message – строка, описывающая ошибку.

Module – название модуля RSL, вызвавшего ошибку.

Line – номер строки модуля, в которой произошла ошибка.

Чтобы в обработчике принудительно сгенерировать некоторую ошибку для передачи ее другому обработчику в цепочке вызова макропроцедур или обработчику RSL по умолчанию, необходимо вызвать стандартную процедуру RSL [RunError](#)²⁹⁵ без параметров.

В случае, когда необходимо получить сведения об ошибках, возникших при выполнении операции с базой данных, в качестве параметра **OnError** следует указать наименование объекта класса **TDbError**.

Объект класса **TDbError** содержит следующие свойства:

ExtMessage – расширенное описание ошибки.

Message – описание ошибки.

Oper – код операции, вызвавшей ошибку. Соответствует операциям Btrieve.

Stat – код ошибки СУБД. Соответствует кодам ошибок Btrieve.

Table – имя таблицы, над которой выполнялась операция.

Пример.

```
onError (er)
  // Printprops (er);
  if (IsEqClass ("TDbError", er.err))
    println ("Код ошибки:      ", er.err.stat);
    println ("Номер операции:   ", er.err.oper);
    println ("Имя таблицы:        ", er.err.table);
    println ("Название ошибки:    ", er.err.message);
    println ("Дополнительно:     ", er.err.extMessage);
  end
```

Обработка транзакций

Обработка транзакций в RSL распределена между макросом, написанным пользователем на языке RSL, и вызывающим модулем, написанным на языке C или C++.

Для выполнения транзакции на языке RSL служат процедуры [ProcessTrn](#)²⁷² и [LoopInTrn](#)²⁷⁴.

Внимание!

Следует отметить следующие особенности работы с процедурой [ProcessTrn](#)²⁷²:

- Процедура **ProcessTrn** всегда выполняется в конкурентном режиме.
- Макропроцедуры, выполняемые в рамках транзакции должны иметь минимальное время их выполнения. Иначе это снижает общее быстродействие системы.

Если во время выполнения транзакции файл или запись, к которым необходим доступ, оказываются захваченными другим пользователем, RSL автоматически выполняет серию повторов операции через определенные промежутки времени. Количество повторов и время задержки задаются в файле *rsreq.ini*. В нем необходимо определить следующие переменные:

- **OPREPCOUNT** – количество повторов операции в случае конкурентного захвата записей.

- **OPSLEEPTIME** – время задержки между повторами.

Действие процедур выполнения транзакции в случае захвата файлов или записей может быть изменено с помощью процедуры **LoopInTrn**. Если данная процедура вызвана перед процедурой выполнения транзакции с параметром, равным TRUE, то в случае блокировки файла или записи процедура выполнения транзакции прервет выполнение транзакции и выполнит ее заново. Количество повторов и время задержки между повторами задается при помощи параметров **REPCOUNT** и **SLEEPTIME**.

Вложенные транзакции не поддерживаются.

Пример.

```
/* В этом примере в транзакции обрабатывается справочник
клиентов. Если отсутствует хотя бы один клиент, выполняется
откат транзакции */
file acc ("account.dbt");
file cl ("client.dbt") write;
if (ProcessTrn (NULL,"MyTrn",cl,acc))
    println ("Transaction OK !")
else
    println ("Transaction Aborted")
end;
macro UpdateClient (client)
    cl.Client = client;
    if (getEQ (cl))
        cl.szShortName = cl.Name_Client;
    else
        println ("Клиент ",client," отсутствует в справочнике
!");
        AbortTrn
    end
end;
macro MyTrn
    while (next (acc))
        UpdateClient (acc.Client)
    end
end
/* Пример демонстрирует использование процедуры LoopInTrn:
*/
ob = TBfile ("client.dbt","w",0);
macro TrnProc
    ob.next;
    .....
    ob.update;
end;
LoopInTrn (true);
if (not ProcessTrn (0,"Test"))
    println ("Ошибка в транзакции ", status)
end;
```

В случае организации транзакции через процедуру **ProcessTrn** все операторы UPDATE, которые генерируются эмулятором, выполняются с полным перечислением всех полей обновляемой таблицы в условиях для **WHERE**. Это в свою очередь может приводить к ошибкам: в транзакции обновляемая запись по условиям для всех полей записи не может быть найдена, и это трактуется как ошибка 80 (запись конкурентно изменена). При этом в штатной транзакции RS-Bank (которая выполняется из C++ кода), в **WHERE** попадают только поля из уникального индекса, который однозначно определяет запись.

В этом случае следует выполнять организацию транзакции в RSL при помощи методов объекта [RslDefCon](#)¹²⁹: транзакция выполняется аналогично штатной из C++, а вышеописанных проблем не возникает.

В связи с этим рекомендуется все новые транзакции в RSL выполнять сразу через методы **RslDefCon**, а старый код, использующий **ProcessTrn**, по-возможности переписывать.

Пример.

```
import RsbDataSet;
macro TrnProc
  var f = TBfile("fininstr.dbt","w", 0 );
  f.rec.FIID = 113;
  if( f.getEQ() == true )
    f.rec.FIID = 114;
    if( f.Update() == false )
      println("error! update");
    else
      println("ok! update");
    end;
  end;
end;
macro RunTest()
  RslDefCon.BeginTrans();
  TrnProc();
  RslDefCon.CommitTrans();
end;
RunTest();
```

Работа с текстовыми и двоичными файлами

В настоящей главе приведено описание стандартных классов языка RSL, используемых для работы с текстовыми и двоичными файлами.

Использование класса TStream

Класс **TStream** используется для работы с двоичными файлами и RSCOM-объектами, поддерживающими интерфейс *IrsStream*.

Конструктор класса может быть вызван одним из следующих способов:

TStream(filename:String [, openmode:String])

Конструктор создаёт объект класса **TStream**, связанный с дисковым файлом, имя которого задано в параметре **filename**.

Параметры:

filename – имя файла. Если не задан абсолютный путь, файл создаётся в каталоге *TXTFILE*. Если параметр не задан или равен **null**, то конструктор создаёт объект в памяти без связи с физическим файлом.

openmode – текстовая строка, задающая режим открытия файла. Возможные значения параметра:

- **R** – открыть файл на чтение. Файл должен существовать.
- **W** – открыть файл на чтение и запись. Файл должен существовать.
- **C** – создать новый файл и открыть его на запись. Если файл уже существует, он перезаписывается.
- **A** – открыть файл для записи в него дополнительной информации. Если файл не существует, он создаётся.

Если параметр не задан, используется режим только для чтения.

TStream(strm:Object)

Конструктор создаёт объект класса **TStream**, соответствующий RSCOM-объекту с интерфейсом *IRsStream*, который задан в параметре **strm**. При помощи данного конструктора можно создать объект для работы с полями типа BLOB источников данных RSD.

Параметры:

strm – RSCOM-объект, для которого создается объект класса **TStream**.

Для класса **TStream** определен ряд [свойств](#)¹⁴⁸ и [методов](#)¹⁴⁸.

Свойства класса

Для класса **TStream** реализованы следующие свойства:

Name – имя файла, в случае если объект был сконструирован при помощи первого варианта конструктора; тип String. В противном случае значение свойства – **null**.

Stream – RSCOM-объект с интерфейсом *IRsStream*. Этот объект может быть передан RSCOM-методам, в которых используется объект с данным интерфейсом, например, источникам данных.

Методы класса

Для класса **TStream** реализованы следующие методы:

- [Write](#)¹⁴⁸.
- [Read](#)¹⁵⁰.
- [Write2](#)¹⁵⁰.
- [Read2](#)¹⁵¹.
- [WriteVal](#)¹⁵².
- [ReadVal](#)¹⁵².
- [Copy](#)¹⁵³.
- [SetPos](#)¹⁵³.
- [GetPos](#)¹⁵⁴.
- [SetSize](#)¹⁵⁴.
- [GetSize](#)¹⁵⁴.
- [Flush](#)¹⁵⁴.

Метод Write

Write(from:Variant [, tp:Integer, sz:Integer, decPoint:Integer]):Bool

Метод предназначен для записи в поток данных, указанных в параметре **from**.

Параметры:

from – переменная языка RSL, данные которой записываются в поток.

tp – тип данных, которые необходимо записать в поток. Если значение переменной имеет тип, отличный от заданного в параметре **tp**, то выполняется автоматическое преобразование типа значения в параметре **from** к типу **tp**. Если параметр **tp** не задан, то тип определяется по типу значения параметра **from**. В параметре может быть указан один из следующих типов данных: V_INTEGER, V_DOUBLE, V_NUMERIC, V_MONEY, V_STRING, V_DATE, V_TIME.

sz – размер данных для записи. Для разных типов данных этот параметр может принимать разные значения:

- **V_INTEGER** – 1, 2, 4 (установлено по умолчанию).
- **V_DOUBLE** – 4, 8 (установлено по умолчанию).
- **V_NUMERIC** – 16.
- **V_STRING** – максимальное количество символов строки для записи. По умолчанию записывается вся строка.

Для остальных типов данных размер игнорируется.

decPoint – количество знаков после точки. По умолчанию – 4. Используется только для типа данных V_NUMERIC. Поскольку этот формат не хранит позицию десятичной точки, её следует задать явно.

Возвращаемое значение:

Метод возвращает одно из значений:

- **TRUE** – при успешном выполнении метода;
- **FALSE** – в случае возникновения ошибки.

Пример.

```
/*Пример записи бинарного файла в обратном направлении, в
blob-поле.*/
import rsd;
macro AddField(ar, name, tp, sz, dec)
    ar[ar.size] = name;
    ar[ar.size] = tp;
    ar[ar.size] = sz;
    ar[ar.size] = dec;
    ar[ar.size] = 0;
end;
var ha = TArray, hd, rs, strm, cmd;
strm = TStream("blob3.jpg", "R");
AddField(ha, "blob", V_STRING, strm.getsize, 0);
hd = TRecHandler("blobread", ha);
strm.read2(hd);
cmd = RSDCommand("select t_imageid, T_FMTBLOBDATA_XXXX from
dimage_dbt where t_imageid = 1 for update");
rs = RSDRecordset(cmd, null, RSDVAL_STATIC);
rs.bloblimitsize = 1; /* должно быть задано обязательно 1,
иначе не работает */
```

```
if(rs != null)
    if(rs.MoveNext())
        rs.edit;
        rs.Fld(1).Write(hd);
        rs.update;
    else
        println("Не найдена запись с блобом!");
    end;
end;
```

Метод Read

Read(out:Variant [, tp:Integer, sz:Integer, decPoint:Integer]):Bool

Метод считывает из потока данные типа, заданного параметром *tp*.

Параметры:

out – переменная, в которой возвращается прочитанный из потока результат.

tp – тип данных, которые необходимо считать из потока. В параметре может быть указан один из следующих типов данных: V_INTEGER, V_DOUBLE, V_NUMERIC, V_MONEY, V_STRING, V_DATE, V_TIME. Если тип данных не задан, то по умолчанию используется V_INTEGER.

sz – размер данных для чтения. Для разных типов данных этот параметр может принимать разные значения. Возможные значения аналогичны значениям, представленным для метода [Write](#)¹⁴⁸.

decPoint – количество знаков после точки. По умолчанию – 4. Используется только для типа данных V_NUMERIC. Поскольку этот формат не хранит позицию десятичной точки, её следует задать явно.

Возвращаемое значение:

При достижении конца файла метод возвращает значение **false**.

Метод Write2

Write2(from:Object):Bool

Метод записывает в поток данные из объекта, заданного параметром *from*.

Примечание.

Язык RSL позволяет определять структуру полей записи объекта **TRecHandler** непосредственно в коде RSL без необходимости определять структуру в словаре базы данных.

Параметры:

from – объект класса **TRecHandler**, из которого переносятся данные в поток.

Возвращаемое значение:

Метод возвращает одно из значений:

- **TRUE** – при успешном выполнении метода;
- **FALSE** – в случае возникновения ошибки.

Пример.

/ Пример чтения данных полей типа BLOB с помощью возможностей стандартных классов RsdField, TStream.*

В примере используется таблица dimage_dbt, которую можно заполнить, например, из режима, вызванного по клавише [Alt+F2] из списка банков ИБС RS-Bank V.6, добавив любое изображение. В примере явно указан размер данных в поле типа BLOB – до 5 Mb.

*В блоке select примера обязательно должно выбираться хотя бы еще одно поле помимо поля t_fmtblobdata_xxxx. В противном случае возникает ошибка выполнения. */*

```
import oralib;
macro AddField(ar, name, tp, sz, dec)
    ar[ar.size] = name;
    ar[ar.size] = tp;
    ar[ar.size] = sz;
    ar[ar.size] = dec;
    ar[ar.size] = 0;
end;
var ha = TArray, hd, rs, strm;
AddField(ha, "blob", V_STRING, 5000000, 0);
hd = TRecHandler("blobread", ha);
rs = execSQLselect("select t_imageid, t_fmtblobdata_xxxx
from dimage_dbt where t_imageid = 1");
if (rs != null)
    if (rs.MoveNext())
        rs.Fld(1).Read(hd);
        strm = TStream("blob2.jpg", "WA");
        strm.Write2(hd);
    else
        println("Не найдена запись с блобом!");
    end;
end;
```

Метод Read2

Read2(to:Object):Bool

Метод считывает из потока данные.

Параметры:

to – объект типа **TRecHandler**, в который считываются данные из потока.

Возвращаемое значение:

Метод возвращает одно из значений:

- **TRUE** – при успешном выполнении метода;
- **FALSE** – в случае возникновения ошибки.

Метод WriteVal

WriteVal(from:Variant):Bool

Метод записывает в поток значение переменной, переданной в качестве параметра *from*. В отличие от метода **Write**, этот метод сначала записывает тип переменной RSL, а затем – значение переменной. В результате метод **ReadVal** восстанавливает из потока значение переменной без указания дополнительной информации о типе.

Методы **WriteVal** и **ReadVal** удобно использовать для сохранения в потоке значений переменных RSL и последующего их восстановления.

Поддерживаются следующие типы данных: V_INTEGER, V_DOUBLE, V_NUMERIC, V_MONEY, V_STRING, V_DATE, V_TIME.

Параметры:

from – значение переменной языка RSL, которую необходимо записать в поток.

Возвращаемое значение:

Метод возвращает одно из значений:

- **TRUE** – при успешном выполнении метода;
- **FALSE** – в случае возникновения ошибки.

Метод ReadVal

ReadVal(to:Variant):Bool

Метод считывает из потока значение переменной, сохранённой методом [WriteVal](#)¹⁵², и помещает значение в переменную, заданную параметром **to**.

Параметры:

to – переменная языка RSL, в которую помещается считанное значение.

Возвращаемое значение:

Метод возвращает одно из значений:

- **TRUE** – при успешном выполнении метода;
- **FALSE** – в случае возникновения ошибки.

Метод Copy

Copy(from:Object [, numBytes:Integer):Bool

Метод копирует данные из объекта, указанного в параметре *from*.

Параметры:

from – объект класса **TStream** или RSCOM-объект с интерфейсом *IRsStream*, из которого копируются данные в поток.

numBytes – количество байт, которое необходимо скопировать из объекта, заданного в параметре *from*. Если параметр не задан, то данные копируются, начиная с текущей позиции и до конца потока.

Возвращаемое значение:

Метод возвращает одно из значений:

- **TRUE** – при успешном выполнении метода;
- **FALSE** – в случае возникновения ошибки.

Метод SetPos

SetPos(pos:Integer [, from:Integer]):Bool

Метод устанавливает позицию в потоке.

Параметры:

pos – позиция в потоке, которую необходимо установить. Интерпретация этого параметра зависит от параметра *from*.

from – параметр определяет, как следует интерпретировать значение из параметра *pos*. Если параметр *from* равен:

- **0** – параметр *pos* задаёт смещение от начала потока;
- **1** – параметр *pos* задаёт смещение от текущей позиции в потоке;
- **2** – параметр *pos* задаёт смещение от конца потока.

Параметр *pos* может принимать отрицательные значения для *from*, равные **-1** и **-2**. Это означает смещение в сторону начала потока.

Возвращаемое значение:

Метод возвращает одно из значений:

- **TRUE** – при успешном выполнении метода;

- **FALSE** – в случае возникновения ошибки.

Метод **GetPos**

GetPos():Integer

Метод возвращает текущую позицию в потоке. Позиция представлена смещением от начала потока.

Метод **SetSize**

SetSize(sz:Integer):Bool

Метод устанавливает размер потока.

Параметры:

sz – размер потока.

Возвращаемое значение:

Метод возвращает одно из значений:

- **TRUE** – при успешном выполнении метода;
- **FALSE** – в случае возникновения ошибки.

Метод **GetSize**

GetSize():Integer

Метод возвращает размер потока.

Метод **Flush**

Flush():Bool

Метод сбрасывает на диск не сохраненные изменения.

Возвращаемое значение:

Метод возвращает одно из значений:

- **TRUE** – при успешном выполнении метода;
- **FALSE** – в случае возникновения ошибки.

Использование класса **TStreamDoc**

Класс **TStreamDoc** применяется для работы с текстовыми файлами в различных кодировках. Этот класс выполняет трансляцию содержимого файла. Он автоматически конвертирует символы между кодировкой, используемой в программе и кодировкой символов в файле.

Конструктор класса может быть вызван одним из следующих способов:

TStreamDoc (filename:String [, openmode:String , encode:String, eolType:Integer])

Создаёт объект **TStreamDoc**, связанный с дисковым файлом, заданным именем **filename**. При вызове конструктора используются параметры:

filename – имя файла. Если не задан абсолютный путь, файл создаётся в каталоге, заданном параметром настройки TXTFILE.

Если данный параметр не задан, или равен **null**, создаётся объект в памяти без связи с физическим файлом.

openmode – текстовая строка, задающая режим открытия. Если параметр не задан, используется режим только для чтения. Строка может содержать один из символов.

- **R** – открыть файл на чтение. Файл должен существовать.
- **C** – создать новый файл и открыть его на запись. Если файл уже существует, он перезаписывается.
- **W** – эквивалентно **C**.
- **A** – открыть файл на дозапись. Если файл не существует, он создаётся.

encode – текстовая строка, определяющая кодировку символов в файле. Может иметь одно из значений.

- **rsoem** – RSOEM кодировка;
- **rsansi** – RSANSI кодировка;
- **lcoem** – LCOEM кодировка;
- **lcansi** – LCANSI кодировка;
- **utf8** – UTF-8 кодировка;
- **utf16le** – UTF-16LE кодировка;
- **utf16be** – UTF-16BE кодировка.

Если этот параметр не задан, то по умолчанию в качестве кодировки символов используется кодировка, заданная в глобальных настройках прикладной системы (по умолчанию **rsoem**).

Если открывается существующий Unicode-файл на чтение или дозапись, то его кодировка определяется из сигнатуры в начале файла и значение параметра **encode** игнорируется.

eolType – стиль завершения строки при записи в текстовый документ. Параметр принимает следующие значения:

- **0** – стандарт DOS и Windows (CR LF) – значение по умолчанию;
- **1** – стандарт UNIX (LF);
- **2** – стандарт MAC (CR).

TStreamDoc (strm:Object [, encode:String, eolType:Integer])

Создаёт объект **TStreamDoc** для RSCOM-объекта с интерфейсом **IRsStream**, заданным параметром **strm**. При помощи данного конструктора, можно создать объект для работы с полями типа BLOB источников данных RSD.

Параметры **encode** и **eolType** аналогичны одноименным параметрам, указанным в вышеописанном варианте конструктора класса.

Для класса **TStreamDoc** определен ряд [свойств](#)⁽¹⁵⁶⁾ и [методов](#)⁽¹⁵⁶⁾.

Свойства класса

Для класса **TStreamDoc** реализованы следующие свойства:

Name – свойство возвращает имя файла, если объект был сконструирован при помощи первого варианта конструктора. В противном случае значение свойства – **null**.

Stream – свойство возвращает RSCOM-объект с интерфейсом **IRsStream**. Этот объект может быть передан RSCOM-методам, ожидающим объект с данным интерфейсом, например, источникам данных.

Str – свойство, доступное только для чтения. В этом свойстве сохраняется строка, которая была считана из файла последним вызовом метода [readLine](#)⁽¹⁵⁷⁾.

Методы класса

Для класса **TStreamDoc** реализованы следующие методы:

WriteLine (str:String):Bool – метод записывает в поток строку символов из параметра **str**. В конец строки символов автоматически помещается признак конца строки:

- **CR LF** – стандарт DOS и Windows (используется по умолчанию).
- **LF CR** – встречается в некоторых текстовых файлах, не является стандартным.
- **CR** – стандарт MAC.
- **LF** – стандарт UNIX.

Примечание.

Запись в файл признака конца строки откладывается и реально выполняется только в том случае, если в выходной поток будут добавляться ещё какие-либо данные.

Если в записываемой строке встречается последовательность символов конца строки, то в выходной поток всегда записывается признак конца строки, стиль которого задан параметром [eolType](#)⁽¹⁵⁵⁾ конструктора.

ReadLine (result:@String):Bool – метод считывает строку из файла и помещает результат в переменную, заданную выходным параметром **result**. Кроме того, считанная строка сохраняется в свойстве **Str** объекта. Символы признака конца строки считываются из потока, но изымаются из результирующей строки.

При достижении конца файла метод возвращает **FALSE**, иначе – **TRUE**.

Пример.

```
macro AddField (ar, name, tp, sz, dec)
  ar [ar.size] = name;
  ar [ar.size] = tp;
  ar [ar.size] = sz;
  ar [ar.size] = dec;
  ar [ar.size] = 0;
end;

ar = TArray;
AddField (ar, "field1", V_NUMERIC, 0, 10);
AddField (ar, "field2", V_INTEGER, 4, 0);
AddField (ar, "field4", V_STRING, 40, 0);
macro WriteProc
  ob = TStream ("file1.txt", "c");
  ob.write (23);
  ob.write (1, V_INTEGER, 1);
  ob.write (2, V_INTEGER, 2);
  ob.write (3, V_INTEGER, 4);
  ob.write (78.23);
  ob.write (5, V_DOUBLE, 4);
  ob.write (6, V_DOUBLE, 8);
  ob.write (7, V_DOUBLE, 10);
  ob.write (Numeric (23.78));
  ob.write (8, V_NUMERIC, null, 6);
  ob.write (date);
  ob.write (time);

  r = TRecHandler ("test", ar);
  r.rec.field1 = 12345.782345678;
  r.rec.field2 = 12345.782345678;
  r.rec.field4 = 12345.782345678;

  ob.write2 (r);
  ob.writeVal (10);
  ob.writeVal (Numeric (23.78));
  ob.writeVal ("Привет");
  ob.writeVal (date);
end;

macro ReadProc
  ob = TStream ("file1.txt", "r");
  ob.read (v, V_INTEGER);
  println (v);
```

```
ob.read (v, V_INTEGER, 1);
println (v);
ob.read (v, V_INTEGER, 2);
println (v);
ob.read (v, V_INTEGER, 4);
println (v);
    ob.read (v, V_DOUBLE);
println (v);
ob.read (v, V_DOUBLE, 4);
println (v);
ob.read (v, V_DOUBLE, 8);
println (v);
ob.read (v, V_DOUBLE, 10);
println (v);
ob.read (v, V_NUMERIC);
println (v);
ob.read (v, V_NUMERIC, null, 6);
println (v);
ob.read (v, V_DATE);
println (v);
ob.read (v, V_TIME);
println (v);

r = TRecHandler ("test", ar);
ob.read2 (r);
println (r.item("field1"):0:10);
println (r.item("field2"):0:10);
println (r.item("field4"):0:10);
while (ob.readVal (@v))
    println (v)
end;
end;

WriteProc;
ReadProc;

macro WriteProcDoc
    ob = TStreamDoc ("unistrm.txt", "c", "utf8");
    ob.writeLine ("Привет, Мир 1!");
    ob.writeLine ("");
    ob.writeLine ("Привет, Мир 2!");
    ob.writeLine ("Привет, Мир 3!");
end;

macro ReadProcDoc
    ob = TStreamDoc ("unistrm.txt", "r");
    while (ob.readLine (@v))
        println (ob.str)
    end;
end;

WriteProcDoc;
ReadProcDoc;
```

Управление файлами и каталогами

В языке RSL предусмотрены средства для манипулирования файлами и каталогами, расположенными на компьютере, в том числе и в сети двух- и трехуровневой архитектуры. К ним относятся стандартный класс [TDirList](#)¹⁶⁰, а также специальные процедуры управления файлами и каталогами:

- [CopyFile](#)²⁷⁷ – процедура создает копию файла, причем исходный файл и файл-копия могут находиться как на терминале, так и на сервере.
- [RenameFile](#)²⁷⁷ – процедура переименовывает файл, причем исходный файл может находиться как на сервере, так и на терминале, а переименованный файл должен располагаться там же, где и исходный файл.
- [RemoveFile](#)²⁷⁷ – процедура удаляет указанный файл, при этом файл может находиться как на сервере, так и на терминале.
- [ExistDir](#)²⁷⁸ – процедура выполняет проверку существования и доступности для записи каталога с указанным именем.
- [MakeDir](#)²⁷⁸ – процедура создает каталог с указанным именем, при этом каталог может быть создан как на сервере, так и на терминале.
- [RemoveDir](#)²⁷⁸ – процедура удаляет каталог с заданным именем, причем каталог может быть удален как на сервере, так и на терминале.
- [GetCurDir](#)²⁷⁹ – процедура возвращает название текущего каталога на сервере.

Для использования средств управления файлами и каталогами в сети трехуровневой архитектуры на терминале должен быть установлен модуль *rsextt.d32*.

Внимание!

Чтобы эти процедуры были доступны, в макропрограмму пользователя следует явно импортировать модуль *rsexts*, т.е. в макропрограмму необходимо включить процедуру импорта: *Import rsexts*

Если при использовании указанных средств управления в качестве имен файлов не используются абсолютные пути, то действуют следующие соглашения:

- поиск файла на сервере осуществляется в каталогах текстовых файлов;
- новый файл на сервере создается в первом каталоге из списка каталогов текстовых файлов;
- поиск файла на терминале осуществляется в каталогах, которые указаны в файле *rsextt.ini* в качестве параметра DNDIR;
- новые файлы на терминале создаются в каталоге, указанном в файле *rsextt.ini* в качестве параметра UPDIR.

Чтобы указать, что файл находится на терминале, перед именем файла необходимо указать символ "\$". Если прикладная программа запущена в сети двухуровневой архитектуры, то символ "\$" игнорируется.

Использование класса TDirList

Класс **TDirList** позволяет получать список файлов и каталогов по заданной маске на сервере или на терминале, а так же позволяет удалять файлы по маске на сервере или терминале.

TDirList ([mask:String], [attr:String], [newSizeMode:Bool]):Object

Конструктор создает новый экземпляр объекта класса. Конструктор может быть вызван с параметрами, в этом случае объект будет наполнен списком файлов, которые удовлетворяют маске поиска:

mask – строка с маской файлов для поиска;

attr – строка с атрибутами для отбора файлов в список.

newSizeMode – задает режим обработки размера файла.

Если параметр **newSizeMode** не задан, или имеет значение **FALSE**, то класс **TDirList** работает в прежнем режиме, а свойство **SizeEx** равно "0". Если же параметр **newSizeMode** задан, и имеет значение **TRUE**, то свойство **SizeEx** содержит целое количество гигабайт, а свойство **Size** – остаток от деления размера файла на 1 073 741 824. Т.е. полный размер файла вычисляется по формуле: $SizeEx * 1073741824 + Size$.

Строка атрибутов может содержать символы F и D:

- Если задан только символ F, то производится поиск файлов, удовлетворяющих маске.
- Если задан только символ D, производится поиск каталогов, удовлетворяющих маске.
- Если заданы оба символа, то производится поиск файлов удовлетворяющих маске и всех каталогов. Этот режим используется по умолчанию.

Пример.

```
ob = TDirList ("$*.*", "f");  
ob = TDirList ("$*.*", "f", TRUE);
```

Свойства класса

Свойства класса **TDirList** служат для получения информации из списка файлов или каталогов:

Count – возвращает количество элементов в списке.

FDate (ind:integer) – возвращает дату последней модификации файла с индексом **ind**.

FTime (ind:integer) – возвращает время последней модификации файла с индексом **ind**.

IsCopy (id: Integer) – возвращает **TRUE**, если файл с индексом **id** успешно скопирован. Параметр **id** принимает значения от 0 до значения, возвращаемого свойством **Count**.

IsDel (ind:integer) – возвращает **TRUE**, если файл с индексом *ind* был успешно удален методами **Remove** или **Copy**.

IsDir (ind:integer) – возвращает **TRUE**, если элемент с индексом *ind* является каталогом.

Name (ind:integer) – возвращает имя файла или каталога с индексом *ind*.

Size (ind:integer) – в зависимости от режима работы **TDirList**, возвращает одно из значений:

- Размер файла с индексом *ind* (возвращаемое значение некорректно для файлов, размер которых превышает 2 Гб) – это старый режим (**newModeSize = FALSE**).
- Остаток от деления размера файла на 1 073 741 824 – это новый режим (**newModeSize = TRUE**). В этом случае полный размер файла вычисляется по формуле: $\text{SizeEx}(i) * 1073741824 + \text{Size}(i)$.

SizeEx (ind:Integer):Integer – при работе **TDirList** в новом режиме (т.е. в конструкторе **TDirList** или в методе **List** был задан параметр **newSizeMode** и установлен в значение **TRUE**), свойство **SizeEx** содержит целое количество гигабайт размера файла (т.е. частное от деления размера файла на 1 073 741 824).

Примеры.

Пример 1.

```
Import rsexts;

...

ob = TDirList ("$*.*", "f");
ob.Sort (0);
i = 0;
while (i < ob.Count)
  if (ob.isDir (i))
    dir = "<DIR>"
  else
    dir = ""
  end;
  if (ob.isDel (i))
    del = "Del"
  else
    del = ""
  end;
  [ #####          #####          #####          #####          #####
##### ]
  (ob.name (i), ob.size (i), ob.fdate (i), ob.ftime (i) ,
  dir , del);
  i = i + 1
end
```

Пример 2:

```
import rsexts;
ob = TDirList; // ("mac\\*.mac");
ob.Copy ("mac\\*.mac", "", "dest\\", false, false, "Test
copy")ш
//ob.Copy ("mac\\*.mac", "", "$dest\\*.*", false, true,
"Test copy");
//ob.Copy ("$dest\\*.mac", "", "$dest2\\*.*", true, true,
"Test copy");
if (ob.count)
  for (i, 0, ob.count - 1)
```

```

        if (ob.isDir (i))
            dir = "<DIR>"
        else
            dir = ""
        end;
        if (ob.isDel (i))
            del = "Del"
        else
            del = ""
        end;
        if (ob.isCopy (i))
            cp = "Copied"
        else
            cp = ""
        end;
        [ #####          #####          #####
#####          #####          #####]
        (ob.name (i), ob.size (i), ob.fdate (i), ob.ftime
(i) , dir , del, cp);
    end;
else
    println ("No files")
end;

```

Методы класса

Copy (srcMask:String, attr:String, dstDir:String, [Force:Integer], move:Bool, indic:Bool, header:String):Bool

Метод выполняет копирование файлов с заданной маской, в указанный каталог.

Параметры:

srcMask – маска, в соответствии с которой производится отбор файлов для копирования. В качестве параметра могут быть указаны как локальные, так и удаленные файлы. Для того чтобы отобразить удаленные файлы, необходимо в качестве параметра передать символ "\$" в имени файла.

attr – атрибуты для отбора файлов. В настоящей версии системы параметр не используется, процедура копирования выполняется только для файлов, каталоги не копируются.

dstDir – каталог, в который требуется выполнить копирование файлов. Наименование каталога указывается одним из следующих способов:

- указывается непосредственно имя каталога с терминирующим символом "\", например, `d:\mydir\`;
- задается путь к произвольному файлу в заданном каталоге, например, `d:\mydir\*.*`.

В качестве каталога может быть указан как локальный, так и удаленный каталог. Для того чтобы задать удаленный каталог, необходимо в качестве параметра **srcMask** передать символ "\$" в имени файла.

Force – признак копирования файлов, открытых на запись (если установлено значение "1"). Если установлено значение "0" или отсутствует, то поведение функции аналогично предыдущему.

move – признак удаления исходных файлов после успешного копирования. Параметр принимает одно из значений:

- **TRUE** – удалить файлы после копирования.
- **FALSE** – не удалять файлы.

indic – признак отображения диалогового окна с индикатором процесса копирования. Параметр принимает одно из значений:

- **TRUE** – отображать диалоговое окно в процессе копирования файлов.
- **FALSE** – диалоговое окно не отображать.

Примечание.

Значение указанного параметра игнорируется, если исходный файл и каталог, в который выполняется копирование, находятся на терминале. В этом случае диалоговое окно не отображается.

header – заголовок окна с индикатором процесса копирования.

Возвращаемое значение:

Процедура наполняет объект класса **TdirList** информацией об отобранных по маске файлах и возвращает одно из значений:

- **TRUE** – если все отобранные по маске файлы были скопированы.
- **FALSE** – если хотя бы один файл не скопировался.

Количество файлов, отобранных по маске, можно получить, используя свойство **Count**¹⁶⁰. Для определения перечня файлов, которые не скопировались, следует воспользоваться свойством **IsCopy**¹⁶⁰. Если для процедуры был установлен признак удаления исходных файлов (**move**), с помощью процедуры **IsDel**¹⁶⁰ определяется, все ли файлы успешно удалены.

Пример.

```
//Пример процедуры копирования содержимого каталога с
подкаталогами:
//
// src - исходный каталог
// dst - каталог назначения
// Каталоги задаются в виде x:\mydir\
//
macro DeepCopy (src, dst)
    var count = 0;
    macro DoCopy
        var i          = 0;
        var errCount   = 0;
        var cp         = TDirList;
        MakeDir (dst);
        if (not cp.Copy (src + ".*", "f", dst + ".*"))
```

```

        while (i < cp.Count)
            if (not cp.IsCopy (i))
                errCount = errCount + 1;
                println ("Error copy file: ", src + cp.name
(i));
            end;
            i = i + 1
        end;
    end;
    count = cp.Count - errCount;
end;
DoCopy;
var ob = TDirList (src + ".*", "d");
var i = 0;
while (i < ob.count)
    if (substr (ob.name (i), 1, 1) != ".")
        count = count + DeepCopy (src + ob.name (i) + "\\ ",
dst + ob.name (i) + "\\ ");
    end;
    i = i + 1;
end;
return count;
end;
// Пример вызова
println (DeepCopy ("test\\", "dest\\"));

```

List (mask:String, [attr:String], [newSizeMode:Bool])

Метод наполняет объект списком файлов и/или каталогов, удовлетворяющих заданной маске **mask**.

Параметр **attr** принимает такие же значения, как и одноименный параметр конструктора – **attr** задает строку с атрибутами для отбора файлов в список.

Параметр **newSizeMode** задает режим обработки размера файла.

Если параметр **newSizeMode** не задан или имеет значение **FALSE**, то класс **TDirList** работает в прежнем режиме, а свойство **SizeEx** равно "0". Если же параметр **newSizeMode** задан, и имеет значение **TRUE**, то свойство **SizeEx** содержит целое количество гигабайт, а свойство **Size** – остаток от деления размера файла на 1 073 741 824. Т.е. полный размер файла вычисляется по формуле: $SizeEx * 1073741824 + Size$.

Строка атрибутов может содержать символы F и D:

- Если задан только символ F, то производится поиск только файлов, удовлетворяющих маске.
- Если задан только символ D, производится поиск только каталогов, удовлетворяющих маске.
- Если заданы оба символа, то производится поиск файлов, удовлетворяющих маске и всех каталогов. Режим используется по умолчанию.

Пример.

```
var lst:tdirlist = TDirList();
```

```
lst.List("../TxtFile\\*.out", "F");
```

Remove (mask:String [, attr:String])

Метод наполняет объект списком файлов и/или каталогов, удовлетворяющих заданной маске **mask**, и удаляет отобранные файлы. Параметр **attr** принимает такие же значения, как и одноименный параметр конструктора.

Sort ([sortBy:Integer] [, dirFirst:Bool])

Метод выполняет сортировку списка файлов. Параметр **sortBy** задает порядок сортировки и может принимать следующие значения:

- **0** – сортировка по имени файла;
- **1** – сортировка по размеру;
- **2** – сортировка по дате модификации.

Если параметр **sortBy** не задан, то выполняется сортировка по имени.

Если задан параметр **dirFirst** и он равен **TRUE**, то каталоги после сортировки будут находиться в списке перед всеми файлами.

Начиная с инструментального комплекта 18.14 (версии модуля *rsextt.d32* не ниже 3.70.051.0) работа метода **Sort** зависит от режима работы **TDirList**: если параметр **newSizeMode** задан и имеет значение **TRUE**, то при сортировке файлов по размеру используется полный размер файла.

Использование "домашних" каталогов пользователей

В прикладных программах реализовано понятие "домашних" каталогов пользователей. В этих каталогах, в дополнение к существующим каталогам, приложение осуществляет поиск персональных конфигурационных файлов. В "домашнем" каталоге также могут размещаться и персональные файлы данных.

Поиск файлов конфигурации будет происходить в следующем порядке:

- "домашний" каталог пользователя;
- текущий каталог прикладной программы;
- список каталогов, заданных в переменной окружения **RSCNFG**;
- каталог запуска прикладной программы.

Переменная **RSCNFG** в трехзвенной архитектуре может быть установлена администратором сервера приложений. В двухзвенной архитектуре эта переменная может быть установлена на компьютере пользователя.

В конфигурационных файлах можно ссылаться на имя "домашнего" каталога с помощью конструкции **%HOME%**. Имя "домашнего" каталога при этом автоматически подставляется в значениях параметров в файлах конфигурации.

При запуске прикладной программы в двухзвенной архитектуре в качестве "домашнего" каталога используется значение переменной окружения **RSHOME**. Если такая переменная не задана, то в качестве "домашнего" используется текущий каталог ".".

Поддержка "домашних" каталогов в сети трехзвенной архитектуры предусмотрена для сервера приложений версии 5.03.105 и выше. "Домашний" каталог при этом задается администратором сервера приложений при настройке сервера приложений следующим образом:

- администратор создает каталоги с именами пользователей, которые присоединяются к серверу приложений;
- администратор при настройке конфигурации сервера приложений присваивает значение параметру **ROOTDIR** (см. раздел "Утилиты настройки конфигурации сервера приложений" Руководства "Сервер приложений. Администрирование сервера приложений"). Если данный параметр не задан, то вместо него используется значение параметра **STARTDIR**.

Имя "домашнего" каталога для пользователя образуется из значения параметра **ROOTDIR** плюс имя каталога, совпадающее с сетевым именем пользователя, присоединенного к серверу приложений.

В сети трехзвенной архитектуры RSL-программам доступны устанавливаемые сервером приложений переменные окружения:

- **RSAPPROOT** – каталог, заданный в настройках сервера приложений как **ROOTDIR**.
- **RSAPDFS32** – полный путь к используемой процессом библиотеке *fs32cm.dll*.
- **RSAPPNAME** – имя сервера приложений, запустившего прикладной процесс.

Для доступа к персональным файлам пользователя предоставляется процедура [GetSysDir](#)²⁸⁰ с параметром, равным 4.

Также предусмотрено использование процедуры **GetUserName**. Эта процедура в трехзвенной архитектуре возвращает сетевое имя пользователя, присоединенного к серверу приложений, в двухзвенной архитектуре возвращает имя текущего пользователя.

Использование RSCOM-объектов из программ на языке RSL

RSCOM – распределенная объектная модель, разработанная в компании R-Style Softlab. Эта модель является удобным средством построения повторно используемых модулей. Такими модулями являются компоненты – RSCOM-серверы.

RSCOM-компоненты доступны из разных языков программирования и из различных программ. RSCOM-компоненты могут использоваться напрямую из языка RSL и C++ как в программах компании R-Style Softlab, так и в разработках сторонних разработчиков.

Технология RSCOM обеспечивает удаленный доступ к прикладным компонентам. С помощью RSCOM-компонентов можно из любого приложения, написанного на языке, поддерживающем RSCOM, получать доступ к прикладным объектам на любом сервере приложений R-Style SoftLab в сети и работать с ними.

Посредством модуля сопряжения с [ActiveX](#)⁽⁶³⁾ модули RSCOM становятся доступными везде, где доступны ActiveX-компоненты.

RSCOM позволяет свободно обмениваться объектами между программой-терминалом, предоставляющей пользовательский интерфейс, и прикладным процессом, используя при этом тот же коммуникационный канал, что используется терминалом и прикладным процессом.

Основным функциональным элементом модели является RSCOM-сервер – специального вида DLL, предоставляющая программам-клиентам RSCOM-объекты. Клиент работает с объектами посредством интерфейсов. Объект может поддерживать множество интерфейсов. RSCOM-серверы загружаются в адресное пространство процесса хоста. Хост-процессом может являться программа-терминал, любая прикладная программа, разработанная при помощи стандартного инструментария либо специальная программа-контейнер RSCOM-серверов *rcomcnt.exe*.

Клиентом RSCOM-серверов может являться код в прикладном процессе либо код в RSCOM-сервере. Для обеспечения взаимодействия клиентского кода с удаленными RSCOM-объектами необходим маршалинг интерфейсов. Инфраструктура RSCOM обеспечивает маршалинг базовых интерфейсов.

Основное назначение RSCOM-серверов – обеспечение удаленного доступа к RSCOM-объектам. Но использование RSCOM-серверов может быть полезно и для использования локальных RSCOM-объектов. В этом случае RSCOM выполняет рутинную работу по поиску и загрузке RSCOM-серверов. Если RSCOM-сервер планируется использовать только локально, то не нужно обеспечивать маршалинг его интерфейсов.

Для того чтобы RSCOM-объект мог использоваться из Object RSL, данный объект обязан поддерживать диспетчерский интерфейс. Диспетчерский интерфейс является основным интерфейсом в RSCOM.

Поддержка RSCOM в RSL реализована во встроенном модуле [RCW](#)⁽¹⁷⁰⁾.

Для создания RSCOM-объектов служит процедура [CreateObject](#)⁽¹⁷¹⁾ из модуля [RCW](#)⁽¹⁷⁶⁾. Для создания специфического коммуникационного канала данный модуль содержит специальный класс [TRslChanel](#)⁽¹⁷³⁾. Объект, созданный при помощи [CreateObject](#)⁽¹⁷¹⁾ может использоваться в RSL-коде так же, как и объект любого другого RSL-класса.

Объекты "обёртки" для RSCOM-объектов в RSL поддерживают стандартный метод `_callHost`.

```
_callHost (methodName:String, par1, par2, ...) : Variant
```

Метод вызывает процедуру или конструктор класса с именем **methodName** либо возвращает значение глобальной переменной с таким именем из экземпляра RSL, которому принадлежит обёртка. Следующие за именем процедуры (класса) параметры **par1**, **par2** и т.п. передаются без изменений в вызываемую процедуру или конструктор класса.

Возвращаемым значением является значение, которое возвращает вызываемая процедура или конструктор класса.

Пример.

```
ob = CreateObject ("rcwhost","TRcwHost","MyInst",false);

ob.GUIMode = false;
rslCls1 = ob.CreateRSLObject ("RslSrv3","TRslClass");
// Вызываем глобальный метод
println (rslCls1._callHost ("TstMac", "Параметр"));
// Устанавливаем и читаем значение глобальной переменной
rslCls1._callHost ("TstVar") = "Проба";
println (rslCls1._callHost ("TstVar"));
// Устанавливаем и читаем значение глобальной переменной
через TRcwHost.
ob.call ("TstVar") = 100;
println (ob.call ("TstVar"));
```

Описание стандартного модуля RCW

Модуль **RCW** реализован вместе с ядром интерпретатора Object RSL в библиотеке *RsScript.dll*. Перед использованием процедур и классов модуля его необходимо импортировать при помощи директивы [Import](#)⁽⁴⁴⁾.

Процедуры

Процедура CreateMarkObj

CreateMarkObj (p1:Variant, p2:Variant, ...):Object

Процедура используется для создания и заполнения объекта **MarkObject** произвольным количеством переменных RSL **p1**, **p2...**, переданных в качестве параметров. Созданный объект служит заместителем при реализации иерархических поставщиков данных или для чтения полей типа **BLOB** по запросу.

Процедура ReadMarkObj

ReadMarkObj (mark:Object, p1:@Variant, p2:@Variant, ...):Bool

Процедура позволяет определить значение переменных объекта, созданного процедурой [CreateMarkObj](#)⁽¹⁷⁰⁾.

Параметры:

mark – наименование объекта, для которого необходимо определить значения переменных.

p1, p2, ... – выходные параметры, в которых сохраняются найденные значения переменных.

Пример.

```
import rcw;  
mark = CreateMarkObj (1, Numeric(1234.567), 3.14, "Строка",  
$5);  
println (ReadMarkObj (mark, @p1, @p2, @p3, @p4, @p5));  
println (p1, " ", p2, " ", p3, " ", p4, " ", p5);
```

Процедура CreateObject

CreateObject (serverName:String, className:String [,objectName:String, (isLocal:Bool | chanel:TRslChanel), par1, par2,...]):Object

Процедура загружает необходимый RSCOM-сервер и создает в нем затребованный RSCOM-объект.

Параметры:

serverName – имя RSCOM-сервера без расширения d32.

className – имя класса RSCOM-объекта.

objectName – имя конкретного экземпляра объекта.

isLocal – признак локальной загрузки RSCOM-сервера: если значение параметра равно **TRUE**, RSCOM-сервер загружается локально, иначе удаленно.

В качестве четвертого параметра может использоваться либо **isLocal**, либо **chanel**. Если данный параметр не задан, то используется параметр **isLocal**, равный **TRUE**, то есть загрузка RSCOM-сервера выполняется локально.

chanel – специфический объект коммуникационного канала класса [TRslChanel](#)¹⁷³.

par1, par2, ... – переменный список параметров, передающихся в метод инициализации объекта.

Обязательным параметром является только имя сервера. Кроме того, обязательно должен быть задан хотя бы один из двух параметров: **className** или **objectName**. Если задан параметр **className**, но не задан **objectName**, всегда будет создаваться новый экземпляр объекта заданного класса. Если задан **objectName** и не задан **className**, объект с заданным именем должен уже существовать. В противном случае во время выполнения программы будет сгенерирована ошибка. Если заданы оба параметра, то если объект с заданным именем существует, он возвращается клиенту, иначе создается новый экземпляр.

Метод инициализации объекта – это диспетчерский метод с фиксированным идентификатором RSDISP_DEFAULT, вызываемым по уровню RSCOM_SYS_LEVEL.

Возвращаемое значение:

Процедура возвращает экземпляр созданного объекта. При невозможности создания объекта во время выполнения программы генерируется ошибка.

Процедура **GetNamedChanel**

GetNamedChanel (name:String) : Object

Процедура проверяет, существует ли канал с именем *name*.

Параметры:

name – имя канала.

Возвращаемое значение:

Процедура возвращает объект класса [*TRslChanel*](#)¹⁷³ для канала, если канал существует. В противном случае возвращается значение **NULL**.

Процедура **ClrRmtStubs**

ClrRmtStubs ()

Процедура **ClrRmtStubs** принудительно удаляет все ненужные удалённые объекты. По умолчанию освобождение ненужных удалённых объектов происходит во время следующей транзакции по коммуникационному каналу, при помощи которого эти объекты были созданы.

Процедура **PrintObject**

PrintObject (object:Object, [CaseSensitive:Bool])

Процедура выводит в стандартный выходной поток информацию об именах всех свойств и методов объекта, переданного ей в качестве параметра.

Параметры:

object – объект, имена свойств и методов которого возвращает процедура.

CaseSensitive – признак возврата информации о свойствах и методах объекта в программу в исходном виде. Параметр может принимать одно из следующих значений:

- **TRUE** – процедура возвращает информацию без изменения регистра.
- **FALSE** – в верхнем регистре.

По умолчанию, если значение параметра не задано, оно считается равным **FALSE**.

Пример.

```
import rcw;  
class TTT  
    var pl = 5;  
    macro Method1  
    end;  
end;  
ob = TTT;  
PrintObject (ob);
```

Процедура LockRcwHost

LockRcwHost (...)

Процедура позволяет принудительно увеличить счетчик ссылок для объекта хоста.

Внимание!

Если программа RSL выполняется не под управлением RSCOM объекта класса **TRcwHost**, вызов процедуры не приводит к требуемому результату.

Процедура UnlockRcwHost

UnlockRcwHost (...)

Процедура позволяет принудительно уменьшить счетчик ссылок для объекта хоста.

Внимание!

Если программа RSL выполняется не под управлением RSCOM объекта класса **TRcwHost**, вызов процедуры не приводит к требуемому результату.

Класс TRslChanel

Объекты класса **TRslChanel** предоставляют RSL-программе коммуникационный канал для связи с произвольным сервером приложений. Использование такого канала обеспечивает возможность загружать RSCOM-серверы в процессы на альтернативных серверах приложений.

Конструктор класса выглядит следующим образом:

```
TRslChanel ([server:String, protocol:Integer, termNumber:Integer,  
pipeName:String, keyPath:String, user:String, domain:String, password:String,  
port:Integer, ip:String, spx:String, NBpref:String, lana:Integer, hostApp:String,  
name:String]): Object
```

Параметры:

Все параметры конструктора являются необязательными:

server – имя сервера, к которому необходимо подключиться. Для локального сервера необходимо указать ".". Это значение используется, если параметр не задан.

protocol – код используемого протокола. Могут применяться следующие значения:

- **1** – именованный канал (named pipe);
- **2** – TCP/IP;
- **4** – SPX;
- **8** – NetBIOS.

Если этот параметр не задан, по умолчанию используется значение **1** (named pipe).

termNumber – номер используемого терминала. Номер терминала определяет имя используемого файла ключей, который задает режим шифрования и паковки передаваемых по сети пакетов.

pipeName – имя именованного канала. Задавать этот параметр необходимо только в том случае, если сервер использует имя канала, отличное от принятого по умолчанию **RsAppServ**.

keyPath – имя каталога, в котором необходимо искать файл ключей. Если параметр не задан, то ключи ищутся в текущем каталоге и каталоге запуска клиентской программы.

userName – имя пользователя, с правами которого необходимо подсоединяться к серверу приложений.

domain – домен пользователя.

password – пароль пользователя.

port – порт для протоколов TCP/IP и SPX.

ip – IP-адрес сервера для протокола TCP/IP. Если данный параметр не задан, то ip-адрес определяется по имени сервера **serverName**.

spx – SPX-адрес сервера для протокола SPX.

NBpref – трехсимвольный NetBIOS-префикс, используемый на сервере для протокола NetBIOS.

lana – номер сетевого адаптера для протокола NetBIOS.

hostApp – задает имя процесса-хоста, в адресное пространство которого загружается RSCOM-сервер. Если данный параметр не задан, используется специальный процесс *rcomcnt.exe*.

name – имя коммуникационного канала.

Созданный объект является не подключенным к серверу приложений. Для выполнения подключения можно явно вызвать метод **Connect**. Если неподключенный объект передать в процедуру [CreateObject](#)⁽¹⁷¹⁾, то подключение выполнится автоматически. После создания

объекта до его подключения к серверу приложений можно задавать значения любым свойствам объекта.

Данный класс имеет ряд [методов](#)¹⁷⁵ и [свойств](#)¹⁷⁵.

Свойства класса TRslChanel

Класс **TRslChanel** имеет свойство **Name**. Если для него задано значение, то новый канал создаётся только в том случае, если канал с таким именем ещё не существует. Иначе объект просто связывается с уже существующим каналом.

Помимо **Name**, класс обладает следующим набором свойств, каждое из которых соответствует назначению одноименного параметра конструктора:

- | | | | |
|------------|--------------|-----------|------------|
| • Protocol | • Server | • KeyPath | • PipeName |
| • Ip | • Spx | • NBPref | • Lana |
| • Port | • TermNumber | • User | • Domain |
| • Password | • HostApp | • Name | |

Методы класса TRslChanel

Для объектов класса **TRslChanel** определены следующие методы:

Connect: bool – метод выполняет соединение с сервером приложений с установленными параметрами соединения.

LoadConfig (iniFile:string): bool – метод загружает параметры соединения из ini-файла, заданного параметром *iniFile*.

Класс TRcwSite

Стандартный класс **TRcwSite** позволяет обрабатывать интерактивные сообщения от RSCOM-объекта **RcwHost**. Стандартная реализация вызывает соответствующий метод в текущем экземпляре RSL. В производном классе можно переопределить реализацию базовых методов.

Пример.

```
import rcw;
class (TRcwSite) MySite
  macro OnTrace (mes)
    MsgBox ("OnTrace in MySite: ",mes);
    _extObj.OnTrace (mes);
  end;
end;
obl = CreateObject
("rcwhost", "TRcwHost", "MyInst", false, MySite, "RslSrv2");
```

Обработка ошибок RSCOM в языке RSL

Для обработки ошибок RSCOM в RSL необходимо использовать стандартную конструкцию **onError**.

Стандартный объект ошибки, передающийся в **OnError**, содержит дополнительное поле **err** с информацией о прикладной ошибке. Данное поле заполняется объектом класса **TRsComError** при возникновении ошибки RSCOM. Объект **TRsComError** содержит свойство **count** с количеством доступных RSCOM ошибок и индексированные свойства:

Message (n) – текст n-ой ошибки RSCOM.

Code (n) – код n-ой ошибки.

Level (n) – прикладной уровень n-ой ошибки.

Пример.

```
onError (er)
  var count, i;
  if (IsEqClass ("TRsComErr", er.err))
    count = er.err.Count;
    i = 0;
    while (i < count)
      println ("*** ", er.err.Message (i), " (Code =
", er.err.Code (i),
              ", Level = ", er.err.Level (i), ")");
      i = i + 1
    end
  end;
```

Обработка сообщений от RSCOM-объектов

RSCOM-объекты могут генерировать события. Их обработка в RSL выполняется точно так же, как и обработка событий от ActiveX- или RSL-объектов, но с некоторыми отличиями. Для обработки событий необходимо присвоить объект-источник событий параметризованному свойству **EvSource** объекта класса [TrslEvHandler](#)⁶⁹:

EvSource (pref:string [,num:Integer])

Первый параметр задает строковый префикс, идентифицирующий источник событий. Второй, необязательный параметр для RSCOM-объектов задает начальный размер таблицы соответствий событий и методов обработки. При необходимости эта таблица динамически увеличивается.

При получении события от RSCOM-объекта автоматически осуществляется привязка к обработчику. Имя обработчика образуется по правилу:

<строковый префикс>+‘_’+<имя события>

Если обработчик не найден, то для событий RSCOM-объектов метод **OnUnbind** не вызывается.

Для явной установки имени обработчика применяется метод **SetHandler**.

SetHandler (pref:String, handler:String, (id:Integer | name:String) : Bool

Параметр **Pref** задаёт префикс для идентификации источника событий. Параметр **Handler** задаёт имя метода обработки события. Последний параметр может задавать или целочисленный номер события, которое будет обрабатываться, или его имя.

Примечание.

Задание имени события поддерживается только для RSCOM-объектов.

Для удаления обработчика применяется метод **RemHandler**.

RemHandler (pref:String, (id:Integer | name:String) : Bool

После установки связи между объектом-источником и обработчиком событий от источника передаются все без исключения события, даже те, для которых нет обработчика. В трехзвенной архитектуре это может привести к неоправданному увеличению трафика. Для решения проблемы можно применить метод **PinEvents**:

PinEvents (pref:String)

Метод разрешает передачу от источника событий только тех событий, для которых установлены обработчики методом **SetHandler**.

Примеры.

В примерах используется RSCOM-сервер *proba3.d32*. В нем создается объект класса *TDemoClass*. Объект поддерживает метод *GetCompName*, возвращающий имя компьютера, на котором выполняется RSCOM-сервер.

Пример 1:

```
/* В этом примере RSCOM-сервер загружается локально. */
import rcw;
ob = CreateObject ("proba3","TDemoClass");
println (ob.GetCompName);
```

Пример 2:

```
/* В этом примере RSCOM-сервер загружается удаленно. */
import rcw;
ob = CreateObject ("proba3","TDemoClass", null,false);
println (ob.GetCompName);
```

Пример 3:

```
// В этом примере RSCOM-сервер загружается удаленно
с //использованием установленного коммуникационного канала к
серверу //TECHMASTER.
import rcw;
cn = TRslChanel ("TECHMASTER");
ob = CreateObject ("proba3","TDemoClass",null,cn);
println (ob.GetCompName);
```

Пример 4:

```
/* В этом примере RSCOM-сервер загружается удаленно с
использованием установленного коммуникационного канала к
серверу TECHMASTER. Параметры соединения задаются при помощи
свойств объекта класса TRslChanel.*/
```

```

import rcw;
cn = TRslChanel;
cn.server = "TECHMASTER";
cn.protocol = 2;
if (cn.connect)
    ob = CreateObject ("proba3","TDemoClass",null,cn);
    println (ob.GetCompName);
end;

```

Пример 5:

/ В этом примере демонстрируется обработка ошибок RSCOM средствами RSL */*

```

import rcw;
var ob = CreateObject ("rcwhost","TRcwHost","MyInst",false);

ob.AddModule ("RslSrv_");
ob.Execute;
onError (er)

var count, i;
println (er.message);
if (IsEqClass ("TRsComErr", er.err))
    count = er.err.Count;
    i = 0;
    while (i < count)
        println ("*** ", er.err.Message (i)," (Code = ",
            er.err.Code (i)," Level = ",er.err.Level
        (i),")");
        i = i + 1
    end
end

```

Пример 6:

/ В этом примере демонстрируется обработка событий RSCOM-объектов */*

```

import rcw;
macro Test_OnGetCompName (p1,p2,p3)
    println ("OnGetCompName ",p1," ",p2," ",p3)
end;
ob = CreateObject ("proba3","TDemoClass",null,true);
RslEvHandler.EvSource ("Test") = ob;
println ("Local computer name is: ", ob.GetCompName);

```

Пример 7:

/ В этом примере демонстрируется обработка событий RSCOM-объектов при помощи задания обработчика методом SetHandler*/*

```

import rcw;
macro HandleOnGetCompName (p1,p2,p3)
    println ("OnGetCompName ",p1," ",p2," ",p3)
end;
ob = CreateObject ("proba3","TDemoClass",null,true);
RslEvHandler.EvSource ("Test") = ob;
RslEvHandler.SetHandler
("Test","HandleOnGetCompName","OnGetCompName");

```

```
RslEventHandler.PinEvents ("Test");  
println ("Local computer name is: ", ob.GetCompName);
```

Стандартные RSCOM-серверы

В этом разделе описываются стандартные RSCOM-серверы *rsax.d32*, *rcwhost.d32* и *rsclr.d32*. Данные серверы могут использоваться как клиентами из кода C++, так и из кода Object RSL. Загружаться они могут любым из трех возможных способов.

Данные модули входят в состав RSCOM SDK.

RSCOM-сервер rsax.d32

RSCOM-сервер **rsax.d32** предоставляет в распоряжение клиентам доступ к ActiveX-объектам. В отличие от стандартной процедуры RSL [ActiveX](#)⁶³, использование RSCOM-сервера **rsax** позволяет создавать ActiveX-объекты на удаленных компьютерах, в том числе клиентам на платформах, не поддерживающих технологии Microsoft ActiveX и COM.

Использование сервера **rsax** заключается в следующем. При помощи процедуры Object RSL [CreateObject](#)¹⁷¹ необходимо создать RSCOM-объект класса **TRsAxServer**, выполняющий роль "фабрики" ActiveX-объектов. Объект этого класса содержит [метод](#)¹⁷⁹ **CreateComObject**, позволяющий создать необходимый ActiveX-объект.

Экземпляр класса **TRsAxServer** остается в памяти до тех пор, пока не будут удалены все созданные им ActiveX-объекты. RSCOM-сервер **rsax.d32** версии 3.03.018.0 и старше всегда создает только один экземпляр класса **TRsAxServer** с именем **RsAxDefaultInst**.

Так как единственный объект класса **TRsAxServer** может обслуживать все запросы от клиентов, то при создании можно дать ему уникальное имя. При необходимости повторного создания объекта класса **TRsAxServer** будет использоваться один и тот же экземпляр, что позволит экономить оперативную память.

Конструктор класса не имеет параметров.

Методы класса TRsAxServer

Для класса **TrsAxServer** предусмотрены следующие методы:

- **CreateComObject**.

Метод **CreateComObject** может быть вызван двумя способами:

CreateComObject (progID:String [, useActive:Bool] [, evId]):Object

Первый вариант метода позволяет создать ActiveX-объект с программным идентификатором **progID**. Если второй необязательный параметр равен **TRUE**, метод пытается вернуть ссылку на существующий активный ActiveX-объект. Если такого объекта нет, создается новый экземпляр.

Параметр **evId** задает уникальный идентификатор интерфейса в виде строки.

CreateComObject (progID:String, moniker:Integer):Object

Применение указанного варианта метода позволяет создавать ActiveX-объекты с использованием моникеров. В параметре **progID** указывается строковое представление моникера. Значение параметра **moniker** всегда должно быть равно 1.

Пример.

```
/* Пример создает удаленный объект Excel.Application и
делает окно Excel видимым */
import rcw;
var          ob          =          CreateObject
("rsax", "TRsAxServer", "RsAxServer", false);
var ex = ob.CreateComObject ("Excel.Application");
ex.visible = true;
MsgBox ("Excel стартован");
```

• Cast.

Метод **Cast** может быть вызван двумя способами:

Cast (obj:Object, intf:String, typeLib:String, verHi:Integer, verLow:Integer):Object

Метод позволяет запросить интерфейс требуемого объекта.

Параметры:

obj – объект, интерфейс которого требуется запросить.

intf – уникальный идентификатор или имя запрашиваемого интерфейса. Если заданный интерфейс поддерживается объектом, метод создаёт диспетчерский объект, реализующий заданный интерфейс. Интерфейс может быть дуальным или произвольным COM интерфейсом. Важно, чтобы указанный интерфейс был совместимым с COM автоматизацией и был описан в библиотеке типов **typeLib**.

typeLib – уникальный идентификатор библиотеки.

verHi – старший номер версии библиотеки.

verLow – младший номер версии библиотеки.

Cast (obj:Object, intf:String, progId:String):Object

Метод позволяет автоматически определить уникальный идентификатор библиотеки типов и её версию по заданному программному идентификатору **progId** любого COM класса из этой библиотеки типов. Параметры **obj**, **intf** аналогичны описанным выше.

Пример.

```
/* Пример создает удаленный объект Excel.Application и
делает окно Excel видимым */
import rcw;
var objRSAX = CreateObject ("rsax", "TRsAxServer",
"RsAxServer", true);
```

```
var rpt = objRSAX.CreateComObject
("FastReport.TfrxReport");
rpt.PrepareReport(true);

// Три варианта получения одного и того же интерфейса
var expIntf = objRSAX.Cast (rpt, "{4859EE7C-4132-4B7B-
87EE-0BA8CAE4A1A3}",
    "{D3C6FB9B-9EDF-48F3-9A02-6D8320EAA9F5}", 4, 0);
var expIntf2 = objRSAX.Cast (rpt, "{4859EE7C-4132-4B7B-
87EE-0BA8CAE4A1A3}",
    "FastReport.TfrxReport");
var expIntf3 = objRSAX.Cast (rpt, "IfxBuiltinExports",
    "FastReport.TfrxReport");
expIntf.ExportToPDF ("D:\\user\\test.pdf", true, true,
true);
expIntf2.ExportToPDF ("D:\\user\\test2.pdf", true, true,
true);
```

RSCOM-сервер rcwhost.d32

RSCOM-сервер **rcwhost.d32** предоставляет функциональность интерпретатора Object RSL. Сервер позволяет:

- создать локально или удаленно экземпляр интерпретатора;
- загружать в него RSL- и DLM-модули;
- исполнять загруженные модули;
- вызывать процедуры;
- создавать экземпляры RSL-классов.

Rcwhost.d32 позволяет передавать в качестве параметров и возвращать как результат кроме скалярных типов данных так же объекты RSL-классов. Такая функциональность реализуется классом [TRcwHost](#)¹⁸².

RSCOM-сервер **rcwhost.d32** считывает свои настройки из собственного настроечного файла *rcwhost.ini*. В этом файле можно указать следующие значения параметров:

- **DATADIC** – имя словаря базы данных;
- **DATAPATH** – список каталогов с таблицами баз данных;
- **INCPATH** – список каталогов для поиска мас- и dlm-файлов;
- **WORKDIR** – каталог для записи выходных файлов;
- **DBFPATH** – список каталогов для поиска dbf-файлов;
- **TXTPATH** – список каталогов для поиска текстовых файлов;
- **LOGDIR** – каталог, в котором создаются файлы протоколирования ошибок;

- **SQLMODE** – признак используемого режима ("Yes" – SQL, "No" – Btrieve). По умолчанию принимается **Btrieve**.

В списках каталоги могут разделяться символом ';'. В именах каталогов можно применять макроподстановку %EXEDIR%, которая заменяется именем каталога запуска прикладной программы.

Файл *rcwhost.ini* ищется по правилам поиска любых других ini-файлов.

Пример файла rcehost.ini.

```

DATADIC      = bank.def
DATAPATH     = d:\dbfile
INCPATH      = %EXEDIR%\ext;%EXEDIR%\sample;%EXEDIR%\mac
WORKDIR      = %EXEDIR%\out
DBFPATH      =
TXTPATH      =

```

В языке RSL предусмотрена специальная процедура **UnderRCWHost**, с помощью которой можно определить, выполняется ли RSL-файл модулем **rcwhost.d32** или нет. Если процедура возвращает значение **True**, то файл выполняется данным модулем, в противном случае возвращается значение **False**.

Использование класса **TRCwHost**случае возвращается значение **False**.

Использование класса TRCwHost

Возможны два варианта вызова конструктора класса **TRCwHost**. Параметры обоих являются необязательными.

TRCwHost ([site:TRCwSite, module:String])

Конструктор устанавливает сайт **site** (см. метод **AttachSite**) и добавляет модуль с именем **module**.

TRCwHost ([module : String])

Конструктор добавляет модуль с именем **module**.

Для класса **TRCwHost** реализован ряд [свойств](#)¹⁸² и [методов](#)¹⁸³.

Свойства класса TRCwHost

Класс **TRCwHost** имеет следующие свойства:

Version – номер версии модуля.

SQLMode:bool – признак, определяющий, является ли данный модуль SQL-модулем. Если признак равен **TRUE**, то является.

Методы класса **TRcwHost**

AttachSite (site: TRcwSite)

Метод устанавливает для объекта **TRcwHost** объект типа **TRcwSite** или производный от него.

Объект **TRcwSite** используется для вызова команд, реализация которых зависит от приложения-хоста.

DetachSite

Удаляет ранее установленный сайт.

AddModule (moduleName:String): Bool

Метод загружает в объект интерпретатора RSL-модуль с именем **moduleName**. В качестве имени модуля может быть указано имя встроенного модуля, например, **system**, имя тас-файла или имя DLM-модуля. Указанный модуль загружается только в том случае, если он не был загружен ранее.

Если модуль был успешно добавлен, процедура возвращает **TRUE**, иначе **FALSE**.

Execute: Bool

Метод исполняет загруженные в объект интерпретатора модули. После исполнения загруженных модулей можно вызывать процедуры и создавать объекты RSL-классов при помощи методов **Call**, **RunProc**, **GetVar**, **SetVar**.

В случае успешного исполнения процедура возвращает **TRUE**, иначе **FALSE**.

Stop

Процедура деинициализирует экземпляр интерпретатора. Можно повторно вызывать метод **Execute** для инициализации RSL.

TestExist (moduleName:String) : Bool;

Процедура проверяет, загружен ли уже в память модуль с именем **moduleName**. Если загружен, возвращает **TRUE**, иначе – **FALSE**.

Call (methodName:String, par1, par2, ...) : Variant

Метод вызывает процедуру или конструктор класса с именем **methodName** либо возвращает значение глобальной переменной с таким именем. Следующие за именем процедуры (класса) параметры **par1**, **par2** и т.п. передаются без изменений в вызываемую процедуру или конструктор класса.

Возвращаемым значением является значение, которое возвращает вызываемая процедура или конструктор класса.

Пример.

В приведенном примере создается удаленный RSCOM-объект класса **TRcwHost**, представляющий экземпляр интерпретатора RSL. Объекту

присваивается имя *MyInst*. В созданный объект загружается RSL модуль *RslSrv.mas*, содержащий RSL класс *TRslClass*. Далее создаем удаленный экземпляр этого класса и работаем с его методами и свойствами. Содержимое модуля *RslSrv.mas*:

```
/*
Этот класс расположен в модуле RslSrv.mas. Мы будем работать
с удаленным экземпляром этого класса.
*/
class TRslClass
    var prop1 = "Property1";
    macro Method1 (p1,p2)
        prop1 = p1;
        return p1 + p2
    end;
end;
import rcw;
/*
Код из модуля TestRcw.mas. Демонстрирует работу с удаленным
RSL классом
*/
ob = CreateObject ("rcwhost","TRcwHost","MyInst",false);
ob.AddModule ("RslSrv");
ob.Execute;
rslCls = ob.Call ("TRslClass");
println (rslCls.prop1);
println (rslCls.Method1 (10, 20));
println (rslCls.prop1);
// Вывод программы:
Property1
30
10
```

Run (methodName:String, par1, par2, ...) Variant

Метод аналогичен методу **Call**, но не позволяет получать значения глобальных переменных. Вызывает только методы и конструкторы классов.

GetVar (varName:String) : Variant

Метод возвращает значение глобальной переменной RSL с именем **varName**.

SetVar (varName:String, value:Variant)

Метод присваивает новое значение **value** глобальной переменной RSL с именем **varName**.

CreateRSLObject (moduleName:String, className:String [, par1, par2, ...]) : Object

Метод создаёт объект RSL-класса с именем **className** из модуля **moduleName**. Модуль **moduleName** загружается только в том случае, если он ещё не был загружен. Параметры **par1**, **par2** и т.д. задают параметры, передаваемые конструктору.

Метод не требует предварительного вызова метода **Execute**. При необходимости он автоматически вызывается. Данный метод похож на аналогичные методы создания объектов из модулей *rsax.d32*, *rsclr.d32*.

Пример.

```
import rcw;  
ob = CreateObject ("rcwhost","TRcwHost","MyInst",false);
```

SetDefFile (name:String)

Процедура задаёт имя файла словаря базы данных.

SetDbtDir (name:String)

Процедура задаёт список каталогов для поиска dbt-файлов.

SetDbfDir (name:String)

Процедура задаёт список каталогов для поиска dbf-файлов.

SetTxtDir (name:String)

Процедура задаёт список каталогов для поиска текстовых файлов.

SetMacDir (name:String)

Процедура задаёт список каталогов для поиска mac- и dlm-файлов.

SetOutDir (name:String)

Процедура задаёт имя каталога для выходных файлов.

RSCOM-сервер rsclr.d32

RSCOM-сервер **rsclr.d32** предоставляет клиентам доступ к объектам классов из сборок платформы .NET. Для его использования на компьютере должен быть установлен Microsoft .NET Framework.

При помощи процедуры Object RSL [CreateObject](#)⁽¹⁷¹⁾ необходимо создать RSCOM-объект класса **TClrHost**. Объект этого класса содержит метод [CreateCLRObj](#)⁽¹⁸⁵⁾, позволяющий создать необходимый объект из сборки .NET. RSCOM-сервер **rsclr.d32** создаёт только один экземпляр класса **TClrHost** с именем **RsClrDefaultInst**.

Конструктор класса не имеет параметров.

Методы класса TClrHost

CreateCLRObj (assembly:String, className:String) : Object

Метод создаёт объект класса с именем **className** из сборки с именем **assembly**. Имя класса должно быть задано с указанием пространства имён. Приватные сборки ищутся в каталоге запуска приложения и в подкаталоге CLR.

Пример.

В примере используется .NET сборка ClassLib2.dll, написанная на C#. Исходный текст:

```
using System;
using System.Reflection;
[assembly: AssemblyVersion("1.0.*")]
namespace ClassLib2
{
    public class Class1
    {
        public Class1()
        {
        }
        public int Method1(int p1, ref short p2)
        {
            short v = p2;
            p2 = 100;
            return p1 + v;
        }
        public string Prop1
        {
            get { return fld1; }
            set { fld1 = value; }
        }
        private string fld1;
    }
}

// На RSL можно использовать объект следующим образом:
import rcw;
ob = CreateObject ("rsclr","TClrHost",null,true);
clr = ob.CreateClrObject ("ClassLib2","ClassLib2.Class1");
println (clr);

val = 2;
println (clr.Method1 (1,@val));
println (val);
clr.Prop1 = "Data for Prop1";
println (clr.Prop1);
```

RSCOM-сервер rsjvm.d32

RSCOM-сервер **rsjvm.d32** предназначен для вызова Java из кода на языке C++ или из кода на языке Object RSL. Данный RSCOM-сервер содержит RSCOM-класс [TJavaHost](#)¹⁸⁷ и инструментальный Java-модуль [rsjvm.jar](#)¹⁹², в котором реализованы инструментальные Java-классы.

Все Java-классы, содержащиеся в инструментальной библиотеке **rsjvm.jar**, находятся в пакете **ru.softlab.tools**. Файл **rsjvm.jar** должен располагаться в том же каталоге, что и модуль **rsjvm.d32**. Данный файл автоматически добавляется в путь поиска Java-классов при инициализации Java-машины.

Во время своей загрузки RSCOM-сервер *rsjvm.d32* инициализирует Java-машину, используя параметры, заданные в настроечном файле [rsjvm.ini](#)²⁰⁵. Java-машина загружается в основном потоке выполнения прикладной программы.

Внимание!

По информации Sun такой вариант загрузки может вызвать проблемы: невозможность при создании Java машины контролировать размер стека главного потока приложения. Рекомендуется загружать Java в отдельном потоке. Таким образом при возникновении проблемы, потребуется реализовать загрузку в отдельном потоке, а RSCOM прокси-объекты будут при вызове Java методов переключать поток выполнения.

Класс TJavaHost

Класс **TJavaHost** используется для доступа к Java из кода на языке C++ или из кода на языке Object RSL. Объект этого класса создаётся аналогично любым RSCOM объектам:

Пример.

```
jvm = CreateObject ("rsjvm", "TJavaHost", "GlobalJavaHost");
```

Благодаря технологии RSCOM данный класс может быть создан как локально, так и удаленно на любом сервере приложений компании R-Style Softlab. Таким образом, прикладной код может использовать одновременно несколько Java-машин.

Объект класса **TJavaHost** всегда создаётся в единственном экземпляре с именем **GlobalJavaHost**. Если при вызове **CreateObject** указано другое имя объекта, или не задано, то всегда используется **GlobalJavaHost**.

Примечание.

Рекомендуется всегда явно указывать это имя. В том случае, когда RSCOM используется из автономного Java приложения, класс **TJavaHost** реализуется в другой библиотеке *jrscom.dll*. Если имя **GlobalJavaHost** не указано, произойдёт вхолостую поиск и загрузка *rsjvm.d32* хотя она и не нужна в данном случае.

Прикладной процесс, работающий в 2-х режиме содержит два RSCOM апартмента – терминальный и серверный. RSCOM сервер *rsjvm* может быть загружен только в один апартмент для одного прикладного процесса. При попытке загрузить его в том же процессе в другой апартмент будет сгенерирована ошибка.

Методы класса

createJavaObject(clsid:String):Object

Метод предназначен для создания Java-объекта конкретного класса.

Параметры:

clsid – идентификатор Java-класса. В качестве значения параметра может использоваться любая строка, которую может интерпретировать реализованный на языке Java класс, используемый для создания Java-объектов.

Пример.

```
ob = jvm.createJavaObject("ru.softlab.tools.KubSample");
```

createJavaObject(clsname:String, methodID:String [, par1, par2, ...]):Object

Метод предназначен для создания Java-объекта конкретного класса.

Параметры:

clsname – имя Java-класса.

methodID – строковый идентификатор конструктора. Имя конструктора должно быть обязательно задано.

par1, par2,... – переменный список параметров, передающихся в метод инициализации объекта.

Пример.

```
ob = jvm.createJavaObject("java.lang.String",  
"<init>@ (Ljava/lang/String;)V", "Hello");
```

callStaticMethod(cls:String, methodID:String [, par1, par2, ...]):Variant

Метод предназначен для вызова статических методов Java-классов.

Параметры:

cls – имя Java-класса.

methodID – строковый идентификатор метода.

par1, par2,... – переменный список параметров.

Пример.

```
jvm.CallStaticMethod("ru.softlab.tools.KubSample",  
"TestIntMethod@ (IILjava/lang/String;)I", 5, 10, "Test  
string");  
jvm.CallStaticMethod("ru.softlab.tools.KubSample",  
"TestIntMethod", 5, 10, "Test string");
```

callObjectMethod(obj:Object, methodID:String [, par1, par2, ...]):Variant

Метод предназначен для вызова метода объекта Java-класса.

Параметры:

obj – объект Java-класса, метод которого необходимо вызвать.

methodID – строковый идентификатор метода.

par1, par2,... – переменный список параметров.

Пример.

```
num = jvm.CallObjectMethod(job,  
"TestNumeric@ (Ljava/math/BigDecimal;)
```

```
Ljava/math/BigDecimal;", Numeric("123.456"));  
num      =      jvm.CallObjectMethod(job,      "TestNumeric",  
Numeric("123.456"));
```

disposeJavaObject(obj:Object [, onlyDead:Bool]):Bool

Метод выполняет очистку Java-объекта, который ссылается на RSCOM-объект, заданный в параметре **obj**. В результате выполнения метода указанный RSCOM-объект освобождается. При попытке приложением использовать в Java-коде освобожденный Java-объект будет сгенерировано исключение `RscomException`.

Параметры:

obj – RSCOM-объект, на который ссылается Java-объект.

onlyDead – параметр, задающий условие освобождения RSCOM-объекта. Возможные значения параметра:

- **True** – освобождение RSCOM-объекта выполняется только, если связанный с ним Java-объект уже был удален "сборщиком мусора".
- **False** – освобождение RSCOM-объекта происходит всегда.

Если параметр не задан, то по умолчанию в качестве его значения используется **"False"**.

Возвращаемое значение:

Метод возвращает одно из значений:

- **True** – если была выполнена очистка RSCOM-объекта.
- **False** – если RSCOM-объект не был освобожден.

disposeAllJavaObjects([, onlyDead:Bool, allCount:@Integer]):Integer

Метод выполняет очистку всех Java-объектов, имеющих ссылки на RSCOM-объекты.

Параметры:

onlyDead – параметр, задающий условие освобождения RSCOM-объектов. Возможные значения параметра:

- **True** – освобождение RSCOM-объектов выполняется только, если связанные с ними соответствующие Java-объекты уже были удалены "сборщиком мусора".
- **False** (установлено по умолчанию) – освобождение RSCOM-объектов происходит всегда.

allCount – общее количество Java-объектов, связанных с RSCOM-объектом.

Возвращаемое значение:

Метод возвращает количество освобожденных объектов.

collect()

Метод запускает принудительную "сборку мусора" Java-машиной.

signatureToRSCOM(sig:String):Integer

Метод предназначен для преобразования сигнатуры из Java-типа в тип RSCOM.

Параметры:

sig – сигнатура Java-типа.

Возвращаемое значение:

Метод возвращает код типа RSCOM.

Класс TJavaObj

Java объект в C++ и RSL представляется диспетчерским RSCOM прокси-объектом класса **TJavaObj**. Этот объект позволяет вызывать любой метод Java объекта посредством реализации диспетчерского интерфейса **IDispObj**.

Пример.

```
jobj.{TestNumeric@(Ljava/math/BigDecimal;)
Ljava/math/BigDecimal;} (Numeric (1));

jobj.TestNumeric (Numeric (1));

GenRun      (jobj,      "TestNumeric@(Ljava/math/BigDecimal;)
Ljava/math/BigDecimal;"," Numeric (1));
```

Примечание.

Так как сигнатура Java метода содержит недопустимые в идентификаторе RSL символы, полное имя метода должно быть заключено в фигурные скобки.

Используя методы отражения Java, объект класса **TJavaObj** реализует получение метаинформации в формате RSCOM. При запросе типа объекта **TJavaObj** возвращает реальное имя типа Java класса с префиксом "java\". При передаче RSCOM-объектов в RSL создаётся дополнительный RSL прокси-объект для представления RSCOM объектов, как объектов RSL. При этом к имени типа добавляется префикс **RSW**. Например, имя типа Java объекта в отладчике RSL:

```
RCW\java\ru.softlab.tools.KubSample
```

Используя соглашения Java Beans **TJavaObj** позволяет обращаться к свойствам Java объектов, используя синтаксис обращения к свойствам. Например, если в Java классе имеются такие методы:

```
public String getTestProperty ()
{
    return testProperty;
}

public void setTestProperty (String testProperty)
{
    this.testProperty = testProperty;
}
```

То на RSL обращаться к свойству **testProperty** можно так:

```
prop = ob.testProperty;
ob.testProperty = "Новое значение";
```

Строковый идентификатор метода

Для указания необходимого метода Java объекта используется строковый идентификатор метода. Полный идентификатор метода состоит из имени Java метода и его сигнатуры в формате *JNI*. Имя и сигнатура разделяются символом "@".

Пример.

```
Method1@(Ljava/lang/String;I)Ljava/lang/String;"
```

Данный идентификатор ссылается на Java метод:

```
String Method1 (String p1, int p2);
```

Идентификатор метода может состоять только из имени метода. В этом случае реальный метод получается с использованием механизмов отражения.

Для вызова метода через диспетчерский интерфейс необходимо по строковому идентификатору получить числовой идентификатор метода. Полученный идентификатор (ID) может быть строгим или нестрогим. Нестрогий – получается при использовании неполного идентификатора метода, когда в Java объекте имеются перегруженные методы. При вызове метода с использованием нестроного ID требуется дополнительный анализ переданных параметров. Если Java метод имеет уникальное имя, или используется полный идентификатор метода возвращается строгий ID. При использовании для вызова строгого идентификатора дополнительной работы для поиска нужного метода выполнять не требуется.

Внимание!

Имена всех Java методов чувствительны к регистру символов.

Кастомизация процесса создания Java объектов

Прикладной код может реализовать собственный класс-креатор объектов по строковому идентификатору класса. Этот класс должен имплементировать интерфейс **ObjectCreator**. Прикладной объект-креатор перед использованием регистрируется. Для этого в настройном файле *rsjvm.ini* указывается имя Java класса, содержащего статический метод *rsInitBridge*. Этот метод будет автоматически вызван во время загрузки Java машины.

Пример регистрации креатора в методе rsInitBridge:

```
public static void rsInitBridge ()
```

```
{  
RsJvm.registerCreator (new MyCustomObjectCreator ());  
}
```

Передача параметров между приложением хостом и Java кодом

Для передачи конфигурационных параметров из C++ в Java код используется поддерживаемая инструментом коллекция свойств. Эта коллекция доступна для использования начиная с инструментального комплекта 15.70. Помещать в коллекцию и получать из неё значения поименованных свойств может прикладной код на C++, RSL, код, код любого RSCOM-сервера и модули Java, загруженные в Java машину, инициализированную сервером *rsjvm.d32*. Интерфейс коллекции реализован в инструментальном классе **RsJvm** и выполнен аналогично интерфейсу коллекции системных свойств Java, реализованному классом **java.lang.System**. Описание Java методов приводится ниже. Прототипы функций для C++ находятся в файле *rsproperty.h*.

Функции C++:

- `const char* ToolsGetProperty (const char* key);`
- `bool ToolsSetProperty (const char* key, const char* value);`

Аналогичные процедуры реализованы и для RSL:

- `SysGetProperty (key:String):String`
- `SysPutProperty (key:String, value:String):Bool`

Доступ к коллекции свойств из кода RSCOM серверов осуществляется через аналогичные методы RSCOM интерфейса **ISiteObj**.

Функция получения значения свойства возвращает либо найденное значение, либо **Null**.

Для удаления свойства из коллекции необходимо вызвать метод **ToolsSetProperty** (или **SysPutProperty** для RSL) и передать в качестве value Null.

Инструментальная библиотека *rsjvm.jar*

Инструментальная библиотека *rsjvm.jar* содержит инструментальные Java классы. Все эти классы содержатся в пакете **ru.softlab.tools**. Одним из стандартных классов является класс **RsJvm**. Этот класс содержит нативные методы для получения информации из C++ приложения, метод для определения режима работы (автономный или из C++), метод регистрации класса-креатора и т.д.

Файл *rsjvm.jar* должен располагаться в том же каталоге, что и модуль *rsjvm.d32* и автоматически добавляется в путь поиска Java классов при инициализации Java машины.

Библиотека *rsjvm.jar* может использоваться и в автономных, WEB и EJB Java приложениях. В этом случае она должна быть добавлена в *classpath* Java приложений. Нативные методы в этом случае реализуются в библиотеке *jrscom.dll*. Эта dll должна находиться в системных путях поиска DLL, либо в настройках Java должен быть задан путь для поиска нативных библиотек.

Класс RsJvm

Методы класса

void registerCreator(p:IObjectCreator);

Метод регистрирует объект-креатор, используемый для создания Java объектов из C++ и RSL. Выполнение регистрации осуществляется в автоматически вызываемом методе **rsInitBridge**. Ниже представлен пример класса-креатора. В этом же классе может быть определён и метод **rsInitBridge**.

Пример.

```
package ru.softlab.tools;
import kubws.*;
public class DemoCreator implements IObjectCreator
{
    public Object createJavaObject (String clsid)
        throws InstantiationException, IllegalAccessException,
        ClassNotFoundException
    {
        if (clsid.equals("DEMOEJB"))
        {
            KubTestEJB ejb = KubTestEJBClient.CreteTestEJB ();
            return ejb;
        }
        return Class.forName(clsid).newInstance();
    }
    public static void rsInitBridge ()
    {
        RsJvm.registerCreator (new DemoCreator ());
    }
}
```

isEmbedded():boolean;

Метод возвращает **True** в том случае, когда Java код исполняется Java машиной, загруженной RSCOM-сервером *rsjvm.d32*.

static native getProperty(key:String):String;

Метод возвращает значение свойства с именем *key* из глобальной коллекции приложения хоста, либо Null, если такого свойства не задано. Имена свойств чувствительны к регистру символов. Метод нельзя вызывать в автономном режиме.

static native setProperty(key:String, val:String):String;

Метод устанавливает значение свойства с именем *key* в глобальной коллекции приложения хоста. Возвращается предыдущее значение этого свойства или Null, если такого свойства ещё не было. Имена свойств чувствительны к регистру символов. Метод нельзя вызывать в автономном режиме.

static native clearProperty(key:String):String;

Метод удаляет свойство с именем `key` из глобальной коллекции приложения хоста. Возвращается предыдущее значение этого свойства или `Null`, если такого свойства ещё не было. Имена свойств чувствительны к регистру символов. Метод нельзя вызывать в автономном режиме.

static getProperty(key:String, def:String):String;

Метод возвращает значение свойства с именем `key` из глобальной коллекции приложения хоста, либо значение параметра **def**, если такого свойства не задано. Имена свойств чувствительны к регистру символов. Метод можно вызывать в автономном режиме. В этом случае метод всегда возвращает значение параметра **def**.

static Object createJavaObject(clsid:String) throws InstantiationException, IllegalAccessException, ClassNotFoundException;

Метод создаёт Java объект заданный строковым идентификатором с использованием заданного пользователем объекта-креатора. Этот метод вызывается native кодом для создания Java объектов.

static void setLogFile(filename:String);

Метод вызывается из C++ при инициализации Java машины для задания перенаправления стандартного вывода и стандартного вывода ошибок в файл с указанным именем. Имя файла задаётся в настроечном файле *rsjvm.ini* в параметре **LOGFILE**.

static void collect ()

Метод вызывается из C++ реализацией соответствующего метода `collect` RSCOM-объекта класса **TJavaHost**.

static Class findJavaClass(clsid:String);

static Class findJavaClass2(clsid:String);

Методы служат для использования из C++ и RSL кода, для поиска Java класса, заданного полным именем класса. Если класс не найден **findJavaClass** возвращает `Null`, **findJavaClass2** генерирует ошибку.

Пример.

```
var cls = jvm.callStaticMethod ("ru.softlab.tools.RsJvm",
    "findJavaClass@ (Ljava/lang/String;) Ljava/lang/Class;",
    "ru.softlab.rsfm.UserController");
```

static String getSignatureFor(cls:Class)

Метод возвращает сигнатуру типа в формате JNI для Java типа `cls`.

static PropertyDescriptor[] getProperties(cls:Class)

Метод возвращает коллекцию свойств определённых в Java типе `cls`.

Класс RSOject

Объекты класса ***RSOject*** являются представителями RSCOM-объектов в Java. Объекты этого класса создаются автоматически при передаче RSCOM-объектов в качестве параметров методам Java объектов, но могут быть созданы непосредственно в коде на Java с автоматическим созданием требуемых RSCOM-объектов. При использовании RSCOM-объектов в Java необходимо соблюдать [определённые правила](#)²⁰². После того как объект не нужен в Java коде, для него вызывается метод ***dispose***, чтобы связанный RSCOM-объект мог быть освобождён.

RSOject(srvDll:String, clsName:String, objName:String) throws RscomException

Конструктор объекта. Позволяет из кода на Java создать RSCOM-объект из RSCOM-сервера ***srvDll***, класса ***clsName*** с именем ***objName***. Когда объект не нужен в Java коде, для него вызывается метод ***dispose***.

RSOject(srvDll:String, clsName:String, objName:String, con:Connection) throws RscomException

Конструктор объекта. Позволяет из кода на Java создать RSCOM-объект из RSCOM-сервера ***srvDll***, класса ***clsName*** с именем ***objName***, расположенный на Сервере приложений, соединение с которым задаётся параметром ***con***. Когда объект не нужен в Java коде, для него вызывается метод ***dispose***. При этом если отсутствуют ссылки на используемый коммуникационный канал, то он закрывается.

Методы класса

void dispose()

Метод выполняет очистку объекта. После вызова метода соответствующий RSCOM-объект будет освобождён. При попытке использования данного объекта будет сгенерировано Java исключение.

Object invoke(id:int, level:int, cmdType:int, Object... args) throws RscomException

Object invoke(name:String, cmdType:int, Object... args) throws RscomException

Метод осуществляет вызов связанного RSCOM-объекта.

Параметры:

id – диспетчерский идентификатор RSCOM метода

level – уровень RSCOM метода.

cmdType – тип вызываемого метода. Возможные значения параметра:

- ***DISP_METHOD*** – вызов RSCOM-метода.
- ***DISP_PROPERTYGET*** – получение значение свойства.
- ***DISP_METHODORPROP*** – вызов метода или получение значение свойства.

- **DISP_PROPERTYPUT** – установка значения свойства.

name – имя RSCOM-метода.

args – список параметров RSCOM-метода.

Возвращаемое значение:

Методы возвращают результат вызова RSCOM-метода. При ошибке генерируется Java исключение.

RSMethod findMethod(name:String, cmdType:int) throws RscomException

RSMethod findMethod(name:String) throws RscomException

Методы используются для поиска по имени метода RSCOM-объекта.

Параметры:

name – имя RSCOM-метода.

cmdType – тип вызываемого метода. Возможные значения параметра:

- **DISP_METHOD** – вызов RSCOM-метода.
- **DISP_PROPERTYGET** – получение значение свойства.
- **DISP_METHODORPROP** – вызов метода или получение значение свойства.
- **DISP_PROPERTYPUT** – установка значения свойства.

Во втором варианте метода **findMethod** в качестве **cmdType** используется значение **DISP_METHODORPROP**.

Возвращаемое значение:

Метод возвращает объект класса **RSMethod**, который используется для вызова RSCOM-метода. Если метод с заданным именем не найден, генерируется Java исключение.

static long disposeAllRSObjects(onlyDead:boolean)

Статический метод, позволяющий освободить созданные RSCOM-объекты. Если параметр **onlyDead** равен **True**, то освобождаются только те объекты, для которых соответствующий Java объект уже был удалён "сборщиком мусора". В противном случае принудительно освобождаются все выданные в Java RSCOM-объекты.

static void setExtPath(path:String, addPath:boolean)

Статический метод, позволяющий задать список каталогов, разделённых символом ";", для поиска RSCOM серверов, загружаемых локально в вызывающее Java приложение. Это может быть полезно для автономных Java приложений, когда затруднён доступ к настроечному файлу *rscom.ini*. Если параметру **addPath** присвоено значение **True**, то

список path добавляется к уже существующему списку поиска. В противном случае, path замещает существующий список.

Пример.

```
Object result = null;
RObject obj = null;
RObject.setExtPath ("d:/user/bin;d:/user/bin2", true);
try
{
    obj = new RObject ("rcwhost", "TRcwHost", null);
    result = obj.invoke ("Version", RObject.DISP_METHODORPROP);
    System.out.println (result);
}
catch (Exception e)
{
    e.printStackTrace ();
}
finally
{
    if (obj != null)
        obj.dispose ();
}
```

Класс Connection

Данный класс позволяет создать соединение с конкретным Сервером приложений R-Style Softlab. Далее, используя это соединение, можно создавать удалённые RSCOM-объекты и работать с ними. Также, как и в случае с [RObject](#)¹⁹⁵, после того как объект класса **Connection** более не нужен в Java коде, для него требуется вызвать метод **dispose**, чтобы связанный с объектом коммуникационный канал мог быть освобождён. Реально канал будет закрыт только после освобождения всех RSCOM-объектов, созданных с применением данного канала. Для создания объектов данного класса служит статический метод **Connection.getConnection()**.

static Connection getConnection(Hashtable<String, String> ht) throws RscmException

Статический метод, позволяющий создать соединение с Сервером приложений R-Style Softlab. Параметры соединения задаются в хэш таблице ht. Все параметры задаются в виде символьных строк. Для хэш таблицы определены следующие ключи:

- **HOST_NAME** – имя хоста, с которым требуется установить соединение.
- **PORT_NUMBER** – номер порта для протоколов TCP/IP или SPX. Используется только при установленном значении USE_USER_CRED = **TRUE**.
- **PROTOCOL** – используемый протокол. Возможные значения:
 - **1** – Named Piped.
 - **2** – TCP/IP.
 - **4** – SPX.
 - **8** – NetBIOS.
- **KEY_PATH** – путь к каталогу с файлами ключей.

- **PIPE_NAME** – имя пайпа. По умолчанию **RsAppServ**. Имя используется для образования полного имени пайпа:

`\\<HOST_NAME>\pipe\<PIPE_NAME>`

Пайп используется при установленном значении PROTOCO = 1 или USE_USER_CRED = FALSE.

- **IP_ADDR** – IP-адрес. Если значение не задано, то автоматически устанавливается из HOST_NAME.
- **SPX_ADDR** – адрес SPX. Например: "0060085d6f6e".
- **NBPREF** – префикс NetBIOS имени, используемый на сервере.
- **LANA_NUM** – номер адаптера для протокола NetBIOS.
- **TERM_NUMBER** – номер терминала, который определяет имя файла ключа шифрования.
- **USER_NAME** – имя пользователя. Может быть задано в виде user@host, если используется Active Directory.
- **DOMAIN_NAME** – имя домена Windows NT.
- **PASSWORD** – пароль пользователя.
- **USE_USER_CRED** – **True** – используется имя, домен и пароль. **False** – используется интегрированная безопасность.
- **HOST_APP** – имя приложения хоста на сервере приложений. По умолчанию установлено значение **rcomcntx.exe/rcomct.exe**.
- **SBF1, SBF2, SBF3** – файлы ключей шифрования СБ РФ.
- **SBPWD** – пароль для шифрования СБ РФ.
- **USETM** – используется touch memory для аутентификации.
- **MASTER_HOST** – имя мастер-хоста для динамической балансировки.
- **MASTER_PORT** – IP-порт для мастер-сервера.
- **SKIPLIST** – список рабочих серверов, к которым не нужно подсоединяться.

Пример.

```
Connection con = null;
RObject obj = null;
try
{
    Hashtable<String, String> ht = new Hashtable ();

    ht.put (Connection.HOST_NAME,      "kubrin");
    ht.put (Connection.PROTOCOL,       "1");
    ht.put (Connection.TERM_NUMBER,    "1");
    ht.put (Connection.KEY_PATH,       "D:/user/keydir");
}
con = Connection.getConnection (ht);
obj = new RObject ("rcwhost", "TRcwHost", null, con);
.....
}
catch (Exception e)
{
    e.printStackTrace ();
}
finally
```

```
{  
  if (con != null)  
    con.dispose ();  
  if (obj != null)  
    obj.dispose ();  
}
```

Класс RSMethod

Данный класс инкапсулирует в себе RSCOM-метод и RSCOM-объект и позволяет выполнять вызов RSCOM-метода. Экземпляры данного класса возвращаются методом **findMethod** класса **RObject**.

Следующие методы используются для вызова RSCOM-метода:

Object invoke (cmdType:int, Object... args) throws RscomException

Метод позволяет явно указать тип команды вызова. Этот тип соответствует одноимённому параметру метода **invoke** класса **RObject**.

Object invoke0 (Object... args) throws RscomException

Метод в качестве **cmdType** использует значение, переданное при вызове метода **findMethod** класса **RObject**. Параметр **args** задаёт список параметров для RSCOM-метода.

Оба метода возвращают результат вызова RSCOM-метода. При ошибке генерируется Java исключение.

Класс CppWrapGen

Данный класс предназначен для генерации C++ обёрток для более простой работы с Java-объектами в RSCOM. Генерируемый C++ класс является смарт-указателем, предназначенный для работы с указателями на прокси-объекты класса **TJavaObj**. В сгенерированном C++ классе находятся функции для вызова всех методов исходного Java-класса посредством запаковки параметров и вызова метода **invoke** у контролируемого прокси-объекта.

```
boolean createWrapper (String className, String outFileName)
```

Метод выполняет генерацию C++ обёртки для Java-класса с именем **className**. Данный Java-класс должен быть определён в контексте вызова метода. Результат помещается в файл, указанный в **outFileName**.

Соответствие типов данных и передача параметров

При передаче параметров из C++ в Java и наоборот выполняется преобразование типов данных, показанное в таблице:

Java тип	RSCOM идентификатор	C++ тип	RSL тип
Boolean	RSCOM_BOOL	bool	V_BOOL
byte	RSCOM_INT	long	V_INTEGER
char	RSCOM_INT	long	V_INTEGER
short	RSCOM_SHORT	short	V_INTEGER
int (32 бита)	RSCOM_INT	long	V_INTEGER
long (64 бита)	RSCOM_INT	long (32 бита)	V_INTEGER
float	RSCOM_DOUBLE	double	V_DOUBLE
double	RSCOM_DOUBLE	double	V_DOUBLE
java.lang.String	RSCOM_STR	const char*	V_STRING
java.util.Date	RSCOM_DATE	bdate	V_DATE
java.sql.Date	RSCOM_DATE	bdate	V_DATE
java.sql.Time	RSCOM_TIME	btime	V_TIME
java.sql.Timestamp	RSCOM_DTTM	btimesta mp	V_DTTM
java.math.BigDecimal	RSCOM_NUMERIC	Numeric	V_NUMERIC
java.util.List java.util.Collection java.sql.Result	RSCOM_OBJ	IRsDataS et	V_GENOBJ (Object)
Другие объекты	RSCOM_OBJ	IDispObj	V_GENOBJ (Object)
Встроенные массивы	RSCOM_PARMAR	IParmInfo	V_GENOBJ (TArray)

Передача параметров:

При передаче параметров из RSCOM в методы Java выполняется преобразование типов, как указано в таблице выше. Если Java метод ожидает в качестве типа параметра **java.lang.Object** или объектный тип-оболочка для примитивного типа, то числовые типы данных автоматически конвертируются инструментом в аналогичные объектные типы Java Integer, Long и т.д.

Возврат значений:

Возвращаемые значения из Java методов преобразуются, как указано в таблице выше. При возврате методом Java объектного типа выполняется проверка, не является ли данный тип объектным представителем для примитивного типа. И если является – автоматически преобразует объектный тип в числовое значение RSCOM.

Передача массивов:

Для передачи массива в метод Java применяется RSCOM тип данных **RSCOM_PARMAR**. Поддерживаются многомерные массивы. В этом случае элементами в массиве IParmInfo

должны являться также значения типа RSCOM_PARMAR.

Для передачи массива из RSL необходимо использоваться стандартный класс RSL **TArray**. Свойство **marshalByVal** должно быть установлено в **True**. По умолчанию значение этого свойства установлено **False**. Для удобства использования, начиная с инструментального комплекта 15.70, добавлен специальный конструктор **TArray**, позволяющий в момент создания установить значение свойства **marshalByVal**:

```
TArray (marshalByVal:Bool, initSz:Integer, delta:Integer)
```

Если необходимо передать массив из одного элемента, то можно не создавать для этого массив в RSL или C++. Можно использовать простой параметр. Инструмент автоматически создает массив единичного размера. В случае многомерного массива будут автоматически созданы все размерности. Например, метод Java:

```
public void Test1 (p:String[]);
```

Метод может быть вызван так:

```
p = TArray (true, 2);
p [0] = "Parm1";
p [1] = "Parm2";
jobj.Test1 (p);
```

Если параметр только один:

```
jobj.Test1 ("Parm1");
```

Значения переданного массива может быть изменено в Java методе. Чтобы изменённые значения попали обратно в C++ или RSL, соответствующий параметр должен быть помечен, как выходной. Например, на RSL можно вызвать метод Test1:

```
p = TArray (true, 2);
p [0] = "Parm1";
p [1] = "Parm2";
jobj.Test1 (@p);
```

Если параметр только один:

```
parm = "Parm1";
jobj.Test1 (@parm);
```

При возврате массива через возвращаемое значения метода, не выполняется замена массива единичного размера на простое значение. При возврате же новых массивов в качестве элементов массива, переданного в метод как параметр, инструмент выполняет замену возвращаемых массивов единичного размера на простые значения.

Если последним параметром Java метода является массив, а соответствующий RSCOM параметр является простым значением, то автоматически создаётся Java массив, в который помещаются все оставшиеся RSCOM значения. Например, если есть такой Java метод с двумя параметрами:

```
public void TestIntArray (String p1, int[] p)
{
    System.out.println(p1);
    for (int i = 0; i < p.length; ++i)
        System.out.println (p [i]);
}
```

Данный метод может быть вызван таким образом:

```
ob.TestIntArray ("Hello", 1, 2, 3, 4, 5);
```

Передача объектов между RSCOM и JAVA

Инструмент позволяет передавать Java объекты в RSCOM и RSCOM-объекты в Java. Java объект представляется в RSCOM диспетчерским объектом [TJavaObj](#)¹⁹⁰. При этом Java объект может быть, как локальным объектом, так и прокси-объектом для удалённого EJB. Объект класса **TJavaObj** держит сильную ссылку на Java-объект. Время жизни объекта класса **TJavaObj** управляется подсчётом ссылок. Когда уходит последняя ссылка на объект класса **TJavaObj**, RSCOM-объект разрушается и освобождает ссылку на Java-объект. При этом Java объект не уничтожается сразу, даже при отсутствии ссылок, а по правилам Java становится доступным "сборщику мусора". Поэтому, если требуется, чтобы при уничтожении RSCOM-объекта соответствующий Java объект освободил какие-либо ресурсы (например, открытые файлы, соединения и т.д.), то необходимо выполнить вызов метода очистки перед удалением последней ссылки на RSCOM-объект.

При передаче в Java RSCOM-объектов, в коде Java создаются представители RSCOM-объектов в мире Java-объекты класса **ru.softlab.tools.RSObject**. RSCOM-объект должен быть диспетчерским (т.е. поддерживать интерфейс **IDispObj**), чтобы он мог быть передан в Java. Объекты класса [RSObject](#)¹⁹⁵ могут быть созданы и явно в Java коде для работы с требуемым RSCOM-объектом. Объекты класса **ru.softlab.tools.RSObject** являются недистанцируемыми. Они не могут быть переданы методам удалённых EJB объектов. Объект класса **RSObject** содержит метод **invoke** позволяющий вызывать диспетчерские методы RSCOM-объектов. Для удобства работы с диспетчерским интерфейсом RSCOM-объектов полезно реализовать производные от **RSObject** классы-обёртки, в которых реализовать специализированные методы для вызова. Для того, чтобы инструмент мог создавать производные от **RSObject** классы при передаче RSCOM-объекта в Java, RSCOM-объект должен сообщить инструменту имя необходимого Java класса. Для этого необходимо реализовать в RSCOM-объекте диспетчерский метод с ID **RSDISP_GETJAVACLASS** по уровню RSCOM_SYS_LEVEL. Метод должен возвращать строку с именем Java класса. При передаче объекта RSL класса, необходимо в данном RSL классе реализовать метод с предопределённым именем **GetJavaClassName**, возвращающий имя Java класса.

Пример.

```
class TRsClass
macro GetJavaClassName
return "ru.softlab.tools.ChildRSObject";
end;
...
end;
```

Объекты класса **RSObject** и производные от него содержат ссылку на RSCOM-объект. Этим объектом может быть локальный RSCOM-объект или прокси-объект для удалённого RSCOM-объекта. Сами объекты класса **RSObject** и производные от него являются Java объектами и, следовательно, подчиняются правилам Java относительно их времени жизни. Когда на эти объекты нет ссылок, они не уничтожаются немедленно, а просто становятся доступными "сборщику мусора". Время проведения сборки мусора не определено. Такое поведение не соответствует правилам RSCOM и поэтому может иметь негативные последствия. Для освобождения ресурсов Java объекта во время сборки мусора, Java объект может реализовать процедуру **finalize**. Но **RSObject** не использует данный подход по следующим соображениям:

- Время создания и разрушения объектов с **finalize** больше, чем объектов без **finalize**. Спецификация Java не рекомендует использование **finalize** без крайней на то необходимости.
- Время выполнения сборки мусора не определено.
- **finalize** может быть вызван в отдельном потоке выполнения, на что может не рассчитывать RSCOM-объект.

- Поскольку инструмент содержит коллекцию всех выданных в Java RSCOM-объектов, инструмент может автоматически выполнять освобождение RSCOM-объектов, соответствующие Java объекты которых уже были утилизированы сборщиком мусора.

Таким образом, если RSCOM-объект держит критические ресурсы, то программист должен побеспокоиться о явном освобождении RSCOM-объекта, если такой объект в коде Java больше не нужен. Аналогично Java программист должен поступать при работе с файлами или соединениями JDBC. К сожалению, в Java отсутствует стандартная языковая конструкция и стандартный интерфейс для автоматического вызова метода освобождения ресурсов, как это сделано в C#. В помощь программисту только конструкция `try finally`. Для явного освобождения RSCOM-объекта в классе **RObject** содержится метод **dispose()**; После вызова этого метода, связанный RSCOM объект освобождается, и при последующих вызове методов **RObject**, обращающихся к RSCOM объекту, генерируется Java исключение.

Пример.

```
public String TestObject (ChildRObject obj)
{
    s = obj.TestMetod();
    obj.dispose();
    return s;
}
```

В коде на C++ или RSL так же имеется возможность явно выполнить очистку Java объекта класса **RObject** и освобождение RSCOM-объекта. Для этого в RSCOM классе **TJavaHost**¹⁸⁷ предусмотрены методы: **disposeJavaObject**¹⁸⁹ и **disposeAllJavaObjects**¹⁸⁹. Эти методы позволяют освободить RSCOM-объект, как принудительно, так и только в том случае, если соответствующий Java объект уже был уничтожен "сборщиком мусора". Также из кода на RSL или C++ можно явно вызвать сборку мусора в Java при помощи вызова метода **collect** RSCOM класса **TJavaHost**. Например, чтобы явно освободить все неиспользуемые в Java RSCOM-объекты:

```
jvm = CreateObject ("rsjvm", "TJavaHost");
...
jvm.collect;
jvm.DisposeAllJavaObjects (true);
```

Освободить все связанные с Java RSCOM-объекты можно и из Java кода при помощи статического метода **RObject.disposeAllRObjects()**.

Вызов перегруженных методов

Если в Java классе имеются перегруженные методы, то выбор наиболее подходящего выполняется с использованием информации о количестве и типах параметров. Для этого для каждого перегруженного метода вычисляется условный вес его параметров.

Если тип параметра в RSCOM точно совпадает с типом параметра в Java, то условный вес такого параметра считается равным нулю. Если RSCOM тип может быть приведен к Java типу, то условным весом будет положительное число. Если хотя бы один параметр не может быть приведён к Java типу, либо задано недостающее количество параметров, такой Java метод считается неподходящим. Подходящим методом является метод с минимальным весом. Если минимальный вес имеет более чем один метод, то считается, что имеется неоднозначность. Для вызова подобных методов требуется явное задание сигнатуры. Вес методов рассчитывается по следующей таблице:

	C H R	U C H R	S H O R T	U S H O R T	I N T	U I N T	D O U B L E	N U M E R I C	B O O L	S T R	O B J	D A T E	T I M E	D T T M
Z									0					
B	0	0	4	4	4	4								
C	1	1	0	0	4	4								
S	1	1	0	0	4	4								
I	1	1	1	1	0	0								
J	1	1	1	1	1	1								
F							2							
D							0							
Boolean									1					
Byte	2	2	5	5	5	5								
Character	3	3	2	2	5	5								
Short	3	3	2	2	5	5								
Integer	3	3	3	3	2	2								
Long	3	3	3	3	3	3								
Float							3							
Double							1							
BigDecimal								0						
String										0				
Date												0		
Time													0	
Timestamp														0
Object	1 0	1 0	1 0	10	1 0	1 0	1 0	1 0	1 0	1 0	1 0	1 0	1 0	1 0

	C H R	U C H R	S H O R T	U S H O R T	I N T	U I N T	D O U B L E	N U M E R I C	B O O L	S T R	O B J	D A T E	T I M E	D T M
Other IsInstanceOf		Object									0			

Поддержка метайнформации RSCOM объектом TJavaObj

Используя механизмы отражения, RSCOM-объект [TJavaObj](#)⁽¹⁹⁰⁾ поддерживает метайнформацию. Возможно получение имени класса Java объекта и информации обо всех его public методах и свойствах. Имена методов возвращаются в виде полного строкового идентификатора в формате <name>@<signature>. Имена свойств возвращаются в виде простой строки с именем свойства, полученного согласно правил технологии Java Beans. Информация о свойствах Java объекта используется отладчиком RSL для показа и модификации значений.

Внимание!

В настоящее время не поддерживается отображение механизмов событий RSCOM и Java друг на друга.

Настроечный файл rsjvm.ini (2)

Файл *rsjvm.ini* используется для задания настроечных параметров Java-машины. Поиск этого файла выполняется согласно правилам поиска любых других ini-файлов.

В файле *rsjvm.ini* можно указать значения следующих параметров:

- **OPTION** – содержит любой параметр командной строки Java.

В файле *rsjvm.ini* может присутствовать произвольное количество параметров **OPTION**. При инициализации Java-машины в ini-файл автоматически добавляется параметр командной строки `-Xrs`, который блокирует установку обработчика сообщений, полученных от экранной консоли.

Пример.

```
OPTION = -Dhttp.proxyHost=myth.softlab.ru
```

- **PATH** – полные пути к каталогам и jar-библиотекам, используемым для поиска Java-классов.

В файле *rsjvm.ini* может присутствовать произвольное количество параметров **PATH**. Пути к каталогам и jar-библиотекам отделяются друг от друга символом ";". Значения всех заданных в ini-файле параметров **PATH** соединяются вместе и указываются при создании Java-машины в параметре командной строки `-Djava.class.path`. Кроме того, в данный параметр автоматически добавляются пути к текущему каталогу и к Java-модулю *rsjvm.jar*, а также содержимое переменной окружения **CLASSPATH**.

Пример.

```
PATH= D:/JAVA/sqljdbc_1.1/enu/sqljdbc.jar;D:/JAVA/myclasses
```

- **INITCLASS** – имя класса, содержащего статический метод **rsInitBridge**.

Метод **rsInitBridge** вызывается автоматически при инициализации Java-машины.

Пример.

```
MAINCLASS = ru.softlab.rsbank.JStartClass
```

- **JVM** – полный путь к файлу Java-машины, которую необходимо использовать.

Если параметр не задан, то используется последняя версия Java-машины, указанная в системном реестре по пути "Software\JavaSoft\Java Runtime Environment". В пути могут использоваться заместители.

- **LOGFILE** – путь к файлу, в который перенаправляются стандартные потоки *stderr* и *stdout*.

Для формирования уникального имени файла на основе идентификатора процесса в имени файла необходимо задать заместитель **%PID%**.

Пример файла rsjvm.ini.

```
OPTION = -Dhttp.proxyHost=myth.softlab.ru
PATH = D:/JAVA/sqljdbc_1.1/enu/sqljdbc.jar;D:/JAVA/myclasses
INITCLASS = ru.softlab.rsbank.JStartClass
JVM = C:\Program
Files\JDeveloper_1.3.3\jdk\jre\bin\ojvm\jvm.dll
LOGFILE = trace\java%PID%.err
```

Отладка Java кода при использовании rsjvm.d32

Так как приложением хостом для Java является прикладная программа, загрузившая RSCOM сервер *rsjvm.d32*, то для отладки Java кода необходимо использовать технику удалённой отладки. Для этого необходимо указать модулю *rsjvm.d32* на необходимость запуска Java машины в отладочном режиме. Это достигается заданием в настроечном файле *rsjvm.ini* соответствующих параметров в соответствии с документацией используемых для отладки Java инструментов. Например, при использовании стандартного протокола для отладки JPDA:

```
OPTION = -Xdebug
OPTION = -Xnoagent
OPTION = -Djava.compiler=NONE
OPTION = -Xrunjdwp:transport=dt_socket,server=y,suspend=n,address=4000
```

Если используется Java машина Oracle (OJVM), то можно использовать более быстрый способ взаимодействия с отладчиком поддерживаемый JDeveloper. Для этого могут быть заданы такие опции:

```
OPTION = -XXdebug,detached,quiet,port4000
```

Из недостатков использования OJVM можно отметить проблемы с одновременной отладкой Java и C++ кода. Java точки останова перехватываются отладчиком C++. Поэтому более удобной в этом случае является отладка с использованием JPDA.

Рекомендации по использованию

Вызов Java методов будет максимально эффективным в случае явного задания сигнатур методов. При этом инструментальный код вообще не выполняет обращения за метайнформацией Java классов.

При первом вызове Java метода без заданной сигнатуры происходит обращение к метайнформации Java класса и построение сигнатур методов. Если вызываемый метод имеет уникальное имя, то именно он и вызывается. В противном случае при каждом вызове метода с неуникальным именем выполняется поиск наиболее подходящего исходя из переданных в методов параметров. В отличие от компилятора Java такой поиск осуществляется не единожды, а при каждом вызове. Поэтому крайне не рекомендуется вызывать перегруженные методы без использования сигнатур.

Так как имя метода с сигнатурой не соответствует формату допустимого в RSL идентификатора, имена таких методов в RSL необходимо указывать в фигурных скобках:

```
obj.{Name@()V}.
```

Возможно использование для вызова стандартной процедуры **GenRun** или метода **CallObjectMethod**. Примеры вызова из RSL Java метода:

```
public int MyCalc (int a, int b)
{
    return a + b;
}

Result = jobj.{MyCalc@(II)I} (10, 20);
Result = GenRun (jobj, "MyCalc@(II)I", 10, 20);
Result = jvm.CallObjectMethod (jobj, "MyCalc@(II)I", 10, 20);
```

Желательно не использовать в качестве параметров объектные типы данных Integer, Long и т.д. (также их массивы), а использовать соответствующие примитивные типы int, long и т.д. При вызове методов с обобщённым параметром типа Object, все примитивные типы автоматически конвертируются в объектные аналоги.

Желательно не использовать возврат значений обобщённого типа Object, так как в этом случае каждый раз выполняется проверка не является ли возвращаемое значение одним из примитивных типов данных, строкой, датой, Decimal и т.д. Целесообразней иметь несколько методов, возвращающих значения конкретных типов.

Для автоматизации процесса генерации C++ обёрток для Java-классов можно написать простой RSL-скрипт, который будет вызывать нужный метод генератора:

```
import rcw;
var jvm = CreateObject ("rsjvm", "TJavaHost", null, true);
```

// Пример генерации C++ обёртки Java-класса

```
var          gen          =          jvm.CreateJavaObject
("ru.softlab.tools.CppWrapGen");
gen.createWrapper ("javax.swing.JButton", "d:\\javagen\
\jbutton.hpp");
```

Использование RSCOM из автономных Java приложений

RSCOM-объекты могут вызываться из автономных DeskTop, Web и EJB приложений. Необходимые для работы классы входят в библиотеку *rsjvm.jar*, которую необходимо включить в classpath. В этом режиме работы все нативные методы класса **RsJvm** не определены, поэтому их вызывать нельзя. Для определения режима работы можно пользоваться статическим методом **RsJvm.isEmbedded()**. В случае автономного приложения метод возвращает **False**. RSCOM сервер *rsjvm.d32* не требуется. Все нативные методы класса **RObject**¹⁹⁵ в этом режиме реализуются в библиотеке *jrscom.dll*. Эта библиотека должна быть доступна Java приложению для загрузки. Библиотека *jrscom.dll* в свою очередь зависит от библиотеки *fs32cm.dll*, которая так же должна быть доступна для загрузки.

При использовании RSCOM из контекста серверов приложений Java, существует проблема с использованием классов с нативными методами. Таким классом является **RObjectBase**. Такие классы должны загружаться корневым class loader-ом Java. Если этого не сделать, то при попытке использовать класс **RObject** из разных приложений генерируется ошибка, информирующая, что *jrscom.dll* уже загружен другим class loader-ом. Самое простое решение проблемы – это поместить библиотеку *rsjvm.jar* в каталог *jre/lib/ext*.

Встроенные процедуры

В этом разделе представлен список всех стандартных процедур RSL. Они охватывают довольно широкий диапазон обработки данных:

- типы и значения переменных;
- работа со строками;
- параметры процедур;
- обработка ошибок и прочее.

Как правило, в результате успешной работы эти процедуры возвращают TRUE, в противном случае – FALSE.

При описании процедур в квадратные скобки заключаются необязательные параметры.

Стандартные процедуры ввода данных с клавиатуры

Процедуры этой группы присваивают переменной, имя которой передано в качестве **первого параметра**, значение, введенное пользователем с клавиатуры. Значение, которое эта переменная имела до вызова процедуры, используется как значение по умолчанию.

Второй параметр процедуры, имеющий тип **String**, задает текст приглашения к вводу. Если он отсутствует, то по умолчанию принимается следующий текст: "Введите значение: ".

Третий параметр задает ширину поля ввода. Если он не задан, то используется значение по умолчанию, указанное при описании каждой процедуры.

Четвертый параметр имеет тип **Bool**. Если он равен **TRUE**, то вводимая информация отображается на экране звездочками.

Пятый параметр имеет тип **Integer** и задает количество знаков после десятичной точки.

Шестой параметр процедуры задает заголовок окна.

Седьмой параметр содержит строку статуса.

В случае успешной работы эти процедуры возвращают **TRUE**. Если пользователь закрыл окно ввода с помощью клавиши **[Esc]**, возвращается **FALSE**, при этом значение переменной не изменяется.

Пример.

```
aa = "Иван";           /* Присвоено значение переменной, которая будет
                        использована как первый параметр процедуры */

if (GetString (aa,"Как вас /* На экране появится запрос: "Как вас зовут?
зовут?"));             Иван". */

println ("Вас зовут: ",aa); /* Печать измененного или прежнего значения
end;                   после ввода ответа */
```

Процедура **GetInt**

GetInt (id [, prompt, len [, hide]])

Процедура присваивает введенное пользователем значение переменной типа **Integer** с именем **id**. Пользователь может ввести в поле значение из диапазона от -2147483648 до 2147483647. По умолчанию ширина поля ввода равна 11 символам.

Процедура **GetDouble**

GetDouble (id [, prompt, len [, hide [, pos]]])

Процедура присваивает введенное пользователем значение переменной типа **Double** с именем **id**. По умолчанию ширина поля ввода равна 24 символам.

Процедура **GetMoney**

GetMoney (id [, prompt, len [, hide] [, pos]])

Процедура присваивает введенное пользователем значение переменной типа **Money** с именем **id**. По умолчанию ширина поля ввода равна 18 символам; количество знаков после десятичной точки равно 2.

Процедура **GetNumeric**

GetNumeric (id [, prompt, len [, hide [, pos]]])

Процедура присваивает введенное пользователем значение переменной типа **Numeric** с именем **id**. По умолчанию количество знаков после десятичной точки равно 4.

Процедура **GetString**

GetString (id [, prompt, len [, hide]])

Процедура присваивает введенное пользователем значение переменной типа **String** с именем **id**. По умолчанию ширина поля ввода равна 70 символам.

Процедура **GetStringR**

GetStringR (id [, prompt, len [, hide]])

Процедура используется для ввода числовых строк (например, номеров лицевых счетов). Она присваивает введенное пользователем значение типа **String** переменной с именем **id**. Кроме этого, она выравнивает введенное значение по правому краю поля ввода. По умолчанию ширина поля ввода равна 9 символам.

Процедура GetTime

GetTime(id [, prompt [, hide]])

Процедура присваивает введенное пользователем значение переменной типа **Time** с именем **id**. Ширина поля ввода всегда равна 10 символам.

Процедура GetDate

GetDate (id [, prompt [, hide]])

Процедура присваивает введенное пользователем значение переменной типа **Date** с именем **id**. Значение, которое эта переменная имела до вызова процедуры, используется как значение по умолчанию. Ширина поля ввода всегда равна 10 символам.

Процедура GetTRUE

GetTRUE (id [, prompt])

Процедура присваивает введенное пользователем значение переменной типа **Bool** с именем **id**.

Если пользователь утвердительно ответил на запрос, то переменной **id** присваивается значение **TRUE**, и процедура возвращает **TRUE**. Если в качестве параметра процедуры **id** используется константа (**TRUE** или **FALSE**), ответ пользователя можно определить только по возвращаемому значению. Ширина поля ввода игнорируется.

Пример.

```
if (GetTRUE (TRUE, "Хотите напечатать отчет?"))
    OutReport;
end;
```

Стандартные процедуры вывода

Процедура Print

Print (...)

Процедура выводит в отформатированном виде значения переменных, переданных в качестве параметров. Количество параметров не ограничено.

При записи параметров могут быть указаны [спецификаторы форматирования](#)⁽⁸³⁾. Если список параметров пуст, данная процедура не выполняет никаких действий.

Процедура PrintGlobs

PrintGlobs (...)

Процедура выводит в стандартный выходной поток информацию обо всех глобальных переменных. Для каждой переменной указывается название, имя модуля, в котором определена переменная, тип и значение.

Процедура PrintFiles

PrintFiles (...)

Для каждого модуля текущего экземпляра RSL процедура выводит в стандартный выходной поток следующую информацию:

- имя модуля (**Module name**);
- тип модуля (**Type**);
- имя файла (**File name**).

В качестве типа модуля могут выступать следующие значения:

- **Internal** – модуль реализован в ядре RSL или непосредственно в использующем его RSL-приложении. В этом случае имя файла не указывается.
- **MAC** – модуль реализован на языке RSL. Указанный тип модуля может загружаться из файла базы данных *btrmac.ddf*. В этом случае в качестве имени файла указывается имя модуля, затем – фраза "from *btrmac.ddf*" (например, *lib from btrmac.ddf*). Если загрузка файла производится не из указанной базы данных, выводится полное имя файла (например, *mac\privinc.mac*).
- **RSM** – прекомпилированный модуль. В качестве имени файла указывается имя соответствующего MAC-файла (например, *d:\user\rsl\l0.mac*).
- **DLM** – DLM-модуль. Указывается полный путь к файлу, реализующему указанный модуль (например, *D:\user\rsl\rsd.D32*).

Процедура PrintLn

PrintLn

Действия данной процедуры аналогичны действиям процедуры [Print](#)⁽²¹¹⁾. Отличие заключается в том, что после вывода значений переменных выполняется возврат каретки.

Процедура PrintLocs

PrintLocs (...)

Процедура выводит информацию о локальных переменных процедуры, из которой вызвана **PrintLocs**. Если процедура вызвана непосредственно из кода инициализации модуля, дополнительно указывается информация о глобальных переменных указанного модуля. Для каждой переменной указывается название, тип и значение.

Процедура PrintModule

PrintModule (modName:String)

Процедура для заданного модуля **modName** выводит информацию о типе модуля и имени файла, из которого загружен указанный модуль.

Процедура PrintProps

PrintProps (object)

Процедура выводит в стандартный выходной поток имя, тип и значение каждого свойства объекта **object**.

Процедура PrintRefs

PrintRefs (ob: Object)

Процедура выводит в стандартный выходной поток информацию об объектах, ссылающихся на объект **ob**.

Процедура PrintStack

PrintStack (...)

Процедура выводит информацию о стеке вызова процедур для контекста, в котором вызвана **PrintStack**:

- выводится название всех процедур в стеке;
- предоставляется информация о локальных переменных каждой процедуры;
- для каждой локальной переменной указывается название, тип и значение.

Процедура PrintSymModule

PrintSymModule (symbolName:String)

Процедура выводит в стандартный выходной поток RSL информацию о переменной, процедуре или классе, имя которого задано параметром **symbolName**. Указывается вид сущности (процедура, переменная или класс) и информация о модуле, в котором объявлена указанная RSL-сущность.

Процедура Message

Message

Данная процедура принимает произвольное количество параметров и выводит в нижнюю строку экрана сформированную из них отформатированную строку. При записи параметров могут быть указаны [спецификаторы форматирования](#) ⁽⁸³⁾.

Если список параметров пуст, данная процедура не выполняет никаких действий.

Процедура SetOutput

SetOutput ([string] [, bool])

Процедура устанавливает новый файл для вывода, имя которого задается параметром **String**.

Если параметр **bool** отсутствует или равен **FALSE**, то все данные из файла удаляются; если он равен **TRUE**, то новая информация дописывается в конец файла, заданного параметром **String**.

После выполнения данной процедуры весь вывод процедур **Print**, **Println** или инструкции вывода направляется в указанный файл.

Для того чтобы вернуть вывод в стандартный файл, необходимо задать процедуру со следующими параметрами:

SetOutput (NULL, TRUE)

Пример.

```
oldFile = SetOutput ("My.out");  
[ Этот текст выводится в файл My.out ];  
SetOutput (NULL,TRUE);  
[ Этот текст выводится в стандартный файл вывода];
```

Процедура **SetOutput** возвращает имя файла, в который выполнялся вывод до ее вызова.

Процедура SetColumn

SetColumn (integer)

Процедура формирует отчет с переменным числом колонок.

Параметр **integer** устанавливает относительный номер колонки, в которую будет осуществляться весь вывод процедур [Print](#)²¹¹, [PrintLn](#)²¹² и инструкции вывода.

Для того чтобы информация попала в выходной файл, необходимо вызвать процедуру [FlushColumn](#)²¹⁵.

Процедура FlushColumn

FlushColumn

Процедура выводит информацию из сформированного при помощи процедуры [SetColumn](#)²¹⁴ отчета в файл вывода.

Пример.

```
macro PrintHead (n)
  if (n == 0)
    ch = "|"; ch2 = "|"
  else
    ch = "-"; ch2 = ""
  end;
  [#-----|
  #  ##      |
  #-----|] (ch,ch2,n,ch)
end;

macro PrintCol (n,str)
  if (n == 0)
    [| ## ##### |] (n,str:w)
  else
    [  ## ##### |] (n,str:w)
  end;
end;

NumColumn = 5;
i = 0;
while (i < NumColumn)
  SetColumn (i);
  PrintHead (i);
  i = i + 1;
end;
FlushColumn;
i = 0;
while (i < NumColumn)
  SetColumn (i);
  PrintCol (i,"Эту очень длинную строку мы будем выводить во
все колонки !!!");
  i = i + 1;
end;
SetColumn (4);
[ 4 колонка ];
SetColumn (2);
[ 2 колонка ];
FlushColumn;
```

Процедура ClearColumn

ClearColumn

Процедура удаляет все данные из буфера печати. Процедура работает совместно с процедурами [SetColumn](#)²¹⁴ и [FlushColumn](#)²¹⁵.

Процедура SetDefPrec

SetDefPrec (integer)

Процедура устанавливает количество отображаемых по умолчанию знаков после точки для вывода значений типа **Double** и **DoubleL**.

Установленное значение будет действовать до следующего вызова данной процедуры.

Процедура SetOutHandler

SetOutHandler (NameMacro)

Процедура устанавливает обработчик стандартного вывода.

Имя или ссылка на макропроцедуру обработки вывода передаются в качестве параметра **NameMacro**. Для установки стандартного обработчика необходимо вызвать **SetOutHandler** без параметров.

Примеры.

```
1) /* Пример asp-файла, в котором все вызовы print
перенаправляются в Response.Write. */
<%@ Language=RSLScript %>
<html>
<head>
<title>New Page 1</title>
</head>
<body>
<%
Macro MyOutProc (str)
    Response.Write (str+"<br>")
End;
SetOutHandler (@MyOutProc);
print (Request.Form ("Name").item);
print (Request.Form ("Address").item);
print (Request.Form ("Info").item);
print (Request.Form ("Password").item);
%>
</body>
</html>

2) /* Пример перенаправления стандартного вывода в пайп. */
```



```
file ff () write txt;
Macro MyOutProc (str)
    insert (ff,str,strlen (str), true)
End;
pipeName = "\\\\" + getEnv ("COMPUTERNAME") + "\\pipe\
\brmon.";
if (open (ff,pipeName))
    SetOutHandler (@MyOutProc);
    println ("Brmon.mac executed");
end;
```

Процедура GetPRNInfo

GetPRNInfo ([escSeq:String, banner:String, frmFeed:Bool]) : String

Процедура возвращает имя используемого устройства печати.

В параметре **escSeq** возвращается используемая ESC-последовательность.

В параметре **banner** возвращается используемый файл заголовка.

В параметре **frmFeed** возвращается признак перевода формата.

Процедура SetPRNInfo

SetPRNInfo (prnName: String [, escSeq: String, banner: String, frmFeed:Bool])

Процедура устанавливает новое имя используемого принтера **prnName**, ESC-последовательность **escSeq**, используемый файл заголовка **banner** и признак перевода формата **frmFeed**.

Преобразование типов значений переменных

Процедуры этой группы выполняют преобразование типа переменной.

Каждая из процедур получает в качестве параметра некоторую величину и возвращает уже преобразованное значение. Если процедура не может выполнить преобразование, она возвращает нулевое значение соответствующего типа.

Процедура ValType

ValType (val)

Процедура возвращает код типа переменной, переданной через параметр **val**. В зависимости от типа данных возвращаемым значением может быть одно из следующих значений:

- | | | | |
|------------|-------------|-------------|-------------|
| • V_UNDEF | • V_INTEGER | • V_MONEY | • V_DECIMAL |
| • V_DOUBLE | • V_STRING | • V_BOOL | • V_DATE |
| • V_TIME | • V_DTTM | • V_FILE | • V_STRUC |
| • V_ARRAY | • V_TXTFILE | • V_DBFFILE | • V_PROC |
| • V_R2M | • V_MEMADDR | | |

Пример.

```
if (ValType (MyVar) == V_UNDEF)
    MsgBox ("Значение переменной MyVar не
           определено")
end
```

Процедура Double

Double (val)

Процедура возвращает преобразованное в тип **Double** значение типа **Integer**, **Money** или **String**, переданное через параметр **val**.

Процедура DoubleL

DoubleL (val)

Процедура возвращает преобразованное в тип **DoubleL** значение типа **Integer**, **Money**, **Double** или **String**, переданное через параметр **val**.

Процедура Int

Int (val)

Процедура возвращает преобразованное в тип **Integer** значение типа **Double**, **String**, **Date** и **Time** переданное через параметр **val**.

После преобразования числа с плавающей точкой процедура возвращает целую часть числа, дробная часть отбрасывается. Для преобразования **Double** в **Integer** с округлением используется процедура [Round](#)²²⁴.

При преобразовании даты в целое число процедура возвращает значение, равное количеству дней от начала нашей эры до заданной параметром **val** даты.

При преобразовании времени в целое число процедура возвращает значение, равное количеству сотых долей секунды во времени, заданном параметром **val**.

Процедура String

String (val {, val})

Процедура принимает произвольное количество параметров и возвращает сформированную из них отформатированную строку.

При записи параметров, так же, как и в процедурах [Print](#)⁽²¹¹⁾ и [PrintLn](#)⁽²¹²⁾, могут быть указаны специальные символы форматирования.

Процедура Money

Money (val)

Процедура возвращает преобразованное в тип **Money** значение параметра типа **String**, **Integer**.

Процедура MkStr

MkStr (val1, val2)

Процедура возвращает строку типа **String** конкретной длины, состоящую из заданного символа-заполнителя.

Вид символа-заполнителя определяется первым параметром **val1**, который может быть числом или строкой. Если это число имеет тип **Integer**, то оно будет расценено как код символа-заполнителя. Если параметр **val1** представляет собой строку (тип **String**), то в качестве заполнителя используется ее первый символ.

Второй параметр **val2** задает длину возвращаемой строки.

Процедура Floor

Floor (val)

Процедура возвращает наибольшее целое число, меньшее или равное заданному параметру для значений типа **Double**, **DoubleL**. Для значений типа **Money** процедура отбрасывает дробную часть до двух знаков после запятой.

Процедура Asize

Asize (val [, newsize])

Процедура возвращает текущий размер массива **val**. Если указан второй параметр, то старый массив удаляется, и распределяется пустой массив размером **newsizе**.

Процедура Date

Date [(day, mon, year)]

Процедура возвращает переменную типа **Date**.

Процедуре можно передать от одного до трех параметров типа **Integer**. Первый задает число, второй – месяц, третий – год. Переданные значения заменят соответствующие им части текущей даты. Если нужно изменить только год и оставить текущий день и месяц, необходимо передать в качестве первого и второго параметров значение любого типа, отличного от **Integer**.

Процедуре также можно передать в качестве первого параметра строку вида "dd.mm.yy" или "dd.mm.yyyy", в результате переданное значение будет преобразовано к соответствующей дате.

Процедура также может принимать в качестве параметра значения типа **Date** и **Dttm**, которые будут преобразованы в переменную типа **Date**.

Если в качестве параметра **IgnoreTestMode** передано значение **TRUE**, то при выполнении процедуры не учитывается режим тестирования, который устанавливается значением "1" параметра **TESTMODE** настроечного файла *rsreq.ini*. В этом случае процедура возвращает текущую системную дату, а не специальное фиксированное значение.

Если при вызове процедуры параметры не указаны, то процедура возвращает одно из значений.

- текущую дату – если в файле *rsreq.ini* параметр **TESTMODE** = 0;
- значение "31.12.2099" – если параметр **TESTMODE** = 1.

Пример.

<code>aa = date;</code>	<code>/*Переменная aa равна текущей дате */</code>
<code>aa = date ("10.03.1994");</code>	<code>/* Переменная aa равна 10 марта 1994 года */</code>
<code>aa = date (10);</code>	<code>/* 10 число текущего месяца и года */</code>
<code>aa = date (10,3);</code>	<code>/* 10 число 3 месяца текущего года */</code>
<code>aa = date ("",3,1994);</code>	<code>/* Текущий день марта1994 года */</code>

Процедура DateSplit

DateSplit (date, day, mon, year)

Процедура получает в качестве первого параметра **date** переменную типа **Date** и возвращает через параметры **day**, **mon**, **year**, имеющие тип **Integer**, соответственно, день, месяц и год для заданной даты.

Если нужно получить только какую-нибудь одну составляющую даты, пропущенные компоненты заменяются нулевыми значениями, а конец списка опускается.

Пример.

```
datesplit (date, NULL, mon);
```

Процедура Time

Time [(hour, min, sec, msec)]

Процедура возвращает переменную типа **Time**.

Процедуре можно передать от одного до четырех параметров типа **Integer**. Первый задает часы, второй – минуты, третий – секунды, четвертый – сотые доли секунды. Переданные значения заменят соответствующие им части текущего времени. Передача не всех параметров осуществляется так же, как для процедуры [Date](#)²²⁰.

Процедуре также можно передать в качестве первого параметра строку вида "hh:mm:ss.nn", "hh:mm:ss" или "hh:mm", в результате переданное значение будет преобразовано к соответствующему времени. Если время задается строкой вида "hh:mm:ss.nn", то процедуре можно передать второй строковый параметр, задающий символ-разделитель между секундами и сотыми долями секунды. По умолчанию используется символ ".".

Процедура может принимать в качестве параметра значения типа **Time** и **Dttm**, которые будут преобразованы в переменную типа **Time**.

Если при вызове процедуры параметры не указаны, то процедура возвращает одно из значений.

- текущую дату – если в файле *rsreq.ini* параметр TESTMODE = 0;
- значение "31.12.2099" – если параметр TESTMODE = 1.

Процедура TimeSplit

TimeSplit (time, hour, min, sec)

Процедура принимает в качестве первого (входного) параметра переменную типа **Time** и возвращает в выходных параметрах **hour**, **min**, **sec** типа **Integer** компоненты заданного времени: часы, минуты и секунды.

Допускается указывать в выходных параметрах только интересующие пользователя компоненты времени.

Процедура DtTm

DtTm (date, time)

Процедура принимает в качестве первого параметра переменную типа **Date**, в качестве второго – переменную типа **Time** и возвращает величину типа **Dttm**, состоящую из двух компонент: даты и времени.

Процедура DtTmSplit

DtTmSplit (d, dt, tm)

Процедура получает в качестве первого параметра величину **d** типа **Dttm** и возвращает две ее составляющие: дату и время через параметры **dt** и **tm**.

В качестве первого параметра также можно передать значение типа **Date** или **Time**, в этом случае процедура вернет дату либо время в соответствующем параметре.

Процедура RubToStr

RubToStr (money [, rub, kop [, full]], kol)

Процедура принимает в качестве первого параметра переменную типа **Money**. Второй и третий параметры процедуры необязательны. Если они заданы, то процедура инициализирует переменную **rub** строкой, содержащей количество рублей прописью, а переменную **kop** – строкой, содержащей количество копеек цифрами. Параметр **kol** задает количество знаков после десятичной точки; по умолчанию значение параметра равно 2.

Процедуре можно указать на необходимость вывода слов рублей и копеек в несокращенном виде. Признаком этого является значение типа **Bool**, равное **TRUE** и переданное в качестве второго или четвертого параметра.

Текст, выдаваемый этой процедурой, можно изменить с помощью соответствующих настроек в файле *locale.ini*. Эти настройки используются в том случае, если национальная валюта отлична от рубля.

Процедура возвращает строку, содержащую сумму прописью.

Пример.

```
/* Вариант 1 */
Var a = 121.11, b, c;
println(RubToStr(a*100.00, b, c)); /* Результат : Сто двадцать один руб. 11
коп. */
/* Вариант 2 */
Var a = 121.11, b = TRUE, c;
println(RubToStr(a*100.00, b, c)); /* Результат : Сто двадцать один рубль 11
копеек */
```

```
/* Вариант 3 */  
Var str, rub, kop;  
str = rubtostr ($12.34, true);  
str = rubtostr ($12.34, rub, kop, true);  
rub = true;  
str = rubtostr ($12.34, rub, kop);  
/*Все три варианта распечатают названия полностью.*/
```

Процедура RubToStrAlt

RubToStrAlt (money, [rub, kop,] kol)

Процедура принимает в качестве первого параметра переменную типа **Money**. Второй и третий параметры процедуры необязательны. Если они заданы, то процедура инициализирует переменную **rub** строкой, содержащей количество рублей прописью, а переменную **kop** – строкой, содержащей количество копеек цифрами. Параметр **kol** задает количество знаков после десятичной точки; по умолчанию значение параметра равно 2.

Текст, выдаваемый этой процедурой, можно изменить с помощью соответствующих настроек в файле *locale.ini*. Эти настройки используются в том случае, если национальная валюта отлична от рубля.

Процедура возвращает строку, содержащую сумму прописью. Рубли и копейки указываются с учетом падежа: рублей (рубля), копеек (копейки).

Процедура CurToStrAlt

CurToStrAlt (money [, rub, kop], ISO, kol)

Процедура принимает в качестве первого параметра переменную типа **Money**, в качестве предпоследнего параметра – цифровой ISO-код валюты. Второй и третий параметры процедуры не обязательны. Если они заданы, то процедура инициализирует переменную **rub** строкой, содержащей количество основных денежных единиц прописью, а переменную **kop** – строкой, содержащей количество дробных денежных единиц прописью. Параметр **kol** задает количество знаков после десятичной точки; по умолчанию значение параметра равно 2.

Процедура возвращает строку, содержащую сумму прописью. Основные и дробные денежные единицы указываются сокращенно.

Для работы процедуры необходим файл *locale.ini*, содержащий цифровые ISO-коды валют с соответствующими им падежными формами основных и дробных денежных единиц для перевода их численного представления в прописное.

Процедура MonName

MonName (mon)

Процедура возвращает название месяца, номер которого (от 1 до 12) передан через параметр **mon** типа **Integer**.

Процедура NumToStr

NumToStr (val, n1:String, n2:String, n3:String, isMan:Bool, prec:Integer) : String

Процедура **NumToStr** возвращает значение числа **val** прописью.

Параметры **n1**, **n2** и **n3** задают три возможных вида сущности, количество которой задается параметром **val**.

Параметр **isMan** имеет значение **TRUE** для сущности мужского рода и **FALSE** для сущности женского рода. По умолчанию параметр принимает значение **FALSE**.

Параметр **prec** задает максимальное количество знаков после десятичной точки. Может быть задано от 0 до 5 знаков. По умолчанию используется значение 2.

Пример.

```
println (NumToStr (125.45,"болт", "болта", "болтов", true,3)
);
/* Выводит: */
сто двадцать пять целых сорок пять сотых болта
```

Процедура SetAutoMoneyFloor

SetAutoMoneyFloor (auto:Bool):bool

Процедура **SetAutoMoneyFloor** устанавливает режим автоматического отбрасывания десятых и сотых долей копеек в денежных типах при выполнении операций над ними, если параметр **Auto** равен **TRUE**. В противном случае десятые и сотые доли не отбрасываются. По умолчанию в системе установлен режим отбрасывания десятых и сотых долей.

Процедура возвращает старое значение режима.

Процедура Round

Round (val:Variant [, pos:Integer] , [round:Integer]):Money

Процедура предназначена для округления введенного значения.

Параметры:

val – значение типа **Money**, для которого следует выполнить округление. Если переданное значение имеет тип **Numeric**, необходимо указать значение параметра **round**.

pos – количество знаков после запятой. Возможные значения:

- **0** – округление до рублей;
- **2** – округление до копеек.

round – характер округления. Возможные значения:

- **1** – округление в большую сторону.
- **2** – округление в меньшую сторону.

Параметр указывается только, если в параметре **val** указано значение типа **Numeric**.

Процедуры для работы с типом DoubleL

Процедуры этой группы предназначены для работы с типом данных **DoubleL (long double)**. Данные указанного типа могут присутствовать в базе данных RS-Bank Pervasive; их невозможно представить в стандартном для RSL формате описания базы данных (в файлах *ddf*).

При конвертировании словаря *def* в *ddf*, все поля типа **long double** (10 байт) преобразуются в три целочисленных поля.

Процедура MakeDouble

MakeDouble (p1:Integer, p2:Integer, p3:Integer):Double

Процедура служит для получения значения типа **Double** из трёх указанных полей (**p1**, **p2**, **p3**).

Процедура MakeMoney

MakeMoney (p1:Integer, p2:Integer, p3:Integer):Money

Процедура служит для получения значения типа **Money** из трёх указанных полей (**p1**, **p2**, **p3**).

Процедура SplitMoney

SplitMoney (mn:Money, p1:@Integer, p2:@Integer, p3:@Integer)

Указанная процедура является обратной процедуре [MakeMoney](#)²²⁵ и предназначена для преобразования величины типа **Money** в три целых числа, возвращаемых в параметры **p1**, **p2**, **p3**.

Работа со строками

Процедуры этой группы выполняют некоторые специфические действия над переменными типа **String**.

При работе с символьными строками считается, что первый символ находится в позиции с номером 1.

Процедура StrLen

StrLen (string)

Процедура возвращает длину строки символов, заданной в качестве параметра.

Процедура Index

Index (srcStr:String, fnd:String [, startPos:Integer]):Integer

Процедура возвращает номер позиции строки **fnd** в строке **srcStr** или **0**, если строка **fnd** не найдена в **srcStr**.

Параметр **startPos** задаёт номер позиции в строке **srcStr**, начиная с которой необходимо выполнить поиск. Минимальный номер позиции равен **1**. Если параметр не задан, поиск выполняется с начала строки **srcStr**.

Процедура StrBrk

StrBrk (string1, string2)

Процедура возвращает позицию первого из символов в строке **string1**, который содержится также и в строке **string2**. Если в строке **string1** нет ни одного из символов строки **string2**, процедура возвращает **0**.

Процедура StrIsNumber

StrIsNumber(str:String):Bool

Процедура предназначена для выполнения проверки, определяющей, являются ли все символы строки цифрами.

Параметры:

str – строка символов.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **TRUE** – если все символы в строке являются цифрами;
- **FALSE** – если в качестве значения параметра *str* указана пустая строка или хотя бы один из символов строки не является цифрой.

Процедура SubStr

SubStr (string, integer1 [, integer2])

Процедура возвращает часть строки **string**, которая начинается с позиции **integer1**.

Третий параметр не обязателен. Если он указан, то извлекается подстрока длиной **integer2**. В противном случае извлекается подстрока с позиции **integer1** до конца строки **string**.

При неправильно заданных параметрах возвращается пустая строка.

Процедура StrSet

StrSet (string1, integer, string2)

Процедура записывает строку **string2** в строку **string1**, начиная с позиции строки **string1**, определенной вторым параметром типа **Integer**.

Процедура Trim

Trim (string)

Процедура удаляет начальные и конечные пробелы из строки типа **String**, заданной в качестве параметра, и возвращает результат.

Процедура StrSplit

StrSplit (string, array, len [, len_first] [, minseg])

Процедура разбивает длинную строку, заданную в качестве первого параметра типа **String**, на несколько сегментов. Результат помещается в одномерный массив, определенный вторым параметром типа **Array**.

Длина каждого сегмента задается третьим параметром **len**, имеющим тип **Integer**. Если к тому же задан параметр **len_first**, он используется как длина первого сегмента массива.

Указав последний параметр процедуры **minseg**, можно задать минимальное число сегментов. В том случае, если реальное число созданных сегментов меньше заданного минимального, лишние элементы массива **array** инициализируются пустыми строками.

Процедура StrSplit2

Возможны варианты вызова указанной процедуры:

StrSplit2 (source:String, len:Integer [, len_first:Integer] [, minseg:Integer]) : TArray

Указанный вариант процедуры аналогичен процедуре [StrSplit](#)²²⁷ и возвращает объект типа **TArray**, содержащий полученные строки.

StrSplit2 (str:String, len:Integer, isFix:Bool, minseg:Integer):TArray

Использование указанного варианта процедуры позволяет разбить строку на сегменты фиксированного размера без переноса по словам. Для этого следует в качестве параметра **isFix** передать значение **TRUE**.

Пример.

```
ar = StrSplit2("Иванов Иван Иванович", 8, true);
for (a, ar)
    println (a);
end;
Результат:
Иванов И
ван Иван
ович
```

Процедура StrUpr

StrUpr (string [,len])

Процедура возвращает копию строки **string**, преобразованную в символы верхнего регистра.

Необязательный параметр **len** задает количество символов, которые необходимо преобразовать.

Если параметр **len** не указан, все символы строки **string** будут преобразованы в символы верхнего регистра.

Пример.

```
TestLine = "Эта Строка Набрана В Разных Регистрах";
println ("Нормальная строка - ",TestLine);
println ("В верхнем регистре - ",StrUpr (TestLine));
```

Процедура StrLwr

StrLwr (string [, len])

Процедура возвращает копию строки **string**, преобразованную в символы нижнего регистра.

Необязательный параметр **len** задает количество символов, которые необходимо преобразовать.

Если параметр **len** не указан, все символы строки **string** будут преобразованы в символы нижнего регистра.

Пример.

```
TestLine = "Эта Строка Набрана В Разных Регистрах";  
println ("Нормальная строка - ", TestLine);  
println ("В нижнем регистре - ", StrLwr (TestLine));
```

Процедура CodeFor

CodeFor (string)

Процедура возвращает код ASCII для первого символа в строке **string**.

Пример.

```
code = CodeFor ("A");
```

Процедура StrFor

StrFor (number)

Процедура возвращает строку, состоящую из одного символа, код ASCII которого равен **number**.

Пример.

```
str = StrFor (65);
```

Процедура StrSubst

StrSubst (source, strToFind, strToReplace)

Процедура ищет в строке **source** подстроки **strToFind** и заменяет их строками **strToReplace**. Возвращаемым значением является результирующая строка.

Пример.

```
Str = StrSubst (oldStr, "\n", " ")
```

Процедура ToOEM

ToOEM (string [, mode])

Процедура конвертирует строку, заданную параметром **string**, из ANSI в OEM. Если значение параметра **mode** равно **TRUE**, выполнение конвертирования осуществляется в любом режиме.

Если параметр **mode** не задан, конвертирование строк в режиме **Unicode** выполняется, только если в файле *rsreq.ini* задан параметр `FORCE_OEM_ANSI_CVT = Yes`.

Возвращаемым значением является преобразованная строка.

Процедура ToANSI

ToANSI (string [, mode])

Процедура конвертирует строку, заданную параметром **string**, из OEM в ANSI. Если значение параметра **mode** равно **TRUE**, выполнение конвертирования осуществляется в любом режиме.

Если параметр **mode** не задан, конвертирование строк в режиме **Unicode** выполняется, только если в файле *rsreq.ini* задан параметр `FORCE_OEM_ANSI_CVT = Yes`.

Возвращаемым значением является преобразованная строка.

Пример.

```
s = ToANSI (oemstr, true)
```

Параметры процедур

Процедуры этой группы предоставляют дополнительные возможности для работы со списком параметров процедур языка RSL. Примеры их использования приведены в разделе ["Передача параметров"](#) ⁵⁸. Первый в списке параметр имеет номер 0.

Процедура GetParm

GetParm (num, var)

Процедура присваивает переменной **var** значение фактического параметра вызывающей процедуры с номером **num**.

В случае успешного выполнения возвращается **TRUE**. При неудаче возвращается **FALSE**, и переменная **var** получает значение неопределенного типа `V_UNDEF`.

Параметр процедуры **var** не должен быть описан как константа.

Примечание.

При использовании данной процедуры в методе класса следует помнить, что

в метод неявно первым параметром с номером 0 передается ссылка на объект (this). Таким образом, первый параметр метода имеет номер 1.

Процедура IsOutParm

IsOutParm (num:Integer):Bool

Процедура определяет, является ли заданный параметр выходным или только входным.

Параметры:

num – параметр, который требуется проверить.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **TRUE** – переданный параметр является выходным (и входным).
- **FALSE** – переданный параметр является только входным.

Пример.

```
Macro Test (a, b c)
  If (IsRef (0))
    SetParm (0, "new value")
  End
End;
```

Процедура SetParm

SetParm (num, expression)

Процедура присваивает фактическому параметру вызывающей процедуры с номером **num** значение, заданное в параметре **expression**.

В случае успешного выполнения возвращается **TRUE**. При неудаче возвращается **FALSE**, и фактический параметр процедуры получает значение неопределенного типа **V_UNDEF**.

Фактический параметр в вызывающей процедуре должен быть описан как переменная. Если же он является константой, его значение не изменится, при этом система не сгенерирует сообщения об ошибке.

Процедура Parmcount

Parmcount ()

Процедура возвращает количество переданных RSL-процедуре параметров.

Математические процедуры

В языке RSL предусмотрены следующие процедуры для вычисления стандартных математических функций:

Exp (number) – экспонента.

Log (number) – логарифм.

Log10 (number) – десятичный логарифм.

Pow (number, number) – возведение в степень.

Sqrt (number) – квадратный корень.

Abs (number) – модуль.

Min (number, number) – минимальное из двух чисел.

Max (number, number) – максимальное из двух чисел.

Mod (val1:variant, val2:variant):variant – остаток от деления величины **val1** на величину **val2**.

В качестве параметров перечисленных процедур по умолчанию используются значения типа **Double**. Если этим процедурам передать значения любого другого типа, то они будут преобразованы в тип **Double**. Исключение составляет значение типа **DoubleL**, которое остается без изменений. Процедура **Mod** может получать параметры типов **DoubleL** и **Integer**.

Все процедуры возвращают значение типа **Double**. Однако если в качестве параметра передано значение типа **DoubleL**, то результатом также будет значение типа **DoubleL**.

Внешние программы

При работе с языком RSL можно использовать внешние программы, написанные на любых других языках программирования. Внешняя программа должна быть представлена в виде файла с расширением **.exe** или **.com**.

Процедура Run

Run (prog, parm, init, finish:String[])

Процедура предназначена для запуска внешней программы.

Параметры:

Примечание.

Параметры **parm**, **init**, **finish** могут быть пропущены, вместо них следует

указать *Null*, например, `run ("myprog.com",NULL,NULL,"Нажмите любую клавишу")`.

prog – имя внешней программы. При записи параметра расширение ".exe" в имени внешней программы можно не указывать. Поиск программы производится сначала в текущем каталоге, потом в каталогах, указанных в переменной среды PATH. Для обработки командных файлов необходимо запустить программу *command.com* и передать ей в качестве параметра имя файла.

Пример.

```
run (GetEnv("COMSPEC"),"/c " + "my.bat");
```

parm – командная строка, передаваемая во внешнюю программу, указанную в параметре **prog**.

init – сообщение, которое необходимо вывести на экран в момент запуска программы. Параметр может включать следующие символы:

Примечание.

Если строка сообщения начинается не с символов "<" или ">" (например, "Строка для отображения>c:\mydir\myout.txt"), все символы строки до указанных символов отображаются в консольном режиме перед запуском программы.

- ">" – символ используется для перенаправления стандартного вывода в требуемый файл; имя файла задается после указанного символа. В случае, когда необходимо использовать стандартный вывод программы в выходной файл RSL, имя файла не указывается.

Пример.

```
">c:\mydir\myout.txt" - выход программы будет произведен в
указанный файл
"Begin">" - выход программы направлен в стандартный выход
RSL.
```

- "<" – символ используется для перенаправления стандартного ввода, имя файла задается после указанного символа.

Пример.

```
"<c:\mydir\myin.txt".
```

- ">>" – символ дозаписи. Если в качестве параметра передать строку типа ">><имя файла>", будет произведена дозапись в указанный файл.

Пример.

```
run (GetEnv("COMSPEC"),"/c dir *.*", ">>e:\remote.out");
```

Имеется возможность одновременно задать имена файлов для перенаправления ввода и вывода. В этом случае порядок указания файлов не имеет значения.

Пример.

```
">c:\mydir\myout.txt<c:\mydir\myin.txt"
"> <c:\mydir\myin.txt"
"Строка для отображения>c:\mydir\myout.txt <c:\mydir\
myin.txt"
```

finish – сообщение, которое необходимо вывести на экран после завершения программы. При использовании указанного параметра перед возвратом в интерпретатор RSL будет сделана пауза.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **код возврата внешней программы** – в случае успешного выполнения;
- **–1**, если указанная программа не найдена.

Пример.

//Пример запуска процедуры *хсору* с перенаправленным входным файлом, содержащим символ ответа на запрос 'у':

```
run ("хсору", "/F d:\user\cmd.mac d:\user\cmd2.*", "> < d:\user\yes.txt", "Press a key");
```

Результат в выходном файле RSL:

```
Overwrite D:\user\cmd2.mac (Yes/No/All)? y
D:\user\cmd.mac -> D:\user\cmd2.mac
1 File(s) copied
```

Удаленный запуск макропрограмм

При работе в трехзвенной архитектуре "Клиент-сервер" пользователь имеет возможность из макропрограмм, выполняющихся на сервере приложений, запускать RSL-программы на терминале. Это осуществляется посредством специальной процедуры [CallRemoteRsl](#)²³⁴, входящей в модуль *rsexts*.

Внимание!

Для того, чтобы RSL-программа запускалась на терминале, необходимо в макрофайле явно импортировать модуль *rsexts* в макрофайле.

Процедура CallRemoteRsl

CallRemoteRsl (fileName [, procName [, parm1, parm2, ...]])

Параметры:

fileName – строка с именем макрофайла, который должен быть выполнен на терминале.

procName – необязательное имя процедуры из файла **fileName**, которая должна быть вызвана.

parm1, parm2 – параметры, передаваемые процедуре **procName**.

Возвращаемое значение:

Возвращаемым значением процедуры **CallRemoteRsl** является значение, возвращаемое процедурой **procName**.

Параметры, передаваемые процедуре **procName**, и ее возвращаемое значение должны быть одного из простых типов данных RSL. Не допускаются параметры и возвращаемое значение объектного типа (**FILE**, **RECORD**, **OBJECT**).

Пример.

```
/* Макрофайл на сервере: */
import rsexts;
println (CallRemoteRsl ("remote.mac" , "Main", $1.33L
,200 ));
/* Макрофайл на терминале (с именем remote.mac): */
MsgBox ("This is remote.mac");
macro Main (p1,p2)
    MsgBox ("Macro Main called with ", p1," ",p2);
    return "Main Return value"
end;
```

В случае возникновения ошибок компиляции или во время выполнения на терминале серверная часть генерирует ошибку.

Следует иметь в виду, что для макрофайла на терминале организуется отдельный файл вывода, таким образом, результаты работы инструкций вывода и др. попадут не в файл вывода вызывающего файла на сервере, а в файл вывода на терминале.

Обработка меню

Процедура Menu

Menu (array [, prompt] [, head] [, x] [, y] [, n])

Процедура формирует на экране меню для выбора варианта.

Текст каждой строки меню необходимо определить в элементах массива **array**, который является первым параметром и имеет тип **V_ARRAY**.

Во втором параметре **prompt** задается сообщение, выводимое в нижнюю строку экрана вместе с меню.

Параметр **head** задает текст, выводимый в заголовок окна. Если этот параметр не задан, в качестве заголовка окна используется строка: "Меню".

Параметры **x** и **y** задают координату на экране левого верхнего угла окна меню. Если эти параметры не заданы, окно меню располагается по центру экрана.

Параметр **n** определяет номер текущего активного пункта меню.

Процедура возвращает номер элемента массива ARRAY, соответствующего выбранному варианту.

Примеры.

1) Пример иллюстрирует формирование меню в заданной позиции экрана с заголовком и подсказкой по умолчанию:

```
array a;
a (0) = "Первый пункт";
Выбор = Menu (a, NULL, NULL, 4, 6)
```

2) Пример иллюстрирует формирование меню, имеющее заголовок "Заголовок" и подсказку "Подсказка", расположенное по центру экрана. В

этом случае активным станет третий пункт экранного меню (нумерация пунктов начинается с нуля):

```
Array mn;
/* Формируем пункты меню */
mn = "Первый пункт";
mn = "Второй пункт";
mn = "Третий пункт";
Menu (mn, "Подсказка", "Заголовок", NULL, NULL, 2)
```

Процедура RunMenu

RunMenu (menuName, procName | procAddr [, lbrName])

Процедура активизирует выполнение меню, созданного в специальной [библиотеке ресурсов](#) ⁸⁹.

Имя меню в библиотеке ресурсов определяется параметром **menuName**.

Во втором параметре задается строка с именем процедуры обработки команд меню **procName** или ссылка на данную процедуру **procAddr**.

С помощью третьего параметра при необходимости можно указать имя альтернативной библиотеки ресурсов.

Пример.

Предположим, что в библиотеке создано меню *Demo*, имеющее два пункта:

- выход, с идентификатором 101;
- печать отчета, с идентификатором 102.

Код обработки такого меню выглядит следующим образом:

```
Macro HandleMenu (id)
    If (id == 101) /* Завершить работу меню */
        Return TRUE;
    Elif (id == 102) /* Напечатать отчет */
        RunReport
    End;
Return FALSE;
/* Возвращать FALSE для необрабатываемых идентификаторов */
End;
RunMenu ("Demo", @HandleMenu);
```

Процедура обработки вызывается автоматически, когда пользователь выбирает пункт меню. В качестве параметра данная процедура получает целочисленный идентификатор выбранного пункта **id**, определенный при создании меню. Если в ответ она возвращает значение **TRUE**, то это же значение возвращает и процедура **RunMenu**, и обработка меню завершается.

При нажатии клавиши **[Esc]** RSL вызывает процедуру обработки команд меню с идентификатором, равным 256. Если она вернет **NULL** или не вернет никакого значения, то обработка меню завершится, и **RunMenu** возвратит значение **FALSE**. Если же завершение обработки меню по клавише **[Esc]** нежелательно, то для идентификатора, равного 256, или для всех необрабатываемых идентификаторов, нужно возвращать значение **FALSE**.

Обработка диалоговых окон

Процедура AddScroll

**AddScroll (dlg:TRecHandler, data:Object [, numCol:Integer, colArray:TArray ,
proc:Variant, rdOnly:Bool, focused:Bool] [, NumBar:Integer])**

Процедура позволяет создать область прокрутки диалоговой панели. Используется при обработке события [DLG_PREINIT](#)⁽⁹³⁾.

Параметры:

dlg – объект класса [TRecHandler](#)⁽¹¹⁶⁾ или производный от него. Указанный объект должен содержать описание диалоговой панели.

data – источник информации для окна прокрутки: объект класса [RsdRecordset](#)⁽¹³³⁾ или класса, производного от [ToolsDataAdapter](#)⁽¹⁰³⁾. Объект класса **RsdRecordset** должен реализовывать статический клиентский курсор.

numCol – количество колонок, отображаемых в области прокрутки, информация для которых содержится в массиве **collInfo**. Если значение параметра равно **0**, колонки создаются автоматически для каждого поля в источнике данных **data**. В этом случае в качестве заголовка колонки используется название поля.

collInfo – массив, задающий атрибуты колонок области прокрутки. Для каждой колонки атрибуты записываются в 6 последовательных элементов массива следующим образом:

- **0** – имя поля в источнике данных, обязательный атрибут;
- **1** – заголовок колонки, если элемент не задан, в качестве заголовка используется название поля;
- **2** – начальная ширина колонки, если элемент не задан, используется автоматическое вычисление ширины колонки;
- **3** – тип поля, возможные значения: 1 – редактируемое (значение по умолчанию), 2 – не редактируемое, 5 – прокручиваемое.
- **4** – количество знаков после точки для числовых полей;
- **5** – зарезервировано.

proc – используемый обработчик сообщений. Если обработчик не требуется, указывается значение **null**. В качестве обработчика могут быть указаны: имя RSL процедуры, ссылка на RSL процедуру, ссылка на метод RSL объекта, полученная при помощи метода [R2M](#)⁽²⁸⁶⁾.

rdOnly – признак, определяющий возможность редактирования сведений в области прокрутки. Возможные значения:

- **TRUE** – редактирование запрещено, независимо от типа колонки.

- **FALSE** – редактирование разрешено.

focused – признак, позволяющий управлять фокусом ввода. Возможные значения:

- **TRUE** – при отображении диалоговой панели фокус ввода устанавливается на область прокрутки.
- **FALSE** – фокус ввода устанавливается на первое поле или поле, заданное при обработке сообщения [DLG_INIT](#)⁽⁹⁴⁾.

NumBar – номер элемента типа **Bar**, расположенного на панели, в который необходимо добавить скроллинг. По умолчанию значение параметра равно "0". Нумерация элементов типа **Bar** и порядок работы инструмента с этими элементами зависит от того, в какой последовательности элементы добавлялись на панель.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **TRUE** – при успешном выполнении процедуры.
- **FALSE** – в случае возникновения ошибок.

Пример 1.

```
dlg = TRecHandler ("test2", null, true);
macro MsgProcScroll (dlg, cmd, id, key)
    if (cmd == DLG_KEY)
        if (key == 319) // F5
            return CM_INSERT
        end;
    end;
end;
macro MsgProc (dlg, cmd, id, key)
    if (cmd == DLG_PREINIT)
        AddScroll (dlg, ob, null, null, @MsgProcScroll,
false, false);
    end;
end;
if (RunDialog (dlg, @MsgProc))
    println ("OK")
end;
```

Пример 2.

```
// Ресурс EXAMPLE представляет собой пользовательскую
панель, в библиотеке usrlib.lbr, которую можно создать при
помощи утилиты rse32.exe.

// Для данного примера, это должна быть панель с любой
рамкой, на которой присутствуют 3 элемента типа BAR
(прямоугольник) с одинарной рамкой, в которых будут
размещаться скроллинги, добавляемые процедурами AddScroll

// В пользовательской схеме также должна быть создана
таблица DPAY_DOC_DBT с полями t_applicationkind,
t_applicationkey, t_oper. Данные из этой таблицы будут
показываться в скроллингах, предварительно отобранных при
помощи SELECT, Подробно смотри в примере:

import RSD;
var dlgPanel = TRecHandler("EXAMPLE", "usrlib.lbr", true);
```

```
/*Диалоговое окно*/
var rs, rs1, rs2, str; /*Основной скролинг*/
const K_ENTER = 13;
const K_ESC = 27;
const K_F5 = 319;
const K_F6 = 320;
const K_F8 = 322;
var Columns = TArray(), NumColumns = 0;
/*Обработка заголовков столбцов*/
macro AddColumn(ar, ind, fld, head, width, edit)
ar.value(ind * 6 + 0) = fld; // fieldName - имя поля в
рекордсете. Этот атрибут необходимо задать обязательно.
ar.value(ind * 6 + 1) = head; // header заголовок колонки.
Если атрибут не задан, то в качестве заголовка используется
название поля.
ar.value(ind * 6 + 2) = width; // width начальная ширина
колонки. Если атрибут не задан, то используется
автоматическое вычисление ширины колонки.
ar.value(ind * 6 + 3) = edit; // fldType (2 = FBT) - тип
поля. 1 - редактируемое, 2 - не редактируемое, 5 -
прокручиваемое. Если атрибут не задан, то используется 1.
ar.value(ind * 6 + 4) = null; // decPoint - количество
знаков после точки для числовых полей.
ar.value(ind * 6 + 5) = 0; // reserve - зарезервировано
NumColumns = NumColumns + 1;
end;
/* обработка клавиш в наложенных через AddScroll
скролингах*/
macro Proc_AddScroll(rs, cmd, id, key, numscrol)
var CM_FLAG = CM_DEFAULT; /* возвращаемое значение по
умолчанию*/
if(cmd == DLG_SWITCH)
// CM_FLAG = CM_IGNORE; // так можно реализовать запрет
переключения между скроллингами и подложкой
elif(cmd == DLG_KEY)
if(key == K_F5)
rs2.Close(); // закрываем рекордсет, потому что необходимо
создать заново
CM_FLAG = CM_UPDATE_ADDSCROLL; // обновить скролинг -
полная перезагрузка, с вызовом AddScroll на событии
DLG_PREINIT
end;
elif(cmd == DLG_SETFOCUS)
UpdateFields(dlgPanel);
end;
return CM_FLAG;
end;
/* обработка нажатия клавиш в диалоговом окне*/
macro Proc_Dialog(dlgPanel, cmd, id, key, numscrol)
var CM_FLAG = CM_DEFAULT; /* возвращаемое значение по
умолчанию*/
if(cmd == DLG_SWITCH)
// CM_FLAG = CM_IGNORE; //так можно реализовать запрет
переключения между скроллингами и подложкой
elif(cmd == DLG_PREINIT)
/* Строим скролинг дополнительных параметров */
str = "SELECT T_iapplicationkind, t_applicationkey, t_oper";
```

```

str = str + "FROM DPAY_DOC_DBT WHERE t_iapplicationkind = 1";
rs1 = RsdRecordSet(RsdCommand(str), RSDVAL_CLIENT, RSDVAL_STATIC);
str = "SELECT T_iapplicationkind, t_applicationkey, t_oper";
str = str + "FROM DPAY_DOC_DBT WHERE t_iapplicationkind > 1";
rs2 = RsdRecordSet(RsdCommand(str), RSDVAL_CLIENT, RSDVAL_STATIC);
AddColumn(Columns, NumColumns, "T_iapplicationkind", "AppKind", 5, 1);
AddColumn(Columns, NumColumns, "t_applicationkey", "AppKey", 200, 1);
AddColumn(Columns, NumColumns, "T_oper", "Oper", 7, 1);
AddScroll(dlgPanel, rs1, NumColumns, Columns, "Proc_AddScroll", false, true, 0);
AddScroll(dlgPanel, rs2, NumColumns, Columns, "Proc_AddScroll", false, false, 1);
AddScroll(dlgPanel, rs2, NumColumns, Columns, "Proc_AddScroll", false, false, 2);
elif(cmd == DLG_KEY)
if(key == K_F5)
rs1.Close(); // закрываем рекордсет, потому что необходимо создать заново
rs2.Close(); // закрываем рекордсет, потому что необходимо создать заново
CM_FLAG = CM_UPDATE_ADDSCROLL; // обновить скроллинг - полная перезагрузка, с вызовом AddScroll на событии DLG_PREINIT
end;
end;
return CM_FLAG;
end;
macro Main_Proc()
if(RunDialog (dlgPanel, @Proc_Dialog, K_F6))
println("OK")
end;
return 0;
end;
Main_Proc();

```

Процедура RunDialog

RunDialog (dlg:TRecHandler [, proc:Variant]):Bool

Процедура предназначена для отображения на экране диалогового окна **dlg**.

Примечание.

Для программного закрытия диалоговой панели, вызванной указанной процедурой, необходимо при обработке сообщений [DLG_KEY](#)⁹³, [DLG_BUTTON](#)⁹⁴ процедурой обработки evProc вернуть значение **CM_SAVE** или **CM_CANCEL**.

Параметры:

dlg – объект класса [TRecHandler](#)⁽¹¹⁶⁾ или производный от него объект, представляющий собой диалоговое окно.

proc – используемый обработчик сообщений. Если обработчик не требуется, указывается значение **null**. В качестве обработчика могут быть указаны: имя RSL процедуры, ссылка на RSL процедуру, ссылка на метод RSL объекта, полученная при помощи метода [R2M](#)⁽²⁸⁶⁾.

SwitchKey – код клавиши, по которой будет выполняться переключение между панелью и скроллингом. Если параметр не передан, то переключение между панелью и скроллингом выполняется только с помощью мыши.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **TRUE** – в случае, если пользователь сохранил данные в диалоговом окне.
- **FALSE** – при возникновении ошибок.

Примечание.

Во время выполнения процедуры RunDialog не осуществляется журнализация возникающих изменений.

Пример.

```
record id (name) dialog;  
RunDialog (id);  
[Имя #####  
Адрес #####]  
(id.Name, id.Address)
```

Приведенный в этом примере диалог имеет два поля: Name и Address.

Процедура DisableValidation

DisableValidation (dlg:Variant, id:Integer)

Процедура отключает проверку корректности даты в поле **id** диалогового окна **dlg**. Процедуру можно вызвать при обработке сообщения **DLG_INIT**.

Процедура UpdateFields

UpdateFields (dlg:TRecHandler [, id:Integer])

Процедура используется для обновления значения поля с **id** диалоговой панели **dlg**. Если параметр **id** не задан, процедура выполняет обновление данных во всех полях диалоговой панели.

Процедура SetFocus

SetFocus (dlg:TRecHandler, id:Integer)

Процедура устанавливает фокус ввода в поле **id** диалоговой панели **dlg**.

Процедура MsgBox

MsgBox (mes)

Процедура выводит на экран диалоговое окно с сообщением, заданным параметром **mes**. В качестве параметра могут быть переданы данные любого типа.

Чтобы сообщение занимало несколько строк, необходимо разделить его текст на части при помощи вертикальной черты "|". Удаление сообщения с экрана производится при помощи клавиши **[Esc]**.

Пример.

```
MsgBox ("Это первая строка|Это вторая строка");
```

Процедура DisableFields

DisableFields (dlg:TRecHandler [, id:Integer])

Процедура делает поле **id** диалоговой панели **dlg** недоступным для корректировки.

Если параметр **id** не задан, процедура вводит запрет на редактирование всех полей указанной диалоговой панели.

Процедура EnableFields

EnableFields (dlg:TRecHandler [, id:Integer])

Процедура отменяет запрет на редактирование поля **id** диалоговой панели **dlg**, установленный процедурой [DisableFields](#)²⁴². Если параметр **id** не задан, процедура разрешает редактирование всех полей указанной диалоговой панели.

Процедура SetTimer

SetTimer (dlg:TRecHandler [, timeOut:Integer, set:Bool])

Процедура устанавливает таймер для диалоговой панели **dlg**. После установки таймера обработчику сообщений данной панели через одинаковые промежутки времени, заданные в миллисекундах параметром **timeOut**, будет посылаться сообщение **DLG_TIMER**. По умолчанию данный параметр принимается равным 3000 (3 секунды).

Чтобы отключить ранее установленный таймер, необходимо вызвать процедуру **SetTimer**, передав ей в качестве параметра **set** значение **FALSE**. В этом случае параметр **timeOut** игнорируется.

При успешном выполнении процедуры она возвращает значение **TRUE**, в противном случае – **FALSE**.

Пример.

В диалоговой панели timer с интервалом в 1 секунду инкрементируется поле Counter:

```
record tm (timer) dialog;
Macro MesProc (dlg,cmd,id)
if (cmd == DLG_INIT)
SetTimer (dlg,1000); /* Устанавливаем таймер на 1сек. */
Elif (cmd == DLG_TIMER)
/* Увеличиваем счетчик и обновляем поля на экране */
dlg.Counter = dlg.Counter + 1;
UpdateFields (dlg);
end;
return CM_DEFAULT
end;
RunDialog (tm, @MecProc);
```

Процедура MsgBoxEx

MsgBoxEx (val:Variant [, flags:Integer, defInd:Integer, title:String, statLn:String]) : Integer

Процедура выводит диалоговое окно с сообщением, задаваемым параметром **val**. Параметр **val** может иметь любой тип, он автоматически преобразуется в строку.

Параметр **flags** представляет собой сумму констант, определяющих набор кнопок, которые будут отображаться в диалоговом окне. При этом используются следующие константы:

- **MB_OK** – кнопка "ОК".
- **MB_YES** – кнопка "Да".
- **MB_NO** – кнопка "Нет".
- **MB_CANCEL** – кнопка "Отмена".
- **MB_ERROR** – константа, задающая цвет фона, применяемый для сообщений об ошибках.

Если параметр равен **0**, то в диалоговом окне не будет ни одной кнопки. Если параметр не задан, то по умолчанию он принимает значение **MB_OK**, то есть в диалоговом окне будет присутствовать кнопка "ОК".

Параметр **defInd** определяет кнопку, выбранную по умолчанию. Возможны следующие значения:

- **IND_OK**.

- **IND_YES.**
- **IND_NO.**
- **IND_CANCEL.**

В параметре **title** задается заголовок диалогового окна.

В параметре **statLn** указывается текст, отображаемый в статус-строке.

Процедура возвращает код выбранной кнопки. При этом используются те же значения, что и для параметра **defInd**. В случае ошибки выполнения процедура возвращает код **IND_ERROR**.

Обработка скроллинга

Макропроцедуры модуля **RslScr**

Раздел содержит описание переменных и макропроцедур обработки скроллинга, определенных в макромодуле **RslScr**.

При написании программ для обработки скроллинга пользователь должен с помощью команды **IMPORT**⁴⁴ подключить к своей программе макромодуль **RslScr**.

Переменные

В макромодуле определены следующие глобальные переменные:

LastUsed – индекс последней заполненной строки в скроллинге.

Nrec – количество строк в скроллинге.

Nfields – количество колонок в скроллинге.

Scrff – ссылка на скроллируемый файл.

Apos – массив физических адресов записей для каждой строки скроллинга.

DlgFields – массив имен колонок скроллинга.

FileFields – массив номеров полей в файле для каждой колонки скроллинга.

Макропроцедуры

Процедура **SetScroll**

SetScroll (FILE, { DlgName, FileName } ...)

Данная процедура устанавливает соответствие имен полей в файле и в диалоговой панели. Эта процедура должна вызываться перед вызовом процедуры [RunDialog](#)²⁴⁰.

Параметры:

FILE – идентификатор файла, по которому будет производиться прокрутка.

DlgName – имя колонки в диалоговой панели.

FileName – имя поля в файле **FILE**, данные из которого должны выводиться в колонку с именем **DlgName**. Если имя поля в файле совпадает с именем колонки, то в качестве этого параметра можно указать значение **NULL**.

Пары “**DlgName** – **FileName**” должны быть указаны для каждой колонки в скроллинге.

Процедура FindRow

FindRow (dlg, id)

Процедура возвращает номер строки скроллинга в диалоге **dlg** для поля с номером **id**. Если поле **id** не принадлежит области скроллинга, то процедура возвращает значение **-1**.

Процедура FindCol

FindCol (dlg, id)

Процедура возвращает номер колонки скроллинга в диалоге **dlg** для поля с номером **id**. Если поле **id** не принадлежит области скроллинга, то процедура возвращает значение **-1**.

Процедура FillDown

FillDown (dlg, pos)

Процедура заполняет область скроллинга в диалоге **dlg** данными из файла. Первая строка скроллинга получает данные из записи файла с физическим адресом **pos**.

Процедура FillUp

FillUp (dlg, pos)

Процедура заполняет область скроллинга в диалоге **dlg** данными из файла. Последняя строка скроллинга получает данные из записи файла с физическим адресом **pos**.

Процедура SetDlgFields

SetDlgFields (dlg, row)

Процедура заполняет строку скроллинга с номером **row** данными из файла для диалога **dlg**.

Процедура ScrollMes

ScrollMes (dlg, cmd, id, key)

Стандартная процедура для обработки сообщений диалога с областью скроллинга.

Эта процедура должна вызываться для всех необработанных программой пользователя сообщений.

Процедура UserFill

UserFill (dlg)

Эта процедура вызывается каждый раз перед перерисовкой области скроллинга. Оригинальная процедура не выполняет никаких действий. Она может быть заменена при помощи процедуры [ReplaceMacro](#)²⁹¹ на процедуру, выполняющую нужные пользователю действия.

Макропроцедуры модуля rsIx

Раздел содержит описание макропроцедур обработки скроллинга, определенных в макромодуле **rsIx**.

Процедура AddMultiAction

AddMultiAction (rs:Object, key:Integer):Bool

Процедура позволяет зарегистрировать код клавиши **key** для запуска перебора выбранных записей в скроллинге. Процедуру необходимо использовать при обработке сообщения [DLG_INIT](#)⁹⁴. Для скроллинга, реализованного с помощью процедуры [AddScroll](#)²³⁷, процедуру **AddMultiAction** необходимо использовать при обработке сообщения [DLG_INLOOP](#)⁹⁴. Для скроллингов, встроенных в диалоговую панель с помощью процедуры [RunScroll](#)²⁴⁸, также возможно выполнение процедуры **AddMultiAction** при обработке сообщения [DLG_INLOOP](#)⁹⁴.

Примечание.

Если необходимо зарегистрировать несколько клавиш, указанную процедуру следует вызвать требуемое количество раз.

Параметры:

rs – объект класса, указанного в процедурах [RunScroll](#)²⁴⁸, [AddScroll](#)²³⁷ или [TrecHandler](#)¹¹⁶.

key – код клавиши, который требуется использовать для запуска перебора записей.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **TRUE** – в случае успешного выполнения.
- **FALSE** – в случае возникновения ошибки.

Процедура GetMultiCount

GetMultiCount (rs:Object):Integer

Процедура возвращает количество выбранных записей в скроллинге; вызывается только при обработке сообщений **DLG_MSEL....**

Параметры:

rs – объект класса, указанного в процедурах [RunScroll](#)²⁴⁸, [AddScroll](#)²³⁷ или [TrecHandler](#)¹¹⁶.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **количество выбранных записей** – в случае успешного выполнения.
- **-1** – если количество записей не известно (в случае использования инвертированного выбора).

Процедура GoToScroll

GoToScroll (obj:Object)

Процедура позиционирует окно прокрутки на текущую запись в источнике данных **obj**. Указанная процедура может использоваться только в процедурах обработки сообщений от окна прокрутки или диалоговой панели.

Параметры:

obj – объект класса, указанного в процедурах [RunScroll](#)²⁴⁸, [AddScroll](#)²³⁷ или [TrecHandler](#)¹¹⁶, данные которого будут отображаться в окне прокрутки.

Пример.

```
macro Proc_RunScroll (rs, cmd, id, key, numscrol)
...
  if (cmd == DLG_INIT)
    rs.movenext ();
    rs.movenext ();
    rs.movenext ();
    GoToScroll ();
  end;
...
end;
```

Процедура RunScroll

RunScroll (data:Object, numCol:Integer, collInfo:TArray, uniqName, proc:Variant, head:String, stLine:String, rdOnly:Bool, x:Integer, y:Integer, cx:Integer, cy:Integer):Bool

Процедура производит запуск окна прокрутки наборов данных RSD; может быть использована для обработки нескольких выделенных строк скроллинга.

Примечание.

Сортировка записей полученного окна прокрутки выполняется при помощи щелчка левой кнопки мыши по заголовку колонки, записи которой необходимо упорядочить. При первом щелчке выполняется сортировка по возрастанию, при повторном – по убыванию.

Для программного закрытия окна прокрутки, запущенного указанной процедурой, и выбора текущей записи необходимо при обработке [сообщения](#)⁽⁹³⁾ **DLG KEY**⁽⁹³⁾ процедурой обработки **evProc** для требуемой клавиши вернуть значение **CM_SELECT**.

Примечание.

Во время выполнения процедуры RunScroll не осуществляется журнализация возникающих изменений.

Параметры:

data – источник данных для окна прокрутки (объект класса [RsdRecordSet](#)⁽¹³³⁾ или класса, производного от инструментального класса [ToolsDataAdapter](#)⁽¹⁰³⁾). Объект класса **RsdRecordset** должен реализовывать статический клиентский курсор.

numCol – количество отображаемых колонок в области прокрутки, сведения для которых содержатся в массиве **collInfo**. Если значение параметра равно **0**, колонки создаются для каждого поля в источнике данных **data**. При этом в качестве заголовка колонки используется название поля.

collInfo – массив, задающий атрибуты колонок области прокрутки. Для каждой колонки атрибуты записываются в 6 последовательных элементов массива:

- **0** – имя поля в рекордсете, обязательный атрибут;
- **1** – заголовок колонки, если заголовок не задан, используется название поля;
- **2** – начальная ширина колонки, если ширина не задана, используется автоматическое вычисление ширины колонки;

Примечание.

Нужно учесть, что в ИБС RS-Bank V.6 значение этого элемента массива, заданное пользователем в макросе, который реализует пользовательский интерфейс, увеличивается в любых скроллингах.

- **3** – тип поля, возможные значения: 1 – редактируемое (значение по умолчанию); 2 – не редактируемое; 5 – прокручиваемое.
- **4** – количество знаков после точки (для числовых полей);
- **5** – зарезервировано.

uniqName – уникальное наименование окна просмотра. Указанное наименование используется для сохранения и восстановления изменений, которые вносятся во внешний вид колонок: ширины, позиции на экране, видимости. Атрибуты колонок сохраняются в ветке `COMMON\TOOLS\COLUMNWIDTH` пользовательского реестра. В результате при повторном вызове процедуры **RunScroll** с заданным параметром **uniqName** колонки скроллинга будут иметь атрибуты, сохраненные в настройке реестра. Атрибуты колонок, передаваемые в процедуру **RunScroll** из макроса, при этом не учитываются. Если наименование окна просмотра не задано, все визуальные атрибуты колонок принимают значения, заданные в макросе.

proc – процедура обработки сообщений окна просмотра (имя глобальной процедуры, ссылка на процедуру, ссылка на метод RSL класса). Если обработчик не требуется, используется значение **Null**.

head – строка, задающая заголовок окна просмотра.

stLine – строка, задающая содержимое статусной строки.

rdOnly – признак "только для чтения". Возможные значения:

- **TRUE** (или не задано) – данные окна просмотра недоступны для корректировки.
- **FALSE** – данные окна просмотра доступны для корректировки.

x, y – координаты левого верхнего угла окна просмотра:

- **x** – расположение окна по горизонтали, если параметр равен -1, выполняется центрирование окна по горизонтали;
- **y** – расположение окна по вертикали, если параметр равен -1, выполняется центрирование окна по вертикали.

cx – ширина окна просмотра.

cy – высота окна просмотра.

Примечание.

Если параметры не заданы, окно просмотра принимает максимально возможный размер в пределах клиентской области окна.

Возвращаемое значение

Процедура возвращает одно из значений:

- **TRUE** – в случае, когда процедура обработки сообщений в ответ на сообщение **DLG_KEY**⁹³ возвращает значение **CM_SELECT**. При этом выбранная в окне прокрутки запись становится текущей и в источнике.
- **FALSE** – в остальных случаях (например, если обработчик сообщения вернул значение **CM_CANCEL**; при этом выполняется закрытие окна прокрутки).

Примеры.

```
import RSD,rcw;
// Создаём клиентский статический курсор
cn = RsdConnection("Northwind");
cmd = RsdCommand (cn,"select * from Customers");
```

```

rs = RSDRecordset(cmd , RSDVAL_CLIENT, RSDVAL_STATIC );

// Пример 1. Просматриваем все поля набора данных данные
RunScroll (rs);

// Пример 2. Просматриваем только заданные поля.
// Вспомогательная процедура подготовки информации с
атрибутами полей
macro AddCol (ar,ind, fld, head, width, rdonly)
    ar.value (ind * 6)      = fld;
    ar.value (ind * 6 + 1) = head;
    ar.value (ind * 6 + 2) = width;
    ar.value (ind * 6 + 3 ) = 0;    // fldType
    ar.value (ind * 6 + 4 ) = -1;   // decPoint
    ar.value (ind * 6 + 5 ) = 0;    // reserv
end;

col = TArray;
AddCol (col, 0, "CompanyName", "Компания", null, true);
AddCol (col, 1, "ContactName", "Контактное лицо",      null,
true);
AddCol (col, 2, "Address", null,      null, true);
RunScroll (rs, 3, col);

// Пример 3. Обрабатываем событие DLG_KEY.
macro EvProc (rs, cmd, id, key )
    if ((cmd == DLG_KEY) and (key == 13))
        return CM_SELECT;
    end;
end;

if (RunScroll (rs,null,null,null,@EvProc))
    println ("Выбрана запись: ", rs.Value (0));
end;

// Пример 4. Использование вспомогательного класса для
обработки в ОО стиле.
class RslScroll
    private var colInfo;
    private macro numCol
        if (colInfo)
            return colInfo.size / 6
        end
    end;

macro AddCol (ind, fld, head, width, fldType,decPoint)
    if (not colInfo)
        colInfo = TArray;
    end;
    colInfo.value (ind * 6)      = fld;
    colInfo.value (ind * 6 + 1) = head;
    colInfo.value (ind * 6 + 2) = width;
    colInfo.value (ind * 6 + 3 ) = fldType;
    colInfo.value (ind * 6 + 4 ) = decPoint;
    colInfo.value (ind * 6 + 5 ) = 0;    // reserv
end;

```

```

macro EvProc (rs, cmd, id, key)
end;

macro Run (rs,name,Head, StLn, rdOnly)
    return RunScroll (rs, numCol, colInfo,name,
R2M(this,"EvProc"), Head, StLn, rdOnly);
end;
end;

// Производный класс задаёт колонки и переопределяет
процедуру обработки сообщений
class (RslScroll) MyScroll
    InitRslScroll;

    macro EvProc (rs, cmd, id, key)
        println ("--- ", "Cmd = ", CmdName (cmd), ", FldId =
", id, ", Key = ", key);
    end;
    AddCol ( 0, "CompanyName", "Компания", null, true);
    AddCol ( 1, "ContactName", "Контактное лицо", null,
true);
    AddCol ( 2, "Address", null, null, true);
end;
MyScroll.Run (rs);
OnError(e)
println (e.message);
if (IsEqClass ("TRsdError", e.err))
    i=0;
    while( i< e.err.environment.errorcount )
        println(e.err.environment.error(i).descr);
        printprops (e.err.environment.error(i));
        i=i+1;
    end;
    e.err.environment.ClearErrors;
end;

```

Процедура UpdateFields

UpdateFields (obj:Object)

Процедура выполняет обновление полей текущей записи на экране. В качестве параметра **obj** используется объект-источник данных. При этом, в отличие от процедуры **UpdateScroll**, данные из хранилища не запрашиваются. Выводятся текущие значения текущей записи, которые могли быть изменены в коде.

Процедура UpdateScroll

UpdateScroll (obj:Object, mode:Integer)

Процедура выполняет обновление окна прокрутки набора данных, заданного параметром **obj**, в соответствии с режимом обновления **mode**. Процедура может быть вызвана только в контексте процедуры обработки сообщений от окна прокрутки или диалоговой панели.

Параметры:

obj – объект класса, указанного в процедурах [RunScroll](#)²⁴⁸, [AddScroll](#)²³⁷ или [TrecHandler](#)¹¹⁶, данные которого будут отображаться в окне прокрутки.

mode – режим обновления. Параметр принимает одно из значений:

- **2** – обновление только текущей записи набора (значение по умолчанию);
- **3** – обновление всей области прокрутки;
- **4** – обновление страницы данных и окна прокрутки, начиная с текущей записи;
- **5** – обновление страницы данных и области прокрутки.

Контроль ввода записи в скроллинг

Раздел содержит описание процедур, используемых для контроля вводимой в скроллинг записи.

Процедура GetScrollFieldValue

GetScrollFieldValue(numfld:Integer, value:Undef):Bool

Процедура позволяет получить значение поля текущей строки скроллинга.

Параметры:

numfld – идентификационный номер поля.

value – возвращаемый параметр, в который передается значение поля, при этом тип значения соответствует типу поля текущей строки скроллинга.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **TRUE** – при успешном выполнении процедуры.
- **FALSE** – в случае возникновения ошибки.

Процедура SetScrollFieldValue

SetScrollFieldValue(numfld:Integer, newvalue:Undef, [strlen:Integer]):Bool

Процедура задает новое значение поля текущей строки скроллинга.

Параметры:

numfld – идентификационный номер поля.

newvalue – новое значение поля. Тип значения должен соответствовать типу поля текущей строки скроллинга.

strlen – длина строки. Параметр используется только для полей строкового формата. Значение параметра не должно превышать максимального размера строкового поля источника данных, по которому построен скроллинг.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **TRUE** – при успешном выполнении процедуры.
- **FALSE** – в случае возникновения ошибки.

Пример.

```
macro EvProc_RunScroll(rs, cmd, id, key, numscroll)
var getval;
    var CM_FLAG = CM_DEFAULT; /*возвращаемое значение по
умолчанию*/
    if (key == EXITFIELD)
        if (GetScrollFieldValue(0, getval) AND (getval ==
12345))
            SetScrollFieldValue(1, "Новое значение", 20);
            SetScrollFieldValue(2, $12345.12);
        end;
        if (GetScrollFieldValue(0, getval) AND (getval == 0))
            msgbox("Значение должно быть ненулевым!");
            CM_FLAG = CM_CANCEL;
        end;
    end;
return CM_FLAG;
end;
```

Процедура IsScrollEditMode

IsScrollEditMode():Bool

Процедура определяет, установлен ли режим ввода/редактирования записи непосредственно в скроллинге, реализованном с помощью процедур [RunScroll](#)²⁴⁸ или [AddScroll](#)²³⁷.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **TRUE** – в случае, если установлен режим ввода/ редактирования записи непосредственно в скроллинге.
- **FALSE** – режим просмотра скроллинга.

Пример.

Пример проверки дублирования записей при вводе/ редактировании записи в скроллинге:

```
var InsertMode = false;
macro EvProc_RunScroll(rs, cmd, id, key, numscrol)
var getval;
    var CM_FLAG = CM_DEFAULT; /*возвращаемое значение по
умолчанию*/
...
    //вводим признак режима ввода
    if ((cmd == DLG_KEY) AND (key == K_F9))
        InsertMode = true;
    end;
    if ((cmd == DLG_OUTREC) AND IsScrollEditMode()) //событие
- выход из записи в режиме ввода/ редактирования
        if (GetScrollFieldValue(0, getval))
            if (FindRecord(getval, InsertMode)) //FindRecord -
пользовательская функция. реализующая поиск записи в
источнике данных, по которому построен скроллинг, с учетом
InsertMode(режима ввода)
                CM_FLAG = CM_CANCEL;
            end;
        end;
    //сбрасываем признак режима ввода
    if (CM_FLAG != CM_CANCEL)
        InsertMode = false;
    end;
end;
return CM_FLAG;
end;
```

Работа с отладчиком RSL

Группа содержит процедуры, используемые для работы с отладчиком RSL.

В отладчике предусмотрена возможность выполнения интерактивных выражений (RSL кода). Вызов диалогового окна, предназначенного для ввода требуемого RSL кода, производится из отладчика с помощью клавиши **[Ctrl+I]**.

Список введенных выражений сохраняется в файле для повторного использования:
<Application Data>\R-Style Softlab\<Product Name>\RSL Debugger\InterExp.dbg.

Процедура Trace

Trace (...)

Процедура печатает свои параметры в окно трассировки отладчика RSL.

Процедура DebugBreak

DebugBreak

Если в системе установлен отладчик RSL, то данная процедура вызывает прерывание исполнения RSL программы и активизацию отладчика на следующей после **DebugBreak** инструкции.

Файлы и структуры

Язык RSL является составной частью информационной системы, разработанной программистами R-Style Softlab. При его работе используется внутренний словарь системы, который поставляется в дистрибутиве. Доступ к таблицам базы данных и структурам, описанным в этом словаре, осуществляется при помощи набора специальных процедур.

Процедуры этой группы работают со следующими видами файлов:

- с таблицами в базе данных;
- с текстовыми файлами,
- с файлами в формате DBF.

В списке формальных параметров этих процедур в качестве первого параметра обязательно указывается ссылка на таблицу или файл или структуру **id** (тип **V_FILE** или **V_RECORD**, соответственно). Любая из этих процедур возвращает значение типа **Bool**, если в ее описании не указано другое значение. При успешном выполнении возвращается **TRUE**, в противном случае **FALSE**.

В описании действий, выполняемых процедурами, встречаются термины "Текущая запись" и "Физическая позиция", которые необходимо пояснить.

Текущей считается запись, которая была прочитана (извлечена) и в этот момент находится в буфере данных. Она может отсутствовать:

- при первом обращении к таблице или файлу;
- после переустановки таблицы или файла процедурой **rewind**;
- в результате сбоя или другой внештатной ситуации.

Физическая позиция – это адрес записи, определяющий ее местоположение в таблице.

Процедура ConvertDDF

ConvertDDF (defName:String, outDir:String):Bool

Процедура позволяет выполнить конвертацию заданного файла словаря **def** в стандартный **ddf** формат.

Параметры:

defName – имя словаря **def**.

outDir – имя каталога, в котором необходимо создать файлы словаря **dff** (*file.dff*, *field.dff*, *index.dff*).

Возвращаемое значение:

Процедура возвращает одно из значений:

- **TRUE** – в случае успешного выполнения.
- **FALSE** – в случае возникновения ошибки.

Процедура CopyTblDef

CopyTblDef (inDef:String, outDef:String, inStruct:String, outStruct:String):Bool

Процедура используется для заимствования описания структуры таблицы из одного словаря в другой.

Параметры:

inDef – имя словаря, из которого производится заимствование.

outDef – имя словаря, в котором необходимо сохранить заимствованную структуру.

Примечание.

Если заимствование производится в пределах одного словаря, значения параметров **inDef** и **outDef** совпадают.

instruct – имя структуры, заимствование которой требуется выполнить.

outStruct – имя структуры, полученной в результате заимствования.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **TRUE** – в случае успешного выполнения.
- **FALSE** – в случае возникновения ошибки.

Процедура ExistFile

ExistFile (string [, integer])

Процедура возвращает **TRUE**, если файл (таблица базы данных), имя которого задано параметром **string**, существует. Параметр **integer** определяет где искать файл (таблицу):

- **0** – в каталоге текстовых файлов;
- **1** – в базе данных;
- **2** – в каталоге DBF файлов.

Если параметр *integer* не задан, поиск осуществляется в текущем каталоге.

Процедура Open

Open (fid, name:String, encode:String, fatal:Bool, eoType:Integer) : Bool

Открытие таблицы базы данных (файла). Следует отметить, что таблица открывается автоматически при первом обращении к ней, поэтому нет необходимости явно вызывать процедуру *open*. После открытия таблицы для неё автоматически выполняется процедура *rewind*.

Параметры:

fid – идентификатор таблицы.

name – имя таблицы, отличное от имени во внутреннем словаре. Это позволяет работать с несколькими таблицами, имеющими одинаковую структуру. Если имя таблицы соответствует имени в словаре базы данных, то в качестве параметра можно указать значение **NULL**.

encode – кодировка символов в таблице (файле). Параметр принимает следующие значения:

- "rsoem" – RSOEM кодировка;
- "rsansi" – RSANSI кодировка;
- "lcoem" – LCOEM кодировка;
- "lcansi" – LCANSI кодировка;
- "utf8" – UTF-8 кодировка;
- "utf16le" – UTF-16LE кодировка;
- "utf16be" – UTF-16BE кодировка.

fatal – признак, определяющий поведение процедуры в случае возникновения ошибки. Возможные значения:

TRUE – процедура аварийно завершает работу RSL-программы, если таблицу невозможно открыть.

FALSE – если таблицу невозможно открыть, процедура возвращает значение **FALSE** (значение по умолчанию).

eoType – стиль завершения строки при записи в текстовый документ. Параметр принимает следующие значения:

- **0** – стандарт DOS и Windows (CR LF) – значение по умолчанию;
- **1** – стандарт UNIX (LF);
- **2** – стандарт MAC (CR).

Пример.

```
File aa ("balance.dbt");  
File bb ("balance.dbt");  
File cc () txt;  
open (aa, "bal.dbt");  
open (bb, "balarch.dbt");  
name = datadir + "data.txt";  
open (cc, name);
```

Если файл (таблица) уже открыт, то последующий вызов **Open** будет проигнорирован до вызова процедуры [Close](#)²⁵⁸.

Пример.

```
open (aa, "data.dbf");           /* Этот вызов игнорируется  
(aa, "olddata.dbf");           */  
  
close (aa);                      /* Этот вызов сработает */  
open (aa, "olddata.dbf");
```

Процедура Close

Close (id)

Процедура закрывает таблицу (файл). Нужно отметить, что таблица (файл) закрывается автоматически при выходе из контекста программы, поэтому нет необходимости явно вызывать процедуру **Close**. Эта процедура используется при наличии в программе большого количества глобальных таблиц (файлов), так как они закрываются только после выполнения всей RSL-программы.

Примечание.

Внутренние словари базы данных, открытые в процессе работы программы, также закрываются только после выполнения всей RSL-программы.

Процедура ClearStructs

ClearStructs

Процедура освобождает из памяти структуры всех таблиц базы данных, открытых во время работы RSL-программы.

Процедура Create

Create (id [, filename] [, isTable:Bool] [, isP:Bool])

Процедура создает новую таблицу в базе данных или DBF-файл по описанию из словаря базы данных, соответствующему идентификатору **id**.

Если имя новой таблицы (файла) отличается от имени, заданного при описании указанного идентификатора, его необходимо определить во втором параметре **filename**.

Процедура не работает с текстовыми файлами. Если в программе определен текстовый файл с доступом на запись, то текстовый файл создается автоматически (если файл уже существовал, то он удаляется и создается новый).

Параметр **isTable** используется только при работе с таблицами базы данных и имеет тип **Bool**. Пользователь может установить два значения данного параметра:

- **TRUE** – при этом процедура **Create** аварийно завершает работу RSL-программы, если файл невозможно создать.
- **FALSE** – если файл невозможно создать, процедура возвращает значение **FALSE**. То же самое происходит и в том случае, если пользователь не установил никакого значения данного параметра.

Параметр **isP** используется только при создании таблиц баз данных. Если этот параметр равен **TRUE**, то процедура создает постоянную таблицу. Такая таблица должна открываться по тому описанию, по которому она была создана.

При успешном выполнении процедуры возвращается **TRUE**, в противном случае **FALSE**.

Пример.

```
file f("person.dbt", "bank.def");
if(not Create( f, "aa", True ))
    println("Error create");
end;
```

Процедура Clone

Clone (id [, filename] [, isTable:Bool] [, isP:Bool])

Процедура позволяет создавать новые или удалять информацию в уже существующих таблицах в базе данных и DBF-файлах.

Если в системе задан список каталогов для поиска DBF-файлов, новый файл создается в первом каталоге из списка, если не задан – в текущем.

Параметры **filename**, **bisTable** и **isP** аналогичны соответствующим параметрам процедуры [Create](#)²⁵⁸.

Процедура не работает с текстовыми файлами.

Пример.

```
File aa ("data.dbf") dbf /* Открывает файл data.dbf с
write;                  идентификатором aa */

clone (aa);              /* Удаляет информацию в
                        файле data.dbf */

clone (aa, "newdata.dbf"); /* Создает и открывает новый
                        файл newdata.dbf со
                        структурой как у data.dbf */
```

Если перед вызовом процедуры **clone** файл с заданным идентификатором **id** уже был открыт, то после клонирования он автоматически закроется.

Процедура ViewFile

ViewFile (id, name [, edit])

Процедура позволяет просматривать текстовые файлы и таблицы в базе данных.

В качестве параметра **id** передается идентификатор таблицы (файла), который нужно просмотреть.

Параметр **name** позволяет задать строку с именем текстового файла, необходимого для просмотра.

Если в качестве параметра **edit** задано значение **TRUE**, то просматриваемая таблица открывается в режиме корректировки. Параметр **edit** для текстового файла игнорируется.

Процедура DelFile

DelFile (string)

Процедура удаляет таблицу баз данных (файл) с именем **string** вместе с ее описанием во внутреннем словаре системы.

Процедура DropTable

DropTable (string)

Процедура удаляет таблицу баз данных с именем **string**, не удаляя ее описания из внутреннего словаря системы.

Процедура Next

Next (id [, integer])

Процедура считывает из таблицы или файла следующую за текущей запись или первую запись, если вызывается после процедуры **rewind**. Если указан параметр **integer**, процедура считает заданное в нем количество байт, символы конца строки в этом случае обрабатываются как и любые другие символы.

При работе с таблицами базы данных, если текущая запись оказывается последней, процедура возвращает значение **FALSE** и выполняет [Rewind](#)²⁶¹. Во всех остальных ошибочных случаях процедура аварийно завершает работу RSL-программы.

Для текстовых файлов процедура читает следующую строку файла.

Процедура Prev

Prev (id)

Процедура считывает из таблицы или файла предыдущую запись или последнюю, если вызывается после процедуры [Rewind](#)²⁶¹.

При работе с таблицами базы данных, если текущая запись оказывается последней, процедура возвращает значение **FALSE** и выполняет [Rewind](#)²⁶¹. Во всех остальных ошибочных случаях процедура аварийно завершает работу RSL-программы.

При успешном выполнении процедуры возвращается **TRUE**, в противном случае **FALSE**.

Процедура не работает с текстовыми файлами.

Процедура Rewind

ReWind (id)

Процедура переустанавливает таблицу или файл таким образом, что текущая позиция не изменяется, но изменяется поведение вызванных после **rewind** процедур [Next](#)²⁶⁰ или [Prev](#)²⁶¹: **next** извлечет первую запись, а **prev** – последнюю.

Процедура Insert

Insert (id [, string | size] [, integer] [, bool])

Процедура помещает в таблицу (файл) новую запись, используя значения полей из буфера данных. Если указан параметр **integer**, процедура запишет заданное в нем количество байт, символы конца строки в этом случае автоматически не добавляются.

Добавление происходит в соответствии с заданными ключевыми последовательностями.

В качестве первого параметра указывается ссылка на таблицу (файл) или структуру **id**.

Второй параметр принимает одно из двух значений:

- для текстовых файлов – строка для записи в файл;
- для таблицы базы данных – размер в байтах переменной части записи, добавляемой в таблицу. Если данный параметр отсутствует, используется размер, который был определен при выполнении операции чтения. Его значение можно узнать при помощи процедуры [GetVarSize](#)²⁶⁸.

Параметр **bool** используется только при работе с таблицами базы данных и имеет тип **V_BOOL**. Пользователь может установить два значения данного параметра:

- **TRUE** – в этом случае при дублировании ключа процедура **Insert** аварийно завершает работу RSL-программы.

- **FALSE** – при этом дублирование ключа не будет считаться ошибкой. Процедура в этом случае возвращает значение **FALSE**. То же самое происходит и тогда, когда пользователь не установил никакого значения данного параметра.

В текстовые и DBF-файлы записи добавляются только в конец.

Возвращаемое значение:

При успешном выполнении процедуры возвращается **TRUE**, в противном случае **FALSE**. Процедура по умолчанию возвращает значение **FALSE** в том случае, если таблица (файл) или запись захвачены или дублируется ключ.

Процедура Update

Update (id [, size] [, bool])

Процедура обновляет текущую запись в таблице баз данных (файле) **id**, используя значения полей из буфера данных.

В качестве первого параметра указывается ссылка на таблицу (файл) или структуру **id**.

Параметр **size** задается только для таблиц базы данных. Он имеет тип **Integer** и содержит размер в байтах переменной части записи, изменяемой в файле. Если данный параметр отсутствует, используется размер, который был определен при выполнении операции чтения. Его значение можно узнать при помощи процедуры [GetVarSize](#)⁽²⁶⁸⁾.

Параметр **bool** аналогичен соответствующему параметру процедуры **Insert**.

Процедура не работает с текстовыми файлами.

Возвращаемое значение:

Процедура по умолчанию возвращает значение **FALSE** в том случае, если таблица (файл) или запись захвачены или дублируется ключ.

Процедура Delete

Delete (id, bool)

Процедура удаляет из таблицы баз данных (файла) **id** текущую запись.

Параметр **bool** аналогичен соответствующему параметру процедуры [Insert](#)⁽²⁶¹⁾.

Процедура не работает с текстовыми файлами.

Возвращаемое значение:

При успешном выполнении процедуры возвращается **TRUE**, в противном случае **FALSE**. Процедура по умолчанию возвращает значение **FALSE** в том случае, если таблица (файл) или запись захвачены или дублируется ключ.

Процедура GetPos

GetPos (id)

Процедура возвращает физическую позицию текущей записи таблицы (файла). Для DBF-файлов это порядковый номер записи.

При успешном выполнении процедуры возвращается **TRUE**, в противном случае **FALSE**.

Процедура не работает с текстовыми файлами.

Процедура GetDirect

GetDirect (id [, recpos])

Процедура извлекает из таблицы (файла) запись, находящуюся в физической позиции, специфицированной вторым необязательным формальным параметром **recpos** (для DBF-файлов это порядковый номер записи). Значение **recpos** должно быть предварительно получено при помощи процедуры [getpos](#)⁽²⁶³⁾.

recpos = getpos (id);

Если параметр **recpos** не указан, то извлекается запись, для которой последний раз выполнялась процедура [getpos](#)⁽²⁶³⁾.

Пример.

```
File ff (account);           /* Запоминает позицию */
getpos (ff)

getdirect (ff)               /* Извлекает запись */
```

В качестве первого параметра указывается ссылка на таблицу (файл) или структуру **id** (тип **V_FILE** или **V_RECORD**).

При успешном выполнении процедуры возвращается **TRUE**. В случае возникновения ошибки данная процедура возвращает значение **FALSE**.

Процедура не работает с текстовыми файлами.

Процедура GetEQ

GetEQ (id)

Процедура осуществляет поиск записи таблицы, значение ключа которой равно указанному.

В качестве параметра указывается ссылка на таблицу или структуру **id** (тип **V_FILE** или **V_RECORD**).

При успешном выполнении процедуры возвращается **TRUE**. Данная процедура возвращает значение **FALSE**, если искомая запись отсутствует в файле. Во всех остальных ошибочных ситуациях происходит аварийное завершение RSL-программы.

Для того чтобы задать значение ключа для поиска, необходимо присвоить нужные значения тем полям записи таблицы, которые соответствуют сегментам текущего ключа.

Пример.

```
FILE          Счета /* Описана таблица Счета с нулевым
("account.dbt") key ключом, который, например, имеет два
0;             сегмента: дату и номер счета */

Счета.Date = date; /* Присвоены значения текущей даты
                  Счета.Acc полю Date и нужного номера счета полю
ount = "    010234"; Account */

getEQ (Счета);     /* Получена запись с нулевым ключом и
                  заданными значениями сегментов */
```

Процедура GetGT

GetGT (id)

Процедура осуществляет поиск записи таблицы, значение ключа которой больше указанного.

В качестве параметра указывается ссылка на таблицу или структуру **id** (тип **V_FILE** или **V_RECORD**).

При успешном выполнении процедуры возвращается **TRUE**. Данная процедура возвращает значение **FALSE**, если искомая запись отсутствует в таблице. Во всех остальных ошибочных ситуациях происходит аварийное завершение RSL-программы.

Для того чтобы задать значение ключа для поиска, необходимо присвоить нужные значения тем полям записи таблицы, которые соответствуют сегментам текущего ключа.

Пример.

```
FILE          Счета /* Описана таблица Счета с нулевым
("account.dbt") key ключом, который, например, имеет два
0;             сегмента: дату и номер счета */

Счета.Date = date; /* Присвоены значения текущей даты
                  С        полю Date и нужного номера счета
чета.Account = "    С        полю Account */
010234";

getGT (Счета);     /* Получена запись с нулевым ключом,
                  у которой значения сегментов больше
                  заданных */
```

Процедура GetGE

GetGE (id)

Процедура осуществляет поиск записи таблицы, значение ключа которой больше или равно указанному.

В качестве параметра указывается ссылка на таблицу или структуру **id** (тип **V_FILE** или **V_RECORD**).

При успешном выполнении процедуры возвращается **TRUE**. Данная процедура возвращает значение **FALSE**, если искомая запись отсутствует в таблице. Во всех остальных ошибочных ситуациях происходит аварийное завершение RSL-программы.

Для того чтобы задать значение ключа для поиска, необходимо присвоить нужные значения тем полям записи таблицы, которые соответствуют сегментам текущего ключа.

Пример.

```
FILE          Счета /* Описана таблица Счета с нулевым
("account.dbt") key ключом, который, например, имеет два
0;             сегмента: дату и номер счета */

Счета.Date = date; /* Присвоены значения текущей даты
                  С полю Date и нужного номера счета
чета.Account = "   полю Account */
010234";

getGE (Счета);    /* Получена запись с нулевым ключом,
                  у которой значения сегментов больше
                  или равны заданным */
```

Процедура GetLT

GetLT (id)

Процедура осуществляет поиск записи таблицы, значение ключа которой меньше указанного.

В качестве параметра указывается ссылка на таблицу или структуру **id** (тип **V_FILE** или **V_RECORD**).

При успешном выполнении процедуры возвращается **TRUE**. Данная процедура возвращает значение **FALSE**, если искомая запись отсутствует в таблице. Во всех остальных ошибочных ситуациях происходит аварийное завершение RSL-программы.

Для того чтобы задать значение ключа для поиска, необходимо присвоить нужные значения тем полям записи таблицы, которые соответствуют сегментам текущего ключа.

Пример.

```
FILE          Счета /* Описана таблица Счета с нулевым
("account.dbt") key ключом, который, например, имеет два
0;             сегмента: дату и номер счета */

Счета.Date = date; /* Присвоены значения текущей даты
                  С полю Date и нужного номера счета
чета.Account = "   полю Account */
010234";

getLT (Счета);    /* Получена запись с нулевым ключом,
                  у которой значения сегментов меньше
                  заданных*/
```

Процедура GetLE

GetLE (id)

Процедура осуществляет поиск записи таблицы, значение ключа которой меньше или равно указанному.

В качестве параметра указывается ссылка на таблицу или структуру **id** (тип **V_FILE** или **V_RECORD**).

При успешном выполнении процедуры возвращается **TRUE**. Данная процедура возвращает значение **FALSE**, если искомая запись отсутствует в таблице. Во всех остальных ошибочных ситуациях происходит аварийное завершение RSL-программы.

Для того чтобы задать значение ключа для поиска, необходимо присвоить нужные значения тем полям записи таблицы, которые соответствуют сегментам текущего ключа.

Пример.

```
FILE          Счета  /* Описана таблица Счета с нулевым
("account.dbt") key  ключом, который, например, имеет два
0;                сегмента: дату и номер счета */

Счета.Date = date; /* Присвоены значения текущей даты
                  С   полю Date и нужного номера счета
чета.Account = "    полю Account */
010234";

getLE (Счета);      /* Получена запись с нулевым ключом,
                   у которой значения сегментов меньше
                   или равны заданным*/
```

Процедура SetRecordAddr

SetRecordAddr (recld, fileld [, ind] [, offset] [, fix])

Процедура позволяет наложить структуру, заданную идентификатором **recld** на таблицу баз данных с идентификатором **fileld**. Это обеспечивает доступ к переменной части записи (так как структура переменной части не описывается при описании структуры файла), а так же использовать для доступа к постоянной части записи структуру, отличную от той, которая описывает структуру записи таблицы. Последняя возможность позволяет хранить в таблице записи разной структуры (union).

Параметры **ind** и **offset** типа **Integer**, а также параметр **fix** типа **Bool** позволяют определить смещение структуры **recld** в таблице **fileld**.

По умолчанию значения **ind** и **offset** считаются равными нулю. Если эти параметры заданы, то смещение вычисляется следующим образом:

$$GetRecordSize (recld) * ind + offset$$

Если значение **fix** равно **FALSE** или не задано, то структура **recld** накладывается с вычисленным смещением относительно начала переменной части записи. Если параметр **fix** равен **TRUE**, то структура накладывается с вычисленным смещением относительно начала постоянной части записи.

Пример.

```

file fl ("varlen.dbt") write
/* Таблица с записями переменной длины */
record rec ("vrec.rec");
/* Структура для доступа к переменной части */
macro ShowVarLen (n)
/* Параметр n задает количество структур rec в переменной
части */
    i = 0;
    while ( i < n )
        SetRecordAddr (rec, fl, i);
        println
            (rec.id, " ", rec.name);
        i = i + 1;
    end;
end;
while ( next (fl) )
    ShowVarLen (file.nRec);
end;

```

Если в переменную часть записи добавляются новые элементы, то для процедур [Insert](#)²⁶¹ и [Update](#)²⁶² необходимо указать новый размер переменной части.

Пример.

В этом примере считается, что в переменной части находится 5 элементов (структур vrec):

```

file fl ("varlen.dbt") write
/* Файл с записями переменной длины */
record rec ("vrec.rec");
/* Структура для доступа к переменной части */
update (fl, GetRecordSize (rec) * 5)

```

Наложить структуру можно и на постоянную часть записи:

```

file acc ("account.dbt") key 1000;
record dkr ("dkr.str");
println (FldOffset (acc,"D0"));
if (next (acc))
    SetRecordAddr (dkr,acc,0,FldOffset (acc,"D0"),TRUE);
    [ Debet      #####
      Credit    #####
      Rest      ##### ]      (dkr.D,dkr.K,dkr.R);
end;

```

Процедура PackVarBuff

PackVarBuff (file [, size])

Процедура "упаковывает" переменную часть записи таблицы баз данных, заданной идентификатором **file**. Размер переменной части определяется параметром **size**. Если параметр **size** не задан, то используется текущий размер переменной части.

Процедура возвращает новый размер переменной части записи и устанавливает данный размер текущим размером переменной части.

Процедура UnPackVarBuff

UnPackVarBuff (file)

Процедура "распаковывает" переменную часть записи таблицы баз данных, заданной идентификатором *file*, и возвращает ее новый размер. Данный размер становится текущим размером переменной части.

Процедура GetRecordSize

GetRecordSize (id)

Процедура возвращает размер в байтах структуры или таблицы (файла) с идентификатором, заданным параметром *id*. Это значение вычисляется на основании информации из внутреннего словаря базы данных.

Пример.

```
file fl ("account.dbt");  
record rec ("document.rec");  
fsize = GetRecordSize (fl);  
rsize = GetRecordSize (rec);
```

В качестве параметра указывается ссылка на файл или структуру *id* (тип **V_FILE** или **V_RECORD**).

При успешном выполнении процедуры возвращается **TRUE**, в противном случае **FALSE**.

Процедура GetVarSize

GetVarSize (id)

Эта процедура возвращает размер в байтах переменной части текущей записи для таблицы базы данных с идентификатором, заданным в параметре *id*.

Процедура FileName

FileName (id)

Процедура возвращает имя таблицы баз данных (файла). В качестве параметра указывается ссылка на таблицу (файл) или структуру *id*.

Если в качестве параметра *id* передать объект типа **TRecHandler**, то процедура вернет имя соответствующей структуры во внутреннем словаре системы.

Процедура NRecords

NRecords (id [, par:Bool])

Процедура возвращает количество записей в таблице базы данных (файле) с идентификатором, заданным в параметре **id**. Если необязательный параметр **par** равен **FALSE**, процедура всегда возвращает **-1**.

Процедура не работает с текстовыми файлами.

Процедура FldNumber

FldNumber (id)

Процедура возвращает количество полей в таблице базы данных (файле) с идентификатором, заданным в параметре **id**.

Процедура не работает с текстовыми файлами.

Процедура FldName

FldName (id, number)

Процедура возвращает имя поля с индексом **number** для таблицы базы данных (файла) с идентификатором, заданным в первом параметре **id**. Второй параметр **number** представляет собой переменную или константу типа **Integer**.

Если поле не найдено, возвращается пустая строка.

Процедура не работает с текстовыми файлами.

Процедура FldIndex

FldIndex (id, string)

Процедура возвращает индекс поля с именем **string** для таблицы баз данных (файла) с идентификатором, заданным в параметре **id** (тип **V_FILE** или **V_RECORD**). Если поле не найдено, процедура возвращает значение **-1**.

Второй параметр **string** представляет собой переменную или константу типа **String**.

Процедура не работает с текстовыми файлами.

Процедура FldOffset

FldOffset (id, string | number)

Процедура возвращает смещение в байтах поля с именем **string** или номером **number** для таблицы баз данных с идентификатором, заданным в параметре **id**.

Если поле не найдено, процедура возвращает значение, равное **0**.

Пример.

```
File ff ("account.dbt");
I = 0;
while (I < FldNumber (ff))
    println (I, " - ", FldOffset (ff, I))
end
```

Процедура ClearRecord

ClearRecord (id)

Процедура обнуляет буфер записи таблицы базы данных (файла), заданной идентификатором **id**, или объект типа **RECORD**.

Процедура не работает с текстовыми файлами.

Процедура SetBuff

SetBuff (id, addr)

Процедура явно устанавливает адрес памяти, заданный во втором параметре, для объекта типа **RECORD** с идентификатором **id**. Второй параметр, представляющий собой переменную с типом, соответствующим константе **V_MEMADDR**, должен содержать адрес памяти, переданный из программы на языке C.

Процедура Copy

Copy (id1, id2 [, flag:Integer])

Процедура копирует содержимое буфера **id2** в буфер **id1**. Параметры **id1** и **id2** представляют собой переменные типа ссылка на таблицу баз данных (файл) (**V_FILE**) или ссылка на структуру (**V_RECORD**).

В качестве параметров процедуры копирования могут также использоваться объекты стандартных классов **Tbfile** и **TRecHandler**.

Если оба буфера имеют одинаковую структуру, то выполняется побайтное копирование.

Если структура буферов различна, то для каждого поля из буфера **id2** будет проводиться поиск поля с таким же именем в буфере **id1**. Если такое поле найдено, и его тип совпадает с типом поля в буфере **id2**, выполняется побайтное копирование.

Для полей с разным типом предварительно, если возможно, выполняется преобразование. Поля в **id1**, для которых не обнаружено соответствующих полей в **id2**, обнуляются.

Процедура позволяет копировать поля типа **BLOB**.

Параметр **flag** задает режим копирования, если буфер представляет собой запись переменной длины, и может принимать следующие значения:

- **0** – процедура скопирует только постоянную часть записи (используется по умолчанию);
- **1** – процедура скопирует постоянную и переменную часть записи;
- **2** – процедура скопирует только переменную часть записи.

Данная процедура позволяет также выполнять копирование буферов DBF-файлов.

Процедура CopyBlob

CopyBlob ()

Данная процедура является аналогом процедуры [Copy](#)²⁷⁰ с параметром **flag**, равным **2**.

Если в таблице-приёмнике имеется поле типа **BLOB**, то при копировании записи необходимо сначала скопировать постоянную часть записи без **BLOB** и вставить новую запись. После этого необходимо скопировать **BLOB**. Если таблица-приёмник имеет просто переменную часть, то необходимо скопировать постоянную и переменную часть записи и после этого вставить запись в файл.

Пример.

```
// Файл-приёмник содержит поле BLOB
f1 = TBfile ("blobdata.dbt");
f1.next;
f2 = TBFile ("blobcopy.dbt","w");
copy (f2,f1);
f2.insert;
CopyBlob (f2,f1);
// Файл-приёмник содержит простую переменную часть
f1 = TBfile ("vardata.dbt","w");
f1.next;
f2 = TBFile ("varcopy.dbt","w");
copy (f2,f1,1);
f2.insert;
```

Процедура KeyNum

KeyNum (id [, newkey])

Процедура возвращает текущий номер ключа для таблицы базы данных с идентификатором *id*.

Если указан второй, необязательный параметр **newkey**, устанавливается новый номер ключа, равный значению **newkey**.

Процедура SetDelim

SetDelim (symbol, space)

Процедура устанавливает символы (максимальное количество 9) **symbol**, которые являются разделителями элементов в строках текстового файла. По умолчанию в качестве символов-разделителей используются символ табуляции "\t" и пробел. Если задано значение параметра **space**, все пробельные разделители обрабатываются аналогично другим разделителям.

Процедура работает только с текстовыми файлами.

Пример.

```
file f ("test.txt") txt;
SetDelim ("\t", true);
next (f);
println ( f (0) );
println ( f (1) );
println ( f (2) );
println ( f (3) );
println ( f (4) );
```

Процедура Status

Status [(parm)]

Процедура позволяет получить код ошибки в случае неудачного завершения процедур, работающих с таблицами баз данных.

Если процедуре передан необязательный параметр **parm**, процедура присвоит ему строку, содержащую сообщение об ошибке.

Пример.

```
if (not next (id))
    println ("Статус:", status);
end;
```

Процедура работает только с таблицами в базе данных.

Процедура ProcessTrn

ProcessTrn (TrnType:Integer, MacroName:String, Variant [, file1:File] [, file2:File] [, ...:File]):Bool

Процедура служит для выполнения транзакции при работе с языком RSL.

Параметр **TrnType** определяет тип транзакции, то есть указывает, что должно быть выполнено в рамках этой транзакции в вызывающем модуле. Значения, которые может принимать данный параметр, должны быть определены в вызывающем модуле. В рамках прикладных систем, разработанных в компании R-Style Software Lab., тип транзакции может быть следующим:

- **1** – транзакция рублевой проводки;
- **2** – транзакция валютной проводки;
- **3** – транзакция проводки валюты без рублей;
- **4** – транзакция проводки вкладчиков;
- **5** – ввод документов, счетов, клиентов, банков;
- **6** – транзакция рублевой проводки "как есть" (то есть без дополнительного заполнения полей рублевого документа);
- **7** – транзакция валютной проводки "как есть";
- **8** – транзакция проводки валюты без рублей "как есть";
- **10** – транзакция ввода параметров сделок.

Если в вызывающем модуле ничего выполнять не надо, то необходимо указать тип транзакции **0** или передать вместо типа транзакции значение **NULL**.

Если в рамках транзакции необходимо выполнить действия только в вызывающем модуле, то, кроме типа транзакции, больше никаких параметров для **ProcessTrn** указывать не нужно.

Параметр **MacroName** задает процедуру, которая вызывается для выполнения транзакции на языке RSL. В качестве параметра может быть передано одно из следующих значений:

- текстовая строка с именем глобальной макропроцедуры;
- текстовая строка с именем локальной макропроцедуры;
- ссылка на локальную макропроцедуру;
- ссылка на метод класса, полученная с помощью процедуры [R2M](#)²⁸⁶.

Макропроцедуры, вызываемые из процедуры **MacroName**, выполняются в рамках той же транзакции. После имени макропроцедуры указывается список идентификаторов таблиц баз данных, участвующих в транзакции на языке RSL. В рамках транзакции можно обращаться только к тем таблицам, идентификаторы которых указаны в этом списке.

Процедура **ProcessTrn** возвращает одно из двух значений:

- **TRUE** – если транзакция завершилась успешно, то есть макропроцедура **MacroName** выполнялась до конца, или была вызвана процедура [Exit](#)²⁹⁶.

- **FALSE** – если транзакция завершилась вызовом процедуры [AbortTrn](#)²⁷⁴, или пользователь ответил отрицательно на запрос повторения операции над заблокированной таблицей.

Вложенные транзакции не поддерживаются.

Процедура LoopInTrn

LoopInTrn (val)

Данная процедура изменяет действие процедур выполнения транзакции [ProcessTrn](#)²⁷² и [ProcessConTrn](#) в случае захвата файлов или записей.

Если параметр **val** равен **TRUE**, то процедура выполнения транзакции прерывает транзакцию и после паузы повторно ее начинает, а также повторно вызывает пользовательскую процедуру обработки транзакции.

Количество повторов задается параметром **REPCOUNT**. Время задержки задается параметрами **SLEEPTIME** и **RANDOMTIME**, причем время задержки определяется как сумма **SLEEPTIME** и случайного числа в диапазоне между **0** и **RANDOMTIME**. Эти параметры устанавливаются в файле *rsreq.ini*.

После выполнения заданного числа повторов процедура выполнения транзакции выводит диалоговое окно с запросом на продолжение повтора выполнения транзакции.

Процедура AbortTrn

AbortTrn()

Процедура завершает выполнение макросов, вызванных для выполнения транзакции, и производит откат всех изменений, внесенных в таблицы баз данных в рамках транзакции как на RSL, так и в вызывающем модуле на С или С++. Процедура [ProcessTrn](#)²⁷² в этом случае возвратит **FALSE**.

Если эта процедура вызвана вне транзакции, то она не выполняет никаких действий.

Процедура используется только при работе с таблицами базы данных.

Процедура SelectFile

SelectFile (file [, mask, head], sort, term)

Данная процедура выводит на экран стандартное диалоговое окно выбора файла.

Процедура возвращает **TRUE**, если файл был выбран. При этом параметр **file** принимает значение имени выбранного файла.

Параметр **mask** содержит строку, определяющую шаблон имени файла, который необходимо выбрать. По умолчанию данный параметр содержит название текущего каталога.

Параметр **head** содержит строку, которая будет выведена на экран в качестве заголовка окна выбора файла. По умолчанию в качестве этого параметра передается строка: "Выбор файла".

Параметр **sort** определяет порядок сортировки файлов по умолчанию. Возможные значения:

- **0** – по имени;
- **1** – по расширению;
- **2** – по времени;
- **3** – по размеру.

Параметр **term** используется при работе в трехзвенной архитектуре и позволяет указать, откуда следует осуществлять выбор файла. Возможные значения:

- **TRUE** – выбор файла выполняется на терминале.
- **FALSE** – выбор файла производится на сервере.

Пример.

```
SelectFile (@fl, "*.mac", "Заголовок", 0, true);
```

Процедура SelectFolder

SelectFolder (folder [, mask, head,] term)

Данная процедура выводит на экран стандартное диалоговое окно выбора подкаталога.

Процедура возвращает **TRUE**, если подкаталог был выбран. При этом параметр **folder** принимает значение имени выбранного подкаталога.

Параметр **mask** содержит строку, определяющую шаблон имени подкаталога, который необходимо выбрать. По умолчанию этот параметр содержит название текущего каталога.

Параметр **head** содержит строку, которая будет выведена на экран в качестве заголовка окна выбора подкаталога. По умолчанию в качестве этого параметра передается строка: "Выбор каталога".

Параметр **term** используется при работе в трехзвенной архитектуре и позволяет указать, откуда следует осуществлять выбор подкаталога. Возможные значения:

- **TRUE** – выбор подкаталога выполняется на терминале.
- **FALSE** – выбор подкаталога производится на сервере.

Пример.

```
SelectFolder (@fl, "*.mac", "Заголовок", true);
```

Процедура NeedFreeDB

NeedFreeDB ()

Данная процедура может быть вызвана в любом месте RSL-программы для указания интерпретатору на необходимость закрытия словарей при завершении выполнения RSL-программы.

Процедура WriteBlob

WriteBlob (file, value)

Процедура записывает значение переменной **value** в поле типа **BLOB** таблицы **file**.

Перед записью при помощи процедур **next**, **prev**, **get**, **insert** и т.п. в таблице следует установить требуемую позицию. После этого процедуру можно вызывать для каждой записываемой в поле **BLOB** переменной.

Процедура ReadBlob

ReadBlob (file, value)

Процедура считывает из поля типа **BLOB** таблицы **file** очередное значение и присваивает его переменной **value**. Если информация данного поля прочитана до конца, процедура возвращает значение **FALSE**, иначе – **TRUE**.

Перед выполнением процедуры необходимо установить позицию в таблице при помощи процедур **next**, **prev**, **get** и т.п. Далее процедуру **ReadBlob** можно вызывать до тех пор, пока она не вернет значение **FALSE**.

Пример.

```
f1 = TBfile ("blobdata.dbt");
v1 = TRecHandler ("var.str");
f1.next;
while (f1.ReadBlob (v1))
  [ ##### ] (v1.rec.f1,
v1.rec.f2);
end;
```

Управление файлами и каталогами

В этом разделе содержится описание процедур, с помощью которых осуществляется управление файлами и каталогами.

Большая часть перечисленных процедур входит в состав модуля **rsexts**, предназначенного для управления файлами и каталогами. В этот модуль также входит стандартный класс [TDirList](#)⁽¹⁶⁰⁾.

Процедура CopyFile

CopyFile (src:String, dst:String [, ind:Bool [, indHeading:String]]) : Bool

Процедура копирует файл **src** в файл **dst**. Исходный файл и файл назначения могут находиться как на терминале, так и на сервере. Если необязательный параметр **ind** равен **TRUE**, то во время копирования отображается индикатор прогресса. В параметре **indHeading** можно задать строку, которая будет отображаться в заголовке окна индикатора. Индикатор прогресса не отображается, если оба файла находятся на терминале.

Примечание.

Процедура **CopyFile** может обрабатывать файлы без указания расширения.

Процедура возвращает **TRUE** при успешном копировании и **FALSE** в противном случае.

Пример.

```
/* Копируем файл с сервера на терминал */
if (not CopyFile ("cvt.c", "$rslDir\cvt.c"))
    println ("Error Copy file");
end;
```

Процедура RenameFile

RenameFile (src:String, dst:String) : Bool

Процедура переименовывает файл **src** в файл с именем **dst**. Исходный файл может находиться как на сервере, так и на терминале. При этом файл назначения должен располагаться там же, где и исходный файл.

Чтобы эта процедура была доступна в макропрограмме пользователя, следует явно импортировать модуль **rsexts**.

Процедура возвращает **TRUE** при успешном переименовании и **FALSE** в противном случае.

Пример.

```
if (not RenameFile ("rslDir\cvt.c", "rslDir\cvt.c2"))
    println ("Error rename file");
end;
```

Процедура RemoveFile

RemoveFile (src:String) : Bool

Процедура удаляет файл, заданный параметром **src**. Файл может находиться как на сервере, так и на терминале. Чтобы эта процедура была доступна в макропрограмме пользователя, следует явно импортировать модуль **rsexts**.

Процедура возвращает **TRUE** при успешном удалении и **FALSE** в противном случае.

Пример.

```
if (not RemoveFile ("$rsldir\cvt.c2"))
    println ("Error remove file");
end;
```

Процедура ExistDir

ExistDir(name:String):Integer

Процедура выполняет проверку существования и доступности для записи каталога с указанным именем. Чтобы эта процедура была доступна в макропрограмме пользователя, следует явно импортировать модуль **rsexts**.

Параметры:

name – имя каталога.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **0** – каталог существует и доступен для записи;
- **1** – каталог существует, но недоступен для записи;
- **2** – каталог с указанным именем не существует.

Процедура MakeDir

MakeDir (name:String) : Bool

Процедура создает каталог, имя которого задается параметром **name**. Каталог может быть создан как на сервере, так и на терминале. Чтобы эта процедура была доступна в макропрограмме пользователя, следует явно импортировать модуль **rsexts**.

Процедура возвращает **TRUE** при успешном создании каталога и **FALSE** в противном случае.

Пример.

```
/* Создаем каталог rslDir на терминале */
if (not MakeDir ("rsldir"))
    println ("Error crating dir");
end;
```

Процедура RemoveDir

RemoveDir (name:String) : Bool

Процедура удаляет каталог, имя которого задается параметром **name**. Каталог может быть удален как на сервере, так и на терминале. Удаляемый каталог должен быть пустым.

Чтобы эта процедура была доступна в макропрограмме пользователя, следует явно импортировать модуль **rsexts**.

Процедура возвращает **TRUE** при успешном удалении и **FALSE** в противном случае.

Пример.

```
if (not RemoveDir ("rsldir"))
  println ("Error remove dir");
end;
```

Процедура GetCurDir

GetCurDir ([isRemote:Bool]) : String

Процедура возвращает название текущего каталога на сервере. Если задан параметр **isRemote** и его значение равно **TRUE**, то возвращается название текущего каталога на терминале.

Чтобы эта процедура была доступна в макропрограмме пользователя, следует явно импортировать модуль **rsexts**.

Пример.

```
/* Печатаем название текущего каталога на терминале */
println (GetCurDir (true));
```

Процедура SplitFile

SplitFile (pathName: String [, name: String [, ext: String]]) : String

Процедура разделяет полное имя файла на составляющие: каталог, имя файла, расширение имени файла.

Параметры:

pathName – строка, содержащая полное имя файла.

name – возвращаемое имя файла.

ext – возвращаемое расширение файла.

Возвращаемое значение:

Возвращаемым значением процедуры является имя каталога – строка типа **String**.

Процедура MergeFile

MergeFile (dirName: String ,name: String [, ext: String]): String

Процедура возвращает полное имя файла, составленное из компонент.

Параметры:

dirName – имя каталога.

name – имя файла.

ext – расширение имени файла.

Процедура FindPath

FindPath (name: String [, dirList: String [, defExt: String [, curDir: Bool]]]): String

Процедура ищет файл с именем **name** в указанном списке каталогов. По умолчанию процедура также осуществляет поиск файла в текущем каталоге.

Параметры:

name – имя файла, который нужно найти.

dirList – список каталогов для поиска, разделенных символом ";". Если данный параметр не задан, то поиск осуществляется в списке каталогов для поиска макрофайлов.

defExt – имя расширения файла по умолчанию, если оно не задано в параметре name. Если параметр не задан, то по умолчанию принимается MAC.

curDir – если этот параметр принимает значение **FALSE**, то поиск в текущем каталоге не производится.

Возвращаемое значение:

Процедура возвращает найденное имя файла либо пустую строку в случае неудачи.

Процедура GetSysDir

GetSysDir ([ndir: Integer]): String

Процедура возвращает используемые интерпретатором списки каталогов для поиска файлов с расширениями MAC, DBT, DBF и TXT.

Параметры:

ndir – тип списка каталогов, который необходимо вернуть. Параметр может принимать следующие значения:

- **0** – каталог txt-файлов;
- **1** – каталог dbt-файлов;
- **2** – каталог dbf-файлов;
- **3** – каталог mac-файлов.

Если параметр не задан, то возвращается список каталогов поиска mac-файлов.

Процедура GetIniFileValue

GetIniFileValue (iniFileName:String, keyName:String):String

Процедура ищет в файле с именем **iniFileName** строку, переданную в качестве параметра **keyName**, и возвращает найденную строку.

При считывании строки для поиска производится замена специального имени "%HOME" на имя домашнего каталога пользователя. Файл настроек ищется в текущем каталоге приложения, в домашнем каталоге пользователя и в списках каталогов, заданных в переменной окружения RSCNFG.

Процедура GetFileInfo

GetFileInfo (src:String [, dt:@Date, tm:@Time, size:@Integer, path:@String]) : Bool

Процедура возвращает информацию о файле с именем **src**. Файл может располагаться на сервере или на терминале. Если файл находится на терминале, то в этом случае его имя должно начинаться с символа '\$'.

Параметры:

src – имя файла.

Dt – возвращаемая дата модификации файла.

Tm – возвращаемое время модификации файла.

Size – возвращаемый размер файла.

Path – возвращаемый путь к файлу.

Возвращаемое значение:

Если заданный файл существует, то возвращается значение **TRUE**. Если файл не существует, то возвращается значение **FALSE**.

Классы и объекты

В Object RSL предусмотрено несколько вспомогательных процедур, предназначенных для поддержки работы с классами и объектами.

Процедура **ActiveX**

ActiveX (mon:String, const=1):Object

Процедура предназначена для создания объекта класса ActiveX с помощью строкового моникера **mon**.

Пример.

```
var strComputer = ".";
var str = "winmgmts:{impersonationLevel=impersonate}!\\\\" +
strComputer + "\\root\\cimv2";
var wmiSrv = ActiveX (str, 1);
var colServices = wmiSrv.ExecQuery ("Select * from Win32_Service");
var ob;
for (ob, colServices)
    [#####]
    (ob.Name, ob.State);
end;
```

Процедура **ClassKind**

ClassKind (obj:Object [, mask:Integer):Integer

Процедура позволяет определить вид указанного объекта.

Параметры:

obj – объект, вид которого требуется определить.

mask – маска поиска.

Возвращаемое значение:

Возвращаемое значение процедуры образуется как результат логической операции "И" для вида класса и значения маски. Процедура возвращает найденный вид объекта. Возможны следующие варианты:

- **1** – обычный класс RSL или DLM;
- **2** – обёртка над RSCOM-объектом для RSL объекта из другого экземпляра RSL;
- **4** – обёртка над RSCOM объектом, отличным от случая 2.

Процедура **GenClassName**

GenClassName (obj)

Процедура возвращает строку с именем класса для объекта *obj*.

Если параметр *obj* не является ссылкой на объект, то возвращается значение типа **V_UNDEF**.

Процедура **GenAttach**

GenAttach (id, methodName, macroName | macroAddr)

Процедура позволяет заменить метод *methodName* объекта, заданного идентификатором *id*, глобальной макропроцедурой *macroName* или ссылкой на нее *macroAddr*.

Если заменяемый метод вызывается из заменяющей процедуры, то рекурсивный вызов не происходит, а вызывается метод предыдущего уровня замены. Это означает, что при вызове метода *methodName* из процедуры *macroName* будет выполнен метод *methodName*, а не заменяющая его процедура *macroName*.

Процедура **GenObject**

GenObject (className [, parm1, parm2, ...])

Данная процедура позволяет создавать объекты, имена классов которых задаются при помощи параметров.

Первый параметр – строка, задающая имя класса, к которому относятся объекты. Далее следуют необязательные параметры, передаваемые конструктору (инициализатору) объекта.

Процедура возвращает ссылку на созданный объект. Если объект заданного класса не может быть создан, произойдет ошибка времени выполнения.

Доступ к свойствам объектов осуществляется аналогично доступу к полям объектов:

```
Ob = GenObject ("MyClass");
```

```
Ob.prop1 = "New value"
```

Вызов методов осуществляется следующим образом:

```
Ob.method1 (par1,par2)
```

Процедура **GenRun**

GenRun (ob, methodName [, par1, par2, ...])

Процедура вызывает метод с именем **methodName** для объекта **ob** и передает ему параметры **par1**, **par2** и т.д.

Процедура возвращает значение, возвращаемое вызываемым методом

```
Val = GenRun (Ob, "Method2", 10)
```

Процедура GenSetProp

GenSetProp (ob, propName, val)

Данная процедура устанавливает для свойства с именем **propName** объекта **ob** новое значение, равное **val**.

Процедура GenGetProp

GenGetProp (ob, propName)

Процедура возвращает значение свойства с именем **propName** объекта **ob**.

Процедура GetObjProps

GetObjProps (obj:Object, [CaseSensitive:Bool]):TArray

Процедура возвращает массив с именами свойств объекта **obj**.

Параметры:

obj – объект, свойства которого возвращает процедура.

CaseSensitive – признак возврата информации о свойствах объекта в программу в исходном виде. Параметр может принимать одно из следующих значений:

- **TRUE** – процедура возвращает информацию без изменения регистра.
- **FALSE** – в верхнем регистре.

По умолчанию, если значение параметра не задано, оно считается равным **FALSE**.

Процедура GetObjMethods

GetObjMethods (obj:Object, [CaseSensitive:Bool]):TArray

Процедура возвращает массив с именами методов объекта **obj**.

Параметры:

obj – объект, имена методов которого возвращает процедура.

CaseSensitive – признак возврата информации о методах объекта в программу в исходном виде. Параметр может принимать одно из следующих значений:

- **TRUE** – процедура возвращает информацию без изменения регистра.
- **FALSE** – в верхнем регистре.

По умолчанию, если значение параметра не задано, оно считается равным FALSE.

Пример.

```
import rcw;
testObj = MyClass;
[Properties:];
obj = GetObjProps (testObj);
for (i, obj)
    println (i)
end;
[Methods:];
obj = GetObjMethods (testObj);
for (i, obj)
    println (i)
end;
```

Процедура IsEqClass

IsEqClass (className, obj)

Процедура проверяет принадлежность объекта, заданного параметром **obj**, классу объектов с именем **className**.

Процедура возвращает **TRUE**, если объект принадлежит классу, и **FALSE** – в противном случае.

Пример.

```
/* Определяем класс AA */
Class AA
End;
/* Наследуем класс BB от класса AA */
Class (AA) BB
End;
If (IsEqClass ("AA",BB))
    Println ("Объект класса BB является и объектом класса AA")
End;
```

Процедура GenPropID

GenPropID (obj, propName)

Процедура возвращает индекс свойства с именем **propName** для объекта **obj**.

Используя полученный индекс, можно получить доступ к свойству при помощи выражения:

A = obj (n);

или:

a = obj [n];

Процедура R2M

R2M (obj:Object, name:String) : MethodRef

Процедура возвращает ссылку на метод с именем **name** объекта **obj**.

Процедура CallR2M

CallR2M (oPtr:MethodRef [, par1, par2,...]) : Variant

Процедура вызывает метод **oPtr**, заданный в виде ссылки типа **MethodRef**.

Перед использованием этой процедуры необходимо убедиться, что объект, для которого вызывается метод, существует. Вызов метода для несуществующего объекта приведет к разрушению программы.

Пример.

```
class Test
  macro Method1 (par)
    return "Ret Val " + par
  end;
end;

var obj = Test;
var ptr = R2M (obj, "Method1");
println (CallR2M (ptr, "Test Param");
```

Процедура GenNumProps

GenNumProps (obj:Object) : Integer

Процедура возвращает количество свойств у объекта **obj**.

Пример.

```
/* Пример процедуры, распечатывающей свойства объекта, переданного в
качестве параметра. */
macro Show (ob:object)
  var n = GenNumProps (ob);
  var i = 0;
  while (i < n)
    println (ob [i]);
    i = i + 1;
  end;
```

```
end;
```

Примечание.

Приведенный в примере способ перебора свойств объекта при помощи оператора `[]` можно использовать не для любых объектов, так как объект может содержать свойства с произвольными идентификаторами (например, объект класса `TArray`).

Процедура `ClrRmtOnRelease`

`ClrRmtOnRelease (proxy:Object) : Bool`

Процедура при удалении прокси-объекта ***proxy*** автоматически удалит связанный с ним удаленный реальный RSCOM-объект.

Примечание.

Эта процедура поддерживается сервером приложений версии не ниже 5.06.118.1

Процедура `GetNamedChanel`

`GetNamedChanel (name:String) : Object`

Процедура проверяет, существует ли коммуникационный канал с именем ***name***. Если существует, то возвращается объект класса ***TRsiChannel*** для этого канала. В противном случае возвращается значение **NULL**.

Индикаторы выполнения процессов

Процедура `InitProgress`

`InitProgress ( maxRecord:Integer [, msg:String, head:String] )`

Процедура инициализирует и выводит на экран индикатор выполнения циклического процесса.

В первом параметре необходимо указать максимальное значение, которое может принимать счетчик отображаемого цикла. Каждый раз при переходе к следующему значению счетчика закрашенная полоска индикатора увеличит свою длину.

Во втором параметре ***msg*** задается сообщение, выводимое в нижнюю строку экрана вместе с индикатором. Третий параметр процедуры должен содержать текст заголовка индикатора выполнения.

Пример.

```
InitProgress (1000);  
var i = 0;  
while ( i < 1000 )  
    i = i + 1;
```

```
UseProgress (i);  
end;  
RemProgress (i)
```

Процедура UseProgress

UseProgress (record)

Процедура приводит изображение на экране в соответствие текущему значению индикатора выполнения циклического процесса.

Единственный параметр **record** задает текущее значение счетчика цикла.

Процедура RemProgress

RemProgress

Данная процедура удаляет с экрана окно индикатора выполнения, выведенное процедурой [InitProgress](#)²⁸⁷.

Процедура BegAction

BegAction ([tm:Integer, text:String, canClose:Bool])

Процедура выводит на экран асинхронный индикатор, предназначенный для отображения занятости приложения. Эта процедура должна использоваться совместно с процедурой [EndAction](#)²⁸⁸.

Процедура работает только в режиме RS-Forms и EasyWin.

Параметры:

tm – время в миллисекундах, через которое появляется индикатор, если не было вызова процедуры **EndAction**. Если параметр не задан, его значение принимается равным 2000.

text – текст, выводимый на индикаторе вместо стандартного.

canClose – признак доступности кнопки "Отменить". По умолчанию кнопка доступна. Если данный параметр равен **FALSE**, кнопка недоступна.

Процедура EndAction

EndAction (tm:Integer)

Процедура убирает с экрана асинхронный индикатор активности, вызванный процедурой [BegAction](#)²⁸⁸.

Если индикатор был выведен, то параметр *tm* задаёт задержку в миллисекундах, по истечении которой индикатор должен быть убран с экрана. По умолчанию параметр имеет значение 200.

Другие процедуры

Процедура CheckBits

CheckBits (n1:Integer, n2:Integer):Integer

Процедура в качестве параметров принимает два целых числа и возвращает результат операции побитового "И" для этих чисел.

Пример.

```
println (CheckBits (#ff, #80));  
//Выводит:  
//128
```

Процедура DateShift

DateShift (inData:Date [, nDay:Integer] [, nMon:Integer] [, nYear:Integer]):Date

Процедура позволяет выполнить смещение заданной даты на требуемое количество дней, месяцев и лет.

Параметры:

inData – дата, смещение которой необходимо выполнить.

nDay – количество дней, на которое требуется выполнить смещение.

nMon – количество месяцев, на которое требуется выполнить смещение.

nYear – количество лет, на которое требуется выполнить смещение.

Возвращаемое значение:

Процедура возвращает дату, полученную в результате необходимого смещения.

Пример.

```
// Вернуть дату, отстоящую от текущей на 5 месяцев:  
Var result = DateShift (date, null, 5);
```

Процедура ExecMacro

ExecMacro (string,.....)

Процедура вызывает для выполнения процедуру, заданную в качестве первого параметра **string**. Это может быть:

- собственно процедура, в этом случае в качестве первого параметра передается ее имя в виде символьной константы или переменной типа **String** либо передается ссылка на данную процедуру;
- метод объекта, в этом случае в качестве первого параметра передается ссылка на метод объекта.

После первого параметра указываются те значения, которые должны быть переданы вызываемой процедуре в качестве фактических параметров.

Если процедура с именем **string** существует, процедура **ExecMacro** возвращает значение **TRUE**, в противном случае возвращается **FALSE**.

Процедуру можно вызвать и по ссылке на нее. В этом случае в качестве параметра процедура **ExecMacro** принимает ссылку на макропроцедуру.

Пример.

```
Macro MyMacro (parm)
    Println ("Вызвана процедура MyMacro с параметром",
parm)
End;
ExecMacro (@MyMacro, "Это параметр для MyMacro")
```

Процедура ExecMacro2

ExecMacro2 (string,)

Процедура эквивалентна процедуре [ExecMacro](#)²⁸⁹, но возвращает значение, которое возвращает запускаемая процедура.

Процедура ExecMacroFile

ExecMacroFile (Module [, ProcName [, Parm1, Parm2, ...]])

Данная процедура производит компиляцию и выполнение макромодуля, заданного строкой **Module**. Строка **ProcName** задает имя макропроцедуры внутри модуля **Module**, которой передается управление. Список параметров **Parm1**, **Parm2** и т.д. передается процедуре **ProcName**.

Возвращаемое значение:

Процедура **ExecMacroFile** возвращает значение, возвращаемое процедурой **ProcName**. Если **ProcName** не задана или не существует, то возвращается значение типа **V_UNDEF**.

Пример.

```
/*Предположим, у нас имеется модуль add2.mac:*/
println ("This is add.mac");
macro MainProc
    println ("This is MainProc");
```

```
i = 0;
while (getParm (i,a))
  println (a);
  i = i + 1
end;
return "RetFrom MainProc";
end;
/* Мы можем запустить на выполнение модуль add.мас из
модуля*/
/* main.мас */
println ("This is main.мас");
println (ExecMacroFile ("add","MainProc",10,20,"par"))
/*Результат выполнения макромодуля main.мас должен выглядеть
так:*/
This is main.мас
This is add.мас
This is MainProc
10
20
par
RetFrom MainProc
```

Процедура ExecMacroModule

ExecMacroModule (codeStr:String [, macroName, par1, par2, ...])

Процедура аналогична процедуре **ExecMacroFile**, но принимает код RSL-модуля в виде строки **codeStr**.

Процедура ReplaceMacro

ReplaceMacro (string1, [string2])

Процедура позволяет заменить процедуру с именем **string1** на глобальную процедуру с именем **string2**. После этого любой вызов процедуры **string1** будет перенаправляться в процедуру с именем **string2**.

Заменить можно не только макропроцедуру, но и любую стандартную процедуру, такую как **println**, **open**, **next**, **prev** и т.д.

Если заменяемая процедура вызывается из заменяющей, то перенаправление вызова не происходит: из заменяющей процедуры вызывается процедура предыдущего уровня замены. Одна процедура может быть заменена несколько раз.

Если параметр **string2** не задан, то это приводит к снятию замены и восстановлению оригинальной процедуры. Если процедура с именем **string1** была заменена несколько раз, то для восстановления оригинальной процедуры необходимо столько же раз вызвать **ReplaceMacro** без параметра **string2**.

Пример.

```
/* Пример замены макропроцедур */
macro Test
  println ("Это Test")
end;
macro ReplTest
```

```

println ("Это ReplTest");
Test
end;
macro ReplReplTest
  println ("Это ReplReplTest");
  Test
end;
/* Выполняем первую замену процедуры Test */
ReplaceMacro ("Test", "ReplTest");
println ("Вызываем Test после выполнения первой замены...");
Test;
/* Выполняем вторую замену процедуры Test */
ReplaceMacro ("Test", "ReplReplTest");
println ("Вызываем Test после выполнения второй замены...");
Test;
println ("Отменяем одну замену...");
ReplaceMacro ("Test");
Test;
println ("Отменяем вторую замену...");
ReplaceMacro ("Test");
Test;
Результат работы этой макропрограммы должен выглядеть так:
Вызываем Test после выполнения первой замены...
Это ReplTest
Это Test
Вызываем Test после выполнения второй замены...
Это ReplReplTest
Это ReplTest
Это Test
Отменяем одну замену...
Это ReplTest
Это Test
Отменяем вторую замену...
Это Test

```

Процедура EхесЕхр

EхесЕхр (string)

Процедура вычисляет выражение, заданное параметром **string**, и возвращает результат вычисления.

Пример.

```
Val = EхесЕхр ("10 + 20*5");
```

Процедура GCollect

GCollect (ncol:@Integer):Integer

Процедура выявляет объекты классов языка RSL и класса **TArray**, на которые нет ссылок в RSL-программе, и пытается освободить такие объекты. При помощи процедуры

[PrintRefs](#)²¹³ для объектов в стандартный выходной поток автоматически выводится информация о ссылках.

Параметры:

ncol – количество объектов без ссылок из RSL-программы, оставшихся после попытки их очистки.

Возвращаемое значение:

Процедура возвращает количество обнаруженных объектов без ссылок из RSL-программы.

Процедура GetCallStack

GetCallStack ()

Процедура возвращает массив строк с названиями процедур в текущем стеке вызовов.

Пример.

```
macro Procl
  macro LocalProc
    a = GetCallStack;
    for (e, a)
      println (e)
    end;
  end;
  LocalProc
end;
```

Процедура GetEnv

GetEnv (string)

Процедура возвращает значение переменной среды с именем, заданным в виде символьной строки **string**.

Пример.

```
println ( getenv ("PATH" ) );
```

Процедура GetMemAddrFrom

GetMemAddrFrom (obj:Variant):MemAddr

Процедура возвращает адрес буфера для объекта, переданного в параметре **obj**, в виде значения типа **MemAddr**. Данное значение может использоваться для передачи в некоторые служебные процедуры RS-Bank V.6.

Процедура GetUIMode

GetUIMode (...)

Процедура возвращает код, обозначающий тип пользовательского интерфейса приложения. Возможны следующие значения:

- **0** – невозможно определить тип интерфейса;
- **1** – консольный интерфейс;
- **2** – EasyWin интерфейс;
- **3** – интерфейс RS-Forms.

Процедура InstLoadModule

InstLoadModule (moduleName:String):Bool

Процедура выполняет динамическую загрузку RSL-модуля в текущий экземпляр интерпретатора RSL. Если модуль с заданным именем уже присутствует в данном экземпляре RSL, процедура не выполняет никаких действий.

Параметры:

moduleName – имя RSL-модуля, загрузку которого требуется выполнить.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **TRUE** – в случае успешного выполнения.
- **FALSE** – в случае возникновения ошибки.

Процедура IsWeakRef

IsWeakRef (obj:Object):Bool

Процедура позволяет определить, является ли объект, указанный в параметре **obj**, "слабой" ссылкой. Если ссылка "слабая", процедура возвращает **TRUE**, в противном случае – **FALSE**.

Процедура WriteByte

WriteByte (stream:Object, byte:Integer):Bool

Процедура предназначена для побайтовой записи в поток IRsStream.

Параметры:

stream – поток, для которого требуется выполнить операцию.

byte – номер байта, начиная с которого требуется выполнить запись.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **TRUE** – в случае успешного выполнения.
- **FALSE** – в случае возникновения ошибки.

Процедура ReadByte

ReadByte (stream:Object, byte:@Integer):Bool

Процедура предназначена для побайтового чтения потока IRsStream.

Параметры:

stream – поток, для которого требуется выполнить операцию.

byte – номер байта, начиная с которого требуется произвести чтение.

Возвращаемое значение:

Процедура возвращает одно из значений:

- **TRUE** – в случае успешного выполнения.
- **FALSE** – в случае возникновения ошибки.

Процедура RunError

RunError ([mes:String] [, userObj:Variant])

Процедура вызывает аварийное завершение выполнения RSL-программы с генерацией ошибки времени выполнения. Кроме этого, производится откат всех изменений, внесенных в файлы в рамках транзакции как на RSL, так и в вызывающем модуле на C или C++.

Параметры:

mes – символьная строка, содержащая текст сообщения об ошибке.

userObj – параметр, задающий специфическую информацию пользователя об ошибке. Может использоваться значение любого типа RSL.

Пример.

```
Class MyError (v)
    Var erCode = v;
End;
RunError ("My Error", MyError (78));
```

Если при аварийном завершении программы управление передается обработчику ошибок **onError**, то в самом обработчике можно вызвать процедуру **RunError** без параметров. В этом случае будет заново сгенерирована та же самая ошибка, из-за которой программа была аварийно завершена. Этот метод позволяет обработать ошибочную ситуацию и снова прервать выполнение программы.

Пример.

```
macro Test
/* Генерация ошибки при выполнении программы и передача
управления обработчику ошибок */
RunError ("My error");
onError
    println ("My handler");
    // Прерывание работы программы и генерация предыдущей
    ошибки
    RunError;
end;
```

Процедура Exit

Exit ([code:Integer] [, mes:String])

Данная процедура прекращает выполнение макропрограммы. Если эта процедура вызвана в рамках транзакции, то она завершает выполнение макросов, вызванных для выполнения транзакции, и актуализирует все изменения, внесенные в файлы в рамках транзакции как на RSL, так и в вызывающем модуле на C или C++. Процедура [ProcessTrn](#)²⁷² при этом возвращает значение **TRUE**, а вся RSL-программа не завершается.

Параметр **code** задает режим просмотра результата работы программы и может принимать следующие значения:

- **0** – результат работы программы будет показываться в окне просмотра; данное значение используется по умолчанию;
- **1** – результат работы программы не будет показываться в окне просмотра;
- **2** – файл вывода не показывается, а сразу распечатывается на принтере.

Параметр **mes** задает сообщение, которое выводится в выходной поток.

Процедура SetExitFlag

SetExitFlag (code:Integer)

Процедура позволяет установить режим просмотра результата работы программы, заданный в параметре **code**, аналогично процедуре **Exit**, но не завершает выполнение RSL-программы.

Процедура MemSize

MemSize()

Процедура возвращает количество свободной оперативной памяти, доступной для работы программы. При работе под Windows всегда возвращает ноль.

Примечание.

Процедура оставлена в RSL для совместимости с ранее написанными программами.

Процедура Version

Version()

Процедура возвращает целое число – номер версии RSL.

Процедура CurrentLine

CurrentLine ([line])

Процедура возвращает номер текущей строки в выходном файле.

Необязательный параметр **line** задает новое значение для счетчика номеров строк.

Пример.

```
If (CurrentLine > 40)
  НапечататьПодвал;
  CurrentLine (0);
end
```

Процедура UserNumber

UserNumber ()

Процедура возвращает номер пользователя, под которым он зарегистрирован в компьютерной сети. Использование этой процедуры позволяет формировать уникальные имена файлов для разных пользователей.

Пример.

```
MyFileName = "DemoFile." + UserNumber;
SetOutput (MyFileName)
```

Процедура Random

Random ([integer])

Процедура возвращает случайное число в диапазоне:

- от 0 до значения *integer-1*, если параметр задан;
- от 0 до 32766, если параметр *integer* не задан.

Процедура SetDefMoneyPrec

SetDefMoneyPrec (newVal:Integer):Integer

Процедура используется для установки нового значения количества знаков после запятой; применяется только для денежного типа.

Возвращает предыдущее значения количества знаков после точки.

Параметры:

newVal — новое значение количества знаков после запятой.

Примечание.

Если в качестве денежного типа используется FDecimal, то допустимы только значения 0, 2, 4.

Возвращаемое значение:

Процедура возвращает предыдущее значение количества знаков после запятой.

Пример.

```
срwin;
var mn1 = $12.3456789;
macro OutPrec (val:money, prec:integer)
    var old = SetDefMoneyPrec (prec);
    var result = string (val);
    SetDefMoneyPrec (old);
    return result
end;
[ Точность 0 #####
  Точность 1 #####
  Точность 2 #####
  Точность 3 #####
  Точность 4 #####
  Точность 5 #####
  Точность 6 #####
  Точность 7 ##### ]
( OutPrec (mn1,0):r,
  OutPrec (mn1,1):r,
  OutPrec (mn1,2):r,
```

```
OutPrec (mn1,3):r,  
OutPrec (mn1,4):r,  
OutPrec (mn1,5):r,  
OutPrec (mn1,6):r,  
OutPrec (mn1,7):r);
```

Процедура ShowDictError

ShowDictError (show:Bool):Bool

Процедура устанавливает признак вывода диагностических сообщений о несоответствии физического Btrieve файла его описанию в словаре базы данных.

Параметры:

show – признак вывода диагностических сообщений. Параметр принимает следующие значения:

- **TRUE** – сообщения выводятся (значение по умолчанию).
- **FALSE** – сообщения не выводятся.

Возвращаемое значение:

Процедура возвращает предыдущее значение признака вывода диагностических сообщений.

Процедура ShowRSCOMError

ShowRSCOMError (obj:TRsComErr)

Процедура отображает ошибки RSCOM в стандартном диалоговом окне.

Процедура StartProg

StartProg (fileName:String [, cmdLine:String] [, detached:Bool]):Integer

Процедура предназначена для запуска приложения на терминале или на сервере (с ожиданием завершения или без ожидания).

Параметры:

fileName – имя запускаемой программы. Если имя программы начинается с символа "\$", то параметр задаёт файл на терминале, иначе – на сервере приложений.

cmdLine – строка с аргументами командной строки для запускаемого приложения.

Detached – признак ожидания завершения. Возможные значения:

- **TRUE** – процедура не ждёт завершения запущенной программы (значение по умолчанию).
- **FALSE** – выполнение приостанавливается до завершения запущенной программы.

Возвращаемое значение:

Процедура возвращает одно из значений:

- В режиме ожидания – код завершения запускаемого процесса.
- В режиме выполнения:
 - **0**, если программа успешно стартована;
 - **-1** – в случае возникновения ошибки.

Процедура StrongRef

StrongRef (par:Object):Object

Процедура возвращает "сильную" ссылку на объект **par**. Основное назначение процедуры – получение "сильной" ссылки на объект, для которого имеется только "слабая" ссылка.

Параметры:

par – "сильная" или "слабая" ссылка на объект.

Пример.

```
class X
end;
ob = X;
weak = WeakRef (ob);
strong = StrongRef (weak);
```

Процедура SysGetProperty

SysGetProperty (key:String):String

Процедура позволяет получить значение глобального свойства **key**.

Процедура SysPutProperty

SysPutProperty (key:String, val:String):Bool

Процедура позволяет в качестве значения свойства **key** установить значение **val**. Если значение **val** равно **null**, указанное свойство удаляется из коллекции. Процедура возвращает **TRUE** в случае успешного изменения значения свойства.

Примечание.

Процедуры **SysGetProperty** и **SysPutProperty** используются для получения доступа к глобальной коллекции свойств приложения. Коллекция свойств доступна из кода на языках C++, RSL, RSCOM и Java. Таким образом, указанная коллекция позволяет выполнять процесс обмена информацией между различными компонентами приложения.

Процедура System

System (Number:Integer, CodeFor:Integer|Type:String [, CmdArgs:String])

Процедура активизирует выполнение системного модуля, заданного идентификатором **Number** и вызванного из активной подсистемы.

Второй параметр задает подсистему: это либо номер (параметр **CodeFor**), либо буквенный код (параметр **Type**) подсистемы, из которой модуль вызывается на выполнение. Номера и буквенные коды подсистем определяются в таблице базы данных, которая во внутреннем словаре имеет имя *typeacc.dbt*.

Необязательный параметр **CmdArgs** задает строку параметров для запуска модуля.

Пример.

```
System(16522, CodeFor("Z"));
/* Для подсистемы "Депозитарий" на экран будет выдана панель
подготовки отчета "Отчет об инвентарном исполнении
операции" (системный модуль 16522).*/
```

Процедура IsStandAlone

IsStandAlone

Программный комплекс RS-Bank может работать в двух- либо трехуровневой архитектуре "клиент-сервер" (см. Руководство пользователя "Установка системы"). Процедура **IsStandAlone** позволяет определить, в каком именно режиме функционирует приложение.

Данная процедура возвращает значения:

- **TRUE** – если программа выполняется в двухуровневой архитектуре.
- **FALSE** – если программа выполняется в трехуровневой архитектуре.

Пример.

```
if (IsStandAlone)
    println ("StandAlone");
else
    println ("Communication");
end;
```

Процедура TestEvent

TestEvent ([pause])

Процедура позволяет узнать, была ли нажата клавиша на клавиатуре или кнопка "мыши", и возвращает:

- код нажатой клавиши – если нажатие было произведено;
- ноль – если нажатий не было.

Необязательный параметр **pause** используется для 32-разрядных программ и определяет время в миллисекундах, по прошествии которого процедура вернет ноль, если нажатий клавиши клавиатуры или кнопки мыши не произошло. Если до истечения заданного интервала времени будет произведено движение мышью, то процедура выйдет из цикла ожидания раньше времени и вернет ноль.

Внимание!

Следует иметь в виду, что максимально допустимое значение параметра **pause** составляет 1000 миллисекунд (т.е. 1 секунду). При указании большего значения, оно автоматически изменяется на 1000.

Если в качестве данного параметра указать значение "-1", то процедура будет дожидаться нажатия кнопки "мыши" или любой клавиши клавиатуры. По умолчанию параметр **pause** принимается равным нулю.

Процедуру **TestEvent** можно использовать для прерывания цикла.

Пример.

```
File ff ("account.dbt");
Const ESC = 27;
Var Done = FALSE;
While ( not Done && next (ff) )
  Println ( ff.account );
  If ( TestEvent == Esc )
    Done = TRUE
End
End
```

Процедура AddEvent

AddEvent (key:Integer)

Процедура добавляет в очередь клавиатурное сообщение с кодом **key**.

Для корректной работы требуется подключение модуля **rsexts**.

Пример.

```
Import RSExts;
Var Дальше = TRUE, Код;
AddEvent (316); // F2
AddEvent (164); // Д
AddEvent (27); // Esc
```

```
While( Дальше )
  Код = TestEvent(100);
  If( Код != 0 ) PrintLn( Код );   End;
  Дальше = (Код != 13);
End;
```

Процедура IsGUI

IsGUI

Данная процедура определяет среду выполнения макропрограммы и возвращает следующие значения:

- **FALSE** – если макропрограмма выполняется консольным приложением Windows NT или программой DOS.
- **TRUE** – если макропрограмма выполняется в графической среде Visual RSL.

Процедура ErrPrint

ErrPrint (...)

Процедура печатает свои параметры в стандартный поток ошибок.

Процедура ErrBox

ErrBox (str:TArray [,caption:String, flags:Integer) : Integer

Процедура отображает в режиме RSFM стандартное диалоговое окно, отображающее список строк. В консольном режиме строки выводятся в обычном диалоговом окне без кнопок.

Параметры:

str – массив строк типа **TArray**.

Caption – заголовок окна.

Flags – вид стандартных иконок и выравнивание текста в окне. В качестве значений параметра передаётся сумма следующих значений:

- **16** – окно будет иметь вид стандартного сообщения об ошибке;
- **32** – окно будет выглядеть как запрос пользователю с несколькими вариантами ответа в виде стандартных кнопок;
- **48** – окно будет иметь вид стандартного предупреждения;
- **64** – окно будет иметь вид информационного сообщения;

- **65536** – выравнивание текста по правому краю;
- **131072** – выравнивание текста по центру;
- **2097152** – в окне будет присутствовать стандартной кнопка минимизации окна;
- **4194304** – окно будет выведено на экран без звукового сигнала.

Пример.

```
cpdos;  
ob = TArray ();  
ob [0] = "Строка1";  
ob [1] = "Строка2";  
ob [2] = "String3";  
ob [3] = "String4";  
println (ErrMsgBox (ob,"Заголовок",2097152+131072+64+0));
```

Процедура ModuleFileName

ModuleFileName ([moduleName:String] [, moduleType:@Integer]):String

Процедура возвращает строку с именем файла для заданного модуля.

Параметры:

moduleName – имя исполняемого модуля. Если параметр не задан, используется имя модуля, из которого была вызвана указанная процедура.

moduleType – тип модуля; возвращаемый параметр. Возможные значения:

- **0** – заданный модуль не найден;
- **1** – макромодуль, загруженный из mac-файла;
- **2** – прекомпилированный модуль, загруженный из rsm-файла;
- **3** – модуль dlm;
- **4** – встроенный модуль;
- **5** – макромодуль, загруженный из базы данных *btrmac.ddf*.

Процедура ModuleName

ModuleName ([symbolName:string]):string

Процедура возвращает строку с именем исполняемого модуля, в котором определена переменная, процедура или класс.

Параметры:

symbolName – имя переменной, процедуры или класса. Если параметр не задан, возвращается имя модуля, из которого был вызван указанный элемент.

Процедура CmdArgs

CmdArgs : String

Процедура возвращает строку с параметрами, которая задается в командной строке при старте приложения.

Пример.

```
/* Запуск утилиты RSL со строкой параметров "Par1 par2  
par3": */  
rsl32.exe bank.def 00.mac "Par1 par2 par3"  
/* В этом случае процедура CmdArgs возвратит значение: */  
/* "Par1 par2 par3" */
```

Процедура GetUserName

GetUserName:String

При работе в трехзвенной архитектуре процедура возвращает сетевое имя пользователя, присоединенного к серверу приложений, в двухзвенной архитектуре – имя текущего пользователя.

Процедура IsSQL

IsSQL : Bool

Процедура возвращает **TRUE**, если код выполняется в SQL-версии RSL, и **FALSE** в противном случае.

Процедура UnderRCWHost

UnderRCWHost :Bool

Процедура возвращает **TRUE**, если RSL-файл выполняется модулем *RCWHost.d32*. В противном случае возвращается значение **FALSE**.

Процедура GetLocaleInfo

GetLocaleInfo (id:Integer, code:Integer [, isLocal:Bool]) : String

Процедура возвращает строку с запрашиваемой региональной настройкой.

Параметры:

id – код языка, для которого необходимо получить настройку. Параметр может принимать следующие значения:

- **0** – язык, установленный для пользователя, работающего с программой;
- **1** – язык системы по умолчанию;
- **2** – русский язык.

code – код региональной настройки, которую требуется получить. Параметр может принимать следующие значения:

- **LOCALE_SDECIMAL** – разделитель дробной части.
- **LOCALE_SMONDECIMALSEP** – разделитель копеек.
- **LOCALE_SDATE** – разделитель даты.
- **LOCALE_STIME** – разделитель времени.
- **LOCALE_SLONGDATE** – формат даты.
- **LOCALE_SSHORTDATE** – короткий формат даты.
- **LOCALE_STIMEFORMAT** – формат времени.

isLocal – признак, определяющий, где необходимо выполнить запрос. Если параметр не задан или задан равным **TRUE**, запрос выполняется на сервере. Если параметр равен **FALSE**, то запрос выполняется на терминале.

Процедура GetLangId

GetLangId (id:Integer [, isLocal:Bool]) : Integer

Процедура возвращает идентификатор языка.

Параметры:

id – код языка, для которого необходимо получить идентификатор. Параметр может принимать следующие значения:

- **0** – язык, установленный для пользователя, работающего с программой;
- **1** – язык системы по умолчанию;
- **2** – русский язык.

isLocal – признак, определяющий, где необходимо выполнить запрос. Если параметр не задан или задан равным **TRUE**, запрос выполняется на сервере. Если параметр равен **FALSE**, то запрос выполняется на терминале.

Процедура ZeroValue

ZeroValue (valtp:Integer):Variant

Процедура возвращает нулевые значения для типа данных, код которого задан параметром **valTp**.

Пример.

```
Println (ZeroValue (V_DOUBLE));
```

Процедура StartShellProgram

StartShellProgram (fileName:Char):Integer

Процедура принимает название файла и открывает его с помощью приложения, зарегистрированного в ОС.

Переданный файл открывается ассоциированным с этим типом файлов приложением. Если путь передан на Сервер приложений, то в режиме трехзвенной архитектуры файл будет скопирован и открыт на терминале. Если путь передан на терминале, начинающийся с "\$", то файл будет открыт на терминале.

Функция встроена в стандартную форму выбора файла на кнопку "Просмотр ОС".

Средство разработки расширений для языка RSL (DLM SDK)

Язык RSL является прекрасным средством настройки программных комплексов, разрабатываемых компанией R-Style Softlab под конкретные требования пользователей. Он позволяет создавать макропрограммы, которые формируют нестандартные отчеты, проводят анализ или диагностику базы данных, выполняют другие функции. Встречаются ситуации, когда для решения поставленных задач стандартных средств языка не хватает. Для устранения этой проблемы был создан инструмент DLM SDK.

DLM SDK предназначен для разработки на языке C/C++ динамически загружаемых модулей. Использование этого инструмента позволяет дополнить RSL новыми эффективными процедурами и даже классами объектов, используя для этого все мощь компилируемых языков C/C++.

В состав дистрибутивного комплекта DLM SDK входят:

- библиотеки;
- заголовочные файлы;
- документация;
- примеры использования.

В этом разделе представлено описание наиболее простых примеров создания DLM-модулей. Полная документация с примерами приведена в дистрибутиве.

Создание и использование DLM-модулей

Двоичные DLM-модули (DLM-ы), созданные при помощи DLM SDK, загружаются в память динамически во время выполнения RSL-программы.

В зависимости от платформ, для которых создаются DLM-модули, компилятор должен удовлетворять следующим требованиям:

- Для создания DLM-ов для защищенного режима необходим компилятор фирмы "Borland" версии 4.02 или выше, а также система Power Pack for DOS.
- Для создания DLM-ов для 32-разрядного защищенного режима DOS и WIN32 необходим компилятор фирмы "Borland" версии 4.0 или выше или компилятор фирмы "Microsoft" Visual C++ версии 5.0.

Создаваемые при помощи DLM SDK модули представляют собой обыкновенные 32-разрядные DLL-библиотеки.

Для того чтобы одна и та же макропрограмма, созданная с использованием DLM SDK, могла исполняться разными интерпретаторами RSL принято следующее соглашение о выборе имен для DLM-модулей:

- имя файла должно удовлетворять требованиям операционной системы;

- расширение файла должно быть следующим: **d32** – для режимов DPMI32 и WIN32.

Для того чтобы DLM модуль стал доступен RSL-программе, имя модуля без его расширения необходимо указать в директиве RSL **import**.

Пример.

```
import demo; //Подключение файла, содержащего код DLM-модуля
```

Рассмотрим пример DLM-модуля, который реализует для языка RSL процедуру с именем "RslHello". Данная процедура выводит в стандартный выходной поток текстовую строку "Вызвана процедура RslHello".

Пример.

```
#include "rsl\rsldll.h"
static void DLMAPI RslHello (void)
{print ("Вызвана процедура RslHello\r\n");}
void DLMAPI EXP AddModuleObjects (void)
{AddStdProc (V_UNDEF, "RslHello", RslHello, 0);}
```

Проанализируем этот код.

Сначала к программе подключается файл *rsldll.h*, содержащий описание интерфейса с интерпретатором RSL.

Затем вводится определение процедуры *RslHello*, которая будет вызываться из RSL-программы. Следует отметить, что все процедуры, которые вызываются из макропрограмм на языке RSL, должны иметь прототип, как приведенная *RslHello*: не принимать параметров, не возвращать значений и иметь тип вызова *DLMAPI*. Для передачи параметров и возвращения значений предусмотрен специальный [механизм](#)³¹¹.

После создания процедуры для RSL ее необходимо зарегистрировать при помощи стандартной функции **AddStdProc**. Функция **AddStdProc** имеет четыре параметра:

- тип возвращаемого значения регистрируемой процедуры;
- строка с именем, которое будет использоваться для вызова процедуры из макропрограммы;
- указатель на функцию, реализующую алгоритм процедуры;
- четвертый параметр зарезервирован и всегда должен быть равен 0.

Функция **AddStdProc** вызывается для каждой регистрируемой процедуры из функции **AddModuleObjects**, которую необходимо определять аналогично определению в примере. Интерпретатор RSL вызывает функцию **AddModuleObjects** автоматически при загрузке DLM-модуля.

Теперь необходимо выполнить компиляцию созданного нами модуля.

Предположим, что исходный текст нашего DLM-модуля находится в файле с именем *demo.c*. Тогда, чтобы осуществить компиляцию модуля для платформы DPMI32, необходимо выполнить команду:

```
bcc32 -WxD -lh -edemo.32 demo.c lib\rsldlm32.lib\dpmi32x.lib
```

В результате компиляции будет получен модуль *demo.d32*.

Пример.

Приведем пример использования созданного DLM-модуля.

```
import demo;  
RslHello;
```

Передача параметров и возврат значений

Для доступа к переданным в макропроцедуру параметрам применяется функция **GetParm**, имеющая два аргумента:

- первый аргумент задает номер параметра, значение которого необходимо получить;
- второй – возвращает ссылку на значение RSL-переменной.

Если запрашиваемый параметр не существует, функция возвращает значение **FALSE**. В этом случае второй аргумент инициализируется ссылкой на временное значение неопределенного типа **V_UNDEF**.

Переменная RSL описывается структурой **VALUE**. В поле **v_type** этой структуры хранится тип переменной, а в поле **value** собственно значение переменной для конкретного типа данных.

Для возврата значений в RSL-программу служит функция **ReturnVal**, которая задает возвращаемое значение и имеет два параметра:

- тип возвращаемого значения;
- указатель на возвращаемое значение.

Покажем на примере, как передаются параметры и возвращаются значения в макропрограмму, созданную на языке RSL.

Пример.

Разработаем процедуру **MultiStrLen**, которая принимает в качестве параметров произвольное количество строк и возвращает их длину:

```
#include "rsl\rsldll.h"  
#include <string.h>  
  
static void DLMAPIC MultiStrLen (void)  
{  
    VALUE *v;  
    int i = 0;  
    long len = 0;  
    for ( i = 0; GetParm (i,&v); ++i )  
        if ( v-> v_type == V_STRING )  
            len += strlen ( v->value.string );  
    Return Val (V_INTEGER,&len);  
}  
  
void DLMAPI EXP AddModuleObjects (void)  
{  
    AddStdProc (V_INTEGER,"MultiStrLen",MultiStrLen,0);  
}
```

```
}
Все параметры в RSL-процедуры передаются по значению.
Однако, используя стандартную функцию PutParm, можно
изменить значение фактического параметра в программе,
вызвавшей эту процедуру.
```

Пример.

Создадим процедуру RetDemo, которая с помощью первого параметра возвращает значение 200:

```
Static void DLMAPIC RetDemo (void)
{
    long val = 200;
    PutParm ( 0, V_INTEGER, &val );
}
```

Функция **PutParm** отличается от функции **RetVal** дополнительным первым аргументом, который задает номер устанавливаемого параметра.

Имеется возможность инициализации и деинициализации DLM-модуля. Для этого служат две специальные функции: **InitExec** и **DoneExec**. Функция **InitExec** вызывается автоматически интерпретатором RSL перед началом выполнения RSL-программы, а **DoneExec** – после выполнения.

Пример.

```
void DLMAPI EXP InitExec (void)
{
    //Выполняем инициализацию
    .....
}
void DLMAPI EXP DoneExec (void)
{
    //Выполняем освобождение занятых ресурсов
    .....
    .....
}
```

Помимо процедур, вызываемых из RSL-программ, при помощи инструмента DLM SDK можно создавать целые классы объектов со свойствами и методами. Объекты классов в RSL создаются и уничтожаются с помощью механизма поставщиков объектов. В DLM-модуль можно включить код, реализующий поставщик объектов требуемого типа, и, используя функцию **AddModuleObjects**, зарегистрировать этого поставщика.

Трансляция макропрограмм

В интерпретаторе RSL предусмотрена возможность генерации прекомпилированных бинарных файлов (пи-код) для ускорения процедуры синтаксического разбора макропрограммы. При использовании пи-кода интерпретатор RSL освобождается от фазы синтаксического разбора, что сокращает время, необходимое для выполнения макрозадания.

Пи-код сохраняется в файле с расширением **.RSM*. Имя этого файла и его расположение на диске те же, что и у соответствующего макрофайла.

Сгенерировать пи-код можно, выбрав соответствующий пункт меню консольного приложения "*Процедуры\Трансляция макрофайла*", или в режиме отладки, воспользовавшись клавишами **[Alt+F9]**.

Если в результате трансляции макрофайла будет обнаружено, что на диске уже есть соответствующий gsm-файл, и он актуален (то есть имеет дату более позднюю, чем макрофайл), то будет выведено соответствующее сообщение, и трансляция будет отложена. Если после этого в режиме отладки внести изменения в макрос и запустить его на выполнение, то будет выдано сообщение о неактуальности существующего gsm-файла.

Отладка макропрограмм

При создании собственных программных модулей с помощью языка RSL пользователь может использовать специальный компонент – отладчик для поиска ошибок и отладки RSL-программ.

Отладчик реализован в виде отдельного модуля и имеет графический интерфейс. С его помощью можно отлаживать любые приложения на языке RSL, как графические, так и консольные.

Если RSL-программа работает на удаленном компьютере-сервере приложений, а пользователь работает с программой-терминалом, то отладчик будет активизироваться там же, где работает программа-терминал. Соответственно, компоненты отладчика должны быть установлены там же, где и программа-терминал.

В состав отладчика входят следующие компоненты:

- основные компоненты отладчика:
 - *RSLDBG.dll* – собственно отладчик; данный компонент должен быть доступным при работе с отлаживаемой программой: он должен находиться либо в текущем каталоге, либо путь к нему должен быть определен в переменной среды PATH;
 - *rdbg.dll* – русские ресурсы отладчика; если этот компонент не найден, то интерфейс отладчика будет только на английском языке;
- модули, необходимые для работы RSL-программ в трехзвенной архитектуре:
 - *rsexts* – встроенный модуль, позволяющий вызывать RSL-программы на терминале. Этот модуль можно импортировать в макрос, который работает на сервере приложений и вызывает макропроцедуры на терминале (см. описание процедуры ***CallRemoteRsl***);
 - *rsextt.d32* – клиентский компонент удаленных вызовов RSL. Данный компонент должен находиться либо в одном каталоге с программой-терминалом, либо в одном из каталогов, указанных в параметре EXPATH в файле *term.ini*;
 - *rsextt.ini* – файл настроек клиентского компонента *rsextt.d32*. Этот файл должен находиться в каталоге запуска терминала или в одном каталоге с программой-терминалом. В файле *rsextt.ini* можно задавать следующие параметры:
 - DATADIC – путь к словарю базы данных;
 - DBFPATH – каталог DBF-файлов;
 - TXTPATH – каталог текстовых файлов;
 - INCPATH – каталог макрофайлов;
 - OUTPUT – задает имя файла, в который будет направляться стандартный вывод RSL. Можно задать "маску" имени файла вида *<dir>*.<ext>*, где *<dir>* – путь к каталогу, *<ext>* – расширение, тогда полное имя файла вывода образуется путем замены "*" на имя выполняемого макрофайла. Если не указано и расширение (*<dir>*.**), то подставляется расширение **.txt*. Например, если задана маска *mac*.out* и выполняется макрос *remote.mac*, то имя файла вывода будет *macremote.out*;

- SRCDIR – список каталогов, разделенных точкой с запятой, в которых отладчик ищет тексты отлаживаемых макрофайлов;
- CASHDIR – каталог, в котором отладчик сохраняет копии текстов отлаживаемых макрофайлов, переданных ему серверной частью, если текст макрофайла ни в одном из каталогов SRCDIR не найден, либо старше, чем его серверный аналог. После использования макрофайлы из этого каталога автоматически не удаляются, что дает возможность использовать их повторно без лишней пересылки. Файлы в этом каталоге создаются с расширением **.ma~*.
- *rsscript.dll* – интерпретатор RSL; этот файл должен располагаться в одном из каталогов поиска DLL, то есть либо в одном каталоге с программой-терминалом, либо в одном из каталогов, заданных в переменной окружения PATH.

Описание отладки макросов транспортной компоненты, вызываемых удаленно, представлено в Руководстве "Транспортная компонента", глава "Взаимодействие ТК и RS-Bank V.6", раздел "Настройка ИБС RS-Bank V.6".

Запуск отладчика

Чтобы активизировать отладчик RSL, прикладная программа должна находиться в режиме "Отладка макрофайла".

Переход в режим отладки макропрограммы можно осуществить одним из следующих способов:

1 способ – находясь в текстовом редакторе, нажать клавишу **[F11]**. Это означает, что программа будет запущена на выполнение, и будет вызван отладчик для первой команды в тексте программы. При этом будет активизировано окно отладчика, и текущей инструкцией будет первая инструкция в программе.

2 способ – запустить программу на выполнение с помощью клавиши **[Alt+F10]** и во время выполнения нажать комбинацию клавиш **[Ctrl+Break]**. В этом случае активизируется окно отладчика, и текущей инструкцией будет та, которая следует за последней выполненной инструкцией.

3 способ – непосредственно в код программы вставить инструкцию **DebugBreak**. Далее программу необходимо запустить на выполнение с помощью клавиши **[Alt+F10]**. В результате этого программа прервет свое выполнение, и в окне отладчика текущей будет инструкция, следующая за инструкцией **DebugBreak**.

4 способ – при возникновении ошибок, возникающих во время выполнения программы. При этом выдается диалоговое окно с информацией об ошибке и предложением запустить отладчик. Если на это предложение ответить утвердительно, то загружается отладчик. Текущей инструкцией при этом устанавливается инструкция, вызвавшая данную ошибку. В режиме отладки можно попытаться устранить причину ошибки и продолжить выполнение.

Основное окно отладчика

После входа в режим отладки на экране появляется основное окно отладчика (см. [Рис. 2³¹⁷](#)). В этом окне мы видим текст программы модуля, в контексте которого был активизирован отладчик. Этот модуль является текущим. Его имя приведено в заголовке основного окна.

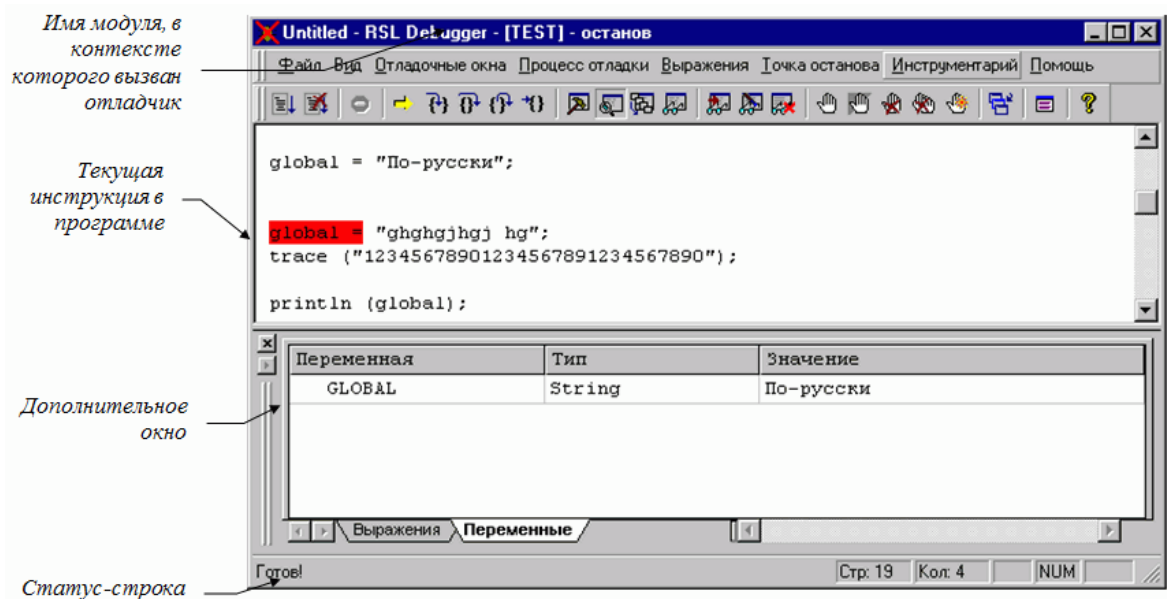


Рис. 2. Основное окно отладчика.

Красным цветом подсвечена **текущая инструкция**, то есть инструкция, которая будет выполняться первой при запуске программы.

В процессе работы с отладчиком данное окно активизируется при нажатии комбинации клавиш **[Alt+0]**, то есть оно автоматически появляется на экране и в него переносится фокус ввода.

В нижней части окна может находиться одно или несколько вспомогательных окон, используемых при отладке программы.

Внизу располагается статус-строка. Чтобы скрыть или вывести ее на экран, нужно в меню **"Вид"** ("View") выбрать пункт **"Строка состояния"** ("Status Bar").

Управление работой отладчика осуществляется с помощью команд, сгруппированных в меню, кнопок, расположенных на панели инструментов, а также "горячих" клавиш. Каждая команда меню имеет аналог вызова с помощью "горячих" клавиш. Действия кнопок дублируют команды меню.

Пользователь может вывести на экран или убрать панель инструментов, выбрав подпункт **"Отладка"** ("Debug Toolbar") пункта **"Панель инструментов"** ("Toolbars") меню **"Вид"** ("View").

Отладка макропрограмм

Основным действием при отладке программ является исследование ее выполнения, поскольку последовательность выполнения инструкций программы не всегда очевидна.

При отладке программы удобно использовать **точки останова**³²⁰, то есть точки, на которых прерывается выполнение программы, и команды пошагового выполнения программы, описание которых приведено в данной главе. Используя команды пошагового выполнения и точки останова, можно эффективно исследовать выполнение работы программы.



— выполнение инструкции с заходом в процедуры. Эта команда позволяет проследить действие всех инструкций на переменные. Аналогом этой команды является выбор пункта

"Шаг с заходом" ("Step into") меню "Процесс отладки" ("Debug actions") или нажатие клавиши [F11].



– выполнение процедуры как единой инструкции. Эта команда используется вместо предыдущей, чтобы продвигаться по вызовам процедур, не входя внутрь. Аналогом этой команды является выбор пункта "Шаг с обходом" ("Step over") меню "Процесс отладки" ("Debug actions") или нажатие комбинации клавиш [F10].



– выполнение оставшейся части процедуры как единой инструкции и остановка на следующей за процедурой выполняемой инструкции. Эта команда используется в том случае, если нет больше необходимости пошагово выполнять процедуру. Аналогом этой команды является выбор пункта "Шаг с выходом" ("Step out") меню "Процесс отладки" ("Debug actions") или нажатие комбинации клавиш [Shift+F11].



– выполнение программы до инструкции, на которой стоит курсор. Эта команда используется, например, для пропуска больших циклов. Аналогом этой команды является выбор пункта "Выполнить до текущей позиции" ("Run to cursor") меню "Процесс отладки" ("Debug actions") или нажатие клавиши [Ctrl+F10].



– продолжение выполнения программы. В результате выбора этой команды программа будет выполнена до конца либо до следующей точки останова. Аналогом этой команды является выбор пункта "Продолжить" ("Run") меню "Процесс отладки" ("Debug actions") или нажатие клавиши [F5].



– прерывание выполнения программы. Эта кнопка активизируется в тот момент, когда начинает выполняться RSL-программа. Нажав на эту кнопку, мы заставляем RSL-программу прервать свое выполнение и активизировать отладчик на текущей исполняемой инструкции. Нажатие данной кнопки эквивалентно выбору пункта "Прервать отладчик" ("Interrupt") меню "Процесс отладки" ("Debug actions") или нажатие комбинации клавиш [Ctrl+Scroll lock]. Это же действие можно выполнить с помощью нажатия комбинации клавиш [Ctrl+Break] в прикладной программе, если программа поддерживает прерывание выполнения RSL-программ по [Ctrl+Break].



– возврат на текущую инструкцию. Эта команда используется при работе с большими программами. С ее помощью можно быстро перейти к текущей инструкции и, тем самым, сократить время поиска нужной строки программы. Аналогом этой команды является нажатие клавиш [Alt+Grey*].

Для дополнительного контроля в режиме отладки предусмотрено использование дополнительных средств контроля, таких как:

- просмотр значений любых [переменных](#) ³²³ и [выражений](#) ³²⁴;
- просмотр результатов выполнения программы в [специальном окне вывода](#) ³²¹;
- просмотр [стека вызовов процедур](#) ³²²;
- просмотр [списка модулей, импортируемых программой](#) ³²¹.



Чтобы прервать работу с отладчиком, необходимо нажать кнопку  или выбрать в меню "Процесс отладки" ("Debug actions") команду "Закончить работу" ("Abort"), эквивалентную нажатию клавиши [Shift+F5]. В результате этого выполнение программы прерывается,

управление передается RSL-программе, в среде RSL появляется сообщение "Прерывание пользователя", и окно отладчика закрывается.

Закрытие окна отладчика в процессе отладки приведет к тому, что режим отладки будет прерван, и программа выполнится до конца или до следующей точки останова.

Внимание!

При возникновении ошибки компиляции макрофайла (если окно с описанием ошибки оставлено на экране) нельзя обновить его в каталоге RSBank стандартной командой операционной системы "move". Для выполнения корректной замены макрофайла необходимо использовать связку команд "delete+copy" или любой файловый менеджер.

Точки останова

В окне с исходным кодом программы можно устанавливать точки останова (см. Рис. 3³¹⁹), чтобы приостановить выполнение программы на определенной инструкции в процедуре, например, там, где может возникать ошибка. Если точки останова больше не нужны, их можно удалить.

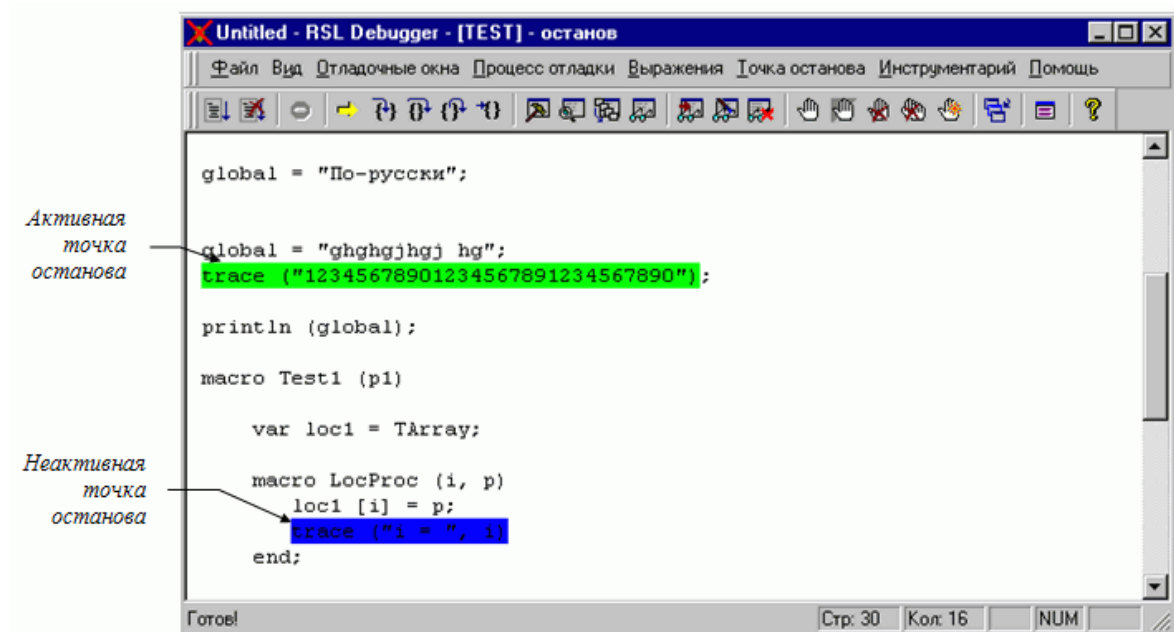


Рис. 3. Точки останова.

Чтобы установить точку останова, необходимо поместить курсор в строку, на которой надо приостановить выполнение, и либо в меню "Точка останова" ("Breakpoint") выбрать команду "Установить" ("Set Breakpoint"), либо воспользоваться кнопкой , расположенной на панели инструментов. В результате выбранная инструкция будет подсвечена зеленым цветом. Помеченная таким образом точка останова является **активной**, то есть при достижении данной инструкции процесс выполнения программы будет остановлен.

В отладчике имеется возможность сделать активную точку останова **неактивной**, то есть разрешить программе не останавливаться на данной инструкции. Для этого необходимо установить курсор на точке останова и воспользоваться командой "Разрешить/Запретить" ("Enable/Disable bp") меню "Точка останова" ("Breakpoint") или

кнопкой , расположенной на панели инструментов. Неактивная точка останова подсвечивается синим цветом. При повторном выполнении этой же команды неактивная точка останова снова станет активной.

Чтобы удалить точку останова, необходимо установить на ней курсор и выбрать команду "Убрать" ("Remove Breakpoint") меню "Точка останова" ("Breakpoint") или нажать кнопку



на панели инструментов. В результате этого подсветка со строки будет убрана. Если точек останова много, и нужно отменить их все сразу, для этого можно воспользоваться командой "Убрать все" ("Remove all") из меню "Точка останова" ("Breakpoint") или кнопкой



В отладчике имеется возможность просмотра информации обо всех установленных точках останова в [специальном окне](#) ³²⁰.

Список точек останова

Чтобы активизировать список точек останова, установленных в программе, необходимо выбрать в меню "Процесс отладки" ("Debug actions") команду "Показать список" ("Manage

BP's"), эквивалентную нажатую комбинации клавиш **[Ctrl+B]**, или нажать кнопку  на панели инструментов.

В результате этого на экране появляется окно (см. [Рис. 4](#) ³²⁰), в котором содержится список всех точек останова, установленных в программе. Для каждой точки останова отображается следующая информация:

- признак активности точки останова;
- наименование модуля, в котором установлена точка;
- номер строки в модуле, на которой установлена точка останова.

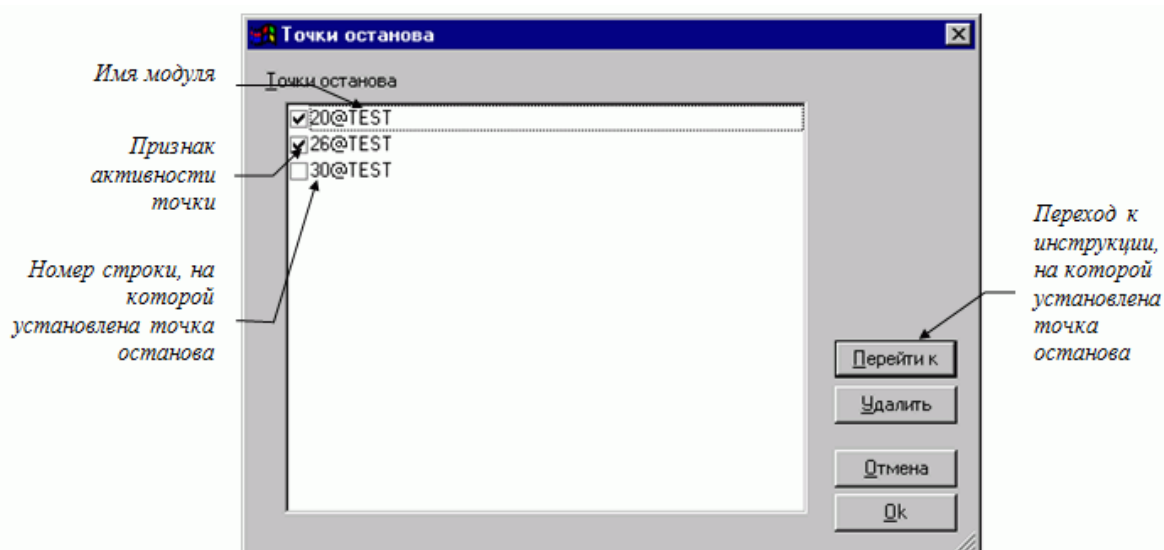


Рис. 4. Список точек останова.

В этом списке можно выполнить следующие операции:

- сделать активную точку неактивной и наоборот; для этого необходимо, соответственно, убрать или установить отметку для выбранной точки, щелкнув мышью в соответствующем поле;
- удалить из списка точку останова, воспользовавшись кнопкой **Удалить (Remove)**;
- позиционироваться на точке останова и перейти на нее в тексте программы в исходном коде, воспользовавшись кнопкой **Перейти к (Go to)**.

Список импортируемых модулей

Чтобы вызвать список импортируемых программой модулей, необходимо в меню "**Процесс отладки**" ("**Debug actions**") выбрать пункт "**Список модулей**" ("**Manage modules**") или нажать кнопку  на панели инструментов. Действие этой команды аналогично нажатию комбинации клавиш **[Ctrl+0]**.

В появившемся на экране окне (см. *Рис. 5*³²¹) будет представлен список модулей, которые написаны на языке RSL. Встроенные и dlm-модули в этом списке не показываются.

С помощью данного списка в процессе отладки можно переключаться из одного модуля в другой. Для этого необходимо выбрать имя нужного модуля в списке и нажать кнопку **Показать**. В результате в основном окне появится текст указанного модуля.

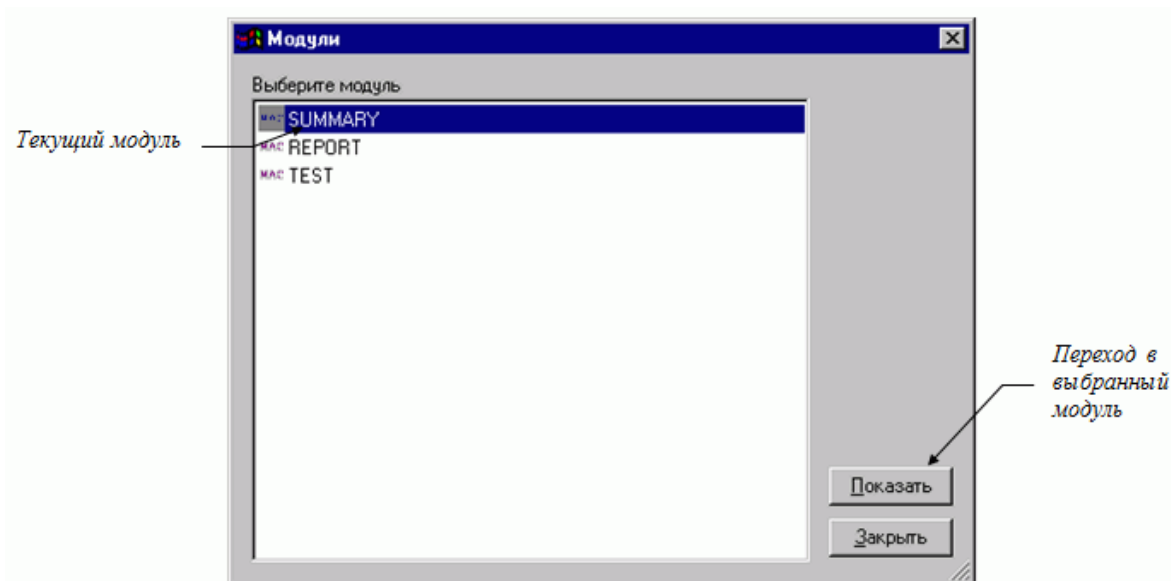


Рис. 5. Список модулей.

Окно проверки

Окно проверки (см. *Рис. 6*³²²) можно вывести на экран или убрать с экрана путем выбора в меню "**Отладочные окна**" ("**Debug window**") пункта "**Окно проверки**" ("**Show output**") или нажатия кнопки  на панели инструментов. Действие этой команды аналогично нажатию комбинации клавиш **[Ctrl+4]**. Чтобы активизировать это окно, то есть вызвать его и перенести в него фокус ввода, нужно нажать комбинацию клавиш **[Alt+4]**.

Это окно служит для отображения информации, выводимой в процессе работы программы с помощью специальной процедуры **trace**. Данная процедура является специальной процедурой отладчика. Синтаксически она аналогична процедуре **Println**. Формат данной процедуры имеет следующий вид:

`trace (<выражение>)`

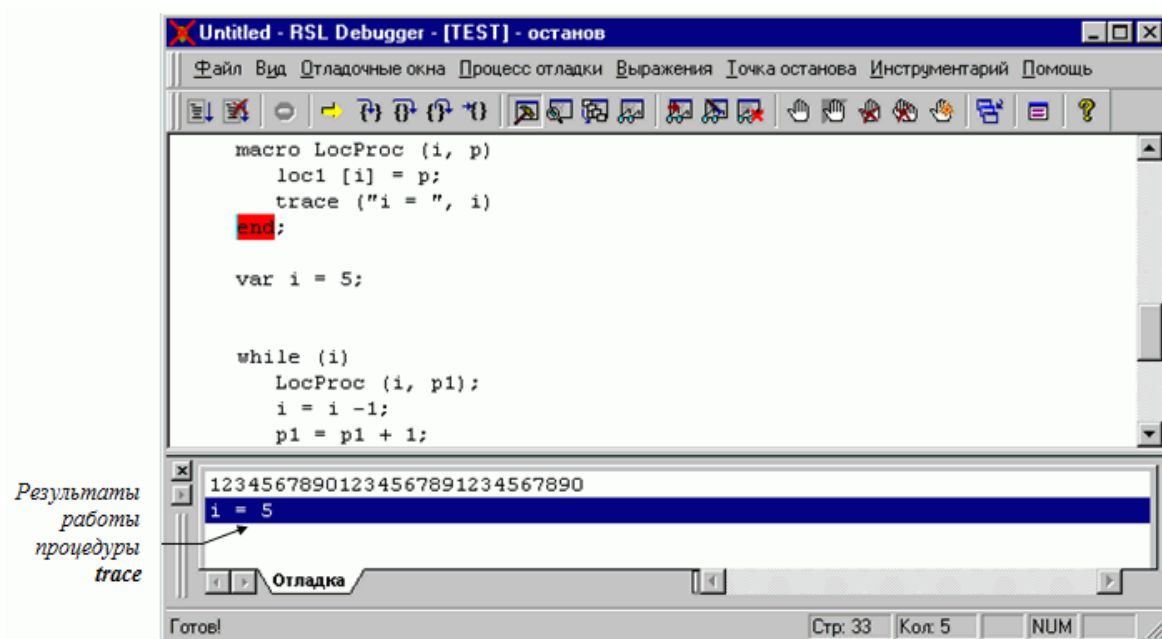


Рис. 6. Окно проверки.

Таким образом, используя окно проверки, удобно наблюдать за работой программы.

Стек вызовов процедур

Окно, содержащее стек вызовов процедур (см. Рис. 7³²³), можно вывести на экран или убрать с экрана путем выбора в меню "Отладочные окна" ("Debug window") пункта "Список вызовов" ("Show call stack") или нажатия кнопки  на панели инструментов. Действие этой команды аналогично нажатию комбинации клавиш **[Ctrl+3]**. Чтобы вызвать данное окно и перенести в него фокус ввода, нужно нажать комбинацию клавиш **[Alt+3]**.

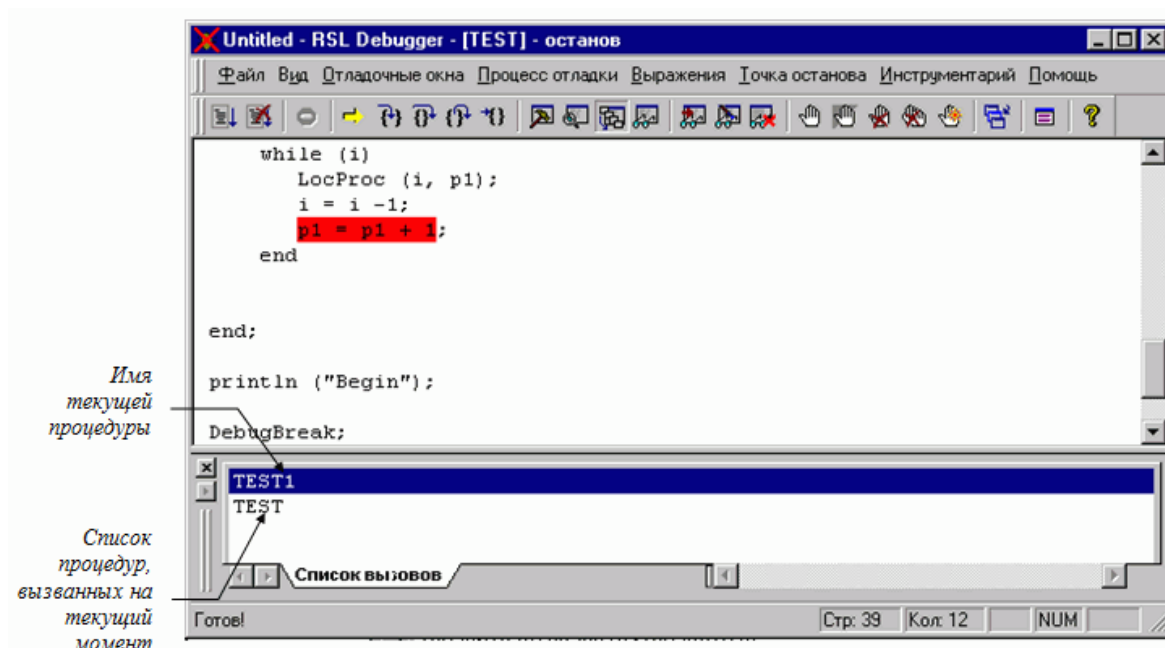


Рис. 7. Стек вызовов процедур.

Стек содержит имена процедур, вызванных в процессе работы программы. На его "вершине" располагается текущая исполняемая процедура. На "дне" стека находится имя модуля, содержащего эту процедуру.

В данном списке можно выбирать любую процедуру.

В результате этого автоматически в окне с исходным кодом будет подсвечиваться инструкция, соответствующая выбранному элементу в стеке. При этом в [окне переменных](#) ⁽³²³⁾ будут отображаться значения переменных выбранной процедуры.

Окно переменных

В процессе отладки удобно пользоваться окном, в котором автоматически отображаются текущие значения локальных переменных для текущей процедуры (см. [Рис. 8](#) ⁽³²⁴⁾).

Чтобы вывести на экран или убрать это окно, нужно в меню **"Отладочные окна"** ("Debug window") выбрать пункт **"Переменные"** ("Debug window") или нажать кнопку  на панели инструментов. Действие этой команды аналогично нажатию комбинации клавиш **[Ctrl+1]**. Чтобы вызвать окно переменных и перенести в него фокус ввода, нужно нажать комбинацию клавиш **[Alt+1]**.

Данное окно становится доступным также при выборе закладки **Переменные (Variables)**, если в окне отладчика присутствует [окно выражений](#) ⁽³²⁴⁾.

В данном окне отображаются:

- для процедур – локальные переменные этой процедуры и их значения;
- для процедуры модуля – локальные и глобальные переменные модуля и их значения.

Если мы осуществляем навигацию по элементам стека вызовов, то в окне переменных отображаются переменные и их значения для выбранной в стеке процедуры.

Значения локальных переменных можно изменять. Для этого нужно щелкнуть мышью в поле **Значение (Value)** на соответствующей строке списка переменных и ввести нужное значение.

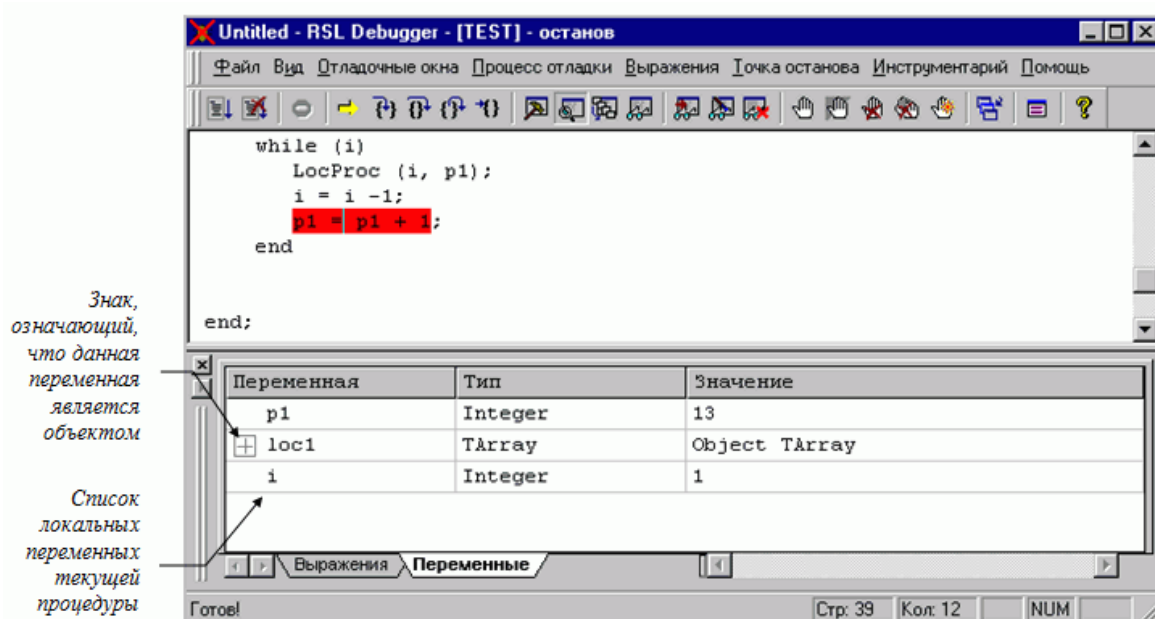


Рис. 8. Окно для просмотра значений локальных переменных.

Если переменная является объектом, то рядом с ее именем устанавливается знак "+". Щелкнув по этому знаку мышью, объект можно раскрыть в виде дерева и просмотреть и при необходимости изменить его свойства (см. Рис. 9³²⁴).

Эта возможность относится как к объектам стандартных классов, так и к любым другим объектам языка RSL.

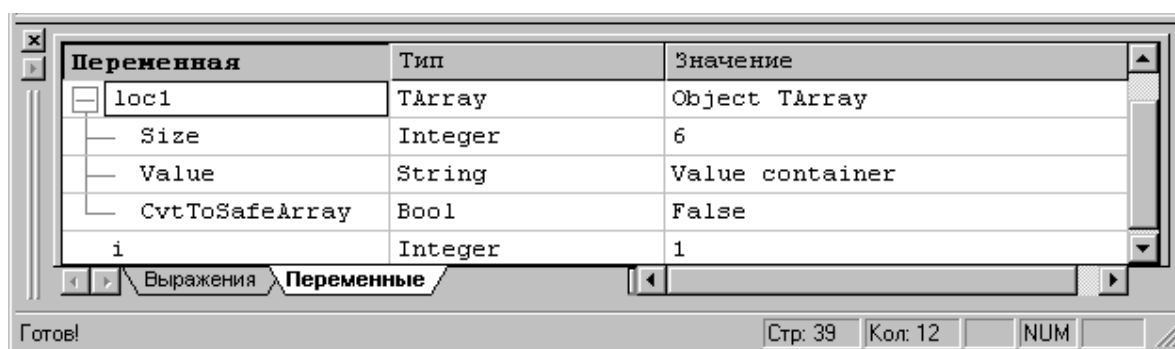


Рис. 9. Пример просмотра свойств объекта.

Окно выражений

Помимо окна переменных, при отладке используется окно, в котором можно просматривать любые контрольные выражения, которые нужны пользователю для дополнительного контроля (см. Рис. 10³²⁵).

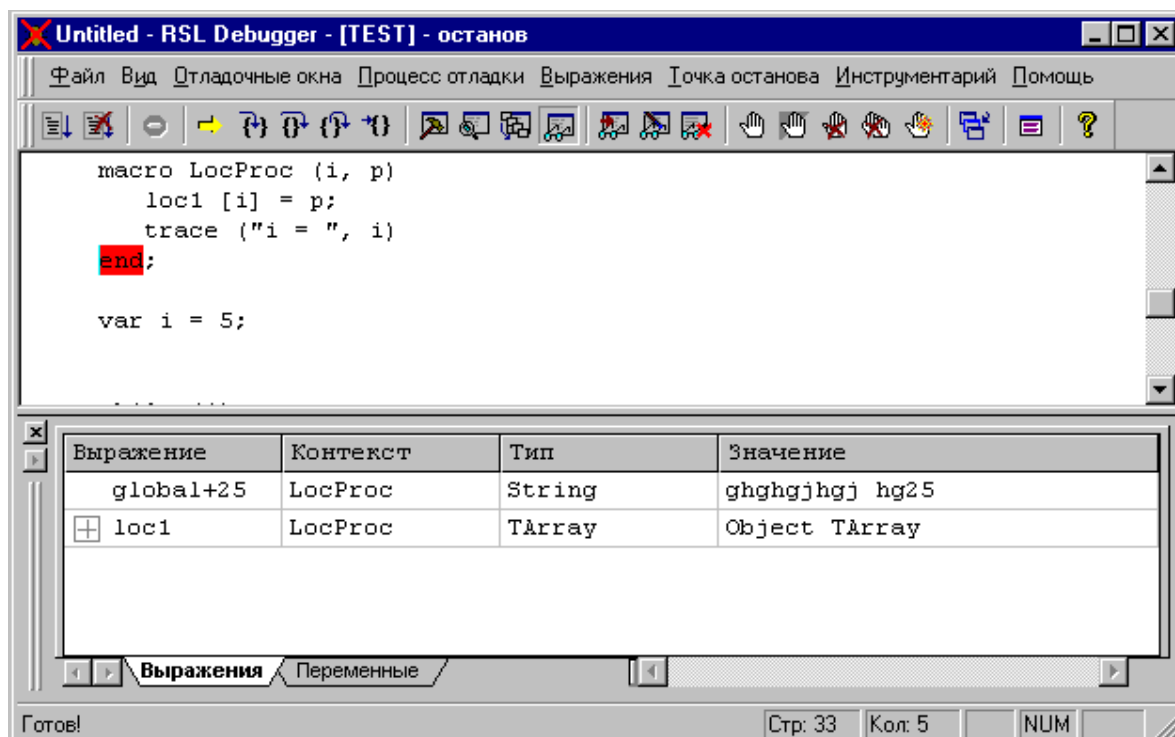


Рис. 10. Окно для просмотра значений выражений.

Чтобы вывести на экран или убрать с экрана это окно, необходимо в меню **"Отладочные окна"** ("Debug window") выбрать пункт **"Выражения"** ("Show watch") или нажать кнопку  на панели инструментов. Действие этой команды аналогично нажатию комбинации клавиш **[Ctrl+2]**. Данное окно становится доступным также при выборе закладки **Выражения** (Variables), если в окне отладчика присутствует [окно переменных](#) ³²³. Чтобы вызвать окно переменных и перенести в него фокус ввода, нужно нажать комбинацию клавиш **[Alt+2]**.

В отличие от окна переменных, в которых автоматически отображаются переменные процедур, данное окно при входе в режим отладки не содержит никаких выражений. Поэтому пользователь должен сам ввести в него нужные выражения с помощью [специального режима](#) ³²⁵.

Как только приложение пользователя входит в режим прерывания, определенные пользователем выражения появляются в данном окне. Если выражение является объектом RSL, то можно просматривать и редактировать его свойства так же, как и в окне переменных.

В данном окне предусмотрен специальный [режим редактирования выражения](#) ³²⁶.

Удалить выражение можно, воспользовавшись пунктом **"Удалить"** ("Delete watch") в меню **"Выражения"** ("Watch"), пунктом **"Удалить"** ("Delete watch") в дополнительном меню, вызываемом при нажатии правой кнопки мыши, или нажав кнопку  на панели инструментов. Данная команда эквивалентна нажатию комбинации клавиш **[Ctrl+D]**.

Ввод выражения

Чтобы ввести выражение, необходимо активизировать специальное окно для ввода (см. [Рис. 11](#) ³²⁶) одним из следующих способов:

- в меню **"Выражения"** ("Watch") выбрать пункт **"Добавить"** ("Immediate window");

- в дополнительном меню, вызываемом при нажатии правой кнопки мыши, выбрать пункт "Добавить" ("Add watch");
- нажать кнопку  на панели инструментов;
- нажать комбинацию клавиш **[Ctrl+A]**.

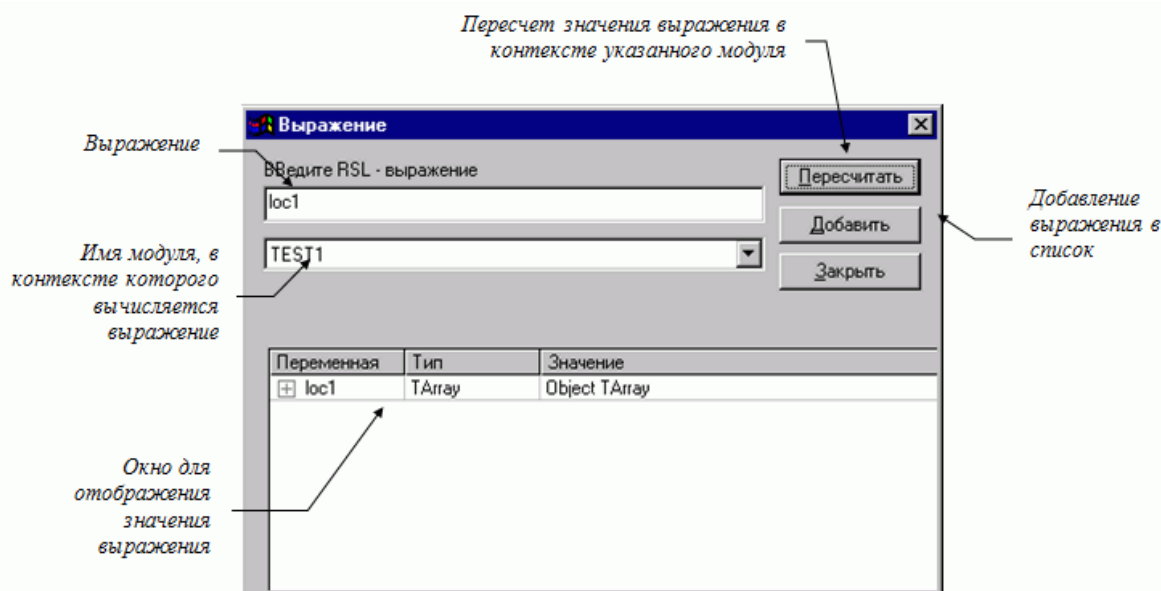


Рис. 11. Ввод и вычисление значения выражения.

Чтобы ввести выражение, достаточно в верхней строке окна ввести его и нажать кнопку **Добавить** (Add). В результате это выражение будет помещено в список.

Перед вызовом окна можно выделить в окне с исходным кодом интересующее выражение, и оно автоматически появится в диалоговом окне.

В качестве выражения можно использовать переменную, вызов функции или любое выражение RSL. Если при вводе присвоить переменной тип какого-либо объекта, то в процессе вычисления его значения будет вызван конструктор этого объекта.

Во второй строке окна указывается контекст, в котором нужно вычислять значение данного выражения. По умолчанию там содержится текущее значение в стеке. В этой строке можно с помощью раскрывающегося списка выбрать любую позицию стека и, нажав кнопку **Пересчитать** (Recalculate), вычислить выражение в контексте этой позиции. Далее, нажав кнопку **Добавить** (Add), мы поместим в список выражение, которое будет всегда вычисляться в указанном контексте.

Редактирование выражения

Чтобы редактировать выражение, необходимо выполнить одно из следующих действий:

- в меню "**Выражения**" ("Watch") выбрать пункт "**Изменить**" ("Edit watch");
- в дополнительном меню, вызываемом при нажатии правой кнопки мыши, выбрать пункт "**Изменить**" ("Edit watch");
- нажать кнопку  на панели инструментов;

- нажать комбинацию клавиш **[Ctrl+E]**.

В результате этого на экране появится окно для редактирования, аналогичное окну для ввода выражения (см. Рис. 11⁽³²⁶⁾).

С помощью этого окна можно:

- изменить значение выражения путем редактирования значений в полях **Тип (Type)** и **Значение (Value)**;
- вычислить значение выражения в контексте какой-либо позиции стека;
- просмотреть значение какой-либо переменной (локальной или глобальной), которая находится в области стека; если этой переменной нет в области видимости, то ее значение будет не определено.

Примечание.

Следует отметить, что изменять значения можно только для выражений, которые принадлежат к так называемому типу LValue. То есть, переменной **A** можно присвоить значение, а значение выражения **A+20** изменить нельзя.

Настройка отладчика

Чтобы выполнить настройку отладчика, необходимо выбрать в меню "Инструментарий" ("Tools") пункт "Настройки" ("Preferences") или нажать кнопку  на панели инструментов.

С помощью данного режима (см. Рис. 12⁽³²⁷⁾) можно выбрать язык, который будет применяться в интерфейсе (русский или английский), а также выбрать шрифт в окне с исходным кодом и во всех остальных окнах. При первом запуске отладчика по умолчанию устанавливается английский интерфейс.

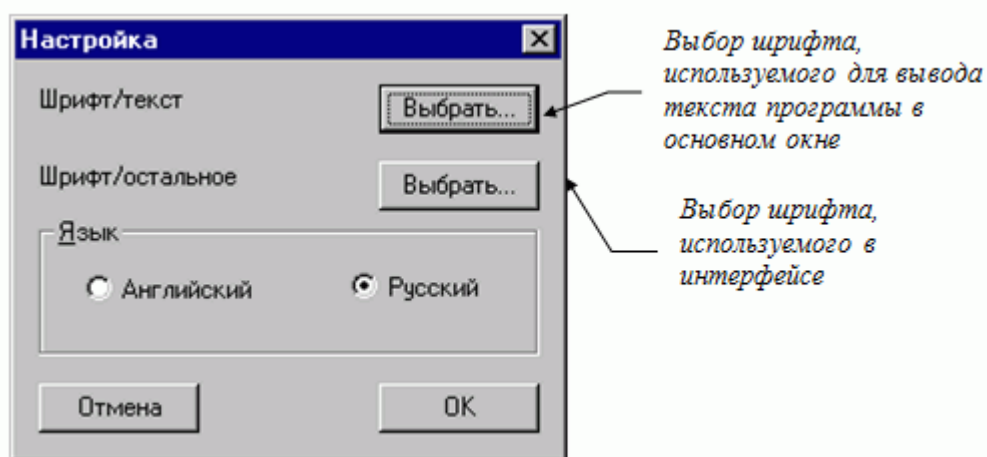


Рис. 12. Настройка параметров отладчика.

Установленные с помощью данного режима настройки сохраняются в системном реестре по следующему пути: `HKEY_CURRENT_USER\SoftWare Lab\ RSL Debugger`.

Список "горячих" клавиш отладчика

Закончить работу отладчика	[Shift+F5]
Шаг с обходом	[F10]
Выполнить до текущей позиции	[Ctrl+F10]
Шаг с заходом	[F11]
Шаг с выходом	[Shift+F11]
Прервать выполнение программы	[Ctrl+Break]
Установить точку останова	[F9]
Удалить точку останова	[Ctrl+F9]
Сделать точку останова активной\неактивной	[Alt+F9]
Стек вызовов процедур	[Ctrl+3]
Окно локальных переменных	[Ctrl+1]
Окно значений	[Ctrl+2]
Окно вывода	[Ctrl+4]
Список импортируемых модулей	[Ctrl+0]
Редактировать выражение	[Ctrl+E]
Удалить выражение	[Ctrl+D]
Добавить выражение	[Ctrl+A]
Список точек останова	[Ctrl+B]
Активизировать окно, содержащее текст программы	[Alt+0]
Активизировать окно выражений	[Alt+2]
Активизировать окно локальных переменных	[Alt+1]
Активизировать стек вызовов	[Alt+3]
Активизировать окно вывода	[Alt+4]
Заккрыть окно	[Alt+F4]

Приложение

В приложении приведен формальный [синтаксис языка RSL на РБНФ](#) ³²⁹.

Сводка синтаксиса RSL

Все терминалы набраны заглавными буквами.

NUMBER – обозначает константу типа Integer, Double, Money.

NAM – последовательность букв и цифр, начинающаяся с буквы, символ подчеркивания '_' считается буквой.

SPNAME – последовательность любых символов, заключенная в фигурные скобки: {34-23-O}.

STRING – строка символов, заключенная в кавычки. В строке могут быть указаны Esc-символы:

- **\n** – новая строка;
- **\r** – возврат каретки;
- **\t** – символ табуляции;
- **\f** – перевод формата;
- **\xHH, \XHH** – символ, заданный кодом (HH две шестнадцатеричные цифры).

Необязательные конструкции заключены в квадратные скобки. Конструкции, которые могут повторяться произвольное количество раз или отсутствовать, заключены в фигурные скобки. Альтернативы разделены символом '|'.

unit	:=	u_decl_list [errhandler] [END].
u_decl_list	:=	u_decl { ';' u_decl }.
errhandler	:=	ONERROR ['(' id ')'] decl_list.
decl_list	:=	decl { ';' decl }.
u_decl	:=	import_def decl.
import_def	:=	IMPORT file_nm_lst.
file_nm_lst	:=	file_name { ';' file_name }.
file_name	:=	id STRING.
id	:=	NAME SPNAME.

decl	:=	macro_def var_def const_def array_def file_def record_def class_def with_def statement.
macro_def	:=	[attr] MACRO id [form_parm] [tpdecl] decl_list [errhandler] END.
form_parm	:=	'([id_list])'.
id_list	:=	tpid { ',' tpid }.
tpid	:=	id [tpdecl]
tpdecl	:=	':' tpname
tpname	:=	id VARIANT INTEGER MONEY DOUBLE DOUBLEL STRING BOOL DATE TIME OBJECT PROCREF DATETIME R2M MEMADDR.
var_def	:=	[attr] VAR init_id_list.
init_id_lst	:=	init_var { ',' init_var }.
init_var	:=	tpid ['=' expression].
const_def	:=	[attr] CONST const_lst.
const_lst	:=	init_const { ',' init_const }.
init_const	:=	tpid '=' expression.
array_def	:=	[attr] ARRAY id_list.
file_def	:=	[attr] FILE id '(' file_name [',' file_name] ')' [fparm_lst].
fparm_lst	:=	fparm { fparm }.
fparm	:=	NORMAL KEY NUMBER WRITE MEM TXT [NUMBER] BTR SORT NUMBER DBF DIALOG BLOB.
record_def	:=	[attr] RECORD id '(' file_name [',' file_name] ')' [fparm_lst].
class_def	:=	[attr] CLASS ['(id ')'] id [form_parm] decl_list [errhandler] END.
with_def	:=	WITH '(' id ')' decl_list END.
attr	:=	PRIVATE LOCAL .
statement	:=	[loop ifstmt retstmt outstmt expstmt].
loop	:=	WHILE condition stmt_list END.

ifstmt	:=	IF condition stmt_list { ELIF condition stmt_list } [ELSE stmt_list] END.
condition	:=	'(' expression ')' .
retstmt	:=	RETURN [expression].
outstmt	:=	'[' control_str ']' ['(' [explist] ')'].
control_str	:=	print_ch format_ch.
print_ch	:=	любой символ, не равный format_ch.
format_ch	:=	'#' .
expstmt	:=	expression.
stmt_list	:=	statement { ';' statement }.
expression	:=	BoolExp { '=' BoolExp }.
BoolExp	:=	SimplExpr [relop SimplExpr].
SimplExpr	:=	['+' '-'] Term { AddOp Term }.
Term	:=	Factor { MulOp Factor }.
Factor	:=	NUMBER STRING '(' expression ')' NOT Factor '@' Factor qualld.
qualld	:=	id { '.' id '(' [expList] ')' }
expList	:=	exp_form { ',' exp_form }.
exp_form	:=	expression [fmt_list].
fmt_list	:=	fmt_spec { fmt_spec }.
fmt_spec	:=	':' fmt_symbol
fmt_symbol	:=	NUMBER 'l' 'r' 'c' 'a' 't' 'd' 'm' 'w' 'z' 'f' 'i' 'iv' 'v'.
relop	:=	'==' '!=' '<' '<=' '>' '>=' .
AddOp	:=	'+' '-' OR .
MulOp	:=	'*' '/' AND .

Существуют следующие правила:

- Все операции, кроме операции присваивания, имеют левую ассоциативность; операция присваивания правоассоциативна.

- Приоритет группы `relOp` самый низкий, `MulOp` самый высокий.
- Порядок вычисления операндов не определен для всех операций, кроме OR (логическое ИЛИ) и AND (логическое И). Для операций OR и AND сначала вычисляется левый операнд, далее, если необходимо, вычисляется правый операнд.

Алфавитный указатель

- _ -

_extObj 53
_callHost 169

- A -

AbortTrn 274
Abs 232
ActiveX 63, 282
AddColumn 109
AddEvent 302
AddModule 183
AddMultiAction 246
AddScrol 237
APPEND 119
ARRAY 60
ArrayRef 33
ASize 220
AttachSite 183
AxCode 80
AxMes 80

- B -

BegAction 288
Bool 32
BREAK 50
BtFileRef 33
BTR 119
BtrError 141

- C -

Call 183
callObjectMethod 187
CallR2M 286
CallRemoteRsl 234
callStaticMethod 187
Cast 179
CheckBits 289
ClassKind 282
ClearColumn 216
ClearRecord 270
ClearStructs 258
Clone 259

Close 258
ClrObj 78
ClrRmtOnRelease 287
ClrRmtStubs 172
CmdArgs 305
cnvMode 116
Code 80
CodeFor 229
collect 187
Connect 175
CONST 57
CONTINUE 50
ConvertDDF 255
Copy 153, 162, 270
CopyBlob 271
CopyFile 277
CopyTblDef 256
Count 160
Create 258
CreateCLRObj 185
CreateComObject 179
CreateEnum 67
createJavaObject 187
CreateMarkObj 170
CreateObject 171
CreateRSLObj 183
CurrentLine 297
CurToStrAlt 223

- D -

Date 32, 220
DateShift 289
DateSplit 221
DateTime 32
DBF 119
DbfFileRef 33
DebugBreak 255
Decimal 32
Delete 262
DelFile 260
DetachSite 183
DIALOG 119
DisableFields 242
DisableValidation 241
disposeAllJavaObjects 187
disposeJavaObject 187
DLM SDK 309
Double 32, 218
DoubleL 32, 218
DropTable 260
DtTm 222

DtTmSplit 222

- E -

EnableFields 242

EndAction 288

ErrBox 303

ErrPrint 303

EvProc 92

EvSource 73

ExecExp 292

ExecMacro 289

ExecMacro2 290

ExecMacroFile 290

ExecMacroModule 291

Execute 183

ExistDir 278

ExistFile 256

Exit 296

Exp 232

- F -

FDate 160

Field 86

FileName 268

FillDown 245

FillUp 245

FindCol 245

FindPath 280

FindRow 245

FldIndex 269

FldName 269

FldNumber 269

FldOffset 270

Floor 219

Flush 154

FlushColumn 215

FOR 48

FTime 160

- G -

GCollect 292

GenAttach 283

GenClassName 283

GenGetProp 284

GenNumProps 286

GenObject 283

GenPropID 285

GenRun 283

GenSetProp 284

GetCallStack 293

GetCurDir 279

GetDate 211

GetDirect 263

GetDouble 210

GetEnv 293

GetEQ 263

GetFileInfo 281

GetGE 264

GetGT 264

GetIniFileValue 281

GetInt 210

GetLangId 306

GetLE 266

GetLocaleInfo 305

GetLT 265

GetMemAddrFrom 293

GetMoney 210

GetMultiCount 247

GetNamedChanel 172, 287

GetNumeric 210

GetObjMethods 284

GetObjProps 284

GetParm 230

GetPos 154, 263

GetPRNInfo 217

GetRecordSize 268

GetScrollFieldValue 252

GetSize 154

GetString 210

GetStringR 210

GetSysDir 280

GetTime 211

GetTRUE 211

GetUIMode 294

GetUserName 305

GetVar 183

GetVarSize 268

GoToScroll 247

- I -

IF 47

IMPORT 44

Index 86, 226

InitProgress 287

Insert 261

InstLoadModule 294

Int 218

Integer 32

IsCopy 160

IsDel 160

IsDir 160
IsEqClass 285
IsGUI 303
IsOutParm 231
IsScrollEditMode 253
IsSQL 305
IsStandAlone 301
IsWeakRef 294
Item 67

- K -

KEY 119
KeyNum 271

- L -

Line 80
List 162
LoadConfig 175
local 41
LockRcwHost 173
Log 232
Log10 232
LoopInTrn 274

- M -

MACRO 57
MakeDir 278
MakeDouble 225
MakeMoney 225
Max 232
MEM 119
MemAddr 32
MemSize 297
Menu 235
MergeFile 280
Message 80, 214
MethodRef 32
Min 232
MkStr 219
Mod 232
Module 80
ModuleFileName 304
ModuleName 304
Money 32, 219
MoneyL 32
MonName 223
MsgBox 242
MsgBoxEx 243

- N -

Name 160
NeedFreeDB 276
newLine 86
Next 141, 260
NRecords 269
Numeric 32
NumToStr 224

- O -

OnError 80, 141
Open 257

- P -

PackVarBuff 267
Parmcount 231
Pow 232
Prev 261
Print 211
PrintFiles 212
PrintGlobs 212
PrintLn 212
PrintLocs 213
PrintModule 213
PrintObject 172
PrintProps 213
PrintRefs 213
PrintStack 213
PrintSymModule 214
private 41
ProcessTrn 144, 272
ProcRef 32

- R -

R2M 286
raise 75
Random 298
rcwhost.d32 181
Read 150
Read2 151
ReadBlob 276
ReadByte 295
ReadMarkObj 170
ReadVal 152
RemHandler 74
Remove 162
RemoveDir 278

RemoveFile 277
 RemProgress 288
 RemTimer 76
 RenameFile 277
 ReplaceMacro 291
 Reset 67
 RETURN 50
 Rewind 261
 Round 224
 rsax.d32 179
 rslr.d32 185
 RsdCommand 129
 RsdConnection 128
 RsdEnvironment 126
 RsdError 137
 RsdField 138
 RsdParameter 138
 RsdRecordset 133
 rsjvm.d32 186
 RslDefCon 144
 RslTimer 75
 RubToStr 222
 RubToStrAlt 223
 Run 183, 232
 RunDialog 93, 240
 RunError 295
 RunMenu 236
 RunScroll 94, 248
 SetRecordAddr 116, 266
 SetScroll 99, 244
 SetScrollFieldValue 252
 SetSize 154
 SetTimer 77, 242
 SetTxtDir 183
 SetVar 183
 ShowDictError 299
 ShowRSCOMError 299
 signatureToRSCOM 187
 Size 160
 Sort 119, 162
 SpecVal 32
 SplitFile 279
 SplitMoney 225
 Sqrt 232
 StartProg 299
 Status 272
 Stop 183
 StrBrk 226
 StrFor 229
 String 32, 219
 StrIsNumber 226
 StrLen 226
 StrLwr 229
 StrongRef 300
 StrSet 227
 StrSplit 227
 StrSplit2 228
 StrSubst 229
 StrucRef 33
 StrUpr 228
 SubStr 227
 SysGetProperty 300
 SysPutProperty 300
 System 301

- S -

ScrollMes 99, 246
 SelectFile 274
 SelectFolder 275
 SetAutoMoneyFloor 224
 SetBuff 270
 SetColumn 214
 SetDbfDir 183
 SetDbtDir 183
 SetDefFile 183
 SetDefMoneyPrec 298
 SetDefPrec 216
 SetDelim 272
 SetDlgFields 245
 SetExitFlag 296
 SetFocus 242
 SetHandler 74
 SetOutHandler 216
 SetOutput 214
 SetParm 231
 SetPos 153
 SetPRNInfo 217

- T -

TArray 61
 Tbfile 111
 TDbError 141
 TDirList 160
 TestEvent 302
 TestExist 183
 Time 32, 221
 TimeSplit 221
 TJavaHost 187
 TJavaObj 190
 ToANSI 230
 ToOEM 230
 ToolsDataAdapter 103

TPattFieldR 89
Trace 254
TRcwHost 182
TRcwSite 175
TRecHandler 116
TRepForm 86
Trim 227
TRsComError 176
TRslChanel 173
TrslError 80, 141
TRslEvHandler 69
TStream 147
TStreamDoc 155
TXT 119
TxtFileRef 33
TypeLib 74

- U -

UnderRCWHost 305
UnlockRcwHost 173
UnPackVarBuff 268
Update 262
UpdateFields 241, 251
UpdateScroll 251
UseProgress 288
UserFill 246
UserNumber 297

- V -

ValType 30, 217
Value 67, 86
VAR 52
Version 297
ViewFile 260

- W -

WeakRef 53
WHILE 48
Write 86, 119, 148
Write2 150
WriteBlob 276
WriteByte 294
writeFields 86
WriteVal 152

- Z -

ZeroValue 307

- K -

Класс Connection 197
Класс CppWrapGen 199
Класс RSMMethod 199
Класс RSOject 195

Контактная информация

R-Style Softlab

**127015 Москва, Новодмитровская улица, дом 2, корпус 1
(Бизнес-центр "Савеловский Сити"), 18 этаж**

<http://www.softlab.ru/>

Связаться с нами можно по телефонам и электронной почте:

Сопровождение

т: (495) 796-9311; E-mail: support@softlab.ru

Корпоративные продажи

т: (495) 796-9310; E-mail: sales@softlab.ru

А также в филиалах:

г. Брянск

т. +7 (4832) 52-4532;
E-mail: Info@bryansk.softlab.ru

г. Вологда

т. +7 (8172) 26-4266, 52-9107;
E-mail: info@vologda.softlab.ru

г. Череповец

т. +7 (8202) 20-12-84;
E-mail: info@vologda.softlab.ru

г. Новосибирск

т. +7 (383) 367-08-01